Onyx — Developer Guide

Onyx · multi-process OS on Raspberry Pi 4

This guide explains how to **build** Onyx, how to **write an application** or a `/bin` **tool**, how to **extend the** `kapi` **ABI**, and the **conventions** + **pitfalls** to be aware of. For the details of how things work internally, see Kernel internals.

Contents

1. Prerequisites and toolchain
2. Building Circle (once)
3. Building the kernel and applications
4. Deploying to the SD card
5. The application model
6. Writing a graphical application
7. Writing a `/bin` tool
8. AppKit's small services (formerly `applib.h`)
9. Packaging an app: `.app`, icons, `config.ini`
10. Extending the `kapi` ABI
11. Coding conventions
12. Debugging on hardware
13. Known pitfalls

---

1. Prerequisites and toolchain

In one command (Linux x86_64 or WSL 2 with Ubuntu, from a fresh clone): `sudo apt install -y git build-essential python3 curl xz-utils`, then **`sh tools/build-sdcard.sh`** [`--card <dir>`] [`--clean`]. The script checks the host tools, finds the `aarch64-none-elf` toolchain (on the `PATH`, in `/opt/toolchains`, or in `~/.cache/onyx`; else it downloads Arm's 14.2.rel1 there, no sudo), fetches and builds Circle (§2, once), builds the kernel and the programs (§3), stages them into `sdcard/` (§4), and with `--card` copies the card to a mounted FAT32 SD card. Jet Browser is not built (WebKit: docs/08) and is not on the card: install the package `jet` on the Pi, from the Package Manager (the Control Panel, the gear on the dock). To build Jet from its sources too: **`sh tools/build-jet.sh`** (hours; docs/08 "Resume"). The sections below are the same steps by hand.

- **AArch64 bare-metal toolchain**: `aarch64-none-elf-` (GCC), used under **WSL** on a Windows development machine.
- *(Only for POSIX / C++ ports)* **the Onyx toolchain** `aarch64-onyx-elf-` (below).

1.1. The two toolchains

Toolchain	Install	Used for
`aarch64-none-elf` — Arm GNU Toolchain 14.2.rel1 (GCC 14.2.1, newlib 4.4; thread	`/opt/toolchains/arm-gnu-toolchain-14.2.rel1-x86_64-aarch64-none-elf` (Arm's	**the kernel and every existing app and `/bin` tool** (`onyx_syscalls.c`,

Toolchain	Install	Used for
model `single`, no TLS)	download)	§5.1). Unchanged.
`aarch64-onyx-elf` — ours (WP-TC, `tools/toolchain/`): GCC 14.2.0 + binutils 2.43 + newlib 4.4, `--enable-threads=posix --enable-tls`, newlib `--enable-newlib-reent-thread-local`	`/opt/toolchains/aarch64-onyx-elf-14.2`	**programs on libonyxposix (§5.4) and the third-party ports (§5.5)**, above all C++: real `std::thread` / `mutex` / `condition_variable`, native `thread_local` (`TPIDR_EL0`), thread-safe statics, `errno` per thread, `steady_clock` on `clock_gettime`. `/bin/posixtest-cxx`.

Its libgcc and libstdc++ are built against libonyxposix's `<pthread.h>` (a header overlay installed into the toolchain): the pthread types are **ABI-frozen** (docs/POSIX-PLAN.md "WP-LIBC resolutions" 7). Any change to `pthread.h`, `sys/_pthreadtypes.h`, `semaphore.h`, `sys/dirent.h` or `sys/features.h` in `user/Runtime/libc/posix/include` means a new toolchain revision.

Getting it (Linux x86_64, or **WSL** on Windows — in the WSL shell, as for the rest of the build):

```
sh tools/toolchain/fetch.sh                  # the prebuilt one, from stephaneweg/onyx-toolchain (git, sha256 checked)
# or build it (about 30 min on 4 cores, ~10 GB of build space in ~/.cache/onyx-toolchain):
sudo apt install build-essential m4 xz-utils curl patch
sudo mkdir -p /opt/toolchains && sudo chown $(id -u):$(id -g) /opt/toolchains
sh tools/toolchain/build-onyx-toolchain.sh   # PREFIX=<dir> elsewhere; resumable; prints the checks
```

Under WSL keep the build directory on the Linux side (the default `~/.cache`), not under `/mnt/c` (ten times slower). Nothing selects it for the kernel: `kernel/` and `user/` keep `aarch64-none-elf-`. What does select it when it is installed: `tools/onyx-env.sh`, `tools/onyx-toolchain.cmake`, `tools/ports/build-all.sh` (`ONYX_TOOLCHAIN_PREFIX=aarch64-none-elf-` forces the interim one), and `user/BinUtils`'s `posixtest-cxx` (skipped with a note when absent). `make -C user/Runtime/libc/posix PREFIX=aarch64-onyx-elf-` builds libonyxposix for it (`build-onyx/`, sysroot `out/sysroot-onyx`).

- **Circle**, pulled in as a **git submodule** at `circle/` (our fork `stephaneweg/circle`, branch `onyx`).
- `make`, `cp`, `mkdir` (standard Unix tools, via WSL).
- A **FAT32 SD card** and a **Raspberry Pi 4** to run on (there is no QEMU `raspi4b` in the reference dev environment; bring-up is done directly on hardware).

> **Important — which Circle?** `circle/` is a **git submodule** → our fork `stephaneweg/circle` (branch `onyx`), **not** any other clone. Patch Circle *there* (commit on `onyx`), rebuild the affected library, then record the new commit in the superproject (`git add circle && git commit`). The patches are listed in Circle Changes.

2. Building Circle (once)

```
# Circle is a git submodule (the fork stephaneweg/circle, branch onyx). Fetch it
# (or clone Onyx with --recurse-submodules in the first place):
git submodule update --init --recursive

# Configure for RPi 4 / AArch64. DEPTH=32 is REQUIRED: our GImage renders 32-bit
# pixels (Circle defaults to 16).
cd circle && ./configure -r 4 -p aarch64-none-elf- -d DEPTH=32 -f
```

```
# Build the libraries used by the kernel (make clean first when the fork's sysconfig.h
# changed -- e.g. the multi-core patch: the .o files do not depend on it):
for d in lib lib/sched lib/fs lib/fs/fat lib/usb lib/input lib/net lib/sound \
         addon/SDCard addon/fatfs addon/wlan; do (cd $d && make clean && make -j4) || break;
done
(cd addon/wlan/hostap/wpa_supplicant && make -f Makefile.circle clean && make -f Makefile.cir
cle -j4)
cd ..
```

> If you change `DEPTH` later, run `make clean` in `circle/lib` before rebuilding (the `.o` files do not depend on `Config.mk`).

The libraries linked by the kernel (cf. `kernel/Makefile`): `libwpa_supplicant.a`, `libwlan.a`, `libfatfs.a`, `libsdcard.a`, `libnet.a`, `libsound.a`, `libsched.a`, `libusb.a`, `libinput.a`, `libfs.a`, `libcircle.a`. Circle is built **multi-core** (`ARM_ALLOW_MULTI_CORE`, fork patch #4): core 1 runs the sound producer.

The Raspberry Pi 5 (a separate distribution, docs/PI5-PORT.md): Circle's libraries are per board, so the Pi 5's are built in their own tree, **`circle5/`** (a copy of `circle/`'s sources configured `-r 5`, not in git): `sh tools/pi5/circle5.sh` (again after any change in `circle/`), then `make -C kernel BOARD=pi5` → `kernel/kernel_2712.img`. The kernel's objects are shared by both boards: switching `BOARD` cleans them first (`kernel/.board`). **`make -C kernel BOARD=pi5 stage`** makes the Pi 5's card **`sdcard5/`**: `tools/pi5/mkcard5.py` copies `sdcard/`'s files without the Pi 4's own (its kernel, boot firmware, device trees, Wi-Fi files, `config.txt`, `cmdline.txt`) and adds `tools/pi5/overlay/` (the Pi 5's `config.txt` — 32-bit framebuffer, no alpha — `cmdline.txt` with `gpiofanpin=45`, the `bcm2712*` device trees and `overlays/bcm2712d0.dtbo` from `raspberrypi/firmware`, the Wi-Fi chip's files under the Pi 5's names, `etc/pkg/pkg.ini` → the repository's `pi5/`), then the kernel and the programs as for the Pi 4 (one userland for both boards so far). Stage the Pi 4's card first: `sdcard/` is the source of what is not built. Its packages: `sh tools/pkg/publish.sh --board pi5`.

> The Onyx-specific patches carried by this fork (branch `onyx`, on upstream tag `Step51.1.1`) are documented in Circle Changes.

3. Building the kernel and applications

> **The kernel's size limit.** The kernel image **and its BSS** must end below `0x280000` (loaded at `0x80000`, Circle's `KERNEL_MAX_SIZE` = 2 MB): past it the BSS runs over the kernel's stacks and the Pi does not boot at all, without a message. `make` checks it after each link (`sizecheck`: `_end` in `kernel8-rpi4.map`) and deletes an image too big. Put big buffers on the heap (`new` at init), not in static arrays.

From `kernel/`:

```
cd kernel
make            # -> kernel8-rpi4.img, THEN builds the apps (../user)
```

The default target (`all`):

1. compiles our sources + links against the Circle libs → **`kernel8-rpi4.img`**;
2. triggers `make -C ../user`, which builds **all the apps** (each from its folder `user/Apps/<name>/`, to `user/<name>.elf`), the **`/bin` tools** (`user/BinUtils/*.elf`), Doom (`user/Ports/doom`) and Koton's plugins. Jet Browser, on WebKit, is built apart (`sh tools/webkit/build-web.sh`, docs/08).

> **Kernel → apps order.** Since the apps go through the **fixed-address ABI table** and are **not** linked against the kernel's addresses, they no longer depend on the kernel build: a *kernel-only*

change (that respects the ABI) does **not** require recompiling the apps. `make` rebuilds them anyway for convenience.

Details of the kernel wiring (which Circle files we replace, the order of the `-I compat -I include` include path before `circle/include`): see Kernel internals §1 and `kernel/Makefile`.

4. Deploying to the SD card

```
cd kernel
make stage     # copie l'image + chaque app + chaque outil vers ../sdcard/
```

`make stage`:

- copies `kernel8-rpi4.img` → `sdcard/`;
- for each `../user/<name>.elf`: creates `sdcard/apps/<name>.app/` and copies the ELF into it as **`main`** (the "Onyx" layout; an app's other files — `app.txt`, icons, resources — are kept in `sdcard/apps/<name>.app/` itself, committed);
- Koton's plugins (`user/Apps/kp_<name>/`) → `sdcard/koton/plugins/<name>/main` + `plugin.json`;
- the BASIC apps (`arkanoid`, `planets3d`) compiled by `tools/basc` → `apps/<name>.app/main.bax`, and the standalone ones (`antigrav`) → `apps/<name>.app/main` (`basc --standalone`: the BASIC runtime with the program inside);
- copies each `../user/BinUtils/<tool>.elf` → `sdcard/bin/<tool>`.

Executables on the card have no extension: `.elf` only exists in the build tree (`user/*.elf`, `user/BinUtils/*.elf`); staging drops it. Programs are recognized by their content (ELF magic `7F 45 4C 46`), e.g. by the file manager.

Then copy **all** of the contents of `sdcard/` onto a **FAT32** card and boot the Pi 4. See the user guide for details about the card and the configuration files.

Keyboard layouts live as binary files in `sdcard/etc/keymaps/<NAME>.kmap` (the `keyb` tool — and the theme editor's dropdown, which lists whatever `.kmap` files are actually present — load one and send it to the kernel via `kapi_set_keymap_data`, v27). **The kernel compiles in no keyboard map at all**: at boot the keyboard table is empty and stays so until `keyb` loads a layout from the card (see Circle Changes §1), so a layout is *only* ever a file — adding one needs no kernel rebuild. Regenerate the `.kmap` files with `python tools/keymaps/genkeymaps.py`; it compiles each `<NAME>` from a `keymap_<name>.h` table in **`tools/keymaps/maps/`** (the 13 layout sources, incl. `BE` = Belgian azerty and, since 2026-10-10, the Raspberry Pi keyboards' `SE` (Nordic), `NO`, `DK`, `PT`, `JP` -- the Japanese Ro / Yen keys, USB codes 0x87 / 0x89 past the table's 128 rows, come at rows 0x7D / 0x7E: fork patch 32, all tracked in the Onyx repo — not in the Circle tree). The `.kmap` format is `"OKM1"` + `u16` rows/cols + the `u16[128][5]` table (see the script header).

5. The application model

The sources of user space are sorted by their use, in `user/`:

Folder	What is in it
`Apps/<name>/`	the graphical applications
`BinUtils/`	the console programs (the card's `SD:/bin`)
`Kits/<kit>/`	the shared libraries: AppKit, UIKit, AudioKit, FileKit, ImageKit, PrinterKit, FontKit (§5.6 to

Folder	What is in it
	§5.10)
`Runtime/`	what every program is linked with: `crt0.S`, `user.ld`, the libraries' `lib.ld` / `lib.h` / `librt.cpp`, `umm.h`, `onyxpp.hpp`, and `libc/` (newlib's glue, the POSIX library)
`Libs/`	the libraries linked into the programs: `av`, `img`, `zlib`, `tls`, `v3d`, `gpucomp`, `pdf`, `mail`, `pkg`, `basic` (and `demo`, the loader's test library)
`Emulators/`	the emulators' cores (`gb`, `gba`, `nes`, `snes`, `n64`, `gc`, `emucore.h`); their windows are apps
`Ports/`	third-party programs ported: `doom`, `stk`
`Include/`	the small headers several programs share (`gamepad.h`, `http.hpp`, `json.hpp`, `trash.h`, `fileassoc.h`, `docguard.h`…)
`Apps/games/`	what the games share (`game.h`, `cards.h`)
`lib/`	the build's outputs for the libraries (not in git)

`user` and these folders (`Kits`, `Runtime`, `Include`, `Libs`, `Emulators`, `Ports`) are on every include path, so a source includes a header the same way whatever its own folder: `"appkit/appkit.h"`, `"uikit/uikit.h"`, `"uikit/clipboard.h"`, `"filekit/fsutil.h"`, `"umm.h"`, `"gamepad.h"`, `"tls/onyx_tls.hpp"`, `"gb/gb.h"`. A build of our sources elsewhere (a PC test) passes `-I user -I user/Kits -I user/Runtime -I user/Include -I user/Libs -I user/Emulators -I user/Ports -I kernel/include`. A header that one program uses is beside that program.

An Onyx application is a folder **`user/Apps/<name>/`** whose `main.cpp` (C++ on the **uikit** toolkit, §6; a big app has more files beside it) is compiled into an **ELF** — freestanding, or against newlib (§5.1) — and run at **EL0** in its own page table, calling the kernel by system calls through the kapi table (docs/02 §6). A `/bin` tool is a single C file, `user/BinUtils/<tool>.c` (§7).

Runtime (`user/Runtime/crt0.S`):

```
_start:
    stp x29, x30, [sp, #-16]!   /* the stack is already set up by the kernel */
    bl  main                    /* call main() */
    ldp x29, x30, [sp], #16
    ret                         /* return -> El0MainReturn (the EL0 code page): exit(0) */
```

No argv is passed via the stack: `main()` takes no arguments. An app retrieves its argument line via `kapi_get_args(buf, size)`, and exits early with `kapi_exit(status)`.

Linking (`user/Runtime/user.ld`): everything is linked at **`USER_VA_BASE` = 8 GB** (`. = 0x200000000;`). This is **not PIE**. The RX (`.text`/`.rodata`) and RW (`.data`/`.bss`) sections are aligned on **64 KB** (`-z max-page-size=0x10000`) so that the loader maps them onto distinct pages. **No kernel symbol** is resolved at link time: everything goes through the ABI table.

Compilation flags (`user/Makefile`):

```
-ffreestanding -nostdlib -fno-pic -fno-pie -mgeneral-regs-only \
-O2 -Wall -Wextra -fno-stack-protector -I../kernel/include
```

- `-ffreestanding -nostdlib`: **no libc**. No `printf`, `malloc`, `string.h`… use AppKit's small services (§8) and static/local buffers.
- `-mgeneral-regs-only`: integer-only codegen — the **default** for apps. Most apps do **integer / fixed-point arithmetic** (see `tinycalc`, `mandelbrot`).

- **Hardware float is available (opt-in).** The kernel now saves the full FP/SIMD state on every trap (see Kernel internals §6, trap frame), and FP/SIMD is enabled at EL0 (`CPACR_EL1`). To use `float`/`double` in an app, build it **without** `-mgeneral-regs-only` and **with** `-mcpu=cortex-a72` (the Pi 4 core). Basic FP (`+ - * /`, int↔double) compiles to hardware instructions with **no** soft-float runtime; transcendental math (`sin`/`cos`/ `pow`…) lives in newlib's `libm` — see §5.1. For the bare-FP recipe, see the `fptest` target in `user/BinUtils/Makefile` (a target-specific `CFLAGS` override) and the `fptest` proof tool. Opt **only** the FP app in — leave the rest integer-only.
- `-I../kernel/include`: to include `<kern/kapi_abi.h>` (the shared ABI structure).

To add an app, create `user/Apps/<name>/main.cpp` and **add `<name>` to the `APPS` list** of `user/Makefile` (`FT_APPS` for a FreeType app, or a rule of its own for a newlib app, as `letters.elf`), and declare its package in `tools/pkg/packages.ini`; for a tool, add `<tool>.elf` to the `PROGS` of `user/BinUtils/Makefile`.

A text tool (a filter as `head`, `sed`, `sort`: `TEXT_PROGS` in that Makefile) is written on `user/BinUtils/tool.h`: it defines `TOOL_NAME` and `int tool_main (int argc, char **argv)`, and gets its argv, a buffered stdout (`t_puts`, `t_putnum`…), files and stdin read by lines (`t_open`, `t_getline`, `t_slurp`), memory (`t_malloc`) and the exit code (`tool_main`'s, through `kapi_exit`: what the shell's `&&`, `||` and `$?` read — a tool on `crt0.S` alone must call `kapi_exit` itself). The variables of the shell reach a program as its **environment** (`kapi_get_env`, `getenv` on the POSIX layer). The regular expressions of `grep`, `sed` and `ed` are `regex.h` and `subst.h`. The same sources build on the PC with `-DTOOL_HOST` (the libc behind the same calls): **`sh tools/tests/run_tools_test.sh`** runs every tool against expected outputs, under the address sanitizer — add a tool's cases there. The shell's parser, command lists and file patterns (`cmdparse.h`), its script language (`cmdscript.h`: variables, `$(…)`, `$((…))`, `test`, `if` / `while` / `for` — no kapi in it: `cmd.c` gives it the pipelines, the variables and Ctrl-C through `struct CsHost`, the test gives it a stand-in) and the consoles' line editor with its history (`user/Include/lineedit.h`, shared by the terminal and `telnetd`) are tested by **`sh tools/tests/run_cmd_test.sh`**. The end of a remote session — `telnetd.c` itself and `user/BinUtils/shellend.h` against a mock kapi (a clock, the kernel's 8 KB pipes, a scripted client, a model of `cmd`): a client that closes or vanishes, at the prompt or while a program runs, leaves no shell behind — by **`sh tools/tests/run_telnetd_test.sh`**.

5.1. Apps using the C library (newlib)

The `aarch64-none-elf` toolchain ships **newlib** (`libc` + `libm`). An app can be built against it to use the real `<stdio.h>` (`printf`, `FILE*`, `fopen`/`fseek`), `<stdlib.h>` (`malloc`/`qsort`/`strtod`), `<string.h>`, and `<math.h>` instead of the freestanding helpers (AppKit's `ax_*` / `umm.h`). This is the foundation for porting large C codebases.

How it works: `user/Runtime/libc/onyx_syscalls.c` implements the handful of POSIX stubs newlib bottoms out in (`_sbrk`, `_read`, `_write`, `_open`, `_close`, `_lseek`, `_fstat`, `_gettimeofday`, …) on top of the kapi ABI. `user/Runtime/libc/crt0libc.S` is the entry point — like `crt0.S` but it calls `exit(main())` so stdio is flushed on the way out. Build flags drop `-ffreestanding`/`-nostdlib` and use `-nostartfiles` (keep our entry, keep `libc`); add `-mcpu=cortex-a72` (FP is required by `printf %f` and `libm`) and link `-lm`. See the `LIBC_PROGS` rule in `user/BinUtils/Makefile` and the proof tool `user/BinUtils/libctest.c`.

A **uikit app** can be a newlib app too (Doom, **Letters**, the **Spreadsheet** — FreeType wants a libc): the `letters.elf` rule of `user/Makefile` is the model — `NL_CFLAGS` / `NL_CXXFLAGS` (hardware FP, `-nostartfiles`, sections for `--gc-sections`), `libc/crt0libc.o` + `libc/onyx_syscalls.o`, the app, `lib/uikit.imp.a`, then its libraries (`lib/fontkit.imp.a`) and `-lm` (the import libraries of the shared uikit and FreeType: §5.6). Take the app out of the generic `APPS` list and add its `.elf` to `all:`; `make stage` stages it as any.

Notes / caveats:

- **One allocator** for a plain C newlib app: newlib's `malloc` owns the heap via `_sbrk`→`kapi_sbrk`; do **not** also link `umm.h`. (A uikit app on newlib has two, side by side: uikit's C++ objects on umm — `onyxpp.hpp`'s `operator new` — and the C libraries on `malloc`; each grows its own arena with `kapi_sbrk`.)
- **Files are buffered in RAM.** `_open` slurps the whole file into memory to give `fseek`/`ftell` full semantics, and a writable file is written back with `kapi_save_file()` on `close`. Fine for resource files; the kapi has had `seek` since v57 (and `file_out` for streamed writes), which `onyx_syscalls.c` does not use yet — a big file is better read with `kapi_open` / `kapi_seek` / `kapi_read` directly (the GameCube discs, the Media Player). A new port or tool should rather use **libonyxposix** (§5.4): real descriptors on the v75 open files, pthreads, sockets, `poll`, `mmap`.
- **Size.** Static newlib pulls a fair amount of code (`printf` float support etc.). Acceptable for big apps; a `nano` variant can be revisited later for small tools.
- **Threads** (§5.2): newlib's locks are defined in `onyx_syscalls.c` (newlib is built with retargetable locking), so `malloc`, the `FILE`s, `atexit`… are safe between threads. `errno` and newlib's other reentrancy state (`_impure_ptr`) are **shared** by the threads.

5.2. Threads (kapi v67)

A process may run **threads**: more tasks in its own address space — the same memory, window, files and sockets —, preempted like the main one, all on core 0. What they are for is **concurrency**: a thread blocks (a socket, a file, a long computation) while the main thread keeps pumping the window's events. (For parallel computing on another core, see the app cores, §6.)

```
static int worker (void *arg)              /* runs in the new thread */
{
    long n = fetch_something (arg);        /* blocks: the UI goes on meanwhile */
    kapi_post (on_done, arg, n);           /* on_done (arg, n) runs on the main thread */
    return 0;                              /* its exit code */
}

int tid = kapi_thread_create (worker, ctx, 0, "fetch");   /* 0: a 256 KB stack */
...
while (!kapi_should_exit ())
    kapi_pump_wait (100);                  /* sleeps until an event, a post or the close box *
/
kapi_thread_join (tid, KAPI_WAIT_FOREVER, &code);
```

- **Threads**: `kapi_thread_create (fn, arg, stack_size, name)` → a tid ≥ 2 (the main thread is 1), −1 no memory, −2 too many (**32** besides the main one). The stack is 256 KB by default, 16 KB .. 16 MB. The name is `<app>:<name>` (in the crash reports; `ps`, the task manager and the taskbar show the app once). `fn`'s return value — or `kapi_thread_exit (code)` — is the exit code, which `kapi_thread_join (tid, timeout_ms, &code)` collects (−1 timeout, −2 no such thread or joined already, −3 itself); a thread never joined is forgotten once its record is needed again. `kapi_thread_self ()` → the caller's tid.

- **The process ends with its main thread** (returning from `main`, `kapi_exit`), or when any thread calls `kapi_exit`: the other threads end with it, wherever they are. Killing the app (the task manager, `kill`) kills them all.

- **Synchronisation objects** — handles, 256 per process, freed with it or by `kapi_sync_close (h)` (its waiters get −2). Timeouts are in ms: 0 only tries, `KAPI_WAIT_FOREVER` waits.
    - **mutex**: `kapi_mutex_create ()`, `kapi_mutex_lock (h, timeout)` (0, −1 timeout), `kapi_mutex_unlock (h)`. Recursive; released if its owner thread ends.
    - **event**: `kapi_event_create (manual_reset, initial)`, `kapi_event_set`, `kapi_event_reset`, `kapi_event_wait (h, timeout)`. A manual-reset event stays set (it releases every waiter); an auto-reset one is taken back by the wait that gets it (one waiter per set).
    - **barrier**: `kapi_barrier_create (count)`, `kapi_barrier_wait (h)` — returns once `count` threads are in: 1 for the last one in, 0 for the others; the barrier is then ready for the next round.
- **Calls posted to the pump**: `kapi_post (fn, ctx, value)` queues `fn (ctx, value)` (256 at most: −1 when full); the pump runs it — `kapi_pump_events`, `kapi_pump_wait`, `kapi_wait_for_exit` — on the thread that pumps. That is how a worker hands its result to the UI: no lock needed on the UI's data. `kapi_pump_wait (ms)` sleeps until a window event, a post or the close box (or the timeout), then pumps → what was pending (0: the timeout). `kapi_wait_for_exit` wakes on a post too.
- **Rules**:
    - **The GUI belongs to the thread that pumps** (the main one): draw, present, open dialogs and change the window from it; other threads post to it.
    - **Memory is shared**: protect what several threads write (a mutex, or post the work to one thread). `umm_malloc` / `umm_free` and newlib's `malloc` take a lock; `errno` is shared.
    - For a small lock of your own there is `kapi_lock (&int)` / `kapi_unlock (&int)` (a swap and a yield: no handle, a zeroed `int` is a free lock; not recursive).
    - A kapi call is not preempted, but it may wait (a file read in pieces, a socket): another thread of the process can run meanwhile, even in another kapi call.
- Example and test: `user/BinUtils/threadtest.c` (`threadtest` in a terminal: every check, then PASS / FAIL).
- **Word waits — a futex (v68)**: `kapi_wait_word (&word, expected, timeout_ms)` sleeps while `word == expected` → 0 (woken, or the value already differs), 1 timeout (0 ms: only check), −1 a bad address (not 4-byte aligned, unmapped). `kapi_wake_word (&word)` wakes its sleepers → how many. Nothing to create: any `unsigned` of your memory, your stack or a **shared surface** — the kernel keys the sleepers by the physical address, so two processes that map the same surface (a plugin and its host) wait / wake on the same word. **Code on an app core** changes words without calling anything (it cannot): the kernel reads every sleeping word at each 10 ms tick and wakes those that changed — so a core-0 thread can sleep until the engine on core 2 has written (≤ 10 ms late; call `kapi_wake_word` when you can, it is immediate). Wakes may be spurious: loop on your condition.

```
while (ring->ready == 0)                        /* set by another thread / process /
 core */
    kapi_wait_word (&ring->ready, 0, 100);
```

- **Priority (v68)**: `kapi_thread_priority (tid, 1)` (tid 0 = the caller, 1 = the main thread) makes a thread **"real time"**: whenever it is ready it runs before the others, and the next tick preempts an app for it. It keeps that only while it sleeps / waits before its 20 ms slice ends — a thread that computes through its slice is an ordinary one until it next sleeps, so it cannot freeze the system.

For an audio pump, a plugin's render loop. `-1` asks → the previous priority; −2 no such thread. Test: `user/BinUtils/futextest.c`.

5.3. Files in memory: the `RAM:` volume (kernel v71)

`RAM:` is a volume in the kernel's memory (docs/02 §16): **every file call works there as on the card** — `kapi_open` / `kapi_read` / `kapi_fsize` / `kapi_seek` / `kapi_close`, `kapi_save_file`, `kapi_file_in` / `kapi_file_out` (append too), `kapi_opendir` / `kapi_readdir`, `kapi_mkdir`, `kapi_remove`, `kapi_rename`, `kapi_chdir` — so newlib's `fopen` / `fwrite` / `remove` do too. Use it for what may be lost and should not wear or wait on the card: caches, temporary files, a download being unpacked.

- **Until the Pi restarts.** Not tied to your app: what it leaves there is still there when it runs again (the same boot), gone after a restart — check for a file before trusting it, rebuild it if it is missing. Choose a folder of your own (`RAM:/<app>/`) and create it (`kapi_mkdir` each level: it does not create parents; −1 when it exists).
- **Fast**: a `save_file` / `read` is a copy into the kernel's pages — no pacing, no SD stall (§*Pitfalls*: the card's writes freeze core 0). Big transfers yield every 1 MB.
- **Bounded**: 128 MB by default (`system.ini` `ramfs=`; less on a 1 GB Pi), and a reserve of free memory is always left to the apps, so a write can fail when memory is short: `kapi_save_file` returns −1 (and leaves no half file), a stream write a short count. **`kapi_vol_info ("RAM:", &vi)`** (v71) says the size / used / free (and `-1`: no `RAM:` — an older kernel or `ramfs=0`: fall back to the card). The memory of a removed file comes back at once.
- Names: up to 127 characters, **case-insensitive** (case kept), as on the FAT card. `rename` works within `RAM:` only (across volumes: copy then remove). Programs cannot be run from it.
- The shell: `ls RAM:`, `cd RAM:/x`, `cp`, `mv`, `rm`, `mkdir`, `cat`, redirections (`> RAM:/log`, `>>`) and **`df`** (the volumes' room). `/bin/ramtest` exercises it on the Pi.
- **On the PC** (the desktop simulator, `tools/tests/desktop_sim/fakekapi.cpp`): `RAM:` is the folder **`SIM_RAM`** — the same folder for several runs is several launches within one boot unset, each run gets a fresh temporary folder, deleted at its end (a boot of its own). `vol_info` answers 128 MB.

5.3.1. USB sticks and the volumes (kernel v93)

A USB stick (or disk) is mounted by the kernel when it is plugged in: **`USB1:`** (then `USB2:`, `USB3:` for a second and a third device; **`USB:`** is `USB1:`) — the device whole, when it has one partition or none. A device with **several** partitions gives **`USB1P1:`, `USB1P2:`**… instead (one per partition; no `USB1:` then). Every file call works there as on the card; a program has nothing to do to support it, but:

- **The stick may leave at any time.** Pulled out without an eject, its files fail: a read or a write returns an error (`-1`, `-KAPI_EIO`), a folder listing ends, and that open file stays dead even if the stick comes back (open it again). Check the results of writes and of `kapi_file_sync` / `kapi_close` when saving to a stick; keep your own copy until the save succeeded.
- **The volumes**: `kapi_vol_list (out, max, flags)` (`struct kapi_volume`: name, state `KAPI_VST_*`, flags `KAPI_VF_*`, sizes, type, label, the files open on it, `gen` — it changes at each event of that volume: poll it once a second to follow the sticks, as the menu bar, the File Viewer and Disks do; `KAPI_VOLS_ROOM` adds the free space, which may read the volume's FAT once: not in a loop). A volume shown to the user: `state == KAPI_VST_MOUNTED`; one that can be ejected: `flags & KAPI_VF_REMOVABLE`; the volumes of one device have the same `device` (`"umsd1"`).
- **Eject**: `kapi_vol_eject ("USB1:", 0)` — the whole device, whichever of its volumes you name → 0 (safe to remove), `-KAPI_EBUSY` (files are open on it — they were synced; ask the user, then

KAPI_EJECT_FORCE). Close your own files on the stick first, and move your view off it (a File Viewer showing USB1: holds nothing open, but its next listing would fail).

- **Format**: kapi_vol_format ("USB1:", &fmt) (the whole device, one partition, whatever it had; "USB1P2:": that partition only) (struct kapi_format: KAPI_FMT_AUTO / FAT / FAT32 / EXFAT, a label of 11 characters at most, a cluster size or 0). The call blocks until done (seconds). The kernel refuses SD: (-KAPI_EPERM) whatever you pass; SD1:..SD3: need KAPI_FMT_CARD — set it only after the user confirmed a second time (Disks does).
- **Mount again**: kapi_vol_mount ("USB1:") — a stick ejected but still plugged in (all its volumes).
- The kernel's side: docs/02 §18; the tools: mount, eject, mkfs, df (user/BinUtils, volutil.h); the app: Disks (user/Apps/disks). **On the PC** the simulator has a 14.9 GB exFAT stick when SIM_USB=1 (SIM_USB=2: two partitions, USB1P1: FAT32 and USB1P2: exFAT; fakekapi.cpp: Eject, Mount and Format change its state as the kernel would).

5.4. The POSIX layer (libonyxposix)

user/Runtime/libc/posix/ is a **POSIX C library layer** over newlib and the kapi (docs/POSIX-PLAN.md §3.4): what large portable code (SQLite, libxml2, curl, and later ICU, Skia, WebKit) expects — real file descriptors, pthreads, BSD sockets, poll, mmap, clock_gettime, posix_spawn. It is the kernel's v75 POSIX ABI (docs/02 §8, "v75") seen from C, and v76's IPC between processes (docs/02 §8 "v76: IPC": local sockets, descriptors passed with SCM_RIGHTS, shared memory — what WebKit2's processes use). MIT, ours.

A program links **libonyxposix or onyx_syscalls.c, never both**: the existing newlib apps (§5.1: Letters, Doom, the TLS tools, pkg, rdpd, the SuperTuxKart port...) keep onyx_syscalls.c and are unchanged; new ports and tools use libonyxposix.

Building with it. make -C user/Runtime/libc/posix builds build/libonyxposix.a, crt0posix.o, onyx-posix.ld and build/onyx.specs (in-tree programs); make -C user/Runtime/libc/posix install SYSROOT=<dir> makes the **sysroot** third-party code builds against (§5.5). A program needs two flags — the headers first on the path, and the specs, which bring the start code, the link script and the libraries:

```
aarch64-onyx-elf-g++ -std=gnu++20 -mcpu=cortex-a72 -O2 -isystem <sysroot>/include -DFD_SETSIZE=1024 \
    -specs=<sysroot>/lib/onyx.specs prog.cpp -o prog.elf    # (C: gcc, the same flags)
```

Either toolchain (§1.1) works for C; **C++ with threads needs aarch64-onyx-elf**. libonyxposix is built per toolchain — make -C user/Runtime/libc/posix with aarch64-none-elf (build/, sysroot out/sysroot), make -C user/Runtime/libc/posix PREFIX=aarch64-onyx-elf- (build-onyx/, sysroot out/sysroot-onyx); the Makefile tells them apart by $(CC) -dumpmachine, the sources by newlib's _WANT_REENT_THREAD_LOCAL (ONYX_NATIVE_TLS). Objects of the two do not mix.

main (int argc, char **argv, char **envp) gets its arguments and environment; exit flushes stdio. In the tree: the POSIX_PROGS rule of user/BinUtils/Makefile (posixtest, aarch64-none-elf) and POSIX_CXX_PROGS (posixtest-cxx, aarch64-onyx-elf: built when that toolchain is installed, else skipped with a note).

What it provides (each piece on its v75 kapi; on a kernel without it, the fallback in brackets — so a program runs on today's kernel too, with less):

Area	Calls	On the kernel	(without v75)
Descriptors	open (O_CREAT/EXCL/TRUNC/APPEND/CLOEXEC/NONBLOCK/DI	file_* (64-bit offsets per	the old calls: read-only through

Area	Calls	On the kernel	(without v75)
	RECTORY), read, write, lseek, pread, pwrite, readv/writev, close, dup/dup2/dup3, fcntl (flags, FD_CLOEXEC, F_DUPFD; locks always granted), flock, ftruncate, fsync, pipe/pipe2, openat…	handle, pread / pwrite, unlink of an open file), the pipe streams, stream_write_nb	kapi_open/kapi_seek; a writable file held whole in memory, written back at fsync/close (as onyx_syscalls.c)
Files	stat/lstat/fstat/fstatat, access, unlink/rmdir/remove, mkdir, rename (replaces its target), utime(s)/utimensat/futimens, statvfs, realpath, getcwd/chdir/fchdir, mkstemp/tmpfile (newlib's), truncate	path_*, file_stat (mtime, a 64-bit id), vol_info	stat probes with opendir/open; kapi_remove/kapi_rename
Directories	opendir/fdopendir/readdir/readdir_r/closedir/rewinddir/dirfd, scandir, alphasort	dir_read (255-character names)	kapi_readdir (127)
Threads	pthread_create (stack size up to 16 MB, detached), join/tryjoin_np/timedjoin_np, detach, exit, self, setname_np/getname_np, getattr_np (the stack's bounds), sched_yield, priorities (> 0: the kernel's "real time")	thread_create_ex / thread_info (lazy 8 MB stacks, the TLS pointer)	thread_create (v67: the stack mapped at once, 1 MB by default), bounds estimated
Synchronisation	mutexes (normal, recursive, errorcheck, timedlock, clocklock), condition variables (timedwait on CLOCK_REALTIME, or CLOCK_MONOTONIC with condattr_setclock; clockwait), once, keys with destructors (128), rwlocks, barriers, spin locks, sem_*	the futex (wait_word / wake_word, v68) + a CAS fast path; on an app core a spin	—
TLS, errno	__thread; errno per thread	TPIDR_EL0 → the thread's TCB + TLS block	—
Memory	mmap (anonymous, PROT_NONE reservations, MAP_FIXED/FIXED_NOREPLACE, MAP_PRIVATE files), munmap (partial), mprotect, madvise, mincore, msync/mlock (no-ops), posix_memalign, getpagesize (65536)	vm_* (lazy regions in the 34–60 GB arena)	heap blocks (64 KB-aligned): no reservations, no MAP_FIXED, protections not enforced
Time	clock_gettime/getres (REALTIME, MONOTONIC(_RAW/_COARSE), BOOTTIME, the CPU clocks = monotonic), gettimeofday, time, nanosleep, clock_nanosleep, usleep, sleep, timegm; localtime_r/mktime/strftime with TZ (newlib's)	CNTPCT_EL0 read at EL0, scaled with one clock_info sample (no system call	get_datetime once (local time taken as UTC, TZ=UTC0); msleep

Area	Calls	On the kernel	(without v75)
		per read); `sleep_us`	
Processes	`getpid`/`getppid`, `posix_spawn(p)` (file actions: a pipe or a file onto 0 / 1, `addchdir_np`; **v76:** every descriptor without `FD_CLOEXEC` is given to the child at its number — files, pipe ends, streams, sockets, shm objects —, `adddup2` onto 3 or more too, `addclose` keeps one back), `waitpid`/`wait` (`WNOHANG`; exit code, `SIGSEGV` for a fault, `SIGKILL` for killed / out of memory); `fork`/`exec*`/`system`/`popen` → `ENOSYS`	`spawn_ex` (argv / envp blocks) + `proc_wait`; v76 `spawn_ex2` + `get_handles` (the child installs them at start)	`kapi_spawn` (one argument line) + `kapi_wait`; (without v76) no descriptor but 0 / 1
Environment	`environ`, `getenv`/`setenv`/`unsetenv`/`putenv` (newlib's) from the process's block; `TZ` set from the kernel's zone when unset	`get_env`, `get_argv`	`HOME=SD:/home`, `PATH=SD:/bin`, `TMPDIR=RAM:/tmp`, `LANG=C.UTF-8`; argv from `get_args`
Sockets	`socket` (`AF_INET` TCP / UDP, `SOCK_NONBLOCK`/`CLOEXEC`), `connect` (non-blocking: `EINPROGRESS`, then `poll` + `SO_ERROR`), `bind`/`listen`/`accept(4)`, `send`/`recv`/`sendto`/`recvfrom`/`sendmsg`/`recvmsg` (`MSG_PEEK`/`DONTWAIT`/`WAITALL`; `MSG_NOSIGNAL` accepted), `get`/`setsockopt` (`SO_ERROR`, `SO_RCVTIMEO`/`SNDTIMEO`, `SO_BROADCAST`; the rest accepted), `getsockname`/`getpeername`, `shutdown`; `AF_INET6` and `socket (AF_UNIX)` → `EAFNOSUPPORT`	`sock_*`	`tcp_*` (TCP only: connect blocks, accept blocks)
Local sockets (v76)	`socketpair (AF_UNIX, SOCK_STREAM / SOCK_SEQPACKET / SOCK_DGRAM [| SOCK_NONBLOCK | SOCK_CLOEXEC])`; every socket call, `poll`/`select` on them; `sendmsg`/`recvmsg` with `SCM_RIGHTS` (up to 256 descriptors a message, Linux: 253; the real `CMSG_FIRSTHDR`/`CMSG_NXTHDR`/`CMSG_DATA`/`CMSG_SPACE`/`CMSG_LEN`), `MSG_CTRUNC`, `MSG_TRUNC`, `MSG_CMSG_CLOEXEC`; `SO_PEERCRED` (the pid), `SO_RCVBUF`/`SO_SNDBUF`, `SO_DOMAIN`; `getsockname` → `AF_UNIX`, no name. A descriptor received is of the sender's kind (the open file and its offset shared, a pipe end, a stream, a local or IP socket — the IP socket becomes the receiver's —, a shm object)	`sock_pair`, `sock_sendmsg` / `sock_recvmsg`	`socketpair` → `EOPNOTSUPP`
Shared memory (v76)	`memfd_create` (`MFD_CLOEXEC`, `MFD_ALLOW_SEALING`), `shm_open`/`shm_unlink` (`O_CREAT/EXCL/TRUNC`, `O_RDONLY`/`O_RDWR`; `FD_CLOEXEC` set), `ftruncate`, `fstat` (size, an id), `fcntl (F_ADD_SEALS / F_GET_SEALS)`, `mmap` (`MAP_SHARED`: the same pages in every process mapping it; `MAP_PRIVATE`: a copy),	`shm_create` / `shm_open` / `shm_ctl` / `shm_map`	`ENOSYS`

Area	Calls	On the kernel	(without v75)
	`munmap`/`mprotect`/`madvise`; `read`/`write` on one: `EINVAL` (map it)		
Names	`getaddrinfo` (numeric, `localhost`, else the kernel's DNS; services by number or a small table), `getnameinfo` (numeric), `gethostbyname(_r)`, `inet_pton`/`ntop`/`aton`/`addr`/`ntoa`, `htons`…	`net_resolve` (v43)	—
Waiting	`poll`, `ppoll`, `select`, `pselect` (`FD_SETSIZE` 1024) over sockets, pipes, files, the console	the `poll` kapi	a user-space loop (non-blocking reads into a carry buffer, 1 ms sleeps)
Signals	`sigaction`/`signal` (a table), `raise`, `kill (getpid (), sig)` and `pthread_kill (self)` run the handler at once; `abort` (status 134); masks kept; `SIGPIPE` never raised (`EPIPE`); `kill` of another pid: `SIGKILL`/`SIGTERM` → `kill_pid`, 0 → exists	—	—
Misc.	`sysconf` (page 65536, **1** processor: a process's threads all run on core 0), `pathconf`, `uname` (`Onyx`, `aarch64`), `gethostname`, `getrandom`/`getentropy`, `/dev/null`, `/dev/zero`, `/dev/urandom`, `isatty`/`ttyname`, `ioctl` (`FIONBIO`, `FIONREAD`, `TIOCGWINSZ`), `getrlimit`/`setrlimit`, `getrusage`, one user `onyx` (`getpwuid`…), `basename`/`dirname`, `err`/`warn`, `syslog` (→ kmsg); stubs: `dlopen` (fails: static programs), `backtrace` (the frame-pointer chain), `iconv_open` (`EINVAL`), `getifaddrs`		

Headers (`user/Runtime/libc/posix/include/`, first on the path with `-isystem`): `pthread.h` and `sys/_pthreadtypes.h` (the types, frozen once WP-TC's toolchain is built on them), `semaphore.h`, `sys/mman.h`, `poll.h`, `sys/socket.h`, `netinet/in.h`, `netinet/tcp.h`, `arpa/inet.h`, `netdb.h`, `sys/uio.h`, `sys/un.h`, `sys/utsname.h`, `sys/ioctl.h`, `net/if.h`, `ifaddrs.h`, `sys/random.h`, `sys/statvfs.h`, `sys/resource.h`, `sys/dirent.h`, `sys/termios.h`, `endian.h`, `byteswap.h`, `sys/sysmacros.h`, `dlfcn.h`, `execinfo.h`, `syslog.h`, `err.h`; and small overlays of newlib's (`#include_next`): `sys/features.h` (the `_POSIX_*` options newlib leaves off for this target: without them `<time.h>` hides `clock_gettime`), `time.h` (`timegm`), `unistd.h` (`pipe2`, `dup3`), `sys/stat.h` (`lstat`, `UTIME_NOW`), `signal.h` (`SA_RESTART`…), `sys/file.h` (`flock`), `fcntl.h` (v76: `F_ADD_SEALS`, `F_GET_SEALS`, `F_SEAL_*`).

Paths. Onyx paths as everywhere (`SD:/x`, `RAM:/y`; relative and `/x` against the working directory, as the kernel resolves them). Two names are mapped: **`/tmp`** → **`RAM:/tmp`** (made at the first use) and **`/dev/null|zero|urandom|random|stdin|stdout|stderr|tty`**. A volume met as a later component restarts the path there (`SD:/x/RAM:/t.db` is `RAM:/t.db`): Unix code that takes `RAM:/t.db` for a relative name and puts the working directory before it (SQLite does) still reaches the file — `:` is not allowed in a FAT name, so no real name is mistaken for a volume.

Threads and TLS, how. `TPIDR_EL0` points at a 16-byte TCB with the thread's static TLS block after it (AArch64 "variant 1"); libonyxposix's `struct __onyx_thread` sits just below, so `pthread_self` costs no system call. `crt0posix` sets the main thread's before any other code; a new thread gets it from

`thread_create_ex` (and sets it itself under v67). The kernel saves `TPIDR_EL0` per task. Under **`aarch64-onyx-elf`** TLS is **native**: `__thread` / `thread_local` compile to `mrs tpidr_el0` + the linker's local-exec offsets, `onyx-posix.ld` gathers `.tdata` / `.tbss` into a `PT_TLS` segment, and `tls.c` copies its image into each thread's block (the main thread's is reserved in `.bss`); newlib's `errno` and the rest of its `_reent` are `__thread` variables (`--enable-newlib-reent-thread-local`), so **errno is per thread everywhere** (newlib's `strtol`, libm included); libgcc's and libstdc++'s gthreads are `gthr-posix.h` over our pthreads (weak references: `onyx.specs` pulls the whole thread layer in with `-u __onyx_pthread_anchor`); `thread_local` destructors run at the thread's end (libstdc++'s `__cxa_thread_atexit` on a pthread key). Function-local statics keep libonyxposix's futex `__cxa_guard_*`. Under the **interim toolchain** (`aarch64-none-elf`, `--disable-tls`, docs/POSIX-PLAN.md §2) `__thread` compiles to *emutls* calls: libonyxposix overrides `__emutls_get_address` with a per-thread vector, and `__cxa_guard_*` (C++ function-local statics, thread-safe), both pulled in by `onyx.specs` before libgcc / libsupc++. **errno** there: the main thread keeps newlib's (`_impure_ptr->_errno`), the others have their own — except what newlib sets through its reentrancy structure directly (`strtol`'s `ERANGE`, the maths functions), which a worker thread does not see. **Code on an app core** (`kapi_core_run`) shares the main thread's TLS and errno, and its locks spin instead of sleeping (no kapi call there); stdio and files from there go through the RPC of §5.1 (`onyx_rpc_enable`).

Limits to know.

- C++ `std::thread` / `std::mutex` / `condition_variable` / `std::async` need the threaded libstdc++ of `aarch64-onyx-elf` (§1.1); under the interim toolchain C code is the target (the SuperTuxKart port's gthreads shim is separate: `user/Ports/stk`).
- `std::filesystem` takes Onyx paths: `RAM:/x` is, for libstdc++, a *relative* path whose first component is `RAM:` (no root name on POSIX) — the operations work (the kernel resolves the volume), but `absolute` / `canonical` / `lexically_*` reason as on Unix.
- `std::thread::hardware_concurrency ()` is 1 (a process's threads all run on core 0).
- newlib's `struct stat` has a 16-bit `st_ino` / `st_dev`: `st_ino` is the kernel's 64-bit id folded to 16 bits.
- No asynchronous signals, no `fork`. `PROT_EXEC` (kernel v78) on anonymous memory only — a JIT's: write the code, `__builtin___clear_cache` over it, run it; a file's private copy too, never a shared object. Shared memory between processes is a shm object (v76: `memfd_create`, `shm_open`); `MAP_SHARED` of a *file* for writing stays `ENOTSUP`, and an anonymous `MAP_SHARED` is private (there is no `fork` to share it with).
- (v76) A pipe end passed to another process (`SCM_RIGHTS`, `posix_spawn`): the pipe's end-of-file waits for every holder of a write end (the kernel counts them: `KAPI_HXF_WRITER`), as on Unix — so mark the descriptors a child must not keep `FD_CLOEXEC`. No named `AF_UNIX` sockets; reference cycles of local sockets queued on themselves through other connections are not collected.
- A pipe's end given as a child's stdin or stdout (`posix_spawn_file_actions_adddup2 (fa, p[0], 0)`, `(fa, q[1], 1)`): the parent may close its own copy at once. Writing into the stdin pipe keeps working (no `EPIPE`: the pipe is read elsewhere), and closing the stdout pipe's write end sends no end of file (the child still writes there). The other side: no `EPIPE` when the child has ended either — watch it with `waitpid (WNOHANG)`, and make the write end `O_NONBLOCK` if the child may stop reading (Jet's downloads window does both). In the child, `fcntl (0, F_SETFL, O_NONBLOCK)` makes `read (0)` answer `EAGAIN` when the pipe is empty (descriptor 0 is the console's: until 2026-10-04 its read waited whatever the flag, and a window that read its stdin each tick froze as soon as its parent had nothing to say).
- `kapi_random` (behind `getrandom`, `/dev/urandom`) is not yet a hardware RNG (user/Libs/tls/README.md).

Testing it: `/bin/posixtest [group…] [dir]` (`user/BinUtils/posixtest.c`; groups `mem thread file io time proc ipc net misc cxx`): one line per check, `PASS`, `FAIL (what was seen)` or `SKIP (kernel ENOSYS)` for a check that needs a v75 piece the kernel does not have yet — it first prints which pieces it found —, then a summary; the exit code is the number of failures. The `file` group runs in `RAM:/posixtest` and `/tmp/posixtest` (give a directory to run it elsewhere, `posixtest file SD:/tmp`); `net` needs the Wi-Fi up; `cxx` runs `posixtest-cxx` from the same directory (SKIP when it is not there). The group `loop` (not in the default run) does TCP / UDP over `127.0.0.1`. The group `ipc` (v76): `socketpair` SEQPACKET / STREAM / DGRAM, `poll`, `SO_PEERCRED`, `memfd_create` + `mmap MAP_SHARED` (twice: the same pages) and `MAP_PRIVATE`, seals, `sendmsg`/`recvmsg` with a memfd and a pipe end (3 iovecs, `CMSG_NXTHDR`), `MSG_CTRUNC` and `MSG_CMSG_CLOEXEC`, `shm_open`, and a `posix_spawn`'d child given the socket at its number (WebKit's launcher: the number in argv, the server end `FD_CLOEXEC` kept out, an `adddup2`) that receives the memfd and writes into it.

`/bin/ipctest [net]` (`user/BinUtils/ipctest.c`, libonyxposix, the kapi calls directly): the v76 kernel itself — local sockets (merging, `EAGAIN`, `POLLOUT`, `SO_RCVBUF`, boundaries, `MSG_TRUNC`, `EMSGSIZE`, a 1 MB packet, the end, `EPIPE`, a blocked receive woken by another thread, `SO_RCVTIMEO`), children spawned with `spawn_ex2` that use a file (its offset shared), a pipe end, a shm object, a local socket and (`ipctest net`) an IP socket sent to them, shared memory read after its writer exited, a futex across processes, 8 MB through a stream, 253 handles in a message, `MSG_CTRUNC`, seals, `shm_open`, a child killed touching beyond its object, the free memory back after 64 MB of shared pages (`meminfo`) and after a queued message is discarded.

`/bin/malloctest [phase…] [-s seed] [-n rounds] [-t threads]` (`user/BinUtils/malloctest.c`, libonyxposix, built with `aarch64-onyx-elf`: the newlib Web links): is every byte `malloc_usable_size` promises the block's own? Each block is filled to its usable size with a pattern of its own; after every phase, and every 1024 operations of the random ones, every live block must still hold its pattern and report the same usable size. Phases `sizes` (every size 0..1024 and around the powers of two up to 4 MB), `align` (`memalign` / `posix_memalign` / `aligned_alloc`, 16 bytes..1 MB), `realloc`, `top` (blocks next to the top of the heap while it grows and newlib trims it: `sbrk` with a negative increment, the kernel drops the pages), `sbrk` (the program's own `sbrk` between allocations: newlib's fenceposts), `mix` (all of it at random) and `threads` (the mix in several threads). A finding prints the block, the first byte changed and the last operations; the exit status is the number of findings. It passes on the bench (`PROG=user/BinUtils/malloctest.c PREFIX=aarch64-onyx-elf- sh tools/tests/posixsim/run.sh`) and on the Pi (about three minutes); it also runs under `tools/webkit/heapcheck.c` (`OBJS=heapcheck.o`, `CFLAGS_EXTRA` with the `--wrap` options of `build-web.sh`), which is how that tool is tested.

`/bin/posixtest-cxx [group…] [dir]` (`user/BinUtils/posixtest-cxx.cpp`, `aarch64-onyx-elf`, C++20): the C++ part, same output and exit code. Groups `thread` (`std::thread`, detach, a move-only argument, a 200 KB frame), `mutex` (`mutex`, `recursive_mutex`, `timed_mutex` timeouts, `shared_mutex`, `scoped_lock`), `cond` (`condition_variable` producer / consumer, `wait_for` / `wait_until` timeouts on `steady_clock` and `system_clock`, `notify_all`, `condition_variable_any`), `tls` (`thread_local` per thread, at `TPIDR_EL0` + offset, objects constructed and destroyed per thread), `static` (a function-local static built once under 8 threads, `call_once`), `fs` (`std::filesystem` in `RAM:` and `SD:/tmp`: directories, iterators, `copy_file`, `rename`, `resize_file`, `remove_all`, errors), `time` (`steady_clock`, `sleep_for` / `sleep_until`), `future` (`async`, `promise`, `packaged_task`, `shared_future`, exceptions through `get`, `broken_promise`), `except` (an `exception_ptr` across threads, 8 threads throwing at once, nested exceptions), `errno` (newlib's `ERANGE` in a worker thread, 8 threads each with its own), `atomic` (`fetch_add`, `shared_ptr` reference counts across threads, `atomic::wait`), `sync` (`latch`, `barrier`, `counting_semaphore`, `jthread`).

Testing it on the PC: the posixsim bench (`tools/tests/posixsim/`). `sh tools/tests/posixsim/run.sh [group…]` builds libonyxposix with `-DONYX_POSIXSIM` (the counter `cntvct` instead of `cntpct`, which Linux traps at EL0) and runs `posixtest` under `qemu-aarch64-static` against `fakekapi.c`: a kapi table built from raw Linux system calls (the volumes are directories of `$POSIXSIM_ROOT`, default `/tmp/posixsim`; threads are `clone`s; the v75 handles, `spawn_ex`, sockets and `poll` are emulated; v76's local sockets are Linux `AF_UNIX` socketpairs carrying the descriptors with `SCM_RIGHTS` and their kinds in a header or a side channel, shm objects are memfds, `spawn_ex2` leaves the descriptors open across `execve`). `POSIXSIM_LEVEL=75` answers `ENOSYS` to the v76 calls, `POSIXSIM_LEVEL=74` to every v75 call too, to test libonyxposix's fallbacks; `PROG=user/BinUtils/ipctest.c sh tools/tests/posixsim/run.sh` runs ipctest there; `POSIXSIM_NONET=0` lets the `net` group out. `sh tools/tests/posixsim/ports.sh` relinks the smoke ports (§5.5) for the bench and runs them on real jobs (a SQLite database on `RAM:` and `SD:`, xmllint, curl over HTTP and HTTPS from a local Python server). The bench checks the library's logic, not the kernel's: the Pi run stays the reference.

With **`aarch64-onyx-elf`** (§1.1): `PREFIX=aarch64-onyx-elf- sh tools/tests/posixsim/run.sh` builds posixtest with it (libonyxposix in `build-onyx-sim`); a `.cpp` `PROG` is linked with `g++` (`PROG=user/BinUtils/posixtest-cxx.cpp`); **`sh tools/tests/posixsim/tc.sh`** runs the lot — posixtest and posixtest-cxx on the v75 calls and on the v74 fallbacks, then `posixtest cxx` (posixtest spawning posixtest-cxx) — and `ports.sh` relinks and runs the ports of whichever toolchain built them (`tools/onyx-env.sh`'s choice).

5.5. Building a third-party library for Onyx (the sysroot, the CMake toolchain file)

The **sysroot** is libonyxposix installed with what ports add to it — one per toolchain (§1.1), their objects do not mix: **`out/sysroot-onyx/`** for `aarch64-onyx-elf`, `out/sysroot/` for the interim `aarch64-none-elf` (not versioned): `include/`, `lib/` (`libonyxposix.a`, `crt0posix.o`, `onyx-posix.ld`, `onyx.specs`, the ports' `.a`), `lib/pkgconfig/`.

```
make -C user/Runtime/libc/posix install PREFIX=aarch64-onyx-elf-   # -> out/sysroot-onyx (SYSROOT=<dir> elsewhere)
make -C user/Runtime/libc/posix install                            # aarch64-none-elf -> out/sysroot
```

CMake — `tools/onyx-toolchain.cmake` (+ `tools/cmake/Platform/Onyx.cmake`: `CMAKE_SYSTEM_NAME Onyx`, `UNIX`, `ONYX`, static only):

```
cmake -S <src> -B <build> -G Ninja -DCMAKE_TOOLCHAIN_FILE=<onyx>/tools/onyx-toolchain.cmake \
      -DBUILD_SHARED_LIBS=OFF [-DONYX_SYSROOT=<dir>]
cmake --build <build> && cmake --install <build>    # installs into the sysroot
```

It sets the compilers (`ONYX_TOOLCHAIN_PREFIX`; by default `aarch64-onyx-elf-` when it is installed, else `aarch64-none-elf-`; the default sysroot follows), the flags (`-mcpu=cortex-a72`, sections, `-isystem <sysroot>/include`, `-DFD_SETSIZE=1024`; the link: `-specs=<sysroot>/lib/onyx.specs`), libraries / headers / packages searched in the sysroot only, programs on the host, `pkg-config` on the sysroot's `.pc` files, the install prefix = the sysroot. Executables link (configure checks that link work) but cannot run on the build machine: a `try_run` question is answered with a cache variable. Do not let CMake also see `CFLAGS` / `LDFLAGS` from the environment (the specs given twice fail): `tools/ports/common.sh`'s `onyx_cmake` runs it under `env -u CFLAGS -u LDFLAGS`. `CMAKE_SYSROOT` is deliberately not set: the sysroot is an overlay on the compiler's own (newlib is in the toolchain's tree; `aarch64-onyx-elf` has its own `--with-sysroot`, `<prefix>/aarch64-onyx-elf`).

Autotools / plain Makefiles — `tools/onyx-env.sh` exports `CC`, `CXX`, `AR`, `RANLIB`, `CFLAGS`, `LDFLAGS`, `PKG_CONFIG_LIBDIR`, `ONYX_HOST`:

```
. tools/onyx-env.sh            # the same choice of toolchain and sysroot (ONYX_TOOLCHAIN_PRE
FIX forces one)
./configure --host=$ONYX_HOST --prefix=$ONYX_SYSROOT --disable-shared --enable-static
```

The ports (`tools/ports/<name>/build.sh`, sources vendored and trimmed in `third_party/` with a `README.onyx`; `sh tools/ports/build-all.sh`, or `make -C user/BinUtils ports`, which also copies the tools to `user/BinUtils/*.elf` — opt-in, a few minutes; `PORTS=1` adds it to `all`). They are built with `aarch64-onyx-elf` when it is installed (out `out/ports-onyx/`), else with `aarch64-none-elf` (`out/ports/`); `make -C user/BinUtils ports ONYX_TOOLCHAIN_PREFIX=aarch64-none-elf-` forces the interim one:

Port	Source	Licence	Built as	Tool
SQLite 3.50.4	the amalgamation	public domain	`libsqlite3.a` (threadsafe, no mmap I/O, no extensions, FTS5, JSON, R-tree)	`sqlite3` (the shell)
mbedTLS 3.6.3	`third_party/mbedtls-3.6.3` (the newlib apps' copy)	Apache-2.0	`libmbed{tls,x509,crypto}.a`, a configuration of its own out of tree (files, time, entropy from `getrandom`), for curl	—
libxml2 2.13.8	trimmed release	MIT	`libxml2.a` (threads, zlib; no iconv / HTTP / modules)	`xmllint`
curl 8.16.0	trimmed release	curl (MIT-like)	`libcurl.a` (mbedTLS, HTTP/2 with nghttp2, zlib, brotli, the threaded resolver; no IPv6)	`curl` (CA bundle `SD:/res/ca-bundle`)

WebKit's graphics / text / i18n libraries (docs/POSIX-PLAN.md "Ports for WebKit"; `aarch64-onyx-elf` only — ICU and Skia need the threaded libstdc++ —, `build-all.sh webkit`, about 12 minutes on 4 cores, Skia the most; sizes of the `.a` with `-ffunction-sections`, the linker keeps what a program uses):

Port	Source (`third_party/`, size in the repo)	Licence	Built as	Tool
ICU 78.3	`icu-78.3`: `common/`, `i18n/` (21 MB) + **the filtered data** `source/data/in/icudt78l.dat` (15.9 MB, made by `tools/ports/icu/gen-data.sh` with `data-filter.json`)	Unicode-3.0	`libicuuc.a` 4.8 MB, `libicui18n.a` 10 MB, `libicudata.a` 15.9 MB (the data linked in, `.incbin`; no data file on the card); `icu-uc.pc`, `icu-i18n.pc`. Our CMake (`tools/ports/icu/CMakeLists.txt`): ICU's autoconf knows no Onyx host	`icutest`
libpng 1.6.44	`libpng-1.6.44` (the apps' copy)	libpng	`libpng16.a` 0.5 MB (NEON filters)	—
FreeType 2.14.3	`freetype-2.14.3` (the apps' copy)	FTL	`libfreetype.a` 0.9 MB: TrueType + CFF/CFF2, sfnt, auto-hinter + PS hinter, smooth + mono, OT-SVG, variable fonts, COLR; zlib (WOFF), brotli (WOFF2), libpng (colour bitmaps) — the apps keep their lean `user/Kits/fontkit`	—

Port	Source (`third_party/`, size in the repo)	Licence	Built as	Tool
HarfBuzz 14.5.1	`harfbuzz-14.5.1` (8 MB)	Old MIT	`libharfbuzz.a` 3.0 MB (hb-ft) + `libharfbuzz-icu.a` (WebKit's `HarfBuzz COMPONENTS ICU`)	`hbtest`
libjpeg-turbo 3.1.4	`libjpeg-turbo-3.1.4` (4 MB)	IJG + BSD-3 + zlib	`libjpeg.a` 1.0 MB (NEON; the turbo extensions Skia and WebKit use — the in-tree jpeg-9f lacks them)	—
libwebp 1.4.0	`libwebp-1.4.0` (the apps' copy)	BSD-3	`libwebp.a` 0.8 MB, `libwebpdemux.a`, `libwebpmux.a`, `libsharpyuv.a` (NEON, threads)	—
Skia m154	`skia-m154` (12.6 MB): **WebKit's own copy** (`Source/ThirdParty/skia`, fetched by a sparse checkout), trimmed to what is compiled	BSD-3	`libskia.a` 15.7 MB: the CPU raster back end (no Ganesh / GL), WebKit's source list (`tools/ports/skia/sources.cmake`), FreeType text, PNG / JPEG / WebP codecs and encoders, `SkFontMgr_New_Onyx` (below)	`skiatest`, `skiademo`

- **ICU's data** is cut to WebKit's needs (TextCodecICU's legacy encodings + the CJK ones: 50 converters; break iterators with the Thai / Lao / Khmer / Burmese / CJ dictionaries; collation; JSC's Intl for 42 languages; no transliteration, spell-out, confusables, stringprep, character names). To change it: edit `tools/ports/icu/data-filter.json`, run `sh tools/ports/icu/gen-data.sh` (fetches ICU 78.3's release and its data sources, a host build of ICU's tools, 3 minutes), commit the new `.dat`.
- **Fonts: no fontconfig.** `#include "include/ports/SkFontMgr_onyx.h"`, `SkFontMgr_New_Onyx("SD:/res/fonts/")`: Skia's custom directory font manager with what WebKit would ask fontconfig for — the CSS generic families and common web names mapped to the card's fonts (`sans-serif` → DejaVu Sans, `serif` → DejaVu Serif, `monospace` → DejaVu Sans Mono, Arial / Helvetica → Liberation Sans, Times New Roman → Liberation Serif, Georgia → Gelasio, Segoe UI → Selawik), family names without regard to case, and `matchFamilyStyleCharacter` (a fallback font for a character: WebKit's FontCache path on Android and Windows).
- **Using them**: `pkg-config` on the sysroot (`icu-uc icu-i18n harfbuzz harfbuzz-icu freetype2 libpng16 libjpeg libwebp libwebpdemux skia`); Skia's headers are under `include/skia` (`#include "include/core/SkCanvas.h"`) and a client needs `skia.pc`'s definitions (`SK_BUILD_FOR_UNIX`, `SK_R32_SHIFT=16`: N32 is BGRA, the canvas' layout — an Onyx window buffer wraps as an `SkSurface` with `SkSurfaces::WrapPixels`). A C file including ICU's headers fails (they want `<uchar.h>`, which newlib lacks): include them from C++. A CMake project that adds `-pthread` itself (libwebp) needs `-DCMAKE_C_COMPILER_LAUNCHER="sh;tools/ports/drop-pthread-flag.sh"`: `aarch64-onyx-elf`'s GCC does not know the option.
- **Bench**: `sh tools/tests/posixsim/ports.sh webkit` (icutest 52 checks, hbtest 10 with the PC's FreeSerif for Devanagari, skiatest 18 — the scene kept as `$POSIXSIM_ROOT/RAM/skiatest.png`).

A new port: a `build.sh` sourcing `tools/ports/common.sh` (it installs the sysroot first; `onyx_install_deps` puts the in-tree zlib, nghttp2 and brotli in it — the prebuilt `.a` of `third_party/` under `aarch64-none-elf`, compiled from their sources under `aarch64-onyx-elf`, whose newlib has no `__errno` / `_impure_ptr` for objects of the interim one; `onyx_cmake` calls CMake with the toolchain file; `onyx_tool_done` copies a tool

to `out/ports(-onyx)/bin` and runs `tools/el0scan.sh` on it). Licences: docs/LICENSING.md — ask before a library that would force its licence on the app.

5.6. Shared libraries (kapi v83): `SD:/lib/<name>.so`

> **Names.** The libraries are "kits", one per use: **UIKit** (the widget toolkit: `user/Kits/uikit/`, `namespace uikit`, the `uk_` functions, `SD:/lib/uikit.so`), **PrinterKit** (§5.7), **AudioKit** (§5.7), **FileKit** (§5.8), **ImageKit** (§5.9). UIKit was named **wtk** until 2026-10-05 (`user/wtk/`, `namespace wtk`, `wk_`, `wtk.so`): the rename was complete — sources, headers, the library, its package (`uikit` replaces `wtk`) — and every program was rebuilt; older notes and commit messages say wtk.

(The design and its reasons: `SHARED-LIBS-PLAN.md`. The kernel side: docs/02 §7 Shared libraries.)

Since kapi v83 the apps no longer carry a copy of the toolkit and of FreeType: **uikit** and **FreeType** are shared libraries — `SD:/lib/uikit.so`, `SD:/lib/fontkit.so` —, loaded once for the whole system, their code mapped into every process that uses them, their data private to each. A fix in a library reaches every app **without rebuilding any app**.

What a library is. Position-independent code (`-fPIC`, linked at 0 by `user/Runtime/lib.ld`; the kernel places it) that publishes its entry points in an **export table**: a struct of function pointers, filled when the library is loaded. A program calls through the table — exactly as it calls the kernel through `KT`. There is no ELF dynamic linker, no symbol looked up by name at run time: an app stays a static, non-PIC program at 8 GB. The table is **append-only and versioned as the kapi table is**: an entry is never moved, removed or changed; a new one is appended and the version (the number of entries) goes up.

The two halves are generated (`tools/libgen/libgen.py`, run by `user/Makefile`) from the library's objects and its list **`user/<lib>/<lib>.abi`** (kept in git; one `<slot> <symbol>` line per entry; the tool only ever appends):

File (in `user/lib/`, the build folder)	Side	What it is
`<lib>_table.S`	library	`onyx_lib_table`: version, size, `init`, then one pointer per entry. The ELF entry point of the `.so` (`ld -e onyx_lib_table`): the kernel finds it without a symbol table.
`<lib>_stubs.S`	program	one **import stub** per entry, under the entry's own (mangled) name: `adrp x16, onyx_<lib>_table; ldr x16, […]; ldr x16, [x16, #slot]; br x16`. The app's source and the library's headers do not change: a call to `FT_Load_Glyph` or `uikit::Widget::invalidate` is bound by the linker to its stub. Stubs are weak (an app's own definition of a name wins, as over a static library). With `--vtables`: a **copy of each of the library's vtables** (a class whose vtable the compiler emits only with its key function is referenced by the apps that construct or derive it) — the same slots, resolved by the stubs; no library address is ever in a program.
`<lib>_bind.cpp`	program	a constructor of **priority 101** (before the app's own: a `static Menu menu;` may use the library) that calls `lib_bind ("<lib>", <version>, imports)` (`user/Runtime/lib.h`) and sets `onyx_<lib>_table`.

`lib/<lib>.imp.a` (the stubs, the bind constructor) is the **import library**: an app links it where it linked `uikit/libuikit.a` or `ft/libft.a`. `make stage` copies `user/lib/*.so` to `sdcard/lib/`.

At run time. The bind constructor calls `kapi_lib_open` (the file read once for the system, then shared), checks the version — the app was built against version N of the table: an older library is refused — and calls the table's `init` with the **imports** (`TLibImports`): the app's allocator (`operator new` / `delete`: one heap in the process, so an object made on one side can be freed on the other) and, for uikit, the addresses of the variables the two share. A library that is missing or too old ends the app with a line on its output (the kernel log for a windowed app): *this program needs the shared library "uikit" (version 667 or later): the one installed is older — update its package*; exit status 126.

Writing a library. Sources compiled with `LIB_CXXFLAGS` (`-fPIC -fvisibility=hidden -DONYX_LIB_BUILD`), linked with `lib/librt.o` (`user/Runtime/librt.cpp`: the imports, the static constructors, `operator new` / `delete` over the importer's allocator, `memcpy` & co. over the kapi) and the generated table; an `init` function (`return onyx_lib_init (imp) < 0 ? -1 : 0;`). `-z text --no-undefined`: no relocation in the code, nothing imported by name — the library reaches the kernel through `KT`, the app through its imports. What the kernel accepts is checked by `sh tools/tests/shlib/check_pic.sh user/lib/<name>.so`. Rules:

- **Functions** — every global function is an entry (`--export` / `--export-file` restrict them: `fontkit.so` exports FreeType's public API only). Never remove one, never change a signature.
- **Global data cannot be imported** by a program (it is linked at a fixed address; the library is not). The generator refuses a library with global variables unless `--allow-data` says they are handled. uikit's (the palette `C_BG`…, the text face) are **the app's variables**: `uikit/globals.inc` lists them (append-only), `uikit/globals.cpp` — compiled into the import library — defines them and the table of their addresses the bind constructor hands over (`--data onyx_uikit_data`, `TLibImports.data`); in the library each one is a reference bound at `init` (`uikit/global.h` `UIKIT_VAR`). The app's code — and the toolkit's inline code compiled into it — reads them directly.
- **C++ classes** (`uikit/abi.h`): the apps allocate, derive and read the classes of the headers, so their **layout** and their **virtual functions' order** are part of the interface, append-only too. Never add a field or a virtual in a class: a new field takes the **reserve** (`Widget::reserved_`, `Widget::ext`, `Canvas`'s, `Root`'s), a new virtual one of the **reserved slots** (`Widget::uk_reserved0..7`, `Root::uk_rootReserved0..7`: renamed, same place). `uikit/layout_lock.cpp` (generated once by `tools/libgen/layout.py`) holds every class's size and the base classes' offsets as `static_assert`s: the library's build fails when one moves (`make -C user uikit-layout-update` only for a class added). **Inline code** of the headers is compiled into the apps: a later change to it reaches only the apps rebuilt after it — code that may have to be fixed belongs in the `.cpp` files.
- A change that cannot keep these rules is **another library** (`uikit2.so`), beside the old one.

Using one from an app's Makefile rule: link `lib/uikit.imp.a` (and `lib/fontkit.imp.a`) in place of the static archives; the package declares `needs = uikit` (and `ft`). Jet's hosted build (`tools/webkit/build-web.sh`) links the library too since 2026-10-05 — it compiles the import side (`lib/uikit_stubs.S`, `lib/uikit_bind.cpp`, `uikit/globals.cpp`) with its own toolchain. The PC builds (the simulator, Koton for Windows, macOS) still compile uikit statically (`user/Kits/uikit/*.cpp`: the same sources — `uikit/globals.cpp` then simply defines the variables).

A library under another name: the alias (kapi v97). Each graphics server has its own UIKit with the same exports (Elegant `SD:/lib/uikit.so`; PocketUI, the server of the pocket and console modes, `SD:/lib/pocket/uikit.so`: §5.10.2, `POCKETUI-TECH-STUDY.md`); a program never chooses — its bind opens `"uikit"`. The server, and only it, calls `kapi_lib_open_as ("SD:/lib/pocket/uikit.so", "SD:/lib/uikit.so", version, &err)` after `KAPI_WS_REGISTER` and before `KAPI_WS_DISPLAY`: from then on every `kapi_lib_open ("uikit")` of any process gets the pocket library (the same image, the same

address in every process). Both names under `SD:/lib/`, never `appkit.so`; one image a name. The alias lives while the server or a program holds the library; the server crashed and started again: kept (it takes it over); another server started (a switch of mode, the fallback to Elegant): dropped. An update of the aliased file applies at the next session. `preload` lists it as `sd:/lib/uikit.so -> sd:/lib/pocket/uikit.so`; `unload` of the alias is refused. The kernel's side: docs/02 §7 *Aliases*.

Tests. `sh tools/tests/run_image_test.sh` (the loader and the aliases, on the PC), `/bin/libtest` (the loader, on the Pi), `/bin/aliastest` (the alias, the server per mode, its switch: on the Pi), `python tools/tests/shlib/pi_apps.py <pi-ip>` (every app of the card started on the libraries, over telnet), `sh tools/tests/shlib/compat.sh` (an app built against version N on the library N+1: a fix reaches it, an added function and a used reserve keep it running; an app built against N+1 is refused by the library N).

5.7. Printing (`SD:/lib/printerkit.so`, the print service `printd`)

An app prints by **drawing its pages once**; the system does the rest. Three parts, all MIT:

Part	Where	What it does
The library	`user/Kits/printerkit/printerkit.h` → `SD:/lib/printerkit.so` (`print.cpp`, `dialog.cpp`; its table `printerkit/printerkit.abi`, append-only)	The **Print dialog** (the same in every app), a **job**: the pages recorded as they are drawn — rectangles, glyphs, images, paths, in points — into `SD:/var/spool/print/<id>.opj` (`printerkit/job.h`), with its ticket `<id>.job`; the printers' list.
The service	`user/Apps/printd` (IPC service `print`; started at boot and on demand)	The **queue**. Replays a job for its printer: a **PDF** (`printerkit/pdfsink.h` → `pdf/pdfwrite.h`) for the PDF printer; for a network printer, pages **rendered at its resolution** (`printerkit/raster.h`: FreeType glyphs from the job's own fonts, images scaled, paths filled with smoothed edges) and **streamed as PWG Raster over IPP** (`printerkit/ipp.h`, HTTP chunked on port 631) while they are made — or the PDF itself to a printer that takes PDF. Follows the job at the printer, tells the user (notifyd).
The printers	`SD:/etc/printers.ini` (`printerkit/printers.h`), the Control Panel's **Printers** applet (`user/Apps/printconf`), `/bin/ipp`	A network printer is added by its address: `printd` asks it what it can do (IPP `Get-Printer-Attributes`: formats, papers, colour, quality, margins) and keeps the answer — no driver of a make. Any **IPP Everywhere / AirPrint** printer works. **Find** (`PD_SCAN`): one mDNS question — who offers `_ipp._tcp.local`? — sent to 224.0.0.251:5353 from an ordinary UDP port, so the printers answer to that port alone (a one-shot query, RFC 6762: no multicast group to join, which the kapi does not have); whoever answers is asked over IPP. (Trying every address of the network instead restarted the Pi: do not.)

A job does not depend on the printer: the same recorded pages become a PDF or a raster. `printerkit.so` is the first library that **uses other libraries**: FreeType (`fontkit.so`, for `print_text`) and uikit (`uikit.so`, for the dialog), through their import stubs linked into it; `print.cpp` opens them when first needed (`kapi_lib_open`), and uikit's variables are the program's (uikit's library build of `globals.o`, `--data onyx_uikit_data` in the bind object) — so a program that links `lib/printerkit.imp.a` also links `lib/uikit.imp.a`.

Printing from an app — link `lib/printerkit.imp.a` (before `lib/uikit.imp.a`), include `printerkit/printerkit.h`. Lengths are **points** (1/72 inch), y goes down from the page's top-left corner, colours are `0xRRGGBB`:

```
#include "printerkit/printerkit.h"

static void cmd_print (void)
{
    PrintSetup s; print_setup_default (&s);                  // the default printer, its pap
er
    PrintDialogInfo di = { sizeof di, "My document", 2, 0, 0, 0, 0 };
    if (!print_dialog (&s, &di)) return;                     // the user chose: printer, pag
es, copies, paper...
    PrintJob *j = print_begin (&s, "My document");
    int f = print_font (j, "DejaVu Sans", 0), fb = print_font (j, "DejaVu Sans", PRINT_BOLD);
    for (int page = 1; page <= 2; page++)
    {
        if (!print_page (j, 0, 0)) continue;                 // 0, 0: the paper chosen; 0: n
ot in the range asked
        print_text (j, fb, 18, 72, 90, "Hello, printer", 0x202020);
        print_line (j, 72, 100, s.paper_w - 72, 100, 0.75f, 0x808080);
        print_text (j, f, 11, 72, 130, "A line of text at 11 points.", 0x000000);
        print_image (j, pixels, pw, ph, 72, 160, 200, 150, PRINT_IMG_PHOTO);
    }
    print_end (j);                                           // queued: printd prints it and
 tells the user
}
```

Call	
`print_setup_default (&s)`, `print_dialog (&s, &info)`, `print_setup_paper (&s, media, orientation)`	The setup: the printer, the paper (`paper_w`, `paper_h`, and `margin_*`: what the printer cannot print on), the pages, the copies, colour, quality. `PrintDialogInfo.flags = PRINT_DLG_OWN_PAPER`: the document has its own page size (a slide, a PDF): no paper choice, each page is **fitted on the paper** (turned if it lies, scaled, centred).
`print_begin`, `print_page (j, w, h)`, `print_end`, `print_abort`	The job. `print_page` returns 0 for a page outside the range chosen: skip its drawing.
`print_rect`, `print_line`, `print_frame`, `print_path_move / line / curve / close`, `print_path_fill`, `print_path_stroke`	Shapes.
`print_image (j, px, pw, ph, x, y, w, h, flags)`	`PRINT_IMG_ALPHA`: the pixels carry alpha; `PRINT_IMG_PHOTO`: kept as a JPEG. More pixels than 300 an inch are averaged down.
`print_font (j, family, style)`,	Text by family (the fonts of `SD:/res/fonts`, `SD:/fonts`; a

Call	
`print_text`, `print_text_width`, `print_font_metrics`	missing bold / italic face is made from the regular one).
`print_font_data`, `print_glyph`	For an app with its own text layout: its font's bytes, a glyph at its pen position.
`print_printers`, `print_printers_find`, `print_printer_media`, `print_printer_add / remove / default / status`, `print_jobs`, `print_job_cancel`, `print_jobs_forget`	The printers and the queue (what the Printers applet uses).

An app that already exports PDF prints with the same code: `printerkit/pdfprint.h`'s **`PrintWriter`** is a `pdfw::Writer` whose pages go to a job (the writer's drawing calls are virtual), and `print_ask (title, pages, current, pageW, pageH)` is the dialog + `print_begin` in one call:

```
PrintJob *j = print_ask (name, npages, current, pageW, pageH);
if (j) { { PrintWriter w (j); export_pages (w); } print_end (j); }
```

That is how Letters, the Spreadsheet, Slides print; Paint, Photos and the PDF Viewer (its pages rendered by MuPDF at 300 dots an inch) use `print_image`; the Printers applet's test page uses the text and shape calls.

Another kind of printer is a branch of `printd`'s `run ()` (`kind` in `printers.ini`): replay the job into a `pjob::Sink` (`printerkit/job.h`: `font`, `begin_page`, `rect`, `glyphs`, `image`, `path`, `end_page`) — `pjob::PdfSink` and `praster::Raster` are the two there are — and send the result.

Tests. `sh tools/tests/run_print_test.sh` (on the PC: a job recorded, replayed as a PDF and as 300 dpi pages, the PWG Raster stream read back pixel for pixel; the IPP messages; with `IPP_PRINTER=<address>`, a real printer asked and a `Validate-Job` — nothing is printed). On the Pi: `ipp <address> validate`, the applet's **Test page**.

5.7. AudioKit: the shared sound library (`SD:/lib/audiokit.so`)

(`user/Kits/audiokit/audiokit.h` is the reference; the library's mechanism: §5.6.)

Everything about sound that more than one program can use lives in **AudioKit**, one copy for the whole system. A program links `lib/audiokit.imp.a` and calls plain functions (`ak_*`); frames are always 16-bit stereo at `AUDIOKIT_RATE` (44100 Hz, the system output's).

Group	Calls	What it does
Files	`ak_open`, `ak_read`, `ak_seek_ms`, `ak_info_of`, `ak_close`	A sound file of any kind — **MP3, FLAC, WAV, Ogg Vorbis, MIDI** (through the SoundFont), **FM Song** (`.fms`, FM Tracker's: on a synthesizer of the stream's own, one at a time per process) — read as frames at the output's rate, whatever its own rate and channels.
Tags	`ak_tags_read (path, &tags)`	What a sound file says about itself, nothing decoded (`struct ak_tags`): title, artist, album artist, album, genre, year, track, disc, length, format, and where its cover picture is inside the

Group	Calls	What it does
		file — ID3v2 / ID3v1 (MP3), Vorbis comments (FLAC, Ogg), LIST INFO (WAV), the first track's name (MIDI), the title and author (FM Song); what is missing comes from the path. **The Media Player's library reads through it** (the code is its `tags.h`, compiled once into the library).
The player	`ak_play (path, loop)`, `ak_play_stop`, `_pause`, `_state`, `_pos_ms`, `_len_ms`, `_seek_ms`, `_volume`, `_wait`, `_error`, `_keep_output`	A file played **in the background** by a thread of the library, on the system's output (it takes the output when it has something to play and lets it go after ~0.6 s of silence; another program playing: `AK_BUSY` until it can). One line to make a sound.
Live notes	`ak_note_on (channel, key, velocity)`, `ak_note_off`, `ak_program`, `ak_control`, `ak_pitch_bend`, `ak_notes_off`	Notes on a **General MIDI synthesizer** (16 channels, 9 the drums), mixed with the file by the same thread.
The output	`ak_out_open`, `ak_out_write`, `ak_out_free`, `ak_out_queued`, `ak_out_close`	The system's output for a program that makes its own frames: the acquire / status / write loop every player and emulator wrote for itself (`ak_out_open (0, 0)`: the output as it is configured). **The Media Player, the six emulators and Doom play through it.**
The input	`ak_in_open (rate)`, `ak_in_read (out, max)`, `ak_in_close`, `ak_in_peak (samples, n)`	**The microphone** the Sound applet chose (kapi v99 `sound_input`: a USB microphone or a headset's, an I2S one on the header), one program at a time: 16-bit **mono** samples at the rate asked (converted from the device's), the DC taken away and the input's gain applied by the kernel. `ak_in_open` → the rate, 0 another program records, −1 no microphone; `ak_in_read` never waits → the samples taken (0 none yet, −1 the device is gone); `ak_in_peak`: the loudest sample, for a level meter. **`/bin/rec` and the Sound applet's test record through it.**
The FM synthesizer	`ak_fm_instrument (voice, ins)`, `ak_fm_start (voice, milli_hz, wave, volume)`, `ak_fm_stop`, `ak_fm_silence`, `ak_fm_render`, `ak_fm_live`	**The system's voices** (`audiokit/fmsynth.h`; they were in the kernel until 2026-10-05, which only puts sound out now): 16 voices, each a plain wave (`SOUND_SQUARE` … `SOUND_NOISE`) or a two-operator FM instrument (`struct kapi_fm_instrument`, `SOUND_FM`). A note sounds until it is stopped; the voices are played by the player's thread, mixed with the file and the MIDI notes (~70 ms from the call to the ear when only notes play) — or, after `ak_fm_live (0)`, rendered by the program itself (`ak_fm_render`: an export, or a program that mixes them with its own frames — Doom's music).

Group	Calls	What it does
		BASIC's `SOUND` / `PLAY` / `NOTEON`, the games' effects (`game.h`), `tone`, FM Tracker, the Sound applet's test play through it.
Mixing	`ak_gain_s16`, `ak_mix_s16` (saturated), `ak_mono_to_stereo`, `ak_volume_gain`, `ak_resampler_new` / `ak_resample`, `ak_f32_to_s16`, `ak_soft_clip`	Gains are 16.16 (65536 = 1). `ak_soft_clip` is the soft limiter (straight up to 0.75, then a `tanh` knee that never passes 1 — Koton's); `ak_f32_to_s16` goes through it.
Effects	`ak_reverb_new (rate)` / `_set (room, damp, wet, width)` / `_process (in, left, right, n)` / `_mute` / `_free`; `ak_chorus_new (rate, delay_s, depth_s, hz)` / `_process` / `_mute` / `_free`	MeltySynth's **reverb** (Freeverb: mono in, stereo out) and **chorus**, as effects of their own on float buffers.
Notes	`ak_note_mhz`, `ak_note_key (note, octave)`, `ak_note_octave_mhz (note, octave)`, `ak_note_name`, `ak_note_parse`	A MIDI key's frequency (69 = A4 = 440 Hz); a note (0 = C .. 11 = B) and an octave's key (C4 = 60) and frequency — **the one table of notes** (FM Tracker's `fms_note_mhz` uses it); a key's name (`C4`, `F#3`), a name's key.
WAV	`ak_wav_header`, `ak_wav_save`, `ak_wav_begin (path, rate, channels, frames)` / `ak_wav_write` / `ak_wav_end`	A 16-bit PCM file written: a buffer at once, or a long one as it is made (its length said first — **Koton's export**).
The synthesizer	`ak_soundfont_default`, `ak_soundfont_name`, `ak_soundfont_prefer (path)`, `ak_soundfont_find (preferred, out, cap)`, `ak_soundfont_load` / `_free`, `ak_synth_new` / `_midi` / `_render` / `_free`	The SoundFont: **one search for everybody** (the preferred file, else the first `.sf2` of `SD:/res/soundfonts`, `SD:/koton/soundfonts`, `SD:/music/soundfonts`, `SD:/music`, `SD:/apps/koton.app`), its load into MeltySynth (Koton's own copy: `_load`; the process's default: `_default`, after `_prefer` — the Media Player's setting). The synthesizer for C; `render` never allocates and never calls the kernel (it may run on an app core).

Also exported **as they are**, for the programs that want them raw: **MeltySynth**'s C++ interface (`Apps/koton/synth/meltysynth.h`: `ms::Synthesizer`, `ms::soundfont_*` — Koton's engine and the Media Player's MIDI use it) and the **decoders' own C interfaces** (`Apps/media/codecs.h`: minimp3, dr_flac, dr_wav, stb_vorbis — the Media Player's `decode.h` uses them). Their sources stay where they were (the PC builds and the tests compile them statically); `user/Makefile` builds them into the library, and Koton and the Media Player no longer carry them.

- **Floating point**: the entries with `float` / `double` parameters (`ak_f32_to_s16`, MeltySynth's `render`) are for programs built with the FPU (the newlib apps, `/bin/basic`, `CXXFLAGS_FP`); all the file, player, output, note and 16-bit mixing calls are integer — `/bin/play` is built `-mgeneral-regs-only`.
- **The SoundFont**: the first `.sf2` of the folders above (`SD:/res/soundfonts`: the package `generaluser-gs`), loaded once per process at the first MIDI file or note (a few seconds for its 32 MB); without one, `ak_play` of a MIDI file and `ak_note_on` fail with the reason in `ak_play_error ()`.

- **Inside**: `audiokit/akcore.cpp` (the files, the player, the output, the effects) compiles the Media Player's `decode.h` / `midi.h` (the decoder classes, the Standard MIDI File reader, the rate converter) behind the C interface; `akmix.cpp` is the pure part (mixing, the soft limiter, the notes, the WAV header), `aksf.cpp` the SoundFont, `akwav.cpp` the WAV files, `akfm.cpp` the FM synthesizer over `fmsynth.h` (one source: the library's and the PC tools'); `audiokit/akso.c` gives the library its `malloc` over the importer's allocator; newlib's `libm` and `libc` are linked in. The table: `audiokit/audiokit.abi` (append-only).
- **Not in it** (and why): FFmpeg and the video side of `user/Libs/av` (GPL, and video: the Media Player's), Koton's engine and plugin host (its own classes), the per-sample inline DSP of `kplug.h`.
- **The PC builds** (no shared library there): Koton compiles `akmix.cpp`, `aksf.cpp` and `akwav.cpp` into itself (`Apps/koton/engine/akhost.cpp`, `ui/audio.h`); the simulator (`tools/tests/desktop_sim/shots.sh`) makes a `libaudiokit.a` of the library's sources, linked into every app it builds.
- **Tests**: `play --notes` (a scale on the synthesizer), `play --fm` (the FM synthesizer: a note rendered off line and checked, then a scale heard), `play --info <file>`, `play <file>` (docs/04 §8); a BASIC program (`PLAYFILE`, `MIDINOTE`: docs/04 *Onyx BASIC*).
- **To come** (IDEAS.md): effects in the kernel's mixer (a reverb send per program), set aside for later.

5.8. FileKit: what programs do with files (`SD:/lib/filekit.so`)

(`user/Kits/filekit/filekit.h` is the reference; the library's mechanism: §5.6.)

Compression, ZIP archives, whole files and trees, paths: one copy for the whole system. A program links `lib/filekit.imp.a` and calls plain functions (`fk_*`). Everything is integer and pointers (a program built without the FPU calls it); a buffer the library returns (`void **out`) is freed with **`fk_free`**.

Group	Calls	What it does
Compression	`fk_deflate (data, n, wrap, level, &out, &n)`, `fk_inflate (data, n, wrap, size_hint, &out, &n)`, `fk_crc32`, `fk_adler32`	**zlib** itself (1.3.1): `wrap` = `FK_RAW` (a ZIP entry's stream), `FK_ZLIB` (PNG, PDF), `FK_GZIP` (`.gz`, HTTP), `FK_AUTO` (inflate: zlib or gzip, told from the bytes). The inflated buffer has a 0 byte after its end.
Archives, any format	`fk_arc_formats (list, max)`, `fk_arc_probe`, `fk_arc_is_name`; `fk_arc_open`, `fk_arc_new (path, "ZIP")`, `fk_arc_format`, `fk_arc_path`, `fk_arc_comment`, `fk_arc_writable`, `fk_arc_method_name`, `fk_arc_test`; `fk_arc_extract_with (a, &opts)`, `fk_arc_extract_bytes`; `fk_arc_add`, `fk_arc_add_conflicts`, `fk_arc_add_bytes`, `fk_arc_delete`, `fk_arc_rename`, `fk_arc_new_folder`, `fk_arc_write_empty`	**The formats are asked, not assumed** (`struct fk_format`: its name, its extensions, what can be done with it — read, written, with a password): today **ZIP** (read and written), **TAR**, **TAR.GZ** and **GZIP** (read; a gzip is unpacked to `SD:/tmp` first). An archive of any of them is one handle (`fk_arc`, the same as `fk_zip`: the `fk_zip_*` calls below work on it). Extraction with choices (`struct fk_extract`: the layout — full, from a folder, flat —, what to do with a file that exists, an `ask` callback, a selection of entries); changes rewrite the archive beside itself, then swap it in. **A format added to the library is one every program gets: the Archiver has no list of its own.**

Group	Calls	What it does
A ZIP on the card	`fk_zip_open` / `_close` / `_error` / `_password`, `fk_zip_count`, `fk_zip_entry (z, i, &e)`, `fk_zip_find`, `fk_zip_read` (to memory), `fk_zip_extract` (to a file), `fk_zip_extract_all (z, prefix, dest_dir, cb, user)`	**The Archiver's engine** (`Apps/archiver/arc.h`, `zip.h`, `ops.h`, compiled once here): ZIP64, old code pages, ZipCrypto passwords, CRC checked, names made safe for the card. `fk_progress` callback: bytes done of the total, the file being worked on; a non-zero answer stops.
A new ZIP	`fk_zipw_create (path)`, `fk_zipw_add (w, disk_path, name)` (a file, or a folder and all it holds), `fk_zipw_add_data (w, name, data, n)`, `fk_zipw_level`, `fk_zipw_close (w, cb, user, err, cap)`	The entries are said first, the archive is written at the close (to `<path>.part`, then swapped in).
A ZIP in memory	`fk_zipmem_count`, `fk_zipmem_entry`, `fk_zipmem_get (zip, n, name, &out, &n)`; `fk_zipbuf_new`, `fk_zipbuf_add (b, name, data, n, level)`, `fk_zipbuf_finish (b, &out, &n)`	A document read whole and built whole — `.docx`, `.xlsx`, `.odt`, OpenRaster: the plain format (no ZIP64, no password). Level 0 stores (an OpenDocument's `mimetype` first).
Files	`fk_exists` (1 a file, 2 a folder), `fk_file_size`, `fk_load`, `fk_save`, `fk_mkdirs`, `fk_copy (src, dst, cb, user)`, `fk_move`, `fk_remove`, `fk_tree_size (path, &files, &folders)`	A file or a whole tree; `dst` is the new path itself. A folder is not copied into itself; a volume's root is never removed.
Paths	`fk_path_name`, `fk_path_ext`, `fk_path_folder`, `fk_path_join`, `fk_path_unique` (`a.txt` → `a (2).txt`), `fk_human_size`, `fk_dos_time_str`	
Key / value text documents (`filekit/kvtext.h`, C and C++, 2026-10-06)	`fk_kv_new`, `fk_kv_parse (text, flags)`, `fk_kv_load (path, flags)`, `fk_kv_free`; `fk_kv_count`, `fk_kv_key`, `fk_kv_value`, `fk_kv_section`, `fk_kv_block`, `fk_kv_line`; `fk_kv_blocks`, `fk_kv_block_name`, `fk_kv_block_line`; `fk_kv_get (kv, section, key, def)`, `fk_kv_set`, `fk_kv_remove`; `fk_kv_text (kv, comment, &len)`, `fk_kv_save (kv, path, comment)`	A file of `[section]` headers and `key = value` lines read, changed and **written back with the keys a program does not know kept** — bigger than AppKit's `.ini` reader takes (no limit but memory). A section may come back (a pack's many `[level]` blocks, one block a header line); `FK_KV_ESCAPES`: values hold new lines written `\n` (a progress file); `FK_KV_PIPES`: a value goes on over the `|` lines that follow (a pack's tables); every value keeps its line for the error messages. Like `fsutil.h`: `kvtext.h` declares, `kvtext.inc` has the code (`kvtext.cpp` in the library; inline on a PC). Circuits' packs and progress use it; Turtle Quest's and Notes' own readers can move onto it. Test: `sh`

Group	Calls	What it does
		`tools/tests/run_kvtext_test.sh`.

- **Inside**: `filekit/fkcore.cpp` (the C interface), the archives' engine — `filekit/arc.h` (the `Archive` interface a format implements), `zip.h`, `tar.h`, `ops.h` (the formats' table `FORMATS`, opening by the first bytes, extracting, the plans of a change); it was the Archiver's until 2026-10-05 —, `arcbase.h` / `arcpath.h` (strings and paths, also included by the programs), `filekit/fkso.c` (the library's `malloc` over the importer's allocator); zlib's eight sources compiled for the library; newlib's libc linked in. The table: `filekit/filekit.abi` (append-only).
- **Who uses it**: **the Archiver** (wholly: `Apps/archiver/arcfk.h` gives its old `arc::` names over `fk_arc_*`; its welcome page lists the formats the library tells) and ImageKit (PNG's deflate). `/bin/zip`, `/bin/unzip` and the package tools still compile the engine into themselves (they use its C++ classes: `filekit/ops.h`); the office formats still use `img/pngsave.hpp`'s small ZIP — moving them here is DocumentKit's first step (IDEAS.md).
- **A new format**: a class of `arc::Archive` in `filekit/` (`open`, `extract`, `rewrite` if it is written), a line in `ops.h`'s `FORMATS` and in `archive_for` — nothing in the Archiver.
- **Test**: `fktest` on the Pi (47 checks: the formats told, a tar / tar.gz / gzip read, an archive changed through `fk_arc_*`; compression there and back, an archive made and read on the card and in memory, a tree copied / moved / removed, the paths; everything under `SD:/tmp/fktest`).

5.9. ImageKit: pictures (`SD:/lib/imagekit.so`)

(`user/Kits/imagekit/imagekit.h` is the reference; the library's mechanism: §5.6.)

What every program that shows or writes a picture needs, one copy for the whole system. A program links `lib/imagekit.imp.a` and calls plain functions (`ik_*`). Every call takes integers and pointers only (a program built without the FPU calls it — the Image Viewer does); an `ik_image` is opaque; a buffer the library returns is freed with `ik_free`. **The first version (2026-10-05) is the base**: the layers, the masks, the brushes and the document of the photo editor come on top of it later (IDEAS.md).

Group	Calls	What it does
The formats	`ik_formats (list, max)`	**Asked, not assumed** (`struct ik_format`: name, extensions, read / written, alpha, animated): a program lists what the library handles; a format ImageKit learns is one every program has.
A picture	`ik_image_new (w, h)`, `ik_image_from (px, w, h, stride)`, `ik_image_copy`, `ik_image_free`, `ik_width`, `ik_height`, `ik_format`, `ik_pixels`, `ik_fill`, `ik_opaque`, `ik_flatten (im, rgb)`, `ik_has_alpha`	`w × h` pixels `0xAARRGGBB`, straight alpha, rows one after the other (`IK_ARGB8`: the only format today; the handle is opaque so that 16 bits a channel can come).
Reading	`ik_load (path, flags)`, `ik_load_mem`, `ik_probe (path, &info)`, `ik_load_preview`, `ik_load_format`, `ik_is_image_name`; `ik_frames_load` / `_load_mem` / `_count` / `_pixels` / `_delay` / `_width` / `_height` / `_free`	**BMP, GIF (animated), PNG, JPEG, PCX, WebP**. `IK_ORIENT`: the camera's orientation (EXIF) applied — the photo as it is seen. `ik_probe` reads no pixel: the size, the orientation, the date taken, the camera and the exposure (`struct ik_info`); `ik_load_preview`: the camera's own small

Group	Calls	What it does
		picture. The frames of an animation with their delays.
Writing	`ik_encode (im, format, quality, &out, &n)`, `ik_save (im, path, quality)`, `ik_encode_pixels (px, w, h, stride, format, quality, alpha, &out, &n)`	**PNG** (zlib's compression through FileKit: smaller files than before), **JPEG**, **BMP**, **GIF**. PNG keeps the alpha, GIF its clear pixels, JPEG and BMP lay the picture on white.
Transforms	`ik_scale (src…, dst…)`, `ik_scale_rgb` (pixels whose top byte is not an alpha: a photo, a canvas), `ik_resize`, `ik_fit (im, max_w, max_h, grow)`, `ik_cover (im, w, h)`, `ik_crop`, `ik_rotate (im, quarter_turns)`, `ik_flip`, `ik_orient`, `ik_straighten (im, millidegrees)`	**One resize, right in alpha**: smaller, each pixel is the average of all those it covers; larger, bilinear; the colours weighed by the alpha (a transparent pixel's colour does not bleed). `ik_scale` writes into any buffer (a window's canvas, the wallpaper).
Adjustments	`ik_adjust_apply (im, &a)`, `ik_adjust_auto (im, &a)`, `ik_filter_name`	Photos' own, as one tone curve and a colour pass (`struct ik_adjust`: exposure, contrast, highlights, shadows, saturation, warmth −100..100, sharpness 0..100, a filter: black and white, warm, cool, vintage, vivid); "enhance" proposes values from the histogram. The alpha is kept.

- **Inside**: `imagekit/ikcore.cpp` compiles the decoders (`img/imgload.hpp`: stb_image, simplewebp, PCX), the encoders (`img/pngsave.hpp`; `PNGSAVE_DEFLATE` sends PNG's compression to FileKit's zlib), Photos' `exif.h` (the picture's facts) and `imgops.h` (the turns, the crop, the adjustments) behind the C interface; the resize is its own. Built with the FPU, newlib's libm and libc linked in. **It uses FileKit**: `lib/filekit_stubs.o` is linked in and `SD:/lib/filekit.so` is opened when a PNG is first written (the package needs `filekit`). The table: `imagekit/imagekit.abi` (append-only).
- **Who uses it** — everybody, since version 2 (2026-10-05):
  - **reading**: UIKit's `img_load` / `img_load_mem` / `img_inflate` / `img_is_image_name` (`img/imgload.hpp`, what every app calls) are **relays to ImageKit** in `uikit.so` (`uikit/imgload.cpp`: the library opened at the first picture; `ik_frames_take` hands the frames over). One copy of the decoders in the system (`uikit.so` lost 70 KB); a format added to ImageKit is shown by every app.
  - **writing**: the apps that write pictures (Letters, Slides, Photos, Paint, Screenshot, the Media Player) are built with `-DPNGSAVE_USE_IMAGEKIT`: `pngsave::png_encode` / `jpeg_encode` / `bmp_encode` / `gif_encode` (`img/pngsave.hpp`) are then `ik_encode_pixels` — the same names in their code, the encoders out of their binaries, PNG files compressed by zlib.
  - **directly**: `/bin/iktest`, the Image Viewer (its window and the wallpaper it paints: photos turned the way the camera says, a wallpaper made smaller averaged).
  - The PC builds (no shared library) compile the decoders and the encoders into themselves, as before.
  - **resizing**: Photos (`imgops.h`'s `scale_into`: its grid, its viewer, its thumbnails), the Media Player's covers and Paint's smooth scaling of a layer are `ik_scale_rgb` / `ik_scale` in the apps (`-DUSE_IMAGEKIT`), as the Image Viewer and the wallpaper.

    - Still their own (each a special case: IDEAS.md): the dock's icons (a colour key, 4 x 4 samples), Letters' and Slides' pictures in a page (cropped, in page units), Screenshot's fitted view, the Media Player's video frames, UIKit's `ImageBox`.
- **Not in it yet**: masks and selections, drawing and brushes, layers and blend modes with `gpucomp`, thumbnails with their cache, 16 bits a channel, ICC profiles, TIFF / RAW / HEIC. Never MuPDF nor FFmpeg.
- **Rule**: `uikit.so` needs `imagekit.so` (its package says so), which needs `filekit.so`.
- **Test**: `iktest [picture]` on the Pi (28 checks: a picture written in the four formats and read back, the resize in alpha, the turns, the crop, the adjustments; with a picture of the card: probed, read, a thumbnail written).

5.9.0. What moved into the kits on 2026-10-05; SystemKit and NetKit

(The guide to the kits, with an example for each: 06-KITS-GUIDE.md.)

Headers of inline functions that every program copied are now a kit's, declared in a header and compiled once in the library. Which kit: **AppKit** makes a program run (its link to the kernel, and what any program needs to stand: strings, console, `.ini`, starting a program); **SystemKit** is what a program says to the system and to the other programs; **NetKit** is the network; **FileKit**, **ImageKit**, **FontKit**, **UIKit** their subject.

It was	It is	In
`user/applib.h` (strings, console, `.ini`, keymap)	`appkit/appkit.h` (`ax_*`, `app_ini_*`)	AppKit
`user/launch.h`	`appkit/appkit.h` (`lx_*`)	AppKit
`user/notify.h`	`systemkit/notify.h` (`notify`, `notify_action`)	SystemKit
`user/clipboard.h`, `user/clipproto.h`	`systemkit/clipboard.h`, `systemkit/clipproto.h` (`clip_*`)	SystemKit (UIKit's text fields open it when they first copy or paste)
`user/wallpaper.h`, `volume.h`, `trash.h`, `preloadini.h`, `fileassoc.h`, `dockconf.h`, `applet_proto.h`	`systemkit/<the same name>` (`wp_*`, `volume_*`, `mixer_set`, `trash_*`, `preload_ini_*`, `fa_*`, `dock_*`)	SystemKit
`user/ftpfs.h`, `user/httpc.h`, `user/http.hpp`	`netkit/ftpfs.h`, `netkit/httpc.h` (`ftpfs_*`, `http_get` / `http_post` / `http_request`), `netkit/http.hpp`	NetKit — `http.hpp` (the HTTP/1.1 class, its TLS transport) is still a header with its code
`user/bmp.hpp`	`uikit/bmp.h`: `ui::icon_load` (and the old name `ui::bmp_decode`)	UIKit, **through ImageKit**: an icon may be any picture ImageKit reads
`user/fsutil.h`	`filekit/fsutil.h` (`fs_*`)	FileKit
`user/Libs/img` (the codecs' sources)	`user/Kits/imagekit/img`	ImageKit's folder; the programs that still include `imagekit/img/imgload.hpp` / `pngsave.hpp` get relays to `ik_*`
`user/ft`	`user/Kits/fontkit`	FontKit

A program includes the kit's one header — `"systemkit/systemkit.h"`, `"netkit/netkit.h"`, `"filekit/filekit.h"` — which brings the subjects' headers named in the table.

How a kit of this kind is made (SystemKit, NetKit, `filekit/fsutil.h`, `filekit/kvtext.h`): each header `x.h` declares (`SK_API int notify (...)`), `x.inc` beside it has the code, and the kit's one source (`systemkit.cpp`) compiles every `.inc` into `SD:/lib/systemkit.so`, exported by name. A program links `lib/systemkit.imp.a` / `lib/netkit.imp.a` / `lib/filekit.imp.a` (the apps' rules of `user/Makefile` do). **A C program links a kit too**: `tools/libgen` writes, beside the C++ bind, a C one (`--bind-c`: `lib/<kit>_bind_c.c`, the archive `lib/<kit>.imp_c.a`) — the same constructor run before `main`, the library given `umm.h`'s allocator (a heap of that object's own), or newlib's `malloc` / `free` when the bind is compiled with `-DONYX_BIND_LIBC`. The console tools do it (`user/BinUtils/Makefile`: `volume.elf preload.elf notifytest.elf: KITLIBS = ../lib/systemkit.imp_c.a`, `ftp` and `wget` with NetKit's); the header a C program includes must be C (SystemKit's and NetKit's are, but the clipboard; `filekit/fsutil.h` is C++, `filekit/kvtext.h` is C). Where there is no shared library at all — a PC build — the code comes inline with the header (also on request: `-DSK_INLINE` / `-DNK_INLINE` / `-DFS_INLINE` / `-DFK_KV_INLINE`).

Still to come: `docguard.h` in the future DocumentKit; `http.hpp`'s class inside NetKit.

GPIOKit (2026-10-06, `user/Kits/gpiokit/`, `SD:/lib/gpiokit.so`; the guide: docs/06 *GPIOKit*, the reference: docs/19) is made like SystemKit: one header that declares (`gpiokit/gpiokit.h`: C functions `gk_*`, and small C++ classes over them, `namespace gpiokit`), one source (`gkcore.cpp`), freestanding, an append-only `gpiokit.abi`, `lib/gpiokit.imp.a` and `lib/gpiokit.imp_c.a` (`--bind-c`). Two things are its own:

- **It reads the kernel's table itself** — `gpio_ctl` (kapi v92, docs/02 §17) — instead of going through AppKit: **the user's explicit exception** to "only AppKit reaches the kernel" (§5.10). So it is built and shipped **with the kernel**: its `.so` is in the package `onyx` (as AppKit's), and a kernel that moves or changes `gpio_ctl` ships the GPIOKit that follows it. It checks `KT->version >= 92` first (an older kernel: `GK_ENODEV`). Its other needs (sleeping) are AppKit's, as for any kit.
- **It carries a simulator**: a board in memory (inputs driven by `gk_sim_input`, the PWM's waves from the clock, an I2C bus with an SSD1306 at 0x3C and a BME280 at 0x76 — its registers and the datasheet's calibration —, SPI looped back). It runs when the system has no GPIO, when a program asks for it (`gk_sim (1)`), and always in a PC build: `gkcore.cpp` compiles as it is against the simulator's stand-in kernel (`tools/tests/desktop_sim`), or on its own with `-DGK_STANDALONE` (a clock of its own: `tools/tests/run_gpiokit_test.sh`, the BASIC tests).

GameKit (2026-10-09, `user/Kits/gamekit/`, `SD:/lib/gamekit.so`; the guide: docs/06 *GameKit*, the reference: docs/20) is made as GPIOKit, through AppKit only: one header (`gamekit/gamekit.h`: C functions `games_*`), one source (`gamekit.cpp`, freestanding; compiled as it is in the PC tests), an append-only `gamekit.abi`, `lib/gamekit.imp.a` and `lib/gamekit.imp_c.a`, its own package `gamekit` (the Game Library's `needs`, and `onyx`'s for the console's home). It holds what the Game Library and `consolehome` share: the consoles of the emulators' `app.txt` (`games =`, `order =`), the watched folders (`SD:/apps/gamelib.app/config.ini`), the ROMs, their names, their pictures (`thumbs/*.thm`). The gamepads stay in `Include/gamepad.h` (the user, 2026-10-09).

BASIC reaches it by statements of its own (`PINMODE`, `PIN`, `PWM`, `SERVO`, `ON PIN`, `I2C…`, `SPI$`; docs/04 §13 *GPIO*): the runtime (`Libs/basic/runtime.cpp`) opens `gpiokit.so` at the first of them (`bas::onyxKitOpen`, `baskits.cpp`) and calls its entries by their places in `gpiokit.abi` (the macros `GKF_*`; `run_gpiokit_test.sh` checks them against the `.abi`), so a program without GPIO never loads it. `#import gpiokit` works too (`SD:/lib/gpiokit.bi`, made at the build like every kit's).

5.9.1. FontKit: FreeType for the apps (`SD:/lib/fontkit.so`)

(`user/Kits/fontkit/`; it was `user/ft`, `SD:/lib/ft.so` and the package `ft` until 2026-10-05 — the package `fontkit` replaces `ft` on a card.)

The shared library is **FreeType** itself (the apps' lean build: TrueType, the auto-hinter, the smooth rasterizer — `onyx_ftoption.h`, `onyx_ftmodule.h`), its functions exported under their own names (`FT_*`, `fontkit.abi`); `ftso.c` gives it its memory and its files. An app links `lib/fontkit.imp.a` and includes:

- `"fontkit/fonts.h"` — the font manager (`namespace fonts`): the card's families (`SD:/res/fonts`), a font by family / style / size, its glyphs cached, text measured and drawn;
- `"fontkit/uikitface.h"` — UIKit's text face on it (`FtTextFace`): the anti-aliased text of the apps.

These two are still headers that carry their code (each app compiles its own manager and glyph cache); moving them into the library — one cache for the system — is a later step.

5.10. AppKit: the programs' interface to the kernel (`SD:/lib/appkit.so`)

(`user/Kits/appkit/appkit.h` is the header a program includes and the reference of the calls; `user/Kits/appkit/appkit_calls.inc` their bodies; `user/Kits/appkit/appkit.c` the library; the kernel's side: docs/02 §8.)

```
#include "appkit/appkit.h"      // (it was "kapi.h" until 2026-10-05: there is no kapi.h any more)
```

A program talks to **AppKit**, never to the kernel: the header says so by its name, and it only declares. The functions keep their `kapi_` prefix: they are the names of AppKit's table, never renamed.

Every `kapi_*` function a program calls is **AppKit's**, reached **by name**. The kernel loads AppKit by itself and binds it to every program: nothing to open, nothing to link by hand (`user/Makefile` puts `lib/appkit_stubs.o` on every link line; `libonyxposix.a` carries it for the POSIX programs). Only AppKit reads the kernel's table — so **the kernel's table can be restructured by rebuilding AppKit alone**.

- **The declarations and the bodies are two files.** `appkit/appkit.h` declares each call (`KAPI_FN int kapi_seek (void *h, unsigned long long pos);`) with its comment, the structures and the constants. `appkit/appkit_calls.inc` has the bodies, one line a call — `KAPI_CALL (result, name, (arguments), { body })` — and is **the only code that reads the kernel's table** (`KT`). It is compiled:
  - into AppKit (`appkit/appkit.c`, `KAPI_IMPL`): the functions themselves, exported by name;
  - with `KAPI_INLINE`, or in a PC build: inline into the program (`appkit.h` includes it at its end), reading the table itself as every program did before AppKit — for the tests of the table (`el0test`, `faulttest`) and the simulator, whose stand-in kernel has no AppKit. The inline helpers of `appkit.h` that make no kernel call (`kapi_sound_ring_write`, `kapi_clock_us`, the spin locks' primitives) stay inline: an app core may use them.
- `KT` **is not for programs any more**: it only exists in AppKit (and `KAPI_INLINE`). The kernel's version is `kapi_abi_version ()` (it was `KT->version`).
- **The one exception: GPIOKit** (the user's decision, 2026-10-06). `SD:/lib/gpiokit.so` calls the kernel's `gpio_ctl` entry (kapi v92) itself, through the table at `KAPI_TABLE_VA`, without an AppKit wrapper — there is no `kapi_gpio_ctl` in AppKit. The price is the one AppKit pays: GPIOKit is rebuilt and shipped with the kernel (the package `onyx`). A program still never reads `KT`: it calls GPIOKit's `gk_*` (§5.9.0, docs/06 *GPIOKit*).

- **Adding a call**: the kernel's entry (`kern/kapi_abi.h`, `sys/kapi.cpp`, `sys/kapitable.cpp`), then its declaration in `appkit/appkit.h` and its `KAPI_CALL` in `appkit/appkit_calls.inc`; the build appends its name to `appkit/appkit.abi` and makes its stub. Commit the `.abi`.
- **Changing the kernel's table** (an entry moved, removed, two merged, a structure changed): adapt the bodies in `appkit/appkit_calls.inc` so that each `kapi_*` name still does what the programs expect; rebuild the kernel and AppKit, ship them together (the package `onyx`), restart. No program is rebuilt. A name of `appkit.abi` is never removed nor renamed: a call that is gone keeps a body that answers `-KAPI_ENOSYS` (or does it another way). (The window calls, moved to UIKit on 2026-10-08: their names left `appkit.h`, their slots stay as retired placeholders — below.)
- **Its small services** (§8): the strings, the console, the `.ini` reader, the keyboard layout loader, the **notifications** (`notify`, `notify_action`) and the **starting of programs** by their runner (`lx_launch`, `lx_open`, `lx_runner`…) — `appkit_lib.inc`, exported by name like the calls.
- **Rules**: the table is append-only by name (the generator refuses a removal); AppKit allocates nothing and has no constructor (its only data: the `.ini` reader's store, private to each program); it is built with the FPU on, its calls passing floats through.
- **Cost**: one more indirect jump a call (the stub), then AppKit's function — nothing beside a system call.
- **The window calls are UIKit's** (2026-10-08, docs/POCKETUI-TECH-STUDY.md phase P2): the windows left the kernel for **Elegant**, the graphics server, a user process (`user/Servers/elegant`, `SD:/bin/elegant`, kapi v89), and since 2026-10-08 the code that speaks to it is **UIKit's port**, no longer AppKit's: the 46 window functions (`kapi_create_window`, `kapi_present`, `kapi_set_menu`, `kapi_win_list`, `kapi_tray_set`…) were **removed from AppKit** — from `appkit.h` and `appkit_calls.inc`; in `appkit.abi` their slots stay under the names `kapi_retired_<slot>` (placeholders answering 0 or the call's old error value: nothing renumbered, a program built before keeps working but for its direct window calls, which fail) — and `appkit_ws.inc` with them. A program calls **`uk_win_*`** (§5.10.1). AppKit keeps the kernel's side: `kapi_ws_ctl` (the transport), the pump, `kapi_screen_size`, and the full screen's kernel primitives (`kapi_fullscreen_begin` / `_end`, which no longer tell the server: `uk_win_fullscreen_begin` does, `kapi_present_fb`, `kapi_fullscreen_direct`).
- **The graphics server's calls** (kapi v89, v97): `kapi_ws_ctl (KAPI_WS_*, ...)` is the transport Elegant uses (the role, the display, the raw input, the programs' requests). Since v97 the server is the one `SD:/etc/system.ini shell =` names (desktop: Elegant; pocket / console: PocketUI, `--mode <m>` on its command line; Elegant if it fails), `kapi_lib_open_as` lets that server publish its own UIKit under `SD:/lib/uikit.so` (§5.6), and anyone may call `kapi_ws_ctl (KAPI_WS_SWITCH, 0, 0, 0)` to end the server and start the one `shell =` names now (→ 0, or 1 when Elegant had to be started instead; the caller waits). A server ends on the input event `KAPI_WS_IN_QUIT`. docs/02 §8 *v97*, §10.

5.10.1. The window API: UIKit's `uk_win_*` (`uikit/win.h`)

(`user/Kits/uikit/win.h` the declarations and the reference; `win.cpp` the entries; `port/` the port — the window driver of `SD:/lib/uikit.so`; docs/POCKETUI-TECH-STUDY.md §4–5.)

```
#include "uikit/win.h"          // C or C++; a C++ app has it through "uikit/uikit.h"
```

A program's windows and what it asks of the graphics server are **UIKit's**, as plain C functions — `extern "C"`, entries of `SD:/lib/uikit.so` like the rest of UIKit (`uikit.abi`). Every graphics server has its own UIKit loaded under the name `SD:/lib/uikit.so` (Elegant's, the desktop's; PocketUI's for the pocket and console modes, `SD:/lib/pocket/uikit.so`: `kapi_lib_open_as`, §5.6, §5.10.2), and **each UIKit's port is the**

only code that speaks its server's protocol — so one binary runs on every server. They were AppKit's `kapi_*` window calls until 2026-10-08; the renaming: `kapi_create_window` → `uk_win_create`, every other `kapi_<name>` → `uk_win_<name>` without a doubled `win_`, and:

AppKit's (gone)	UIKit's
`kapi_create_window`, `_ex`, `kapi_win_new`, `kapi_win_select`, `kapi_win_destroy`	`uk_win_create`, `uk_win_create_ex`, `uk_win_new`, `uk_win_select`, `uk_win_destroy`
`kapi_move_window`, `kapi_resize_window`, `kapi_resize_window2`, `kapi_win_resizable`, `kapi_set_window_alpha`, `kapi_win_geometry`, `kapi_get_chrome`	`uk_win_move`, `uk_win_resize`, `uk_win_resize2`, `uk_win_resizable`, `uk_win_alpha`, `uk_win_geometry`, `uk_win_chrome`
`kapi_present`, `kapi_draw_text`, `kapi_set_key_handler`, `_click_`, `_pointer_handler`, `kapi_set_cursor`, `kapi_cursor_pos`, `kapi_cursor_shown`	`uk_win_present`, `uk_win_draw_text`, `uk_win_on_key`, `uk_win_on_click`, `uk_win_on_pointer`, `uk_win_cursor`, `uk_win_cursor_pos`, `uk_win_cursor_shown`
`kapi_set_menu`, `kapi_get_menu`, `kapi_menu_command`	`uk_win_menu_set`, `uk_win_menu_get`, `uk_win_menu_command`
`kapi_win_list`, `kapi_win_raise`, `kapi_win_close`, `kapi_win_minimise`, `kapi_win_move`, `kapi_win_desk`, `kapi_desk`, `kapi_win_read`, `kapi_list_windows`, `kapi_raise_app`, `kapi_toggle_app`	`uk_win_list`, `uk_win_raise`, `uk_win_close`, `uk_win_minimise`, `uk_win_place`, `uk_win_to_desk`, `uk_win_desk`, `uk_win_read`, `uk_win_apps`, `uk_win_app_raise`, `uk_win_app_toggle`
`kapi_wallpaper_buffer`, `_commit`, `_generate`; `kapi_drag_begin`, `_data`; `kapi_tray_set`, `_clear`, `_list`, `_icon`, `_activate`; `kapi_get_wheel_speed`, `kapi_set_wheel_speed`	`uk_win_wallpaper_buffer`, `_commit`, `_generate`; `uk_win_drag_begin`, `_data`; `uk_win_tray_set`, `_clear`, `_list`, `_icon`, `_activate`; `uk_win_wheel_get`, `uk_win_wheel_set`
`kapi_fullscreen_begin` / `_end` (the server told, then the kernel's call)	`uk_win_fullscreen_begin` / `_end` (AppKit's `kapi_fullscreen_*` are now the kernel's primitives alone; `kapi_present_fb`, `kapi_fullscreen_direct` stay AppKit's)
—	`uk_win_server` (the server's name, mode, screen, work area, scale, size class: `struct uk_win_server_info`), `uk_shell_*` (PocketUI's shell calls; the desktop's UIKit answers `-KAPI_ENOSYS`)

The flags (`WIN_FLAG_*`), the events (`GUI_EVENT_*`, `GUI_PTR_*`, `KEY_*`, `MENU_QUIT`) and the structures (`struct kapi_chrome`, `kapi_win_geom`, `kapi_win_info`, `kapi_tray_info`) keep their names and their place (`appkit/appkit.h`, `kern/kapi_abi.h`): the events still come through the kernel's pump (`kapi_pump_events`, `kapi_pump_wait`, `kapi_should_exit`: AppKit's). The demos' aliases `create_window` / `present` are in `uikit/win.h` now.

- **Linking**: a C++ program links `lib/uikit.imp.a` (every app of `user/Makefile` does), with `onyxpp.hpp` in one of its files (the bind hands the library its `operator new`); a **C program** links **`lib/uikit.imp_c.a`** (UIKit's stubs and the C bind, `libgen --bind-c`; `BinUtils/Makefile`'s `KITLIBS`: `el0test`, `gpcdemo`); a newlib C program compiles `lib/uikit_bind_c.c` with `-DONYX_BIND_LIBC` (its `malloc`) beside `lib/uikit_stubs.o` (`rdpd`). A program that never had UIKit maps one more library (its code shared; about 64 KB of private data a process).

- **The structure of UIKit**: the common sources (every widget, the canvas, the window API's entries `win.cpp`) and **the port** (`user/Kits/uikit/port/`, internal: `namespace uikit::port`, not in the table — libgen's `--exclude '^_ZN5uikit4port'`): `port/port.h` the interface (one function per `uk_win_*`), `port/port_desktop.cpp` the desktop's — the window driver for Elegant, with **`port/elegant.h`, the protocol** (private to the desktop port and Elegant; `wstest` its test) — and `port/port_pocket.cpp` PocketUI's (`port/pocket.h`, §5.10.2). Both include **`port/client.inc`**, the client of a server that speaks Elegant's operations (the requests through `kapi_ws_ctl (KAPI_WS_CALL, ...)`, `KAPI_WS_KICK` at a present, the per-program window state and **the replay** when the server is started again: it was `appkit_ws.inc`), and on a PC **`port/host.inc`**. `user/Kits/uikit/port.cpp` compiles the port chosen (`UK_PORT_POCKET`: PocketUI's; else the desktop's), so every build that compiles `user/Kits/uikit/*.cpp` (the PC's simulators, the tests' hosts, Koton for Windows) has it. **On a PC** the port relays each call to the stand-in kernel's window manager (`kern/kapi_abi.h`'s *NOT ON ONYX* entries, `tools/tests/desktop_sim/fakekapi.cpp`) — what AppKit's `KAPI_HOST` bodies did; with **`UK_PORT_WIRE`** it speaks to a graphics server built for the PC instead (`tools/tests/server_sim`, §5.10.3).
- **Adding a window call**: its declaration in `uikit/win.h`, its entry in `win.cpp`, its port function in `port/port.h` and in **both** ports (`port/client.inc` when it is an operation both servers share: its number in `port/elegant.h`, its decoding in `user/Servers/common/ops.cpp`; PocketUI's meaning, if another, in its policy `user/Servers/pocketui/wm.cpp`); the desktop's build appends it to `uikit.abi`, the pocket one is `--frozen` (§5.10.2). Then `python tools/docgen/kitdocs.py`.

5.10.2. One UIKit per server: the pocket UIKit, `abi_same.py`

(docs/POCKETUI-TECH-STUDY.md §5; phase P3, 2026-10-08.) Each graphics server has its UIKit with **the same export table**: a program is built once (against the desktop's `lib/uikit.imp.a`, whose stubs call the entries **by slot**) and runs on both.

- **The build** (`user/Makefile`): the common objects `lib/uikit/*.o` are compiled once; `lib/uikit.so` links them with `lib/uikit/port.o` (the desktop port), **`lib/pocket/uikit.so`** with `lib/pocket/port.o` (`Kits/uikit/port.cpp` compiled with `-DUK_PORT_POCKET`). The pocket table (`lib/pocket/uikit_table.S`) is generated by libgen against the **same `Kits/uikit/uikit.abi`** with **`--frozen`** (a function new to one library is an error: an entry enters the `.abi` through the desktop's build first) and no stubs nor bind of its own. `make stage` copies it to `SD:/lib/pocket/uikit.so`; the package **`uikit`** carries both (`tools/pkg/packages.ini`): they cannot drift apart on a card.
- **What may differ**: only the port's `.cpp` (`namespace uikit::port`, excluded from the table). The headers are the ABI (`uikit/abi.h`: the table, the classes' layout, the virtual slots, the inlines, the program's globals): one set for both, byte for byte. The pocket port today = the desktop's client (`port/client.inc`)
    - PocketUI's hello (`PK_OP_HELLO`, its first request: a server that does not answer it — Elegant, for a daemon of the other session that kept this UIKit — gets no window, the reason logged once) + `uk_win_server`'s answer (`PK_OP_SERVER`: mode, work area under the status band, size class) + the shell's calls (`uk_shell_*` → `PK_OP_SHELL`..`PK_OP_DIM`). The frames need nothing: PocketUI says which windows have one (insets 0 for a filled window; a card keeps UIKit's Milk title).
- **The identity test**: **`tools/libgen/abi_same.py lib/uikit.so lib/pocket/uikit.so --tables lib/uikit_table.S lib/pocket/uikit_table.S`** reads both export tables from the ELF files (`onyx_lib_table`: version, size, then each slot's pointer through the dynamic relocations, named by the symbol table) and requires the same version and the same symbol at every slot, and the

two generated tables line for line. `make libs` runs it (`lib/pocket/abi_same.stamp`: the build fails on a difference); `sh tools/tests/shlib/abi_same_test.sh` runs it and checks that it tells two different tables apart.

5.10.3. PocketUI, the pocket and console modes' server (`SD:/bin/pocketui`)

(`user/Servers/pocketui/`; docs/02 §10 "Two servers, one common code"; docs/POCKETUI-TECH-STUDY.md §7.) The kernel starts it for `shell = pocket` / `console` (`SD:/etc/system.ini`): `pocketui --serve [--restart] --mode pocket|console`. It is built from **`user/Servers/common/`** — the code it shares with Elegant (`serve.cpp` the loop, `route.cpp` the routing, `core.cpp` + `wm/` the window store and compositor, `ops.cpp` the requests; `common.mk`) — and its own part: **`main.cpp`** (the arguments; in the policy's `registered` hook, between `KAPI_WS_REGISTER` and `KAPI_WS_DISPLAY`, `kapi_lib_open_as ("SD:/lib/pocket/uikit.so", "SD:/lib/uikit.so", 1, &err)`, the table kept while it lives), **`wm.cpp`** (the policy, `common/policy.h`), **`band.cpp`** (the status band). Its protocol: **`user/Kits/uikit/port/pocket.h`** (private to it and the pocket port) — Elegant's operations by their numbers and structures (`port/elegant.h` included) with PocketUI's meanings, and its own from `0x100`.

Policy	What it does
The work area	the screen less the top band: the desktop's **global menu bar** when it runs (pocket), else PocketUI's own **status band** (24 px, a topmost system window — the Onyx gem, the app in front, the time; the kernel's bitmap font) — the whole screen in console
The global menu bar (pocket)	a topmost window across the screen's top edge (`user/Apps/menubar`) is accepted as the top band (`PK_KIND_BAR`): PocketUI's status band is taken away while it is there, made again when it goes; `EL_OP_MENU_GET` / `EL_OP_MENU_COMMAND` answer with the **front app's** topmost window (a filled window is borderless, which Elegant's "active window" skips: `CWindowManager::GetActiveMenu`'s `pFor`). Console: refused (P9 shows the menus on demand)
A program's window	(the user's rule, 2026-10-08) its **main window** (its first) is **full screen**: **filled** (`WIN_FLAG_BORDERLESS` forced, at the work area's top left; bigger than the work area: cut) — or, of a fixed size smaller than the work area, **centred** (`PK_KIND_CENTRE`: frameless, in the middle, over PocketUI's **matte** — a window of its own just under it, the work area in the app's background colour: its client area's top left pixel); it fills once it says it is resizable. **Cards** (framed, centred) are a program's other windows (a dialog, a second window) when they fit, and the main windows of the apps listed in **`SD:/etc/pocketui.ini`** `[cards]` (one app's name a line, `#` comments; read again at each program's first window — shipped: `tinycalc`) or whose `app.txt` says **`pocket = card`**
`EL_OP_RESIZABLE` on	the window is **filled**: its frame dropped (`CWindow::DropChrome`), `GUI_EVENT_WINRESIZE` to the work area's size (UIKit's `Root::frameResize`); a program that ignores it (its own pointer handler: the Terminal, Media, PDF, Screenshot) gets `GUI_EVENT_WINCTL` maximise once 0.25 s later
Popups	a borderless or topmost window stays where it asked (placed by PocketUI when it asked none), on the screen
The shell's windows	(P5) the program registered by `PK_OP_SHELL` (`pocketshell`): its **backmost** window is its **home** (`PK_KIND_HOME`: at the work area, frameless, moved with it), its topmost or borderless ones its overlays and toasts (`PK_KIND_SHELL`: where asked, even off the

Policy	What it does
	screen or across its top edge); none of them is an app (not fronted, not set aside, not a task); they never count as a band of the work area (`CWindow::SetNoInset`)
Refused	a backmost window of another program than the shell (the agenda, the stickies), a topmost one on the bottom edge whatever its width (the dock — narrower than half a 1920 px screen), a second menu bar (pocket) or any on the top edge (console)
Moves	`EL_OP_MOVE`, `EL_OP_WIN_MOVE` ignored for a filled window, a card and the menu bar; cards and filled windows are **pinned** (`CWindow::SetPinned`): their title bar neither drags nor maximises them, no edge resizes them
Workspaces	one (`EL_OP_DESK`: count 1; `EL_OP_WIN_DESK`: 0)
One app in front	kept by the policy, not read back from the z-order: a program that **opens a window** or is **raised** (`EL_OP_WIN_RAISE`, `EL_OP_APP_RAISE` — `run` of a running app —, its status icon opened, the switcher's `PK_OP_FRONT`) is fronted (its windows raised, first of the recent ones); the others' windows **set aside** (`CWindow::SetAside`: hidden as off-desk) and kept under its own; the front program gone or minimised: the one fronted before it. **Home** (P5, with a shell): no app in front, every app set aside, the shell's home shown with the keys — asked by `PK_OP_FRONT (0, 0)`, or when no app shows a window; any program fronted leaves it. `EL_OP_MENU_GET` at home: no menus (the menu bar's Onyx menu)
Keys	with a shell: the keys it registered (`PK_OP_KEYS`: Alt+Tab, Super alone, Super+N...) go to it as `UK_SHELL_KEYEV` events, every key and the modifiers' changes while it grabs them (`PK_OP_GRAB`: an overlay up); a lone **Super** is seen through the policy's `mods` hook (the kernel gives the server the Super bit, `KAPI_WS_MOD_SUPER`, which `serve.cpp` keeps from the window manager). Without a shell: Alt+Tab / Alt+Shift+Tab cycle the programs. **The keys follow the front program**: typed keys go to its topmost window (home: the shell's home) even when the window manager would give them to another window standing above it — logged once `pocketui: the keys to <app> ...` — and its window is raised when another one has the window manager's keys (`KAPI_WS_FOCUS`: `kapi_key_held`, the pads). A program in full screen keeps every key
Full screen	as Elegant (`KAPI_WS_IN_FULLSCREEN`: `el_core_fullscreen`): the window at 0,0, `KAPI_WIN_FULLSCREEN` in the list; the policy moves, fills, sets aside nothing meanwhile; logged `pocketui: full screen: <app>` / `the full screen given back`. (Both servers, common `ops.cpp`, 2026-10-08: the list gives the full-screen window no frame and a gen moving every 50 ms, `EL_OP_WIN_READ` of it what the screen shows — `el_sys_screen_grab` — so rdpd shows the game, not its stale canvas)
The remote desktop	a window set aside is listed with `KAPI_WIN_OFFDESK` (rdpd hides it on the PC: only the app in front shows); the Super key comes from vncd / rdpd too (`remotekeys.h`, `kapi_inject_modifiers`)
The viewport (P6)	a filled window **bigger than the work area** that will not shrink (a fixed size, a least size over the screen: Setup's 800 × 600 at 800 × 480) keeps its canvas; PocketUI moves it up / left under the work area's edges and draws thin indicators at the work area's right and bottom (its own topmost windows): the wheel over an indicator, or an indicator dragged, scrolls; the focused control the pocket UIKit tells

Policy	What it does
	(`PK_OP_FOCUS_RECT`, each time the focus moves) is kept in view. The policy's new **`pointer`** hook (`common/policy.h`, `WS_POLICY_HAS_POINTER`; Elegant's is 0) sees the pointer before the window manager. `PK_OP_TEXT_HINT` (the focused field's type: P10's on-screen keyboard) and `PK_OP_UNITS` (`uk_logical_units`: P10's native scale) are kept per window
Split view (P8)	two programs side by side (landscape) or one over the other (portrait): the **front** one and the **side** one, a divider between them (PocketUI's own window, the accent's colour on the front one's side). Asked by the shell (`PK_OP_SPLIT`: the switcher's **S**) or the keys **Super+**← / **Super+**→ (the front program to that half, the one fronted before it to the other), ended by **Super+**↑, by home, or when one of the two goes. **Super+[** / **Super+]** move the divider (40 / 50 / 60 %), **Super+Tab** — or a press in the other half — passes the front (the keys, the menus) to the other program. Each program is laid out in **its half**: `uk_win_server` answers the half as its work area and the size class of that half (`UK_SC_NARROW` under 480 px), so the adaptive widgets fold by themselves; a full-screen program ends nothing but covers both. `pk_split ()` tells the rest of the server; tested by `server_sim`'s `pocket-split` and `pocket-split-portrait` scenes
Who has the pads (P9)	the kernel gives the pads' buttons and `kapi_key_held` to the program the server names (`KAPI_WS_FOCUS`); the policy's new **`focus`** hook (`common/policy.h`, `WS_POLICY_HAS_FOCUS`; Elegant's is 0) lets PocketUI name **its shell** while the shell grabs the keys (an overlay up) and at home: the console's menu over a game takes the pad, the game reads none
The on-screen keyboard (P10)	**the shell's** (`pocketshell`'s `keys.h`), the server knows two things of it. **Its window is a band**: a window of the shell that is opaque (no `WIN_FLAG_ALPHA`), as wide as the screen, on the screen's bottom edge and no taller than half of it ends the work area above it (the policy's tick clears its `SetNoInset`; parked off the screen it is no band) — the filled windows are sized again, the shell gets `UK_SHELL_EV_AREA`. **The focused field**: the pocket UIKit tells each window's (`PK_OP_TEXT_HINT`, from `uk_set_input_type` / a Textbox taking the focus); when the front program's changes, the shell gets `UK_SHELL_EVENT` with `UK_SHELL_EV_TEXT` in the value's low byte and the type + 1 in bits 8–15 (0: none) — pocket mode only. The keys typed are `kapi_inject_key` (the server routes them to the program in front; the keyboard's window never has the keys). Tests: `run.sh`'s `shell-osk` (the work area 450 → 266 px at 800 × 480 and back)
Not yet	the scale (P10: 2 × composition, native scale, `icon@2x`), portrait sizes / rotation / touch (the kernel's part of P10: no device yet), a dim behind a card (`PK_OP_DIM`), the console's quick menu for games (save states), the top edge's reveal

On the PC (`tools/tests/server_sim/run.sh [out]`): the servers' code built for the host with a real app as their client in one process (`server_sim.cpp` stands for the kernel's side: `kapi_ws_ctl`'s operations, the shared buffers mapped twice, the events) — the app built against the **wire** port (`-DUK_PORT_WIRE`, with `-DUK_PORT_POCKET` for PocketUI's: the port of `SD:/lib/pocket/uikit.so`, not the PC relay), its script's pointer and keys in screen coordinates, `dump` the composed screen, `screen` the screen as the turns composed it (only the damage, as on the Pi), `expect` the policy's checks (`kind`, `frame`, `area`, `client`, `pos`, `front`, `aside`, `made`, `menu`, `bar`, `band`, `opos`; P5: `home`, `shell`, `tasks`, `kindN` / `okind[N]` a window's kind, `shownN` an app's window N on the screen, `focus` who has the keys, `keys` the keys the app received, `matte`, `full`

the full screen), `other` another program's window (its flags and place: the desktop's bands), `otherpic` the same painted from a dump (a real app's picture), `othermenu` its menus, `owin` its second window, `oclose` its end, `raise` / `place` that program's own requests, `mods 8` the Super key. It runs the Terminal, the Calculator and TinyPad under PocketUI at 800 × 480 (and 640 × 480, 1920 × 1080, console: a program started over another comes in front, a card does not move, the menu bar, the dock refused), then **Elegant before and after** the extraction of `user/Servers/common/` on the same scripts — the same pixels required. Since P5 also: the full-screen rule at 800 × 480 and 1920 × 1080 (Letters, the File Viewer, the Terminal, TinyPad filled and typed into — also under a window of no app; the Calculator a card, centred over its matte without the list; a dialog a card), a program in full screen (`tools/tests/server_sim/fsapp.cpp`, a BASIC program's way), **the pocket shell** (the real `pocketshell` with the real menu bar's band and the real apps' pictures: the launcher, the search, the switcher, quick settings with notifications, at 800 × 480, 1280 × 720, 640 × 480, 1920 × 1080, portrait 480 × 800, and in French — the pictures `pocketshell-<shot>-<size>[-fr].png`; `SHELL_PNG=docs/compact-shell/real` copies them), and **Elegant built from the previous revision's `user/Servers/common/`** (HEAD, or `COMMON_BEFORE`) against the working tree: the same pixels. Its pictures of PocketUI are in `docs/compact-shell/real/` (copied from its output folder when they change).

5.10.4. The sessions: the mode's programs, the switch (`systemkit/session.h`, `/bin/session`)

(docs/POCKETUI-TECH-STUDY.md §8, phase P4; the user's side: docs/04 §5 Sessions.) The programs started at boot are split in two: **`SD:/etc/autostart`**, the **system's part** (`wait pkg commit`, the services `clockd`, `pkgd`, `clipd`, `printd`, `telnetd`, `vncd`, `rdpd`, `keyb`, `preload /boot`) with one line **`session`**, and the **session files `SD:/etc/session/desktop`**, **`pocket`**, **`console`** — the interface's programs of each mode, in the autostart's syntax (`run <app>`, a `/bin` tool, `sleep`, `wait`, Setup's `#setup:` lines). Shipped: desktop = `voronoy`, `setup` (a new card), `menubar`, `notifyd`, `dock`, `agenda`, `stickies`; pocket = `menubar` (PocketUI's top band) and `pocketshell` (P5: it serves the notifications, no `notifyd`); console = `terminal`, `gamelib` until `consolehome` (P9) — no menu bar, no dock. A program of one mode only goes into that mode's file; a service of every mode into the autostart.

`/bin/session` (`user/BinUtils/session.c`, C: `lib/systemkit.imp_c.a` + `lib/uikit.imp_c.a`):

- alone (`init` runs the autostart's line): the mode is **the running server's** (UIKit's `uk_win_server`: a `shell=pocket` whose PocketUI failed at boot got Elegant — the desktop's file runs), its file run line by line as `init` does (fire-and-forget; `sleep`, `wait`; a `session` line skipped). The desktop's file missing: its defaults (`voronoy`, `menubar`, `notifyd`, `dock`, `agenda`). `start MODE`, `mode`, `list [MODE]`.
- **`switch MODE [--force] [--no-ask] [--wait S] [--keep PID,...]`**: (1) every program owning a window that is not `WIN_FLAG_SYSTEM`, not one of the session files' programs, not kept, is asked to close (`uk_win_close` on its first window: its close box — an unsaved document asks, `Include/docguard.h`), then waited for (`kapi_list_procs`, 5 s); the ones left: `SD:/tmp/session.wait` (`name<TAB>title` lines) and exit status 2, nothing changed, unless `--force` (`kapi_kill_pid (pid, 1)`). The tool's **parents** (a program whose `kapi_proc_tree` holds it: the Mode applet, the Control Panel, the Terminal it was typed in) and `--keep`'s are not asked: ended at the very end (ending a terminal kills its tree, the tool with it). (2) the processes named in the three session files and `notifyd` are ended (`kapi_kill_pid (pid, 0)`, 2 s, then 1), and `printd` (it maps the server's UIKit through PrinterKit: started again after). (3) `session_set_mode`, (4) `kapi_ws_ctl (KAPI_WS_SWITCH)` (kapi v97): 0 the new server, **1 it failed → Elegant**: `shell=desktop` put back, the desktop's file, a notification; `-KAPI_EBUSY` (full screen): `shell=` back, the old session's file again, exit 3; `-KAPI_ENOSYS` / `-KAPI_EINVAL` (a kernel before v97): `kapi_reboot` (shell= written). (5) the new

mode's file, `printd`. Exit status: SystemKit's `SESSION_*` (0 switched, 1 fell back, 2 programs waiting, 3 refused, 4 bad arguments, 5 failed).

- `migrate`: SystemKit's `session_migrate ()`.

SystemKit `systemkit/session.h` (C and C++; docs/12): `session_modes`, `session_mode_name`, `session_mode_find`; `session_mode ()` / `session_set_mode (m)` (`system.ini`'s `shell =`, through `locale_ini_get` / `_set`; no line or an unknown word: desktop), `session_file (m)`, `session_programs (m, out, cap)` (the names the processes of a mode's file have: `run <app>` → the app, a path → its file's name, `wait <cmd>` → the command's tool, `sleep` none), **`session_switch_start (m, flags, keep_pid)`** (spawns `SD:/bin/session switch …`: the handle for `kapi_proc_done` / `kapi_wait`; `SESSION_FORCE`, `SESSION_NO_ASK`; **the caller must stay until it ends**: the kernel ends a process whose parent has ended — Setup left at once and its switch died, fixed 2026-10-09) and `session_switch (m, flags)` (waited for), `session_waiting (out, cap)` (after `SESSION_WAITING_APPS`), **`session_migrate ()`**. **`autostart_has` / `autostart_ensure`** (`autostart.h`) look in the autostart **and** the session files, and add a line to the file that holds its anchor (Notes' `run stickies` after `run agenda`: `session/desktop`; the Clock's `run clockd` after `session`: the autostart).

The migration of a card: an autostart from before the sessions (no `session` line — active, `#session` or `#setup: session` count) is split by **`session_migrate`**: the desktop's lines (`run voronoy`, `setup`, `menubar`, `notifyd`, `dock`, `agenda`, `stickies`, `wifimenu`, `imageview`, held or not, Setup's `#setup#` comments) and the comment lines just above each go to `SD:/etc/session/desktop` — **made anew from them** (the package's own `session/desktop`, written beside as a new config file, would hold `run setup`) —, the line `session` (with two comment lines) where the first of them was (before `preload` when there is none), every other line where it was; the desktop file is written first, then `autostart.old`, then the autostart (a failure leaves the autostart as it was: the next try makes it anew). It is called by **`pkg commit`** (`user/BinUtils/pkg.cpp`, at every boot's `wait pkg commit`; compiled in with `SK_INLINE` — the boot needs no library): the commit that brings this onyx version runs the old `pkg`, so the split happens at the next boot's commit, `init` still running the old lines from its buffer — the boot after that one is in the new layout. **Setup** (`user/Apps/setup/system.h` `autostart_finish`) gives back the `#setup:` lines of every session file (those of the mode chosen started now), removes its `run setup` line and comments there; the keyboard's and the services' lines stay the autostart's.

The Control Panel's Mode applet (`user/Apps/modeconf/`, `SD:/apps/modeconf.app`, `applets/12-modeconf.lnk`): three cards (`ModeCard`: a 160 × 100 picture from `res/<mode>.bmp` — made by `tools/screenshot/modeconf_previews.py` from the design study's mock-ups —, the name, *in use*, a line), **Apply** → `uk_messagebox` → `session_switch_start (mode, 0, host)` (the applet's host pid from `--applet <surface> <host>`), polled in `Root::onTick`; on `SESSION_WAITING_APPS` a `Modal` names the program (`session_waiting`) with **Wait** (`SESSION_NO_ASK`), **Force** (`| SESSION_FORCE`), **Cancel**. On a switch the applet and the Control Panel are ended with the old session.

Tests (PC): **`sh tools/tests/session/run.sh`** — `session_test.cpp` (the split of the card's autostart of before, `tools/tests/session/autostart.before`: every line kept, in order, in one of the two files; once; the user's lines; `\r\n`; too big, read-only; the files' programs; the modes; `autostart_*` across the files; Setup's end on the new card) and `session_tool_test.cpp` (`/bin/session` with `main` renamed, against stand-in processes, windows and `KAPI_WS_SWITCH`: the boot, the commands, the parsing's errors, a program waiting, `--force`, `--keep`, a parent kept, the fallback, `EBUSY`, an old kernel, `migrate`); `tools/tests/pkg/test.py` (the commit splits the card's autostart, once); `run_notes_test.sh`, `run_notes_sim_test.sh` (Stickies' line in `session/desktop`).

5.10.5. The pocket shell (`pocketshell`) and the shell protocol (`uk_shell_*`, `PK_OP_*`)

(docs/POCKETUI-TECH-STUDY.md §4.3, §7.2, phase P5; the look and rules: docs/COMPACT-SHELL-STUDY.md §6; the user's side: docs/04 §5 The pocket mode.) **`user/Apps/pocketshell/`** (`main.cpp`: the windows, the launcher, the switcher, quick settings, the toasts, the keys, the screen and the scale followed; `home.h`: the launcher v2; `look.h`: its drawing -- rounded boxes of any radius by their signed distance, outlines, soft shadows and glows, pictures clipped to a rounded box, the match in a text; `today.h`: the agenda; `catalog.h`: the apps, the applets, the categories, the recent apps, the search — one unit; FT_APPS; `sdcard/apps/pocketshell.app/`, category `Shell`: the `onyx` package) is the pocket mode's shell, started by `SD:/etc/session/pocket` after the menu bar. One program, three windows (v94): its **home** (window 0, `WIN_FLAG_BACKMOST`: the launcher, its canvas the screen's size, resized to the work area), the **overlay** (window 1, topmost, `WIN_FLAG_ALPHA`, the work area: the switcher, quick settings or the clipboard's sheet -- `clipsheet.h`: clipd's widget protocol, the paste typed with `kapi_inject_key`; parked off the screen and see-through when hidden — made on the screen first: a place off it would be "placed by the server") and the **toast** (window 2). It serves the IPC services **`shell`** (SystemKit's `systemkit/shell.h`: `shell_ask (SHELL_MSG_HOME | _SWITCHER | _QUICK | _SEARCH | _CLIPBOARD)`, `shell_running ()` — the menu bar's Onyx (the Home button in pocket) and its time ask it) and **`notify`** (SystemKit's `notify.h`: in pocket no `notifyd` runs — a notification is a toast, then a line of quick settings; the message types are apart: `NOTIFY_MSG_SHOW` 1, `SHELL_MSG_*` from 101). Everything is drawn by hand (`look.h`, `uikit/paint.h`; FreeType faces of 10 to 22 lp times the scale, opened again when the scale changes) and laid out from the logical size: the launcher v2 as its approved mock-ups (`docs/COMPACT-SHELL-STUDY.md` §6.2, `tools/screenshot/mockup_compact.py pocket_home_v2`) -- the search field 34 lp, Today beside it (380 lp field) or under it below 700 lp, the Today column (300 lp) from 1100 lp, chips 30 lp, the card with plates of 60 lp in cells of 94 × 100, the Documents of Recent when a row of them fits, the Running strip (104 lp) from 400 lp of height with thumbnails of 134 × 62 (`uk_shell_thumb`, the window's top, taken again when the tasks change or after a second); the search's best match (216 lp) at the left from 700 lp, else one column. The Today data: `SD:/apps/calendar.app/agenda.txt` (`YYYYMMDD|[HH:MM ]what`, which the Calendar writes for the Agenda widget) and the notifications it serves; the documents: SystemKit's **`recent.h`** (`recent_docs`, `recent_doc_add`; `SD:/etc/recent-docs`; `fa_open` notes every file it opens). The switcher's row — as many cards as fit: 5 at 800, 4 at 640 — or column in portrait, the panel 340 lp or the width under 420 lp. The scale: `SD:/etc/theme.txt` `scale =`, else 1.5 from a 1000-line screen, 2 from 1800 (the apps are not scaled yet: P6/P10), of the screen PocketUI gives (`uk_win_server`; `kapi_screen_size` until it answers): read again at `UK_SHELL_EV_AREA`, a display resize and every second -- a bigger screen grows the canvases, another scale reopens the faces; each change logged `pocketshell: the screen W x H, the work area ..., the scale S%` (the Pi's report: after a live switch from the desktop the shell was laid out for a stale screen -- server_sim's `shell-late-1080`: `SIM_SERVER_LATE`, `SIM_SCREEN_STALE`). Its words: `TR` and `sdcard/apps/pocketshell.app/lang/fr.txt` (the categories and the applets' names by `// TR:` lines). The recent apps: `SD:/etc/pocket/recent` (the app fronted, whoever started it).

The shell's calls (UIKit, `uikit/win.h`; the desktop's UIKit answers `-KAPI_ENOSYS` to each — Elegant has no shell — and the pocket port speaks them as `PK_OP_*`, `uikit/port/pocket.h`, protocol revision 2):

Call	`PK_OP_*`	What
`uk_shell_register ()`	`SHELL` 0x110	"I am the shell" → 1; `-KAPI_EBUSY` another running program is. First, before its windows; the pocket port asks it again (with the handler and the keys) of a PocketUI started again, before its windows' replay
`uk_shell_events (fn)`	`EVENTS` 0x111	`fn` gets **`UK_SHELL_EVENT`** (value `UK_SHELL_EV_TASKS`: a window opened, closed, retitled, minimised, the front program changed, home — a

Call	PK_OP_*	What
		signature compared each turn; UK_SHELL_EV_AREA: the work area changed) and **UK_SHELL_KEYEV** (value UK_SHELL_KEY (mods, code): 8 = Super; UK_SHELL_KEY_SUPER a lone Super, UK_SHELL_KEY_HELD the modifiers changed while grabbed)
uk_shell_keys (keys, n)	KEYS 0x112	the system keys taken before the front app (pocketshell: Alt+Tab, Alt+Shift+Tab, Super, Alt+F1, Ctrl+Esc, Super+N, Super+Space)
uk_shell_grab (on)	GRAB 0x118	every key and every change of the modifiers to the shell (an overlay is up: Alt released → switch)
uk_shell_tasks (out, max)	TASKS 0x117	the running programs, the most recently in front first: struct uk_shell_task (its topmost window's id, pid, UK_TASK_FRONT / _CARD / _MINIMISED, its client size, title, the program's name) — none in front: home
uk_shell_thumb (id, dst, w, h)	THUMB 0x113	a window's client area scaled (a box filter, ≤ 512 × 512) by the server into the caller's transfer buffer (KAPI_WS_VA_XFER: el_core_xfer), copied to dst by the port: the switcher's pictures, no app redraws
uk_shell_front (id, f)	FRONT 0x114	f 1: the window's program to the front (from minimised too); 0 and id 0: **home**; 0 and an id: that program behind the others
uk_shell_split (a, b)	SPLIT 0x115	(P8) the programs of windows a and b side by side, a's in front (b 0: a's with the program fronted before it; both 0: the split ended); the shell alone may ask
uk_shell_dim	DIM	not yet (-KAPI_ENOSYS: the overlays draw their own dim)

EVENTS, KEYS, FRONT, GRAB from another program than the shell: -KAPI_EPERM; TASKS and THUMB: anyone (a task manager, a remote desktop). uk_shell_tasks and uk_shell_grab were appended to uikit.abi (848, 849: 850 entries, uikit >= 1.850); shell_ask, shell_running to systemkit.abi (89, 90: systemkit >= 1.91).

The kernel: PublishKeys (kernel/kernel.cpp) gives the graphics server the Super keys (the USB modifiers' bits 3 and 7) as KAPI_WS_MOD_SUPER (8) in KAPI_WS_IN_MODS (no kapi version change); kapi_get_modifiers and the kernel's own state stay Ctrl, Shift, Alt. **The common code** (user/Servers/common/, Elegant unchanged — server_sim compares the pixels): serve.cpp keeps the Super bit from the window manager and calls the policy's new **mods** hook (policy.h, WS_POLICY_HAS_MODS); core.h gains el_core_xfer (a program's transfer buffer) and el_core_next_key (the window manager's reading of a cooked key string, for a policy); CWindow::SetNoInset (a topmost window that is not a band: the work area ignores it). **The Terminal** takes a command as its arguments (typed into its first tab once the prompt came): the search's "run" line.

5.10.5b. The console home (consolehome, phase P9)

user/Apps/consolehome/main.cpp — one source, a FreeType app (FT_APPS), category **Shell** (the onyx package takes it), started by SD:/etc/session/console. It is PocketUI's shell exactly as pocketshell is (uk_shell_register, a **backmost** window = its home, a **topmost alpha** window = its menu's overlay parked off the screen when hidden, uk_shell_keys for Super and F10, uk_shell_grab while the menu is up) and shares pocketshell's catalogue and drawing helpers (Apps/pocketshell/catalog.h: scan_apps, g_cats, recent_*, tasks_read, open_app, draw_icon; look.h: lk_fill, lk_ring, lk_shadow).

- **The home** (`xmb.h`, since 2026-10-09 — Lakka's XMB, docs/COMPACT-SHELL-STUDY.md §17 "v3", the user's choice): the columns (`xmb_build`): one per console that has ROMs (**GameKit**, `roms_read`: `games_systems`, `games_folders`, `games_scan`), then Onyx (the apps of category *Games* but the Game Library), Apps (the categories of `scan_apps` but Recent / Settings / Games / Shell / Emulators; the chosen one's apps `g_sub`, `g_inSub`) and Settings (the console's eight pages, `xset.h`). `draw_home ()`: the background made once per size (`xmb_bg_make`: the gradient, a soft light, the ribbon from an integer sine table — the apps are built `-mgeneral-regs-only`), the white icons drawn into a cached mask (`xi_mask`: generic console shapes with their short names, the gem, the grid, the gear, a folder; `look.h`'s rounded boxes), the lists (`draw_list`: the chosen item at a fixed height), a ROM's title screen (`games_thumb_load`), the hints right-aligned (`hint_w`). `go (key)` is the one function every input ends in (the keyboard's handler, the pad's poll; `home_ptr` the pointer's). A ROM opens with `lx_open` (its emulator by its extension), the screen switched to the emulator's `resolution =` first (`play_rom`). The ROMs are read again when the home comes back, at most every 30 s. Sizes: the mock-up's metrics (`xm ()`), a compact set under 560 logical lines. **The moves are animated** (2026-10-09): `anim_step ()` eases what is drawn towards the chosen column and item (`g_ax`, `g_ay`, `g_asub` in 1/256: `draw_list` places, sizes and fades each item by its fractional distance; a new column's list slides in, `g_slide`); while it moves the home is drawn every 16 ms (`g_anim`).
- **The menu**: `menu_build ()` — *Resume*, the front app's menus (its spec read by `uk_win_menu_get` when the menu comes up; one of them chosen: `g_level`, its items, `uk_win_menu_command (id)` once the overlay is hidden), *Home* (`uk_shell_front (0, 0)`), the other tasks (`uk_shell_front (id, 1)`), *Close* (`uk_win_close`), *Settings*, *Shut Down*.
- **The pad**: `pad_poll ()` reads every pad **without the focus rule** (`pad_any`: `kapi_pad_state` + `pad_map_for` + `pad_apply`) to see the menu's button over an app — Home, or Select + Start —, and moves only at home or in the menu, where PocketUI names it as the program that has the pads (the policy's `focus` hook).
- **The pad as a mouse and keys**: over an app whose `app.txt` category is not *Games* nor *Emulators* (`front_look ()`, after each `tasks_read`), `pad_app ()` points and types for the pad (the server routes them to the app in front): the left stick moves the pointer (`pad_app_read`: the left stick not as the d-pad; `stick_step`: the square of the push past a dead zone, the screen's width in about a second), A / X its buttons, L1 / R1 and the right stick the wheel (`kapi_inject_pointer`); `pad_type ()` the keys (`kapi_inject_key`): arrows, Enter (Start), Esc, Tab, Page Up / Down. The loop polls every 16 ms while the stick is pushed (`g_mMoving`).
- **Sizes**: logical units × the scale (`D ()`, `F (lp)` a face per size), as pocketshell.
- **Tests**: `tools/tests/server_sim/run.sh`'s `conshots` (800 × 480, 1280 × 720, 1920 × 1080, 640 × 480 in French, with `gamelib_samples.py`'s made-up ROMs): the home is the whole screen; a console's games, Apps and its unrolled apps, Settings; F10 shows and Esc hides the menu; over an app with menus: the menu, its File menu, Esc twice — the app still in front. Pictures `consolehome-<home|apps|appsub|settings|menu|appmenu>-*.png` (`CONSOLE_PNG=<folder>`).
- **The tiles** (`tiles.h`, 2026-10-10 — docs/COMPACT-SHELL-STUDY.md §19, the Switch-like v4; the user chose V1 light and V2 dark): `g_style` (`ST_XMB`, `ST_LIGHT`, `ST_DARK`; `SD:/etc/console.ini` `[look]` `style =`, `style_read` / `style_save` in main.cpp, the Display page's *Look* row) and its palette `g_pal` (`Pal`: the background, the words, the accent, the lines... the settings pages' colours too). Over XMB's model (the columns `g_x`, `sel`, `g_inSub` / `g_subSel` for a folder — tile 0 Back —, `sub_read`, `play_rom`, `open_app_at`, `choose_col`), `tiles_draw` draws the two rows (`TMet`: the mock-up's metrics, the bottom rows from the screen's bottom), `draw_tile` a tile by its kind (a ROM's title screen made square — `tile_thumb`, 24 cached —, an app on a plate, a folder with its first four apps, Back,

a settings page), `tiles_ring` the accent ring, `tiles_top` / `tiles_hints` the bars; `tiles_go` the two rows' moves (`g_trow`), `tiles_ptr` the pointer. `draw_home`, `go` and `home_ptr` hand over to them when the style is not XMB. **The settings in the theme** (`draw_settings_sw`): the pages at the left, the rows at the right with lines between them and the ring (`draw_set_rows`, shared with XMB's pages), the dialog and the virtual keyboard in the palette. **The menu as a panel** (`draw_menu_sw`: the same items as `menu_build`, drawn at the right over the dimmed app, the volume / network / time at its foot). Not built yet (§19): a game's options (X), favourites (Y), the last-played order and the All games tile, the ring's pulse.

- **The settings** (`xset.h`, 2026-10-09; the **Games** page — the watched folders, GameKit's `games_folders_save`, a folder browser over `kapi_vol_list` / `kapi_opendir` — added the same night — docs/COMPACT-SHELL-STUDY.md §18): A on an item of the Settings column calls `set_enter (page)`; while `g_setOn`, `draw_home` draws `draw_settings` and every input goes to `set_key` (`pad_poll` gives the pad's buttons that have no key their own codes `K_A`, `K_B`, `K_X`, `K_Y`, `K_START`, `K_L3`; `set_ptr` the pointer's). A **screen stack** (`g_ss`: the page, then deeper screens — a pad's page, the wizard, the keyboard's keys, a network's page, a package list, a package, the games' resolutions); each screen's **rows** are built again for every draw and key (`rows_build`: `R_CHOICE`, `R_SLIDER`, `R_TOGGLE`, `R_ACTION`, `R_SUB`, `R_INFO`, `R_NET`, `R_RADIO`, `R_HEAD`; a label, a value, a help line, an icon, a progress), so they always show the files' and the kernel's state; Left / Right go to the page's `*_change`, A to its `*_act`. **The dialog** (`dlg_open`: two or three choices, the safe one for B, an optional countdown that chooses it — Display's 15 s) and **the virtual keyboard** (`osk_open`: its rows QWERTY / AZERTY / QWERTZ after `input_keymap_kind` of the keymap in use, a symbols page, a secret's dots, `osk_done` per purpose) draw over the page. The logic is the kits': SystemKit (`volume.h`, `locale.h`, `session.h` — `session_switch_start`, `display.h`, `input.h`), NetKit (`wifi.h`; the scan in a thread), `gamepad.h` (`pad_learn_*`, `pad_keyboard_*`, `pad_forget`), `pkg/pkgjob.h` (a `pkg::Manager`; HTTPS: the Makefile links mbedTLS and zlib into consolehome), AudioKit (the test chime). `set_tick ()` each turn: the dialog's countdown, the scan's end, the package job's progress, the mode switch's answer, the wizard (`wiz_tick`: the raw pad, 8 s per step), a redraw when a live value changed. Tests: `conshots`' `console-set-*` (Sound, Gamepad, Wi-Fi, a network, the virtual keyboard, Packages, Mode, Display, the games' resolutions); `tools/tests/gamepad` (the wizard, the keyboard's keys).
- **The tip**: a third window (`W_TIP`, see-through, parked): `front_look ()` shows it for 3 s when another app comes to the front (`tip_show`, `tip_tick`).
- **A size asked while running**: SystemKit's `display_game_ask (w, h)` (a `SHELL_MSG_SCREEN` message to the home; one of `display_modes`' sizes, `0, 0` gives it back) is the front app's size over the two below, forgotten when that app is no longer among the tasks. To be asked BEFORE going full screen (`kapi_screen_set` refuses under a full-screen window): BASIC's runtime does it for `FULLSCREEN` (`consoleScreen` in `Libs/basic/runtime.cpp`: the smallest size that holds the `SCREEN` mode's picture; out and in again when the mode changes).
- **A game's resolution**: an app's `app.txt` may say `resolution = 800x600` (the emulators do), overridden by `SD:/etc/console.ini` `[screen]` `<app> = WxH | system`. `screen_follow ()` (every 0.1 s) puts the screen at the front app's size (`screen_of`, cached: SystemKit's `display_game_size`) with `kapi_screen_set`, and back at the system's when none is wanted; `open_tile` switches before the app starts (`g_launch`, 10 s at most). The system's size is the screen's whenever no app's size is on (`g_ours`): a change made meanwhile (the Display applet) is followed, only a size set by the shell is undone. A refusal (`-2`: a full-screen program has the display) is said once and tried again every 2 s. The kernel's switch is not simulated by `server_sim` (its stand-in has no `screen_set`).

- **The apps' on-screen keyboard** (`consolehome/appkeys.h`, 2026-10-14): PocketUI now gives the console's shell the focused field's type too (`wm.cpp`: `UK_SHELL_EV_TEXT` in pocket AND console mode); `ak_hint` shows the keyboard when a pad is plugged in (`g_padKeys`: not over a game) and hides it unless it was asked for (a Select tap: `pad_poll`'s `selTap`, `ak_toggle`). Its window (`W_KEYS`, opaque, the screen's width, on its bottom edge) is a band: the app laid out above it. The pad goes to `ak_pad` while it is up; the keys are `kapi_inject_key` -- no grab (a grab would send the injected keys back to the shell). An app's `app.txt` `pad_y = f1..f12` gives the pad's Y that key instead of Tab (`front_look`; Jet: `f6`, its address bar), `pad_dpad = pointer` makes the d-pad move the pointer (`pad_app`: 2 px a turn, faster after 0.25 s held) instead of typing the arrows (Jet). `server_sim`: `pad BITS` (a known pad with those `PAD_*` held), `hint T` (the latest other's field), `kapi_inject_key` routed (`run.sh`, *console-keys*).
- **Not done** (the study's §7.5): the quick menu of games (save / load state — the emulators have no common call for it yet; the mock-ups' *Run / Favourites / Save states* level), the pointer held at the top edge, History and Favourites (Lakka's), `PK_OP_TOOLS`. A full-screen game (the kernel's direct path) hides the menu: the emulators are to stay windowed in console (a window fills the screen there).

5.10.6. The adaptive widgets: one binary laid out for every mode (`uikit/adapt.h`, `sidepanel.h`, `form.h`)

(PocketUI's phase P6; docs/POCKETUI-TECH-STUDY.md §6.5–6.14; the reference: docs/11 `adapt.h`, `sidepanel.h`, `form.h`, `toolbar.h`, `tabstrip.h`, `datagrid.h`, `treeview.h`.) An app is **one binary for the desktop, pocket and console**: it runs on the desktop's UIKit (Elegant, `SD:/lib/uikit.so`) and on PocketUI's (`SD:/lib/pocket/uikit.so`, loaded under the same name), the two built from one source with the same table (`abi_same.py`). Instead of laying itself out for one screen, an app **says what its parts are** and each UIKit shows them for its mode. **On the desktop every one of them looks and behaves as it always did** (the size class is *regular*, the metrics today's): an app that adopts them is unchanged there (its screenshots pixel-identical).

What comes without a source change (library code, rebuilt with UIKit): in pocket and console the popup menus (touch-size rows at the pointer in landscape, an **action sheet** at the window's foot in portrait, a deep-blue list in the middle in console), the dialogs (`Modal::run`: the window **dimmed** behind the box; a box taller or wider than the window becomes a **sheet** that scrolls — the wheel nobody took, Page Up / Down, the focused control kept in view —; console: the PS2's panel), the lists' and grids' **rows of the profile**, the **overlay scroll bars** (a thin thumb at the edge, no groove; `uk_scroll_bar`), the **focus ring** (always drawn in pocket and console) and the **arrows moving the focus** to the nearest control in their direction (`uk_focus_move`, for the keys nobody took), **L1 / R1** (Ctrl+Page Up / Down: the window's tabs, else its navigation panel's sections), `TabStrip`'s renderings (the Terminal's tabs adapt by themselves). A window bigger than the screen that will not shrink is shown in PocketUI's **viewport** (§5.10.3).

The size class and the metrics — for an app's own layout code:

```
int sc = uk_size_class ();          // UK_SC_REGULAR (the desktop), _COMPACT (pocket, landsca
pe), _NARROW (portrait), _CONSOLE
const UkMetrics &m = uk_metrics ();  // m.row, m.button, m.field, m.tab, m.toolbar, m.rail, m.
pad, m.touch, m.scale
int gw = uk_scroll_gutter ();       // what a scrolled view leaves for its bar: UK_SBW (10) o
n the desktop, 0 under overlay bars
class MyRoot : public Root {
    void onSizeClass (int sc) override { lay_out (); }   // the class changed (the screen tur
ned): every window told
    void onResized () override { lay_out (); }           // PocketUI fills a resizable window:
```

```
 GUI_EVENT_WINRESIZE
};
```

The class is PocketUI's (`uk_win_server`: by the work area's logical size — portrait when it is taller than wide), asked once and again after a resize; console's whatever the shape. The metrics' profile is `SD:/etc/theme.txt` `metrics = regular | compact | touch` and `scale = 1 | 1.5 | 2` in pocket (compact when not said), console's own in console. `uk_lp (v)` scales a logical size. `uk_logical_units (true)` says the app draws only through UIKit (P10's native scale may be given). Replace a compiled `UK_SBW` by `uk_scroll_gutter ()` where the app subtracts the bar.

SidePanel — a side panel that says whether it is a **navigation** list (left: the places of a library, the mailboxes, the applets) or an **inspector** (right: layers, properties):

	the desktop	pocket, landscape	pocket, portrait	console
navigation	`UK_SP_FULL` (today's sidebar)	a **rail** of icons (48 lp; it expands over the content under the pointer or the focus) — **whole** when the window is at least four times its width, or `setRail (false)`	a **drawer** (a tab at the left edge, or the app's `toggleButton ()`)	a **column** of big rows; L1 / R1 the sections
inspector	`UK_SP_FULL`	a **slide-over** from the right (its edge's tab)	a **bottom sheet** (half; its grabber: whole)	a slide-over

```
SidePanel *sp = new SidePanel (0, 52, 208, h, UK_SP_LEFT, UK_SP_NAVIGATION);
sp->setIconFn (my_icon);                    // the app's icons (else an icon is a WKG_* glyph)
sp->addHeading ("LIBRARY", UK_SPI_FOLDABLE);
sp->addItem (1, "Songs", I_SONGS);  sp->setBadge (1, 12);  sp->setTrailing (2, "42");
sp->setSubtitle (1, "Every song");          // a help line: shown whole, 240 px wide or more,
when every item fits at two lines
sp->onSelect = go;  sp->onItemMenu = menu;  sp->onDrop = dropped;  sp->onPresentation = relay
out;
root.addChild (sp);
void relayout () {                          // the window's resize, onSizeClass, onPresentatio
n
    sp->place (0, 52, 208, height - 52);    // the panel's rectangle as the app means it (NOT
left / top / resizeTo)
    content->left = sp->reservedWidth ();  // the width it takes now: whole 208, a rail 48, a
n overlay 0
}
```

Items carry a small fixed set of accessories (icon, indent, badge — a count or a dot —, a trailing value, a picture, a toggle such as a layer's eye); anything richer goes in a **header** (`setHeader`, uikit 1.950: over the items -- whose books these are, a period's chooser; hidden in a rail of icons, shown when it expands), a **footer** (`setFooter`), **pages** (`addPage`: an inspector's tabs) or a **content** under the items (`setContent`: a tree, a month — such a panel becomes a drawer, never a rail). `place ()` is the API for the app's own geometry: writes to `left` / `top` / `resizeTo` are taken as what the parent's anchors did to the panel shown. Pilots: the **Control Panel** (its applets as links, on the desktop too — docs/04), the **File Viewer** (its places in pocket and console; the desktop keeps its own sidebar).

ToolBar — the priorities of its tools; the desktop as built (every tool, every row), pocket **one row** of what fits by priority and a **»** button opening a panel that hosts the other tools themselves (a toggle stays a toggle, a drop-down a drop-down; a group a row):

```
bar->setPriority (save, UK_TB_ALWAYS);          // the default
bar->setPriority (cut, UK_TB_IF_ROOM, 2);       // shown when there is room; rank 2 leaves be
```

```
fore rank 1
bar->setPriority (gear, UK_TB_OVERFLOW);        // only behind »
second->foldInto (bar);                         // a second row joins the first when compact
(and is hidden)
int rowsNow = bar->rows ();                     // the app's layout: the desktop 1 + the fold
ed bars, else 1
```

TabStrip — `presentation ()`: the desktop's strip; pocket landscape a strip that **scrolls** (each tab its title's width) with a ⋯ list when they do not fit; portrait **the current tab's title (n)** ▾ opening the list (its marks, close crosses, "New Tab"), or a **segmented control** for 2–3 short fixed tabs; console a **header row** (the current tab big, L1 / R1). `setEdge (UK_TAB_BOTTOM)` (a spreadsheet's sheets), `setStyle`, `setNav` (this strip takes L1 / R1; the first one made does), `showList ()`.

DataGrid — the columns' **roles** and **priorities**: `setColumnRole (c, UK_COL_PRIMARY | _SECONDARY | _DETAIL, priority)`; in pocket the highest priorities give way when the grid is narrow (0: never); in portrait a grid with a primary column draws **two-line cards** (the primary, then the secondary ones) and `setDetailView (w)` opens the app's detail on a click (Esc back); `setMultiSelect`, `isSelected`, `setSelected`, `selectedCount`. Pilot: the **Task Manager** (a task a card in portrait). *Beware: DataGrid has fields of its own named `left` and `top` (its scroll) — place it with `grid->Widget::left` / `Widget::top`, or anchors.* **TreeView** `setDrillDown (true)`: in portrait one level at a time, "‹ parent" at the top.

FormDialog — a dialog of rows the UIKit lays out: two columns in a box on the desktop and in landscape (the buttons at the bottom right, the default then Cancel), **a sheet the window's size** in portrait (each label over its control, Cancel at the title's left, the default at its right), the PS2's panel in console (✕ / ○ beside the buttons):

```
FormDialog f (TR ("New Playlist"));
Textbox *name = new Textbox (0, 0, 240, 26, "");
f.addSection (TR ("Playlist"));  f.addRow (TR ("Name:"), name);  f.addText (TR ("A smart play
list fills itself."));
f.addButton (TR ("Create"), UK_FB_DEFAULT, 1);  f.addButton (TR ("Cancel"), UK_FB_CANCEL, 0);
if (f.run () == 1) ...                  // Enter: the default's result, Esc: the Cancel's
```

A field's type — `uk_set_input_type (w, UK_IN_NUMBER | _DECIMAL | _URL | _EMAIL | _PASSWORD | _TERMINAL | _SEARCH)` (a `Textbox` with `password` is `UK_IN_PASSWORD` by itself): told to PocketUI with the focus (the on-screen keyboard's layout in pocket, P10; console never raises one).

The state behind `Widget::ext` — the classes' fields are frozen (`uikit/abi.h`): what a widget gained at P6 is kept in an extension (`uikit/internal/adapt_int.h`, `namespace uikit::internal`, left out of the table) freed with the widget. `ToolBar` gained a `layout ()` override: a program built before P6 keeps its bar as built until rebuilt. `Root::onSizeClass` is the former reserved slot `uk_rootReserved2` (the name kept as a plain entry of the table). P6 appended 99 entries to `uikit.abi` (850–948: `uikit >= 1.949`), the same both UIKits (P7: 949, `SidePanel::setHeader` -- `uikit >= 1.950`); the layout lock gained `SidePanel`, `FormDialog`, `UkMetrics`.

On the PC: `sh tools/tests/server_sim/adaptive.sh [out] [control gallery pilots viewport fileviewer]` — the widgets' host tests (`tools/tests/server_sim/adaptive.cpp` with `ADAPT_TEST=1`: each widget's layout in the four classes, forced) and the real apps under PocketUI at 800 × 480, 1280 × 720, 1920 × 1080, portrait 480 × 800 and console 640 × 480, their pictures dumped: the Control Panel (and in French), the gallery (a toolbar and its », tabs, both panels, a grid with roles, a form, a menu, in every mode, and on Elegant), the Task Manager, the Terminal's tabs, the viewport (Setup), the File Viewer. A selection is in `screenshots/pocket/`. In the desktop simulator (`tools/tests/desktop_sim`) the class is the desktop's: `shots.sh` checks that nothing changed there.

5.10.7. The flat look, a handheld's buttons, the Menu entry (`uikit/theme.h`, `padhint.h`, `toolbar.h`; the branch `PocketTerm`)

What an app needs to know of PocketUI on a small screen without a pointer (`docs/COMPACT-SHELL-STUDY.md` section 20):

- **The flat look.** The theme *Onyx Noir* sets `UK_FLAT` (`uk_flat ()`): UIKit's painter then draws plain faces -- an app that uses the widgets and `uk_raised` / `uk_sunken` / `uk_popup` / `uk_hilite` / `uk_rbox` has nothing to do. An app that paints chrome in colours of its own asks `uk_flat ()` and takes the tokens `UK_N_BG0..3`, `UK_N_LINE`, `UK_N_TEXT1..3`, `UK_N_ACCENT`, `UK_N_ACCENT_HI`, `UK_N_DANGER / SUCCESS / WARNING` (or simply the palette: `C_BG`, `C_FIELD`, `C_BUTTON`, `C_BORDER`, `C_TEXT`, `C_DIS`, `C_ACCENT`). A document's page, a picture, a terminal keep their own colours. The pocket mode's theme is `SD:/etc/pocketui.ini [look] theme =`.

- **The menus.** Nothing changes for the app: `uikit::Menu` as ever. In pocket the menu bar shows them as ONE `Menu` entry (the CMD key, F10). Every command a toolbar offers should also be a menu item: the menu is what the keys always reach.

- **A toolbar that adapts** (`uikit/toolbar.h`): `ToolBar::add` any widget, `setPriority (w, UK_TB_IF_ROOM, rank)` for what may leave the row (the higher the rank, the sooner), `setFlex (w, minW, foldW)` for a list or a value box (it shrinks to `minW`, then folds to `foldW`: draw an icon when your width is that small), `foldInto (first)` for a second row, `rows ()` to lay the window out. On the desktop the bar stays as built.

- **The keys a handheld's buttons type** (so that an app is driven without a pointer): the arrows, Enter (A), Esc (B), Delete (X), Shift+F10 (Y: UIKit clicks the focused control with the right button -- a context menu), Ctrl+Page Up / Down (L / R: UIKit steps the window's tabs or sections). Start and Select type nothing (they are the games'); the menu is the CMD key (Super tapped) or F10, Home is CMD held. An app must not need another key for its main path. `uikit/padhint.h` draws the buttons (`uk_pad_glyph`, `uk_pad_hints`) for a line of hints.

- **PocketUI's protocol**: the menu bar's program may call `uk_shell_events` and `uk_shell_grab` (they were the shell's only): it is sent `UK_SHELL_KEY_MENU` for the menu key and, while it grabs, every key; `UK_SHELL_KEY_CANCEL` when a key of the shell's closes its menu.

- **Without a mouse on a 640 x 480 screen (2026-10-11).** What UIKit gained for it, pocket and console only (the desktop's pictures compared before / after: the same): (1) **a drawer by the keys** -- a SidePanel that has no place to step (free content: `setContent`, an inspector's pages) and is shown as a drawer, a slide-over or a sheet is opened and closed by Prev / Next (L / R, Ctrl + Page Up / Down): **L the left panel, R the right one**, a window's only panel either (`sp_step`; `nav_step` now asks every owner in turn, then once more with `dir * 2`: "nobody took it"); opened by the keys it gives them to its content (its page) and, closed, back to the widget that had them; a closed overlay takes no key (it ate the arrows); the arrows scroll the items to the one they are on. (2) **The keys' pointer** (`uikit/adapt.h`): `UkKeyPointer`, `uk_keys_drive ()`, `uk_keypointer_move (p, key, step, x0, y0, x1, y1)`, `uk_keypointer_draw (canvas, p)` -- for a view worked with the pointer (Circuits' board: the arrows move it a cell at a time, Enter is the button; the view gives itself the place through its own `onMouse`). (3) **`ToolBar::setShown (w, shown)`**: a tool the app takes off the bar for now -- the desktop hides it as the app did; compact, the row is laid out without it (setting `hidden` by hand did nothing there: the bar's layout shows what it places -- Circuits' palette showed the gates a level does not allow). Four entries appended to `uikit.abi`. (4) **The**

hint line (`uikit/padhint.h`, pocket and console; nothing on the desktop): `int uk_hint_line (Root *, const UkPadHint *hints, int n, const char *mode = 0)` -- a 30 px line UIKit keeps at the window's foot (a widget of its own, above an open drawer's foot: the drawer ends over it) with the app's hints, copied, and a mode's name as a pill; it returns the height the app's layout leaves (30; 0 on the desktop and when n = 0); `uk_hint_line_height (root)` says it later. The app calls it each time its state may have changed (its `onTick` is fine: nothing is redrawn when the hints are the same) and lays itself out again when the height it returns changes. `uk_hint_line (root, 0, UK_HINTS_STANDARD)`: "F6 tools, Alt + D-pad scroll, CMD Menu" in UIKit's own words (Letters). (5) **`uk_segmented_nav (SegmentedControl *)`** (`uikit/segmented.h`): a segmented control that switches the window's views is its tabs -- Prev / Next step it, round, as a TabStrip. (6) **The drawer's key first**: in pocket and console `nav_step` asks the LEFT drawer before the tabs (`dir * 3`): L opens a left drawer that has no place to step, everywhere; R then steps the tabs, round. A window of tabs without such a drawer is as before (L / R step its tabs back and forth, an inspector at its right or not). (7) A pointer "exists" (the closed drawer's tab comes back) only after a real pointer event: a window resized, the screen's size and a title button come through `Root::ptrEvent` too and used to count (GPIO Lab showed its tab at once). Three entries appended to `uikit.abi` (`uk_hint_line`, `uk_hint_line_height`, `uk_segmented_nav`). (8) **A canvas of objects by the keys** (`uikit/objnav.h`, written once for Slides and QBStudio's forms): the view keeps a `UkObjNav` (a plain structure) and gives its keys, with its objects' rectangles in its own units, to `uk_objnav_key (nav, key, rects, n, bounds)` -- the D-pad chooses the nearest object in that direction, M / S begin the move / the size mode, in which the D-pad nudges `nav.now` by `nav.step` (Shift: `nav.fine`), Enter keeps, Esc cancels; the result says what the view does (`UK_OBJ_PICKED`, `_BEGAN`: open an undo step, `_CHANGED`: the object takes `nav.now`, `_KEPT`, `_CANCELLED`: it takes `nav.from` back, `_USED`, `_NO`: the view's own key). `uk_objnav_begin` puts a new object in the move mode; `uk_objnav_draw` draws the frame, the handles, the outline of where it was, a pill with its place and the bounds' centre lines; `uk_objnav_hints` gives the hint line its hints and its pill (MOVE, SIZE). Used where `uk_keys_drive ()`; five entries appended to `uikit.abi`. (9) **`uk_fit_pane (pane, layoutW, layoutH, groups, n)`** (`uikit/adapt.h`): a fixed layout brought to a smaller pane -- the children of the pane (and of the GroupBoxes named) scaled in place and size, the texts as they are; nothing happens when the pane is the layout's size at least (the desktop). The Control Panel gives its applets the pane as it is in pocket under 800 px (`surface_fit`), and Theme, Mode, Preload, App Settings, Panel and the Package Manager's own window call it. **The pattern for an app whose left column is a list of places it draws itself** (Circuits, Turtle Quest, GPIO Lab): when `uk_mode () == UK_MODE_POCKET` and the screen (`kapi_screen_size`) is under 800 px wide, the column's widget is made the content of a navigation SidePanel instead of a child of the Root (decided once, in `main`: the mode never changes under a program), the panel added LAST, `place ()`d in the layout, the rest laid out from `reservedWidth ()` (0: a drawer) plus 24 px for its tab; the content's `onKey` does Up / Down / Enter, and a mouse choice closes the drawer. Mail's three screens (`g_three`, `layout_three`) are the other pattern: one SidePanel whole across the window (`setRail (false)`), then the list, then the message, a bar with "<" over each. Tests: `sh tools/tests/server_sim/pocket640.sh [out] [mail circuits gpiolab turtle audit]` (keys only, pictures compared by `pngcmp.py`); the audit of every app at that size: `docs/compact-shell/real/pocketterm/AUDIT-640.md`. **An app with a hand-made toolbar** (a class of its own with `add` / `sep`, as Letters had and the Spreadsheet and Slides had until this work: `user/Apps/sheet/ui_base.h`) takes UIKit's instead with three lines -- `#define ToolButton UkToolButton_`, `#include "uikit/toolbar.h"`, `#undef ToolButton`, then `using uikit::ToolBar;` in its namespace: the desktop's rows are the same pixels (checked), pocket gets "»" and F6. Cardfile's is the next one.

- **A closed drawer without a pointer (the designer's Q3).** In pocket and console a closed drawer, slide-over or sheet draws NO tab until a pointer exists -- `internal::pointer_seen ()`, set by the first pointer event that reaches a Root (`Root::ptrEvent`), which lays the adaptive widgets out again. The shell's band names the drawers instead: `SidePanel::setName ("Levels")` (else *Panel* / *Inspector*); SidePanel sends `D<L|R><0|1>\t<name>` lines before the app's menus in the spec given to `uk_win_menu_set` (`uikit/menu.cpp`: `internal::menu_extra`), and the menu bar (`user/Apps/menubar`: `g_drawer`) draws `[L] name` after Menu and `name [R]` before the status icons, lit while open. An app writes one line (`setName`) and leaves no gutter for a tab.

5.10.8. InputKit and UIKit's navigation: a pad and the keys as actions (`inputkit/inputkit.h`, `uikit/adapt.h`; the branch `PocketTerm`)

InputKit (`user/Kits/inputkit/`, `SD:/lib/inputkit.so`; docs/06 §14, `docs/21-INPUTKIT.md`) turns the keys and the gamepads into **actions** bound in `SD:/etc/input.ini`. UIKit links its import stubs (as SystemKit's) and takes all its navigation from it — in every `Root`, with no code in the app:

- **the pad** (`adapt.cpp` `pad_turn`, each `Root::step`): InputKit's events are acted on as the keys they stand for — the directions are the arrows, Accept is Enter, Back is Esc, Alt1 is Delete — through `Root::handleKey`, so every control answers a pad as it answers the keyboard; what no control takes moves the focus (pocket, console);
- **Prev / Next** (L / R; Ctrl+Page Up / Down, Ctrl+Tab / Ctrl+Shift+Tab): the window's tabs — the registered owner of the highest priority: a `TabStrip`, else a navigation `SidePanel`'s sections. In pocket and console `Root::keyEvent` tries them **before** the focused control (`key_first`), so they work while a view that takes every key has the focus;
- **Tab** that no control took (pocket, console; `Widget::handleKey` at the root, `internal::tab_steps`): the window's tabs stepped, Shift+Tab back -- before it would move the focus, which it still does in a window without tabs; Ctrl / Alt with it: not this (2026-10-26: the PocketTerm35's L and R type letters, its Tab steps the tabs);
- **Alt2** (Y; Shift+F10): a right click at the focused control's middle — its context menu;
- the desktop with a mouse and a keyboard is unchanged; a pad there acts on the focused control the same way.

Only the program that **has the keys** gets a pad's events (the kernel's `kapi_pad_state` gives a pad's buttons to it alone; under PocketUI the shell at home and under its overlays, the menu bar while its Menu is open). A program whose pad is its own is left alone: one that reads InputKit's **state** (`in_state`: `in_claimed`), and — until the emulators and games move to it — one whose `app.txt` has the category Games or Emulators, or `pad = own` (`in_app_owns_pad`).

A program that runs **its own loop** (`pump_events`, no `Root::run` / `step` — the Terminal) calls `uk_input_turn (&root)` each turn for the pad, and one that hands its keys to a view itself calls `uk_nav_key (root, key)` first for Prev / Next. A program that is not on a `Root` (the pocket shell, the menu bar) reads InputKit itself: `in_poll`, `in_key_action`.

6. Writing a graphical application

Notifications and clipboard (ABI v40). Notifications are AppKit's (`appkit/appkit.h`; it was `notify.h`): `notify ("My App", "Done.")` shows a bubble (the `notifyd` service, reached by IPC; launched on demand); `notify_action (title, text, "app args")`: a click on it runs that app with those arguments (`pkgd`: `"control pkgman"`), and it stays longer. `#include "uikit/clipboard.h"` (UIKit's; it was `user/clipboard.h`): `clip_set_text`, `clip_get_text`,

`clip_set_files (paths, cut)`, `clip_get_file`, `clip_clear`, `clip_set_image (px, w, h)`, `clip_get_image (&w, &h)`, and for several formats of one copy `clip_put (fmts, datas, lens, n)` / `clip_get (fmts, nf, got, cap, &data, &len)` (formats: `text`, `rtf`, `image`, `files`, `files-cut`, `url`, `x-<app>`; Letters would give `rtf` + `text`). **The shared clipboard** (`docs/clipboard/README.md`): the service `clipd` keeps the last 10 copies in its memory, a cursor on the one Ctrl+V pastes (the cursor's item in a format the app takes, else the newest that has one); every copy shows a notification; the dock's clipboard button opens its widget (`apps/clipboard`). The protocol (`clipproto.h`): mailboxes for the messages, files of `RAM:/clip` for the bytes (an app never reads its own mailbox); the kernel's v40 clipboard is kept as the fallback. Test: `sh tools/tests/run_clipboard_test.sh` (clipd and an app as threads over the simulator's in-process mailboxes, `SIM_IPC=1`). **Full-screen apps (ABI v41)**: `unsigned *fb = uk_win_fullscreen_begin (&w, &h);` gives a screen-sized buffer; draw into it and call `kapi_present_fb ()` once per frame (it also yields). The desktop is not drawn meanwhile and all input comes to your key/pointer handlers in screen coordinates. `uk_win_fullscreen_end ()` (or exiting) restores the desktop. See `user/Apps/plasma`. **v55**: `unsigned *scr = kapi_fullscreen_direct (&w, &h, &stride);` (after `fullscreen_begin`) gives the **displayed framebuffer itself** (uncached; `stride` in pixels): draw there — or render there with `kapi_gpu_render` — and nothing is copied any more (`kapi_present_fb` only yields); tearing is possible. 0: keep the back buffer. Used by `n64emu` in full screen.

Files: `filekit/fsutil.h` (FileKit's — link `lib/filekit.imp.a`; it was `user/fsutil.h`) (`fs_join`, `fs_exists`, `fs_is_dir`, `fs_copy_tree`, `fs_remove_tree`, `fs_unique_name`) and `trash.h` (`trash_move`, `trash_restore`, `trash_purge`, `trash_empty`, `trash_count` — layout `SD:/.Trash/files` + `info/*.trashinfo` holding `Path=<original>`). Your own IPC service: `kapi_ipc_register ("name")`, clients `kapi_ipc_lookup ("name")` + `kapi_mailbox_send (pid, type, data, len)` (≤ 512 bytes); the service drains with `kapi_mailbox_recv`.

Drag & drop (ABI v42). A **source** calls `uk_win_drag_begin (DND_FILES, paths, len, label)` while the left button is held (from `onMouse`, once the cursor moved a few pixels from the press); `paths` is `\n`-separated. A **target** overrides the `uikit::Root` virtuals: `onDrop (x, y, type, data, len, flags)` (data NUL-terminated; `flags & DND_F_COPY` = Ctrl held), `onDragOver (x, y, leave, flags)` (highlight the drop spot), and the source gets `onDragDone (targetPid, flags)` (`DND_F_DESKTOP` = dropped on the desktop, `DND_F_CANCEL` = Esc). `kapi_get_modifiers ()` returns `MOD_CTRL` / `MOD_SHIFT` / `MOD_ALT`. Examples: `fileviewer` (source + target) and the document apps.

Several windows (kapi v94). A program may have up to 16 windows besides its first (Elegant holds 64 in all). In UIKit, one more window is a `Root` made with the tag `NewWindow`: `new MyWin (...)` whose constructor is `Root (NewWindow (), x, y, w, h, title)` (x, y negative: placed by Elegant); `winOpened ()` says whether it could be made (no graphics server's windows left, no memory: fall back to one window — on the PC's simulator too, unless it is the multi-window one, below). The first `Root`'s `run ()` serves every window: their events (each comes with its window's number as `sender`; UIKit routes it to that `Root`, which becomes `Root::current ()` — the dialogs open in it), their `onTick`, their drawing (`Root::paintAll`). Its **close box** calls the virtual `onClose ()` — by default `closeWindow ()`, the window gone (the object stays: delete it later, not from inside its own handler); closing the **first** window still ends the program. The menu bar shows the first window's menu for all of them. The window calls (`uk_win_present`, `uk_win_resize2`, `uk_win_chrome`, `uk_win_cursor`...) act on the window `uk_win_select (n)` chose: a `Root` selects itself (`winSelect ()`) before it acts; an app that calls them directly selects the window it means. Without UIKit: `uk_win_new (x, y, w, h, title, flags, &canvas)` → the

number, `uk_win_destroy (n)`; its close box sends `GUI_EVENT_WINCTL` with `KAPI_FRAME_CLOSE` to its pointer handler. **The simulator** (`fakekapi.cpp`) has the windows too: the script's `win N` sends the next events (and the dumps) to window N, `winclose` is its close box. The first app with several: Telegram (one window a conversation). The design: `docs/MULTI-WINDOW-STUDY.md`.

The status area (kapi v95). A program may put **one icon** in the menu bar's status area: `uk_tray ("SD:/apps/<app>.app/icon.bmp", "tip")` (UIKit, `uikit/root.h`: any picture ImageKit reads, brought to `KAPI_TRAY_PX` = 20 pixels square, the icons' magenta see-through) → true; call it again to change the tip (Telegram: its unread messages), `uk_tray_clear ()` to take it away (the program's end does it). A **double click** on it makes Elegant show the program's **first window** again (back from minimised, its workspace shown) and calls the first `Root`'s virtual `onTray (KAPI_TRAY_OPEN)`; a right click `onTray (KAPI_TRAY_MENU)` (a menu of its own: a `PopupMenu` in the window). The icons are Elegant's (`EL_OP_TRAY_*`), kept for an Elegant started again by AppKit; the menu bar reads them (`uk_win_tray_list`, `uk_win_tray_icon`) and reports the clicks (`uk_win_tray_activate`). Without UIKit: `uk_win_tray_set (pixels 0xTTRRGGBB, tip, handler)` — the handler gets `GUI_EVENT_TRAY`. On the PC's simulator: `SIM_TRAY="tip"` gives the menu bar an icon, `SIM_TRAYDUMP=file.elsm` writes the program's.

File associations: `#include "fileassoc.h"` — `fa_open (path)` opens a path like a double-click (folder → File Viewer, `.app` / ELF → run, else the app `SD:/etc/fileassoc.ini` maps its extension to, as `SD:apps/<app>.app/main <path>`); `fa_app_for (path, app, cap)` only looks it up. **Unsaved changes**: `#include "docguard.h"` — keep `doc_hash (data, len)` of the document as loaded / saved, and before replacing it call `doc_confirm (name, changed, save_fn)` (a Yes / No / Cancel `MB_YESNOCANCEL` box; false = cancelled). A document app should accept a path argument (`kapi_get_args`) and a dropped file (`onDrop`), so it works with the File Viewer and `fileassoc.ini`. **Images**: `#include "img/imgload.hpp"` (in one TU) — `img_load (path, &frames)` decodes BMP / GIF (all frames + delays) / PNG / JPEG (stb_image, public domain), WebP (simplewebp, BSD-3) and PCX (our decoder) into `0xAARRGGBB` frames from the app's umm heap (`new unsigned[]` frames); `img_free (&frames)`; `img_is_image_name (name)`. The codecs are compiled once into `libuikit.a` (`uikit/imgload.cpp`, with FP/SIMD like `uikit/canvas.o`) and linked only into the apps that call them. Users: `imageview`, `fileviewer` (preview), `uikit::ImageBox`. `img_load_mem (data, len, &frames)` decodes a file's bytes already in memory (Letters' RTF pictures, Paint's OpenRaster layers); `img_inflate (data, len, zlib, &n)` inflates a deflate stream (stb's: a ZIP entry, a zlib stream). **Writing images** (`user/Kits/imagekit/img/pngsave.hpp`, header-only, integer only — freestanding apps use it): `pngsave::deflate` (LZ77 over 32 KB with hash chains, the fixed Huffman codes; zlib's wrapper or raw), `png_encode (px, w, h, alpha)` (RGBA / RGB, each row's best filter), `jpeg_encode (px, w, h, quality)` (baseline 4:2:0, the standard tables, an integer DCT), `gif_encode` (GIF89a: the exact colours up to 256, else a median cut; the clear pixels one transparent index; LZW), `bmp_encode` (24-bit), `on_white` (a pixel laid on white); `ZipOut` (`add` stored or deflated, `finish`) and `zip_find` (an entry's bytes and method). Paint's exports and its OpenRaster files use them. **TrueType text** (`user/Kits/fontkit/`): the apps' FreeType — the upstream sources of `third_party/freetype-2.14.3` built lean by `user/Makefile` into `ft/libft.a` (`ft/onyx_ftoption.h`, `ft/onyx_ftmodule.h`: TrueType fonts only — truetype + sfnt —, anti-aliased — smooth —, hinted by the auto-hinter only — autofit, no bytecode interpreter —, their kerning read — GPOS too —; no compressed, web, bitmap, colour or variable fonts, no PostScript names). FreeType wants a C library: an app using it is a **newlib** app (§5.1; Letters' rule in `user/Makefile` is the model). `#include "fontkit/fonts.h"` (header-only, one TU): `fnt::init ()` finds the families of `SD:/res/fonts` and `SD:/fonts` (each file's family name and

style read from its own `name` / `head` / `OS/2` tables — no FreeType, three small reads), sorted; `fnt::count / name / find / styles`; `fnt::get (family, fnt::BOLD | fnt::ITALIC, size in 1/64 px)` a sized font (a style the family lacks is made: thickened, slanted) with its `ascent`, `descent`, `height`, underline place and thickness; `fnt::advance (f, cp)` — the design's advance, 1/64 px: exact at any size, so a line breaks at the same word at every zoom —, `fnt::kern`, `fnt::draw (cv, f, x64, baseline, cp, colour, clip)` — the glyph rendered once per quarter-pixel position, cached, blended with a slight gamma —, `draw_str / str_w` for Latin-1 labels. A character a font lacks comes from DejaVu Sans. `fnt::trim ()` drops the least used sizes when the cache is big — only at a moment no `Font *` is held (after a redraw); `fnt::g_onTrim` lets the app forget its own. The screenshots' build compiles the same sources for the PC (`shots.sh`). **Vector shapes** (`uikit/vpaint.h`, in `libuikit.a`, integer only): a `VPath` gathers outlines — `poly`, `rect`, `rrect`, `circle`, `ellipse`, `hole` (a disc cut out), strokes `line`, `polyline`, `arc` (round ends and joins), `arrowHead` — in 1/16 px (`V (px)`), then `fill (cv, colour, alpha)` paints their union (the non-zero rule: every outline turned the same way, a hole the other way), anti-aliased (four sub-rows a pixel, the spans' ends to 1/16 px). Letters' toolbar icons are drawn with it (`user/Apps/letters/icons.h`). `uk_sin / uk_cos (degrees)` × 16384. **File-system providers (ABI v44)**: an app can serve a whole path prefix to every other app — `kapi_vfs_register ("XYZ:")`, then loop on `kapi_vfs_next (&req, 1)` and answer each request (`req.op` = `VFS_OP_OPEN` / `READ` / `CLOSE` / `LIST` / `SAVE` / `MKDIR` / `REMOVE` / `RENAME`, see `user/Kits/appkit/appkit.h`) with `kapi_vfs_reply (req.id, status, data, len)`; a SAVE's payload is read with `kapi_vfs_req_data`. Example: `user/BinUtils/ftpfs.cpp` (FTP / FTPS). The ordinary file kapis then work on `XYZ:...` paths in every app, unchanged. `ftpfs.h`: `ftpfs_login` / `ftpfs_login_site` (hand a login to ftpfs, optionally remembered), `ftpfs_forget`, `ftpfs_load_sites` (the remembered servers of `SD:/etc/ftpfs.ini`). **Sound (ABI v46; a mixer since v85)**: `kapi_sound_acquire ()` first (1 = your program has a **channel** of the mixer — several programs play together, each with its own volume; 0 = the 8 channels are taken) — the channel is freed by `kapi_sound_release ()` or when your process ends. `kapi_sound_clients (list, max)` lists the channels (`struct kapi_sound_client`: pid, name, volume, mute, level) and `kapi_sound_client_volume (pid, 0..100, mute)` sets one (`volume.h`'s `mixer_set` also writes `SD:/etc/mixer.ini`). Then `kapi_sound_write (frames, n)` streams PCM (s16 L/R at `SOUND_RATE` 44100 Hz; non-blocking, returns the frames taken — loop with a short sleep while it returns 0) for audio / MIDI players. **A program that only wants to make a sound does not do this itself: AudioKit (§5.7) does** — `ak_fm_start (voice 0..15, milliHz, SOUND_SQUARE / SINE / TRIANGLE / SAW / NOISE, volume 0..255)` plays a note until `ak_fm_stop (voice)` (-1 = all), `ak_play (file)` a file, `ak_out_*` your own frames. (The kernel had 16 voices of its own, `kapi_sound_start` / `_stop`, until 2026-10-05: they answer −1 now.) Example: `user/BinUtils/tone.cpp`. **Low-latency sound (ABI v68)**: the output normally lags ~116 ms (1024-frame chunks, 4 rendered ahead). The owner may ask for less: `kapi_sound_config (chunk_frames, ahead)` (64..1024 frames, 1..4 chunks; 0 = the default) → the latency in frames, (ahead + 1) × chunk — 256 × 2 = 768 frames ≈ 17 ms, 128 × 2 ≈ 9 ms (smaller means more DMA interrupts and no slack for a late core 1: try 256 × 2 first). Then keep little queued: with `kapi_sound_write`, write only while `kapi_sound_status`'s free frames show less than a chunk or two waiting. **The mapped ring**: `struct kapi_sound_ring *r = kapi_sound_map ();` (0 if you are not the owner) — 8192 frames in shared memory that the kernel mixes (with the stream) until you release the output. `kapi_sound_ring_write (r, frames, n)` → frames taken (`kapi_sound_ring_free (r)`: the room). It makes no kapi call, so **code on an app core** (`kapi_core_run`) can fill it: the audio needs no pump thread on core 0. `r->dry` counts the underruns; `r->rd` moves as the kernel plays — a thread can sleep on it with `kapi_wait_word`

`(&r->rd, old, ms)` (§5.2). All of this is undone by `kapi_sound_release` or the process's end. Example: `user/BinUtils/ringtest.c` (a tone from an app core at 256 × 2). **FM instruments** (AudioKit; the kernel's `kapi_sound_instrument`, ABI v47, is retired): fill a `struct kapi_fm_instrument` (2 operators, OPL2-style parameters, see `kern/kapi_abi.h`), `ak_fm_instrument (voice, &ins)`, then `ak_fm_start (voice, milliHz, SOUND_FM, volume)` / `ak_fm_stop (voice)`. The FM Song formats (.FMS / .FMI) and their conversion are in `user/Apps/fmtracker/fms.h` (portable; host tests: `sh tools/tests/run_fms_test.sh`). **Held keys (ABI v48)**: key events only report presses; an action game polls `kapi_key_held (KEY_LEFT)` (also `KEY_RIGHT/UP/DOWN`, `KEY_ENTER`, 27, `' '`, `'a'..'z'` = the US position of the key, `'0'..'9'`; since 2026-10-26 `KEY_BACKSPACE`, `KEY_TAB`, `KEY_HOME/END/PGUP/PGDN/DEL`, `KEY_F1`..`KEY_F12`, and **`KEY_PRTSC`, `KEY_PAUSE`** -- held keys only, they make no key event) every frame — 1 while held and your window has the keyboard (USB keyboard and VNC). Returns 0 on an older kernel. `gamepad.h`'s `[keyboard]` names them `prtsc`, `pause`, `f1`..`f12`, `backspace`, `tab`... (the Gamepad applet maps them). **USB gamepads (ABI v50)**: `#include "gamepad.h"` — `pad_buttons (-1)` = the `PAD_*` buttons held on every pad (or `0..3` for one): `PAD_UP/DOWN/LEFT/RIGHT`, `PAD_A` (bottom face button), `PAD_B` (right), `PAD_X` (left), `PAD_Y` (top), `PAD_L/R/L2/R2`, `PAD_SELECT/START`, `PAD_L3/R3`, `PAD_HOME`; `pad_read (i, &in)` adds the sticks (`in.lx/ly/rx/ry`, -1000..1000), the USB ids and the raw bits. Nothing is pressed while your window has not the keyboard. The pad's raw state (`kapi_pad_state`) goes through **`SD:/etc/gamepad.ini`** (a `[vvvv:pppp]` section per pad model, `[default]` for the other generic pads; analog triggers as `l2_axis` / `r2_axis`; written by the Gamepad app, `padconf`) or the built-in mapping (pads Circle knows). Used by gbemu, gamelib, BASIC (`PAD`, `STICK`, `STRIG`). **The keyboard as pad 0** (2026-10-09): a program that calls **`pad_keyboard (1)`** (the emulators) gets the keys of `[keyboard]` (a key for each `PAD_*` button and the right stick: `rs_up`...; by default by place -- arrows, Z bottom, X right, A left, S top, Q / W, E / R, Backspace Select, Enter Start, I J K L the right stick) added to pad 0, and pad 0 is there with no pad plugged in (`pad_keys` alone: the keyboard's state; `pad_key_code` / `pad_key_word` the words). Off by default: an app that reads the keys itself would see each twice. The Gamepad applet's **Keyboard** tab writes the section. Host test: `tools/tests/run_gamepad_test.sh`. Host test: `sh tools/tests/run_gamepad_test.sh`. **USB MIDI input (ABI v68)**: class-compliant USB MIDI devices (keyboards, interfaces) are found when plugged in, at boot or later. `kapi_midi_read (ev, max)` takes up to `max` queued `struct kapi_midi_event` (oldest first; never waits) → how many: `time_us` (arrival, the kernel's µs clock — `kapi_clock_us ()` reads the same clock, also on an app core), `cable`, `status`, `data1`, `data2` (`length` of them valid: 1..3; a SysEx arrives in 3-byte pieces), `device` (its `umidiN`). One queue for the whole system (256 events; the newest are dropped when nobody reads): one reader at a time. `kapi_midi_devices ()` → attached now. Poll it from your pump or a thread (every 1–2 ms for live playing). Example: `user/BinUtils/miditest.c`. **The GPU (ABI v52)**: `kapi_gpu_info (buf, cap)` (1 = the V3D is usable; the first call brings it up) and `kapi_gpu_draw (verts, n, clear, pixels, w, h, stride)`: a triangle list of `struct kapi_gpu_vertex { float x, y, z; unsigned char r, g, b, a; }` in normalized device coordinates (y up, z −1 near … 1 far), depth-tested and Gouraud-shaded by the GPU into the app's pixels (e.g. its window canvas: `canvas.px`, `canvas.stride`). The app transforms and lights its geometry itself (floats: build it with `CXXFLAGS_FP`, like `teapot`); keep a software path for `r < 0`. See `user/Apps/teapot` (`teapot.h` is portable: `tools/tests/teapot/teapot_host.cpp` renders it on a PC) and docs/02 §15. **The GPU's full pipeline (ABI v53)**: textures — `int t = kapi_gpu_texture (-1, argb, w, h, stride);` (0xAARRGGBB, up to 2048 × 2048, 256 handles; the same call with `t` replaces the pixels, with `pixels = 0` frees it; freed anyway when the app ends) — and `kapi_gpu_render (&frame,`

`verts, nv, batches, nb)`: `struct kapi_gpu_vertex3 { x, y, z, w, s, t, r, g, b, a }`, each `struct kapi_gpu_batch` draws `count` vertices from `first` with its own **4 × 4 matrix** (row by row, clip = M · (x y z w): build model × view × projection once per object on the CPU, the GPU transforms every vertex; `KAPI_GPU_B_NOMATRIX` if they are already in clip space), texture (−1 none), `KAPI_GPU_B_*` flags (depth test / writes, culling, blending alpha / add / multiply, linear filter, wrap). Draw opaque batches first, then the blended ones with `KAPI_GPU_B_NOZWRITE`. `frame.flags = KAPI_GPU_F_KEEP` draws over the pixels. **v54**: each vertex also carries `r2 g2 b2 a2`, a colour **added** after texel × colour (0 = unchanged; the N64 emulator's colour combiner is `texel × c1 + c2`), and `KAPI_GPU_B_ALPHATEST(t)` discards the fragments whose alpha is below `t` / 255 (cut-out foliage, fences, text) without blending or depth sorting. **ABI v98 — a generated vertex shader**: a program whose vertex and coordinate shaders are made at run time from an emulated console's (`user/Libs/v3d/picavs.h`: a PICA200 shader specialized for a draw, with its lighting) is given `KAPI_GPU_P_VS_1WAY` (64 registers, its VPM output block apart: an output may be written before the last input is read) and drawn by `kapi_gpu_render3` batches flagged `KAPI_GPU_B_NOCLIP` (the vertices are the floats the shaders read — the coordinate shader the first `csInputs` —, the V3D clips) and, better, `KAPI_GPU_B_INDEXED` (`b.first` vertices then `b.count` 32-bit indices: a vertex shaded once). Test `kapi_abi_version () >= 98` first. The 3DS core does it (`user/Emulators/n3ds/n3ds_pica.cpp`, `vsChunk`; the host: `n3ds/onyxhost.h`); `tools/tests/v3d/picavs_test.cpp` checks the generator in the simulator.

ABI v61 — your own shaders: `kapi_gpu_program (-1, &prog)` uploads a vertex, a coordinate and a fragment shader (V3D 4.2 QPU words, generated at run time with `user/Libs/v3d/qpu.h`: `qpu::Prog p; p << qpu::I ().a (V3D_QPU_A_FADD, rf (3), r1, r5).ldvary (r0); …`; every app links `v3d/libv3d.a`; **on the Pi 5** (V3D 7.1) the same code is translated as it is built after `qpu::setVersion (qpu::versionOf (info))` (`info`: `kapi_gpu_info`'s text), keep to rf0..rf(`qpu::rfLimit ()` − 1) and give the program `qpu::programFlags (flags)` (docs/02 §15.1); test them on the PC with `tools/qpu/qpusim` — `tools/tests/run_qpu_test.sh`), then `kapi_gpu_render2 (&frame, verts, nv, stride, batches, nb, uniforms, nuni)` draws `struct kapi_gpu_batch2` batches of vertices of `stride` floats (the clip-space x y z w first, clipped by the kernel) with their program, uniform ranges, up to 8 textures (the kernel writes each one's TMU words at `fsUni + texUni[i]`), blending (`KAPI_GPU_BLEND2`), write mask and scissor. The vertex shader writes Xs Ys (24.8 fixed point: x × w/2 × 256 …) Zs 1/Wc then the varyings; the coordinate shader Xc Yc Zc Wc Xs Ys; the fragment shader ends with its TLB writes after the last thread switch — see the v53 shaders in `kernel/sys/v3d_shaders.qasm` for the recipes, or take the ready-made ones of `user/Libs/v3d/shaders.h` (`passVS` / `passCS`: the position as it is, the other floats handed on as varyings; `flatFS`, `varyFS`, `texFS`; `viewUniforms (w, h)`). `/bin/v3dprog` checks them on the Pi (PASS / FAIL); `tools/tests/run_qpu_test.sh` checks the same programs on the PC (the instruction restrictions, then the fragment shaders in the simulator). The GameCube's TEV is generated by `user/Libs/v3d/gxtev.h` (`gxtev::build (config, shader)`: the stages in integers as the hardware, the texture lookups, the alpha test, the EFB format -> QPU code + the uniforms and varyings the draw must give); `tools/tests/v3d/gxtev_test.cpp` checks thousands of random configurations in the simulator against `user/Libs/v3d/gxtev_ref.h` (the GLSL TEV of `gxgl.cpp` in C++), `/bin/v3dprog` a few on the GPU. `gcemu` draws with them: `user/Apps/gcemu/gxv3d.h` is its `gc::GxGpu` -- the app core records each draw (the vertex stage of `gxgl.cpp` in C++, the TEV program by its configuration, the uniforms, the state), the main thread gives the frame to `kapi_gpu_render2`; `tools/tests/gc/gcv3d.cpp` runs a `.dol` / `.iso` on the PC with the same recorder and draws its frame with a software V3D (the shaders in `qpusim`) into a `.ppm` (`GCV3D_EVERY=n` more frames,

`GCV3D_DUMP=1` / `2` the batches / their programs, `GCV3D_SAVE=<f.gxf>` the last frame dumped, `GCV3D_SKIPLOG=<file>` each draw the recorder leaves out and why -- `Rec::skipWhy`, `Rec::onSkip`; `GC_CARD=<f.sav>` a memory card, read only, `GC_PAD="f0-f1:hex;..."` the pad's buttons held from field f0 to f1, as `gcrun`'s: the games' menus scripted -- The Wind Waker's first steps on Outset: Start at 900, A at 1050, 1200, 2200, 2300, 2400, Start at 2500, A at 2600, then A every 60 fields to 22000; `GC_HASH=n`, in `gcrun` too: MEM1's hash every n fields, two runs compared -- the GL and the TEV renderers gave the same game, so a difference was the drawing's; `GCV3D_VHASH=1`: each finished frame's recording hashed, a line a frame -- a change of the recorder or the vertex decoder checked bit for bit over a whole run). The recorder's vertex loop and `gxPrimitive`'s decoder work from plans made once a draw (the matrices of the last index, the lit channels' lights, the texgens, the varyings; the attributes present with their formats and arrays, a vertex template) with the same arithmetic as before. The recorder's plan (`Rec::Plan`) is **kept from a draw to the next**: `planState` (the TEV's configuration and program, the textures, the batch and its uniforms, the vertex stage's plan) is made again only when `GxState::serial` changed, `planXf` (the lights, the texgens' post-transform matrices) when `Machine::xfSerial` did (every XF memory write, the indexed loads too), and the matrices of the vertices' indices stay while the XF memory is the same; a draw of the same plan right after its last one goes on in that batch. The Wind Waker's draws are 4.6 vertices on average, ~90 % of them with the last one's state and XF memory: its recorder 15.7 -> 8.2 ms a field on the Pi (the GX core 92 -> 61 % busy). A colour's varying (`/ 255`) comes from a table (the colours are integers 0..255) and a texture coordinate's `/ size` is a product when the size is a power of two (both exact: the recordings bit for bit the same, `GCV3D_VHASH` over 2400 fields). The GX's reset keeps its state's serial and `xfSerial` going (a plan made before must not match). A triangle whose three vertices are before the near plane or behind the eye (the kernel's first two clipping planes, `V3DClipDistN`: the kernel would drop it) is not recorded — ~10 % of The Wind Waker's, copied for nothing by the recorder, `Out::prepare` and the kernel before (the pictures the same: gcv3d; F12's fourth line counts them). The recorder keeps the vertices in the clip space of the whole EFB (640 x 528) and the scissors in its pixels; the frame's XFB copy rectangle, known at its end (`Frame::rect`), is applied when it is drawn (`frameView`, `frameScissor`: `Out::prepare`, gcv3d) -- the copy registers of the draw's time were used before, and a half-size copy to a texture in the middle of a frame (The Wind Waker's effects) framed what followed as a quarter of the picture (the camera "in Link's head"). **On the Pi**, gcemu's F12 third line says what the TEV renderer did (`gpu_render2`'s result, the batches drawn / recorded, the ones left out: program refused, texture missing, nothing visible, over the vertex limit), F9 dumps the frame shown into `SD:/gcdump/frame_<n>.gxf` (the recorded frame, its programs and textures, and the picture the GPU made), `--diag[=<folder>]` writes F12's lines of every second into `<folder>/diag.txt` (saved every 5 s), dumps a frame every 60 s (the machine waits meanwhile: a few seconds over FTP) and quits after 200 s; `--statlog[=<folder>]` writes the same lines into `<folder>/statlog.txt` without the dumps and the end (the last ~200 KB kept, saved every 5 s: a measure of a game played, F12's lines whole whatever the window's width); `--nodraw` (a measure) records the TEV frames but neither prepares nor draws them (the window stays still): the machine's speed without the display's work beside it; F12's fourth line is where a field's time went on the app core — the GX (`Machine::timeFifo`, all in), its primitives (`timePrim`: decoding, state, textures, the recorder), the textures (`timeTex`), the recorder (`Rec::drawTicks`) — and a frame's on the main thread (`Out::prepare`, `gpu_render2` — the kernel splits the latter in kmsg every 256 frames: `render2, a frame: clip … lists … GPU … target …`); the counters read the ARM's clock (`gcClock`, nothing on other hosts) — the folder may be an FTP one (`--diag=FTP:<pc>:<port>/<dir>`) so the files land on the PC at once (`--tevbuf`: the frames drawn

into a buffer of gcemu's, then copied — the GPU no longer writing the window's pixels itself). F10: the game's frames a second (its XFB copies) and the speed alone, in a corner. `--pmu[=e1,e2,e3,e4]`: the app cores' performance counters, a fifth line (millions of cycles a field, instructions a cycle, and a thousand instructions' L1D refills, L2 refills, branch mispredictions -- the events, hex, default 08, 03, 17, 10), the CPU's part and the GX's, then the share of the machine's cycles inside the JIT's code (`in JIT`, its helpers too) and why its runs ended a field (`ends`: a VI line, DI, the audio DMA, AID, DSP, EXI, card, an AI sample; then the JIT's entries from C); `--noren` (a measure) prepares the TEV frames but does not draw them. `--jitprof` (with `--diag`): the JIT's profile every 60 s -- `jitprof.txt` (the costliest blocks, the host instructions each guest instruction costs, the instructions left to the interpreter, the slow memory accesses) and `jitprof.bin` (the blocks' code: `python tools/gc/jitprof.py jitprof.bin --top N --ppc <objdump> --a64 <objdump> --host` disassembles both sides). The machine's cores set FPCR to 0 (IEEE: to nearest, no flush-to-zero; nothing sets it at boot): F12 shows `FPCR was ...` when it was not. **The GX on its own core** (when a second app core is free -- `netcore=0` in `cmdline.txt` leaves cores 2 and 3 to the apps): the machine's core writes the FIFO (the write-gather pipe's bursts: `gatherFlush` publishes the PI's write pointer), the GX's core runs it meanwhile (`Machine::gxStep`: the commands up to that pointer, then `gxDoneW`), reading them in place (a command cut by the FIFO's end only is copied). The machine waits for it (`gxSync`) where the game can see its progress: a CP / PE / PI-FIFO register, a polling loop the JIT skips (the game waiting for the GPU: `idleHit`), a full FIFO (`gxRoom`); the PE's token / finish come back through `gxIrqBits` (raised on the machine's core); `gxLock` keeps the frame's flip and the texture pool from the main thread's `Out::prepare`. F12's fourth line gives the GX core's busy share and the machine's waits; `--gxone` keeps the GX on the machine's core, to compare. `gcv3d --replay <frame.gxf> <out.ppm>` draws a dump on the PC at the Pi's size next to the Pi's own picture (`out_pi.ppm`): the same picture → the GPU is right, the recording is the question; another one → the GPU side. `v3dprog ww <w> <h> <flags> <percent> <variant>` draws The Wind Waker's first frame (its TEV program, one quad) into a w × h target — the experiment that found the `P_FS_FINAL` hangs (docs/02 §15; variants: +1 the plain varyings shader, +2 RGB8, +4 no scissor, +8 / +16 the flat shader final / with a thread switch). Example: `user/Apps/gpudemo`. The kernel's shaders are QPU assembly (`kernel/sys/v3d_shaders.qasm`): after editing, `cd tools/qpu && make` re-assembles and checks them into `v3d_shaders.inc` (committed; the kernel build does not need the tool). docs/02 §15.

GPU compositing (`user/Libs/gpucomp`, ABI v70): to assemble layers (a browser's, a desktop's) link `gpucomp/libgpucomp.a` (built by `user/Makefile`; pure C without libc, FP on — freestanding or newlib apps alike) and include `gpucomp/gpucomp.h`:

```
gpc_config cfg = { my_alloc, my_free, 0 };          // (umm_malloc / malloc...; GPC_F_CPU, GP
C_F_ASYNC)
gpc_ctx *g = gpc_create (&cfg);                      // the GPU when usable (kapi v70), else
the CPU
gpc_tex *page = gpc_tex_create (g, 1920, 4000, argb, 1920);   // premultiplied 0xAARRGGBB, an
y size
gpc_layer L; gpc_layer_init (&L, page);              // whole texture, identity, opacity 255
L.src_y = scroll; L.src_h = 1080;                    // scrolling: nothing uploaded again
gpc_matrix_translate (&L.m, cx, cy); gpc_matrix_rotate (&L.m, a); gpc_matrix_translate (&L.m,
 -w / 2, -h / 2);
L.clip[0] = x; ...; L.opacity = 200; L.flags = 0;    // GPC_L_NEAREST, GPC_L_OPAQUE
gpc_target t = { canvas, w, h, stride, 0 };          // GPC_T_ALPHA: an ARGB off-screen layer
gpc_composite (g, &t, &L, 1, 0xFFFFFF, GPC_C_CLEAR); // bottom layer first; the pixels are th
ere on return
gpc_tex_update (g, page, x, y, w, h, px, stride);    // a damaged rectangle only (gpu_texture
_rect)
```

The matrices are CSS's `matrix(a, b, c, d, e, f)` (`gpc_matrix_translate / scale / rotate / skew / multiply` post-multiply: the last one given applies first, as a CSS transform list). A layer shows its texture's source rectangle (`src_x/y/w/h`, fractions allowed: smooth scrolling); its texels past the texture are not drawn. **Targets**: the window's canvas (the GPU renders straight into it) or `gpc_target_alloc (g, w, h, &stride)` (a `gpu_vbuf` block: the same; kept until the program ends); any other buffer costs the kernel a copy there and back. A damaged rectangle only: give a target of that rectangle (`pixels + y * stride + x`, its w / h) and translate the layers by −x, −y. **Results**: 0; `GPC_LOST` (1) — the GPU stopped (a hang turns it off for everybody): the CPU from then on, the textures that were on the GPU lost (`gpc_tex_lost`), upload them again; < 0 `GPC_EINVAL` / `GPC_ENOMEM`. A texture the GPU refuses (no handle, no low memory) stays the CPU's and is drawn by the CPU in its place in the order. `GPC_F_ASYNC` + `gpc_submit` / `gpc_wait`: the composite on a thread of the program (kapi v67) — the app goes on meanwhile; the target and the textures are not touched until `gpc_wait` (the texture calls wait by themselves). `gpc_get_stats`: the last composite's µs, its `gpu_render` calls, the layers the CPU drew. **Blend modes** (a layer's `blend`, `gpc_layer_init` sets `GPC_B_NORMAL`): `GPC_B_MULTIPLY`, `SCREEN`, `ADD`, `SUBTRACT`, `LIGHTEN`, `MASK` (the target kept where the layer is opaque: `d · sa`; its opacity unused), `CUTOUT` (`d · (1 − sa)`) — premultiplied maths, exact over a transparent (`GPC_T_ALPHA`) target too. The GPU does them from kapi **v72** (`gpu_render`'s presets `KAPI_GPU_BLEND_MULCOL` … `DSTOUT`; multiply and subtract are two batches over the same vertices, the second `UNDER` adding `s (1 − da)`, left out on an opaque target); with an older kernel such a layer is drawn by the CPU between the GPU runs. `gpc_blend_pixel (s, d, blend, alpha)` is the CPU's own pixel: an app flattening its layers itself (Paint's Export) gets the screen's pixels with it. Tests: `sh tools/tests/run_gpucomp_test.sh` on the PC — the CPU path and the GPU path (on a software V3D, `tools/tests/gpucomp/hostkapi.cpp`: the kernel's `FS_TEX` run in `tools/qpu/qpusim`) against a reference in doubles, then partial updates across tiles, a refused texture, the GPU lost, `gpcdemo test` / `bench` built for the PC (first `tools/tests/run_v3d_cl_test.sh`: the control-list packets' fields against Mesa's positions — the software V3D decodes the kernel's own target load / store packets); with `aarch64-none-elf-gcc` on the PATH (or `A64_GCC=`) and `qemu-aarch64`, the CPU path built for the Pi (NEON loops) must give the PC's pixels bit for bit. On the Pi: `/bin/gpcdemo test` (the GPU's pictures against the CPU's), `/bin/gpcdemo bench` (ms a frame at 1920 × 1080, GPU then CPU), `/bin/gpcdemo` (a window). **Jet Browser** composites its pages with it (WebKit's `LayerTreeHost` for Onyx: docs/08 *The compositor on the V3D* — tiles, layers for animations and videos, one composite a frame). **App cores (ABI v51)**: an app may take a whole core (2 or 3) for a function of its own — `int c = kapi_core_acquire ();` (−1: none free), `kapi_core_run (c, fn, arg, stack_top)` (the stack is the app's memory, 16-byte aligned), poll `kapi_core_state (c)` (`KAPI_CORE_IDLE` once `fn` returned, `_RUNNING`, `_FAULT`), `kapi_core_release (c)` (stops `fn` if needed; done at exit anyway). **`fn` makes no kapi call and no allocation** (neither the kernel nor `malloc` is multi-core safe) and never masks the interrupts: it computes and talks to the main thread through memory — `volatile` flags with barriers (`dmb ish`), `wfe` to wait, the main thread's `sev` to wake it, the clock read directly (`cntpct_el0`). Stop it cleanly with a flag it polls. Keep working without a free core (run the same function on the main thread). **Emulators**: `#include "emucore.h"` does all that — `ec_init (&ec, w, h, frame_fn)` (acquires a core if one is free), `frame_fn (EmuCore *)` runs one frame, writes the picture into `ec_back ()` then `ec_publish ()`, pushes its sound with `ec_audio_push`; the main thread `ec_request (&ec, n)`, `ec_pump` (frames made here when there is no core), `ec_take` / `ec_front` (the latest picture: a triple buffer, nobody waits), `ec_audio_pop`, `ec_hold` / `ec_resume` around anything that touches the machine (reset, battery save), `ec_shutdown`. Two rules the app core imposes: **its code**

allocates nothing (the heap, `user/Runtime/umm.h`, is not shared safely between two cores, and `kapi_sbrk` from an app core grows the heap of whatever task core 0 is running — make every buffer before, as gc::Machine's texture pool of `TEX_POOL` texels); and **take the new picture before asking for the next frame** (in the main loop: `if (ec_pending (&ec) == 0 && ec_take (&ec)) ...` first, then `ec_request`) — the other way round, a machine slower than real time always has its next frame asked first and the window is never drawn again. What reads the machine (its frame, its textures) runs while it waits, or under a lock it respects; what only reads a copy may run while it works: gcemu asks for the next field, then makes the kernel's batches of the TEV frame (`Out::prepare`, under the GX's lock: the recorder's finished frame and the textures kept still; it marks that frame `held`: of the recorder's three frames, the GX builds into the other two meanwhile) and draws it (`Out::submit` → `gpu_render3`, kapi v62: the kernel reads the frame's vertices where they are and frames them onto the XFB copy's rectangle on the way — kapi v63: the recorder's three frames are `gpu_vbuf` memory, drawn in place, nothing copied, the frame framed there once (`Frame::framed`: drawn again with `view` 0; F9's dump then says the whole EFB); an older kernel: the vertices copied into one array for `gpu_render2`, as before) while the field runs — two fields asked for at once when behind real time (the drawing, ~30 ms with ~100k vertices, is longer than a field: the machine, its field done, no longer idles until the next request; the picture is then taken after the second) — and not again when the same frame is already in the window (a 30 fps game: every other field). While a field runs the main thread yields (its end seen at once), but naps (`kapi_msleep (1)`: to the scheduler's next tick) while that end, expected from the last second's time a field, is further than 12 ms — a yield in a loop kept core 0 busy (heat: the Pi 4 throttles its clock above ~80 °C, kmsg's `power:` lines). See `gbemu` / `gbaemu` / `nesemu` / `snesemu`; `/bin/coretest` exercises the raw kapi. **Game kit** (`user/Apps/games/game.h`): `GameView` (a full-window widget: `paint`, `press` / `release` / `move` edges, `key`, `tick (dt)` at ~60 Hz), `GameRoot` (ticks it, routes every key to it), sound effects on voices 12..15 (`sfx (hz, ms, wave, vol)`, `sfx_later` for jingles, `sfx_win` / `sfx_lose`, `sfx_set_mute`; the output is acquired on first use), `rng` / `rng_n`, text helpers (`gtext`, `gtext_c` centred with a shadow, `gitoa`, `gcat`). Cards (`user/Apps/games/cards.h`): `card_face` / `card_back` / `card_slot` (64×88) and the bouncing-cards victory animation (`win_start` / `win_step`). Used by invaders, pipes, solitaire, freecell (Arkanoid is now the BASIC game). **Host test**: `sh tools/tests/run_games_test.sh [INVADERS …]` builds each game on the PC against a fake kapi table (`tools/tests/uikithost/host_kapi.h`: every slot a stub, files from `sdcard/`), plays a scripted scenario (UBSan) and saves real screenshots to `/tmp/onyx_games`. **Rich Text Format** (`user/Apps/rtfview/rtf.h`): `rtf::load (box, data, len)` parses an RTF document into a `RichTextBox` (styles, colour table → the 16-colour palette, `\'hh` / `\uN` → Latin-1, skipped destinations), `rtf::save (box, out, cap)` writes it back; `rtf::is_rtf`. Used by `rtfview` (Letters reads and writes RTF itself, with everything: `Apps/letters/fileio.h`). **Letters** (`user/Apps/letters/`, one TU: `main.cpp` includes the rest) — `doc.h` the document (paragraphs of code points, each with an index into the table of character formats — font of the font table, size in half-points, flags, colour, highlight, a field (`FIELD_CHAR` U+FFF9 whose format names a `Field`: page, pages, date / time with its picture, a merge field) or an image (the character U+FFFC whose format names the image and its size) —, each paragraph its `ParaFmt` — style, alignment, indents and spacing in twips, line spacing, list, page break, keep with next / lines together / widow control, tab stops (position, alignment, leader) and, in a **table**, the table's index and the cell's row and column); the **stories** — the body, the header, the footer, the first page's own — one of which is edited at a time (`d.p`, `doc_story` swaps them); a **table** is a run of paragraphs saying their cell, the `Table` itself (columns' widths, rows' heights, the cells' spans, shading, lines, the heading row) a value in `d.tbl` replaced whole by an edit; the tables only

grow, so an edit's undo just puts the paragraphs it copied back: `doc_begin` / `doc_end_edit`, typing coalesced), `layout.h` (lines at the zoom in 1/64 px from the fonts' design advances in boxes — the text's width or a cell's —, each character's x kept — `Para::xs` — for drawing, hit-testing and the caret alike; tab stops, fields' texts, lists' numbers; the headers and footers placed first — the body's top and foot follow them —, then the **pages**: a `Pager` keeps a heading with its next line, widows and orphans off, a table broken between its rows — the rows its cells span together — with its heading row repeated), `edit.h` (the selection, the edits — the tables' rows, columns, merges and splits as one undoable edit of the table's paragraphs and its `Table` —, the stories, the fields, the page numbers, the **table of contents** (laid out twice: its own place moves the pages), the clipboard — the system's text plus the piece of document kept here —, find), `view.h` (the page view widget: drawing — the tables' shading and lines, the fields shaded, the leaders, each page's header and footer, the body greyed while one is edited —, caret blink without a redraw, mouse, keys, images' resizing, a table's column border dragged), `ui.h` (tool buttons, pick boxes dropping a `ListPopup`, the size box, colour popups, the ruler — a table's columns —, the status bar), `icons.h` (`VPath` icons), `fileio.h` (RTF in / out — tables `\trowd`... — their lines as the cells give them: every edge, the rows' only, none —, headers and footers, fields `{\field}`, tab stops, the TOC's field, pictures as `\pict\pngblip` / `\jpegblip`, a PNG made when the image came as something else —, text, HTML), `xml.h` (a pull reader over a zip entry — uikit's `img_inflate` —, the units; the zip written with the PNG writer's deflate), `docx.h` (WordprocessingML: styles with their inheritance and the theme's fonts, numbering, tables — grid, spans, vertical merges —, simple and complex fields, the headers and footers, the section, images, `w:docVars` for the mail merge's data), `odt.h` (ODF: named and automatic styles, lists, tables — spanned and covered cells —, frames, fields with their data styles, the TOC, the master page), `merge.h` (the **mail merge**: the data is a Cardfile form read with Cardfile's own `model.h`; the fields filled per record into a copy of the letter read back from its bytes; `letters --merge JOB` — the job file's keys at the top of `merge.h` — makes a merge's documents for Cardfile and Ledger: a job's `lines` form — a record a line of a document — has the letter's table row holding `Line...` fields repeated for each of its records, `merge_lines`; one file written is shown without a message), `dialogs.h`. **Host test**: `sh tools/tests/run_letters_test.sh` (`tools/tests/letters/files_test.cpp`: a document with all of it written as RTF, `.docx` and `.odt` and read back the same — each through the others too —, and with LibreOffice installed — `soffice` — our `.docx` and `.odt` converted by it and read back; UBSan, `VG=1` valgrind); `tools/tests/letters/conv.cpp` converts a file by the names' extensions (`conv a.docx b.odt`). The samples (`SD:/docs/letters-tour.rtf`, `new-year-letter.rtf`) are made by `tools/gen_letters_sample.py`. **Paint** (`user/Apps/paint/`, a newlib app: FreeType, gpucomp; its own `paint.elf` rule; the mock-ups and the decisions: `docs/paint/README.md`) — `pdoc.h` (up to 32 layers of 0xAARRGGBB pixels, straight alpha, bottom first, each with its blend mode `GPC_B_*` and `clip` — a Mask / Cut out on the layer below only; what changed since the screen showed it: `D.dirty` for the current layer, `D.dirtyAll`; `comp_px` the CPU's composite of a pixel with `gpc_blend_pixel`, the same as the GPU's; undo: a stroke keeps the 64 × 64 tiles it touches — `rec_orig` reads a pixel as it was before the stroke —, a change of size or of layers the whole picture), `psel.h` (the selection: a rectangle or a 0..255 mask; lasso polygons, colour regions with a tolerance, contiguous or not; add / subtract / invert), `pbrush.h` (the stroke engine: a coverage buffer a stroke — max of the dabs, the airbrush adds —, each pixel = the original with the colour over it at coverage × opacity × selection; the brushes' dab shapes, the patterns), `pgrad.h` (gradients: stops + midpoints, the presets, GIMP's `.ggr` read / written; the shapes and repeats along a line), `ptext.h` (the Text tool: `fontkit/fonts.h`'s glyphs laid out into the overlay), `padjust.h` (the colours and filters on the

selection, the layer or every layer: the originals kept, each setting applied to them), `raster.h` (the shapes, flips, turns, scaling), `pview.h` (the canvas: the layers as gpucomp textures keyed by their pixels' address — the changed rectangles sent with `gpc_tex_update`, premultiplied, the current layer with what floats over it, a layer under a clip mask multiplied by the mask's alpha —, composited at the zoom into a view-sized ARGB target, laid over the checkerboard; the tools; the overlay — a shape, a gradient, a text being placed — put down by `ov_commit`), `picons.h` / `pui.h` (the icons, the ribbon, the options bar — items laid out per tool, sliders dragged in place —, the layers' panel, the status bar), `pfile.h` (OpenRaster with `composite-op` and `onyx:clip`, the pictures, the exports flattened by `comp_px`). The simulator: `sh tools/tests/desktop_sim/shots.sh paint` (`paint_scene.py`: the scripts); the samples `SD:/docs/pictures/*.jpg` by `tools/gen_paint_samples.py` (package `paint-samples`). **Cardfile** (`user/Apps/cardfile/`, one TU: `main.cpp` includes the rest; integer only, no libc) — `model.h` the document: a form (`Field`: display name, column name, type `FT_*`, decimals, choices) and its records (an array of heap strings a record, the empty value shared), each value kept in one canonical text a type (integer `-12`, decimal `12.50` at the field's decimals — a `long long` scaled by 10^decimals when computed —, date ISO `2026-09-29`, colour `#3366CC`, yes / no `yes` or empty); the values read as typed (`parse_num`: `.` or `,` as the point, group separators; `parse_date`: D/M/Y, D.M.YY, D/M, DDMMYYYY, ISO), shown (`value_show`), made (`value_new`), compared (`value_cmp`: numbers, dates, a choice by its list's order, text folded — Latin-1 case and accents — with its runs of digits compared as numbers); the file (`doc_write` / `doc_read`: an INI-like head, the records tab-separated with `\t \n \\` escapes, read leniently — the format is described at the top of `main.cpp` and in docs/04); a type changed (`convert_count` / `convert_field`: through the value's shown text); the fields inserted, removed, moved (`doc_remap`: every record laid out again); CSV (`csv_write`, `csv_read` with each column's type guessed); the order (`build_order`: the search's words, a stable merge sort, empty values last). `widgets.h`: the editors — `LineEdit` (any length, selection, clipboard, a filter of the characters; over a `TextCore` it shares with `MemoEdit`, wrapped at the words), `DateEdit` + `CalPopup` (uikit's `Calendar`), `ColorEdit` + `ColorPopup`, `ChoiceBox` + `PickList` (a scrolling list over the window), `YesNoBox` —, the toolbar, the view switch, the search box, the navigator (`NavBar`), `VPath` icons. `formview.h` (the card: an editor a field, the commit and its validation, Tab order, scrolling), `listview.h` (a `DataGrid`), `designview.h`, `app.h` (the state the views share). Undo keeps the whole document written before each change (100 steps, 24 MB at most; one control's edits coalesced into one step). **Record ▸ Mail Merge** (`MergeBox`, `cmd_mail_merge`): the records to merge written as a `.card` (`SD:/apps/cardfile.app/merge.card`), a job file beside it, then `kapi_exec` of `letters --merge JOB` — Letters' `merge.h` reads both; the form's `merge` key keeps the letter. **Host test**: `sh tools/tests/run_cardfile_test.sh` (the values as typed, a round trip byte for byte, a file edited by hand, a type changed, fields moved, CSV, the order; ASan + UBSan). **Ledger** (`user/Apps/ledger/`, one TU: `main.cpp` includes the rest; **in English or French**: its words `TR(...)`, `sdcard/apps/ledger.app/lang/fr.txt` — uikit's `lang.h` above —, the language the system's (Language & Region), taken at its start; the books' own words — the chart, the printed documents — follow the company's and the party's language as before; integer only — money in **cents** (`money`, a `long long`), VAT rates in hundredths of a percent, quantities in thousandths, dates `yyyymmdd` —; Cardfile's `model.h` and `widgets.h` are reused: strings, `Out`, the editors). **The engine** (plain C++, the same on the PC, no uikit): `core.h` (money and dates typed and shown the Belgian way, the checks — VAT numbers, IBAN, BIC, structured communications `+++…+++` —), `model.h` (the `Book`: company, fiscal years, accounts, parties, journals, entries of lines — account, amount, party, VAT code, role, due date, matching group —, VAT returns, the commercial documents; matching: a group of lines adding up to zero), `pcmn.h` (the PCMN in

French and Dutch, generated by `tools/ledger/gen_pcmn.py` from `pcmn.tsv`), `setup.h` (a new company's chart and journals, the fiscal years added, closed — the result appropriated —), `vat.h` (the VAT codes — Odoo's Belgian ones: a code, its rate, its side, the grids its base and tax go to, a reverse charge's both sides —, the grids of a period, Intervat's checks, the return's XML — `NewTVA-in` v0.9 —, the settlement, the customer and intra-Community listings' XML), `post.h` (an `Invoice` and a `Statement` as typed ↔ their `Entry`, the checks, the numbers, the structured communication made from a sale's number, the matches applied), `reports.h` (a `Report`: rows of cells — journals, general ledger, trial balance, balance sheet and income statement by the PCMN's headings, balances, ages, a party's account —), `fileio.h` (the `.ledger` file: its format described at its top; read leniently, written whole), `commerce.h` (quotes, orders, delivery notes, purchase orders: numbered by kind and year, their totals, one made from another, the invoice made from one), `coda.h` (Febelfin's **CODA** read — the records 0, 1, 21 / 22 / 23, 8, 9; a globalisation's details skipped —, a statement made of it: the parties and the invoices paid found by structured communication, IBAN, name; a writer for the demo and the tests), `sepa.h` (the suppliers' open lines, **pain.001.001.09** written: hybrid addresses, the EPC's Latin set, a structured communication as `SCOR` / `BBA`), `print.h` (the merge's data for a document's template: the fields, the lines, the VAT's detail with the legal mentions in French, Dutch or English — a party's language —, then `letters --merge`), `export.h` (a `Report` as RTF — A4, landscape when wide, a header and page numbers, tables — or `.xlsx` — `img/pngsave.hpp`'s `ZipOut` — or CSV, opened in Letters or the Spreadsheet). **The window**: `ui.h` (the `Page` a view is — its header, `sync` when the books or the year shown change —, the icons as `VPath`s, `FlatButton`, `HeadRow`, `Segmented`, pills, tiles, `TickGrid`, `AskBox`, `FormBox`), `pick.h` (`PickEdit`: a party, an account, a VAT code found as typed, its suggestions below), `editgrid.h` (`EditGrid`: a document's lines typed in place over an `EGModel` — Tab / Enter / arrows, F4, a line added at the end —), `docs.h` (`DocPage`; the invoice, the statement — its open items ticked, a CODA statement to complete —, the misc. operation), `lists.h` (the overview, the journals' lists, the parties' cards and accounts, the chart), `pages2.h` (reports, VAT, settings — the templates' pane —, the new company), `commerce_ui.h` (the quotes' list and page, *Next step*), `payui.h` (the SEPA dialog), `main.cpp` (the side bar, the pages made when first shown, the books' file saved at each change, the CODA import's queue, the menus). **Host test**: `sh tools/tests/run_ledger_test.sh` (`tools/tests/ledger/engine_test.cpp`: money, dates, the checks, invoices and credit notes, a statement, the VAT grids, matching, the file byte for byte, the reports, the year's end, the commercial documents, CODA read and made a statement, a SEPA file — 200 checks, ASan + UBSan —; the XML files validated by `xmllint` against the official schemas in `tools/tests/ledger/xsd/`: Intervat v0.9, ISO 20022 `pain.001.001.09`). **Generators**: `tools/ledger/make_demo.cpp` (the demo company through the engine itself — `sdcard/docs/demo-company.ledger` — and its next CODA statement, `demo-bank-statement.cod`: see its header), `tools/ledger/gen_templates.py` (the documents' templates in French, Dutch, English, and `templates/fields.card`), `tools/ledger/gen_pcmn.py`. **Spreadsheet** (`user/Apps/sheet/`, a **newlib** uikit app — FreeType and `libm` — built as Letters is; one TU: `main.cpp` includes the rest, a chain of headers each including the one before, all in `namespace ss`; `app.txt` asks for a **4 MB stack**: the formulas are evaluated recursively). **The engine** is plain C++ over libc, the same code on the PC: `core.h` (a growing `Buf`, UTF-8 — the cells' text is UTF-8, uikit's font and the keyboard Latin-1 with the euro at 0x80: `latin1_cp`, `latin1_to_u8`, `u8_to_latin1` —, numbers ↔ text, Excel's serial dates, the `Arena` a formula's temporary values live in, released after each cell), `book.h` (the `Book`: its sheets, the interned `Style`s — a cell keeps an index —, the defined names; a `Sheet`: the used cells in a hash `CellMap` by row and column, the widths / heights / hidden flags, merges, frozen panes, charts anchored to a cell, conditional

formats, the AutoFilter; a `Cell`: what was typed — number, text, formula — and its value), `formula.h` (Excel's syntax read into tokens — kept to print the formula back as typed — and a tree over them; each reference its row / column and `$` flags; `formula_copy (dr, dc)` moves the relative parts, the shifts of inserted / deleted rows and columns move every reference, a deleted one becomes `#REF!`; the names' table), `eval.h` (the tree walked: `IF` and its kind lazy, operators over numbers / texts / arrays element by element, a function taking one value lifted over an array; `recalc` recomputes every formula at each change, a formula computing the cells it reads first — the book's `epoch` marks what is done —, a chain deeper than `MAX_DEPTH` pushed on an explicit stack, a cell met again on it `#CIRC!`), `numfmt.h` (Excel's format codes compiled once — `fmt_get` caches them — and applied), `input.h` (what a typed entry is: number, percentage, amount, date, time, boolean, error, text — with the format it implies), `fn_core.h` / `fn_more.h` (the 237 functions, Excel's names, arguments and results), `funcs.h` (the table `FNS`: name, arguments, flags — lifted, lazy, volatile —, category, the argument list and the line the tips and Insert Function show), `ops.h` (what the window does to the book: a cell set, a value shown, styles over a range, rows / columns inserted, deleted, sized, cells moved / copied / filled / sorted / merged, sheets; the AutoFilter), `undo.h` (before a change, the range, sheet or book it touches written in a small binary form; Undo writes the present the same way for Redo; 100 steps, 64 MB). **The files**: `xml.h` (a pull reader, the escaping), `xlsx.h` (Excel's `.xlsx`, read and written — the archive through `img/pngsave.hpp`'s `zip_find` / `ZipOut` and `img_inflate`; under `SHEET_APP` the files go through kapi, on the PC through stdio), `ods.h` (LibreOffice's `.ods`, read: OpenFormula turned into Excel's syntax, the number styles into format codes), CSV in `main.cpp`. **The window**: `condfmt.h` (the rules' look for a cell — their figures over the range computed once a recalculation), `ui_base.h` (Letters' toolbar look, the icons Letters lacks), `render.h` (a cell drawn: FreeType fonts — the Office families mapped to the card's —, the formatted value, wrap and overflow, fill, borders, merges), `chart.h`, `grid.h` (`GridView`: headers, up to four panes, the selection, the fill handle, the charts, the in-place editor with its coloured references, pointing, the functions offered and their tips, the AutoFilter's buttons), `bars.h` (`NameBox`, `FormulaBar`, `SheetTabs`, `StatusBar`), `dialogs.h`, `main.cpp` (the menus, the commands, the clipboard — the cells kept here, their text on the system clipboard: a paste from elsewhere is read as a tab-separated table —, the files, `docguard.h`, the recovered workbook). **Host test**: `sh tools/tests/run_sheet_test.sh` — `tools/tests/sheet/engine_test.cpp` (formulas, functions, formats, typed entries, references moved, names, undo) and `files_test.cpp` (`.xlsx` round trips; with LibreOffice installed, its `.ods` and `.xlsx` of the same workbook read back alike); ASan + UBSan. The sample `sdcard/docs/cafe-2026.xlsx` is made by `tools/tests/sheet/make_sample.cpp` (built with the engine itself: see its header). **Slides** (`user/Apps/slides/`, a **newlib** uikit app — FreeType, `gpucomp`, Letters' PDF writer —, built as Letters and the Spreadsheet, their toolbar icons shared through `sheet/ui_base.h`'s `g_appIcon`): `model.h` (the deck: slides, objects — text, shape, line, picture, table, chart —, text bodies of paragraphs and character runs in hmm and tenths of a point, the theme, the master's text styles and layouts, effects, transitions; a format inherits from the master: `cf_resolve`, `pf_resolve`), `text.h` (a box's text laid out — lists, autofit shrinking —, drawn with FreeType into a premultiplied layer, the caret, the edits), `render.h` (each object drawn into its own **layer**: VPath shapes, gradients, blurred shadows, pictures, tables, charts; the `Compositor` keeps them as GPU textures, cached by a hash of the object, and `composite_frame` places them — moved, faded, rotated, clipped — with `gpc_composite`; the CPU path for thumbnails and exports: `flatten_slide`), `editor.h` (selection, commands, undo as deck snapshots), `view.h` (`SlideView`: the slide composited by the GPU, handles, guides, the caret; a drag only moves the layers), `panes.h` (thumbnails, sorter, notes, status bar), `sidebar.h`, `show.h` (the full-screen show:

`fx_plan` / `fx_moves` -- a text by paragraph: its paragraphs' steps in `FxTime`, shown through a clip down to the one playing --, the transitions as two textures, the presenter's console), `find.h` (Find and Replace: `find_next`, `replace_all`), `odp.h` (OpenDocument read and written: styles, gradients, list styles, table-cell styles; what ODF cannot say kept in `onyx:` attributes and `onyx.xml`, so a deck comes back the same), `master.h` (the master view: `g_deck.slides` swapped for the master and the layouts as slides, edited with every tool; `master_sync` after each change -- `done_change`'s `g_onDone` -- folds the samples' formats into the styles and writes the decorations and layouts back; `master_close` puts the slides back, their placeholders following their layout's, and pushes one undo step; `g_accepts` turns what is added to a layout into placeholders), `pptx.h` (PowerPoint's format, written as PowerPoint writes it — every part Slides needs has its native word: the theme, one master and a layout per Slides layout (placeholders by type and idx), `p:sp` / `p:cxnSp` / `p:pic` / `p:graphicFrame` (tables in PowerPoint's default style, charts as `ppt/charts` parts with their data cached), the footers as the slides' placeholders, `p:transition` with `p14:dur`, `p:timing` from a table of PowerPoint's presets both ways (`FX_PRESET`), sections as `p14:sectionLst` — so a deck comes back the same; read generically: relationships followed, `mc:AlternateContent`'s Fallback (a `p14` Choice for transitions), the colour map and theme colour modifiers, a placeholder's position and formats from its layout and master, style references, groups flattened, every master's layouts — LibreOffice writes a master per layout), `main.cpp` (`deck_load`: by what the archive holds; Save writes `.pptx` when the name says so). **Host test**: `sh tools/tests/run_slides_test.sh` — `tools/tests/slides/slides_test.cpp` (the sample read and checked, the `.odp` and `.pptx` round trips, a stress deck — every shape, effect, transition, chart kind — written as `.pptx` and read back the same; with LibreOffice installed, its conversions read: the sample as `.odp` and as `.pptx`, our `.pptx` resaved). `tools/tests/slides/make_pptx.py` makes `powerpoint.pptx`, a deck from PowerPoint's template (python-pptx), for the reader and the screenshot. The `.pptx` written also passes the OOXML schema validator of the `pptx` skill (`scripts/office/validate.py`). The sample `sdcard/docs/cafe-2026.odp` is made by `tools/tests/slides/make_sample.cpp`, and `cafe-2026.pptx` from it by Slides' writer (`MAKE=1 sh tools/tests/run_slides_test.sh`); the icon by `tools/icons/slides_icon.py`. **Game Boy / Color core** (`user/Emulators/gb/gb.h`, `gb/libgb.a`, linked into every app): `gb::Machine` — `load (rom, size)` (CGB mode from the header), `runFrame ()` → `fb` (160×144, 0x00RRGGBB), `setButtons (gb::BTN_* mask)`, `setAudioRate (hz)` + `audioRead (lr, n)` (s16 stereo), `setSaveRam` / `sram` / `sramDirty` (battery saves), `setDmgPalette`. The SM83 CPU with instruction timing, a scanline PPU (DMG + CGB), timer, OAM DMA / HDMA, MBC1/2/3(+RTC)/5, the 4-channel APU; integer only, no libc. Used by `gbemu` (the emulator: paced by the audio queue, else the clock; the machine on an app core through `emucore.h`) and `gamelib` (the library: title-screen thumbnails made headless). The ROM windows are pointers set when the game switches banks (`mapRom`), the background and window are drawn a tile at a time. **Host test**: `GB_TEST_ROMS=<unzipped c-sp game-boy-test-roms> sh tools/tests/run_gb_test.sh` (Blargg cpu_instrs / instr_timing / halt_bug, dmg-acid2, cgb-acid2 pixel-exact); `tools/tests/gb/gbtest.cpp <rom> <seconds> [out.ppm] ["t:mask,..."]` runs any ROM headless. **Game Boy Advance core** (`user/Emulators/gba/gba.h`, `gba/libgba.a`): `gba::Machine` — the same shape as `gb::Machine` (`load`, `runFrame` → `fb` 240×160, `setButtons (gba::BTN_*)`, `setAudioRate`, `audioRead`, `setSaveRam` / `save` / `saveSize` / `saveDirty` / `saveType`). The ARM7TDMI (ARM + Thumb, the two prefetched opcodes kept: self-modifying code sees them), the memory map with WAITCNT wait states and open bus, the frame loop driven by events (line phases, timer overflows), DMA (immediate, V/H-blank, sound FIFO), 4 timers (cascade), the PPU a line at a time (modes 0-5, affine, sprites, windows, blending, mosaic), the PSG +

Direct Sound A / B, the BIOS calls done in C++ (`gba_bios.cpp`: no BIOS image), SRAM / Flash 64-128 KB / EEPROM (found from the ROM's library string). Used by `gbaemu` and `gamelib`. **Host test**: `GBA_TEST_ROMS=<jsmolka gba-tests> sh tools/tests/run_gba_test.sh` (arm, thumb, memory, bios, nes, unsafe, saves); `tools/tests/gba/gbatest.cpp <rom> <seconds> [out.ppm] [keys]` runs a game headless (`GBA_SHOTS`, `GBA_AUDIO`, `GBA_SAVE`, `GBA_LOAD`, `GBA_REGS`; built with `-DGBA_DEBUG`, `GBA_WATCH=<addr>` prints every write there). **NES core** (`user/Emulators/nes/nes.h`, `nes/libnes.a`): `nes::Machine` — the same shape (`load` an iNES / NES 2.0 file, `runFrame` → `fb` 256×240, `setButtons (nes::BTN_*)`, `setAudioRate`, `audioRead`, `setSaveRam` / `sram` (8 KB) / `battery` / `sramDirty`, `setPal`). The 6502 (official

- stable unofficial opcodes, from the generated table `nes_optable.inc`), stepped an instruction at a time with the PPU and APU brought up to each access; the PPU (`nes_ppu.cpp`) renders a line at dot 1 from the loopy `v` / `t` registers, with the sprite 0 hit dot, vblank / NMI and the A12 edges the MMC3 counts; the APU (`nes_apu.cpp`: 2 pulses, triangle, noise, DMC, the frame counter and its IRQ) mixes through the nonlinear tables `nes_mix.inc`. Mappers 0, 1, 2, 3, 4, 7, 66; NTSC and PAL (312 lines, 3.2 dots per CPU cycle, the PAL APU tables; `load` reads it from the header, `nesemu` also from the file name). Used by `nesemu` and `gamelib`. **Host test**: `NES_TEST_ROMS=<christopherpow nes-test-roms> sh tools/tests/run_nes_test.sh` (nestest against its trace, blargg all_instrs / instr_timing, apu_test 1-8, mmc3_test 1-3 and 5); `tools/tests/nes/nestest.cpp run <rom> <seconds> [out.ppm] [keys]` runs a game headless (`NES_PAL=1`, `NES_AUDIO=<file>`). **Super Nintendo core** (`user/Emulators/snes/snes.h`, `snes/libsnes.a`): `snes::Machine` — the same shape (`load` a `.sfc` / `.smc`, a copier header skipped; `runFrame` → `fb` 256 × `height` (224, 239 with overscan); `setButtons (snes::BTN_*)`, the pad's 12 buttons as the joypad word; `setAudioRate`, `audioRead`; `setSaveRam` / `sram` / `sramSize` / `sramDirty`; `setPal`; `title`, `hirom`, `chip`). `snes.cpp`: LoROM / HiROM (the header that scores best), the bus as 4 KB pages (direct reads, the access speed of each page, FastROM), the 5A22's registers, DMA and HDMA (direct / indirect, the 8 transfer modes), a line = 1364 master clocks with its events (HDMA start, DRAM refresh, the H/V IRQ, the line drawn + HDMA) run between the instructions. `snes_cpu.cpp`: the 65C816 (every opcode and mode, the emulation-mode wraps, decimal mode, block moves). `snes_ppu.cpp`: modes 0-7 a line at a time (tile rows decoded 8 pixels at a time, offset-per-tile, mosaic, direct colour, Mode 7 + EXTBG, 32 sprites a line, the windows, colour math, brightness; modes 5 / 6 at half their width). `snes_apu.cpp`: the SPC700 with its IPL ROM and timers, the S-DSP (BRR, gaussian interpolation — `snes_gauss.inc`, a generated gaussian kernel —, ADSR / GAIN, noise, pitch modulation, echo + FIR), resampled from 32 kHz; the sound unit catches up with the CPU when the CPU touches its ports and every 16 lines. No enhancement chip (`load` refuses them). Used by `snesemu` and `gamelib`. **Host test**: `SNES_TEST_ROMS=<PeterLemon SNES> sh tools/tests/run_snes_test.sh` (the CPU, SPC700 and PPU test ROMs against their reference pictures; `SNES_CPUTEST=` gilyon's cputest-full.sfc, 649 tests); `tools/tests/snes/snestest.cpp <rom> <seconds> [out.ppm] [keys]` runs a game headless (`SNES_SHOTS`, `SNES_AUDIO`). **Nintendo 64** (`user/Emulators/n64/`, in progress): `n64_cpu.cpp` the R4300i interpreter (MIPS III, COP0 with the TLB / exceptions / Count-Compare, COP1), `n64_bus.cpp` the RCP interfaces (MI, VI, AI, PI DMA as ares does it, SI + PIF, SP / DP registers) and the boot (the IPL3's work done directly, CIC 6101-6106), `n64_gfx.cpp` the graphics tasks at a high level (F3DEX2: matrices, lit vertices, triangles; the RDP: TMEM loads, tiles, the texture formats decoded into a cache, the combiner evaluated per vertex, blending / depth — the decal z mode drawn $2{\cdot}10^{-4}$ nearer in NDC depth, against z-fighting —, fill and texture rectangles; the rectangles into an off-screen colour image drawn by the CPU straight into RDRAM, and the renderer's framebuffers written back into RDRAM from the host's copy of the last frame (`Machine::fbSnapshot`, called by n64emu / NintendoEMU after each frame shown) when a game loads a texture from one — Ocarina of Time's pause background,

copied into the z-buffer, its 8-bit coverage copy written as full so that the CPU anti-aliasing filter leaves it alone: the menu opens in ~1 s instead of ~4) into a `GFrame` of clip-space vertices + batches in the kapi v54 layout; `n64_audio.cpp` the audio tasks at a high level (the "nead" microcode of Zelda OoT / MM, recognised by its data: VADPCM, resampling, envelope mixer, interleave on a 4 KB DMEM image; other audio microcodes: silent) and the AI output (each DMA's frames resampled into a ring, `audioRead`). `n64emu` draws the frames with `kapi_gpu_render` (the machine on an app core, lockstep). **Host test**: `N64_TEST_ROMS=<PeterLemon N64> sh tools/tests/run_n64_test.sh` (89 of the 94 CPU test ROMs match their pictures); `tools/tests/n64/n64test.cpp <rom> <frames> [out.ppm]` runs a game headless (`N64_GFX=out.ppm`: the last graphics frame drawn by the BASIC 3D's software renderer; `N64_STATE`, `N64_THREADS` (libultra's threads found in RDRAM), `N64_WHO`, `N64_MEM`, `N64_WAV=out.wav`: the sound, 32 kHz stereo). **GameCube** (`user/Emulators/gc/`, in progress): `gc_cpu.cpp` the Gekko (PowerPC 750CL) interpreter: integer, branch, the SPRs (BATs, HIDs, GQRs, the locked-cache DMA, timebase, decrementer), the exceptions, the FPU (exact single <-> double conversions, 25-bit multiplicands), the paired singles and the quantized loads / stores; `gc_mem.cpp` MEM1 (big-endian), the locked cache, the BAT translation (a map of 128 KB blocks); `gc_hw.cpp` PI, VI (lines, display interrupts, the YUV framebuffer), SI pads, EXI (RTC / SRAM), DI (disc reads), AI, the DSP interface (audio / ARAM DMA); `gc_gx.cpp` the CP / PE and the GX FIFO (linked mode: read as the CPU writes it; the commands, display lists, vertex sizes from VCD / VAT, tokens, "draw done"); `gc_gxdraw.cpp` the drawing at a high level: vertices decoded and transformed (XF matrices, projection, viewport), lit, texgens, the TEV per vertex (texel x c1 + c2), the GX texture formats (TLUTs in TMEM; their mipmaps for a GPU) -> a `GFrame` in the kapi v54 layout, ended by the EFB -> XFB copy — or, with a GPU backend (`Machine::gpu`), the draws as the game gave them (`gc_gxgpu.cpp`, below); `gc_dsp.cpp` the DSP at a high level (as Dolphin's HLE): the ROM's boot mails, the microcode told by its CRC (AX, the "Zelda" one and its variants' flags, the memory card's), the mail queue with its interrupts (a mail's IRQ raised when the previous one is read, none seen while the DSP is halted), the task switch (DSP_YIELD, a new microcode, DSP_RESUME); AX's protocol acknowledged (silent), the Zelda protocol (command lists acked with sync mails, the frames rendered as the CPU says the voices are ready, DSP_FRAME_END) with its **audio renderer** `gc_zelda.cpp` (Dolphin's ZeldaAudioRenderer: the 0xC0-word voice blocks, AFC ADPCM and PCM8 / PCM16 from ARAM or MRAM, 4-tap resampling, volume ramps, Dolby, the reverbs, 0x50-sample frames into the game's buffers — a game on this microcode, e.g. The Wind Waker, waits for its streams to play: without it, a black screen after the intro); `gc_card.cpp` the **memory card** (an EXI device: the Nintendo commands 00 / 52 / 81 / 83 / 85 / 89 / F1 / F2 / F4, DMA with the card's delays, EXIINT, the EXT bit; a blank card formatted as the SDK's CARDFormat would, its serial from this console's SRAM, the SRAM's flash ID taken back from the card; the flash is the front end's: `cardInsert`, Dolphin's `.raw` layout, `card[s].dirty` after a write); `gc_boot.cpp` the IPL's state, `.dol` loading, discs booted by running their apploader on the CPU. The **sound out**: the audio DMA's 32-byte blocks (8 frames, right channel first, big-endian; 32 or 48 kHz by AICR's AIDFR, 32 kHz as the IPL leaves it) resampled to the front end's rate (`setAudioRate`) into a ring that `audioRead` empties. `runFrame` runs **one field** (the lines 1..313 / 314..625 in PAL): a front end calling it 50 / 60 times a second plays in real time (it ran a whole frame before: twice too fast). `gcemu` reads a disc image on demand (`kapi_seek`, the main thread serving the app core). The **write-gather pipe** (0x0C008000) keeps its bytes (`gather`) and sends them to the GX FIFO 32 at a time, as the hardware (`gatherFlush`: the burst at the PI's write pointer, then the commands run); the JIT appends a store to it inline (a call only at each burst). **The JIT** (`gc_jit.cpp`, AArch64 hosts): `Machine::jitEnable()` (code memory from the host's `gc::codeAlloc` hook: `kapi_code_alloc` (ABI v58) on Onyx, `mmap` RWX in the host test), then `run()` goes through `jitRun()`: the PowerPC code

is translated a **block** at a time (to its branch, ≤ 64 instructions, within a 4 KB page) by a small built-in AArch64 assembler. The guest registers live in the `Machine` (x19 = it, x20 = MEM1, x21 = the helpers + a 64K-entry direct-mapped table of blocks, w22 = MEM1's size); **within a block** the GPRs, CR, XER, LR and CTR it uses are cached in host registers (x9–x15, x18: loaded at first use, the dirty ones written back at the exits, before the interpreter, and around a slow-path call); **LR, CR, r3 and r0** — the most used — **live in x24, x27, x28, x29 across the blocks** (`SRA_G` / `SRA_H`: callee-saved, the helpers keep them; loaded by the enter stub, stored by the exit stub and around the interpreter's calls, `lmw` loading them straight: no load at a block's first use, no store at its exits); the integer unit, CR logic, rotates, shifts, compares, divw / divwu, the branches (CTR / CR conditions, LR), mftb, the loads / stores, lmw / stmw and dcbz (MEM1 in one go) are native (a load / store whose address maps MEM1 through the OS's standard BATs, or in real mode, reads the host memory directly and byte-swaps -- with the BATs, the 0x80000000 mirror checked by one EOR and a compare, the value loaded straight into the destination's register or stored from the source's, the uncached 0xC0000000 one tried first on the cold path; a D-form access through a base register the translation finds in MEM1 (and not yet written in the block) makes that register's host pointer (`basePtr`: checked there -- not MEM1's then: the instruction by the interpreter, and out of the block) and the block's later accesses through it are one load / store with the displacement (`ptrAccess`; MEM1 has a guard zone of `MEM1_GUARD` bytes on each side for the displacements past its ends); the rest calls `read32`… with the cycle count exact), every other instruction (the FPU, the paired singles for now) calls the interpreter's `exec`. The FPU and the paired singles are native too, on NEON: `ps[32][2]` keeps both halves of an FPR side by side, so an FPR is one q register, cached like the GPRs (q8–q31, lane 0 = ps0); the paired singles compute both halves at once (FADD / FMLA .2D, by-element for muls0 / madds1, FCVTN + FCVTL for the single rounding, ZIP / EXT for the merges, FCMGE + BSL for ps_sel); the plain FPU works on lane 0 (fadd… fmadd, fres / frsqrte (ps_res / ps_rsqrte on both), the 25-bit multiplicand — skipped when the operand is known to hold a single exactly —, fsel, fcmp, frsp, fctiwz, the moves), lfs / lfd / stfs / stfd, psq_l / psq_st and their indexed forms, **specialised for the GQR's value found at translation** (a float type: one 8-byte access for the pair; u8 / u16 / s8 / s16 with a scale: both elements in one access, converted and scaled, or rounded to single, scaled, truncated and clamped), the GQR checked at run time (another value: the interpreter); a NaN result re-runs the instruction in the interpreter (from the cache's state at that branch), FPRF is set lazily (the source FPR noted at translation, `fprfVal` / `fprfPending` written only when needed). The cycles are a countdown in x26 (to `jitEnd`, = `jitUntil` when set; written to `cycles` when leaving and before a helper that reads them, resynced after one that may change `jitUntil`). The rare paths (a slow memory access, a NaN, a conversion's odd case, the interpreter leaving, an exit's stub) are emitted apart, each with the cache's state at its branch: the code buffer is in 1 MB chunks, the blocks' main code from a chunk's start up, their rare paths from its end down (a b.cond's reach) -- the main code dense in the I-cache (`--pmu=08,02,05,01` counts the TLB and L1I refills: the L1I ones were ~20 a thousand instructions, 17.5 with the split; the TLBs, with Onyx's 64 KB pages, ~0.3). In The Wind Waker's game (Outset, ~96k vertices a frame) `--pmu=08,01,52,53` gave 11.4 L1I refills and 3.2 L2 read refills a thousand instructions (0.1 write ones) — about as many as the L1D refills: the machine's data comes from the RAM, its IPC ~0.84; with `--nodraw` 37 → 30 M cycles a field for the same instructions (IPC 1.02): the display's copies on core 0 (through the L2 the four cores share) cost it ~18 %. A block's exit to a known address is **linked**: once that block is translated the exit's branch is patched to jump straight to it (back to its stub when that block is dropped), while `cycles < jitUntil` (the next event / the decrementer; `piUpdate` and `decWrite` zero it to come back); an exit to a register (blr, bctr) goes through the table; `jitRun` (C) takes the interrupts, translates what is missing. **Calls and returns are the host's own**: a call (`bl`, `bcctrl`)

pushes a pair {its landing, the guest's return address} on the host's stack (`stp x1, x24, [sp, #-16]!`) and calls the target's block with `bl` (linked like an exit) / `blr` (`bcctrl`: the block looked up in the table there); a return (`blr`, `bclr`) pops the pair and, LR being its address, is a `ret` to the landing — which the core's return stack predicts —, else the stack is emptied and the dispatcher goes on (`retTo`, `callTo`, `callReg`); the landing, in the caller's block, is a linked exit to the instruction after the call (the host registers are the callee's there: the cache dropped). The stack holds a sentinel at `jitSpBase` (no LR is 1); it is emptied when the JIT leaves (the exit stub), at a mismatch, and past 8 KB (`jitSpLimit`: calls that never return, a return through `bctr`); a landing stays valid while the JIT runs (a dropped block's code stays in the buffer until `flushAll`, which runs from `jitRun` only). `bl $+4` (the pc read) pushes nothing. The Wind Waker: the branch mispredictions 5.6 -> 2.0 a thousand instructions, 37 -> 42 fields/s (its hottest loop, a list searched through two function pointers, went through the dispatcher four times a node). `gctest calltest` checks it against the interpreter. Blocks are keyed by address + MSR IR / DR; `icbi`, the DVD / ARAM / locked-cache DMAs drop the blocks of the 4 KB pages written; a BAT change, HID0's ICFI or a reset drop everything. `b .` jumps to the next event (idle), and so does a **polling loop** (a block of loads, compares, masks branching back to its start, each register it reads set earlier in the same pass or not by it at all: nothing changes until an interrupt / the hardware — the VBlank waits, the DSP's mail). A compare (or a record form) right before a conditional branch on its field's LT / GT / EQ: the branch tests the host's flags, and the CR field is made only on the paths where it may still be read (`crSet`, `crDead`: the guest code of the block's 4 KB page scanned from each path -- set again before a read: not made; a call (`bl`) sets CR0 again as far as the caller knows — the ABI's volatile field: the callee may set it, the caller does not read it after without setting it again —; a return, leaving the page, a CR logic op on it: made) -- an exception that does not come back then leaves the older field, which only a crash report shows. `gcemu`: Game ▸ *Interpreter (no JIT)* / `--interp` to compare. Tested by `run_gc_test.sh` under `qemu-aarch64` (`GC_JIT=1`): cputest / pstest / hwtest / gxtest identical to the interpreter's, `gctest fuzz` (random sequences of FPU / paired-single / load-store / integer / branch instructions from random states with NaNs, infinities, denormals, run by both, every register, the FPSCR and the memory compared: it found a cache bug and an interpreter bug — `srawi` / `sraw` with rA = rS took CA from the result; `GC_FUZZC=1` their data through the uncached mirror, `GC_FUZZSTART=n` from the n-th, `GC_FUZZDUMP=<f>` each one's code, `GC_FUZZPROG=<f>` + `GC_FUZZLEN=n` a failing one cut down, `GC_FUZZG="3 11"` those GPRs, CR and XER printed, `GC_FUZZBLOCK=<pc>` the block there's code; `gctest calltest`: a program of calls built there (fib, a chain of 1500 calls, calls through a table, a tail call, `bl $+4`, a return to LR + 4, returns through `bctr`) run by both, twice (the blocks kept the second time), every register and the stack compared; `GC_FUZZ2=1` each one run a second time with the JIT's blocks kept and the data through the other mirror (the blocks' base pointers meet another value); the CR is not compared after an exception that stopped the run -- see `crDead`), `tools/tests/gc/bench.c` (sort, CRC, copies, calls) against qemu-ppc's result: ~40× the interpreter's speed (integer), ~10× (float); `GC_PROFILE=1` prints the host instructions per guest instruction, `GC_DUMP=prefix` the hottest blocks' code (for `aarch64-linux-gnu-objdump -b binary -m aarch64`); `gctest jitsize <jitprof.bin>` translates again the hot blocks of a Pi's profile (gcemu `--jitprof`) and prints their host instructions a guest one weighted by the runs counted there (6.56 → 5.85 with the base pointers, the hot / cold chunks and the kept registers), `GC_JITDUMP=<pc>` one's code. **The x86-64 JIT** (`gc_jit_x64.cpp`, NintendoEMU / x86-64 Linux hosts; `gc_jit.cpp` is the AArch64 one): the same design (blocks, links, the dispatcher's table, idle loops, the rare paths after the block, the cycles as a countdown) with a small x86-64 assembler (REX / ModRM / SIB, SSE2, VEX for FMA3): the guest GPRs cached in rbx, rbp, rsi, rdi, r12, r10, r11, the FPRs in xmm6–15 (a paired single = one xmm, both halves), the constants in the code buffer

(RIP-relative); the helpers take only the `Machine *` (their arguments in `jitArg[]`: the Windows and System V ABIs alike). The same fuzz test (`gctest fuzz`) checks it against the interpreter. Two fixes found with it and shared: HID0's ICFI / DCFI read back 0 (a game setting ICFI flushed every block), and each AI sample is an event while the AI plays (`__AI_SRC_INIT` polls its counter). **The GX on a GPU with shaders** (`gc_gxgpu.cpp`): a front end sets `Machine::gpu` (a `GxGpu`: `draw`, `copy`); each primitive then reaches it as the game gave it — its vertices in model space with their matrix indices (`GxVertex`), its triangles / lines / points as indices — with a `GxState`: the XF's state (colour channels, texgens, projection) and the BP's (the 16 TEV stages, swap tables, konst colours, alpha test, z texture, fog, indirect texturing, depth / blending / culling / scissor, the maps' textures) laid out as a std140 uniform block, rebuilt when a register changed (`gxsDirty`; the XF memory — matrices, lights — has its own `xfSerial`). The EFB copies go to it too: to the XFB (the picture), or to a texture — kept by the GPU, not written to MEM1: a slot (`efbCopies`) remembers the address and a hash of MEM1 there, so a texture read from that address (MEM1 not written since) is the copy. NintendoEMU's backend is `pc/NintendoEMU/core/gxgl.cpp` (below); without a backend (the Pi today) nothing changes. **Host test** (`sh tools/tests/run_gc_test.sh`, needs gcc-powerpc-linux-gnu + qemu-user): `tools/tests/gc/cputest.c` compiled once runs under `qemu-ppc -cpu 750` and in the interpreter (10485 result words identical), `pstest.S` the paired singles against the manual, `hwtest.c` and `gxtest.c` bare-metal programs (built with `tools/gc/elf2dol.py`): the VI's picture and interrupts, the FIFO / PE, a textured quad and a shaded triangle drawn by the GX path. **Nintendo DS** (`user/Emulators/nds/`, `nds/libnds.a`; the app `ndsemu`): `nds::Machine` — `load` a `.nds` (the ROM stays the caller's), `runFrame` (up to the next VBlank: 263 lines of 4260 master cycles, 59.83 Hz) → `screen[0]` (top) / `screen[1]` (bottom), 256 × 192 each, `setButtons (nds::BTN_*)`, `setTouch (down, x, y)`, `setAudioRate` / `audioRead`, `setSaveData` / `save` / `saveSize` / `saveDirty`, `setTime`, `setUser` (the firmware's name and language), `setBios7` (optional: only its KEY1 table, for a dump whose secure area is still encrypted). Integer only, no libc, no allocation once made (it runs on an app core). `nds_cpu.cpp` the two interpreters: the ARM946E-S (ARMv5TE: BLX, CLZ, the saturating and the 16-bit multiplies, LDRD / STRD, the CP15 with its ITCM / DTCM) and the ARM7TDMI, in turns of 256 master cycles (the ARM7 at half the clock); `nds_mem.cpp` the maps, the VRAM banks A–I by VRAMCNT (16 KB pages: the BG / OBJ / LCDC / ARM7 views, the texture and extended-palette slots) and the **page tables** both the interpreter and the JIT read and write through (`Machine::rdPage` / `wrPage`, updated by range when WRAMCNT, VRAMCNT or the TCMs change); `nds_io.cpp` the interrupts, timers (cascade), DMA (immediate, V/H-blank, the card, the GX FIFO, main-memory display), the IPC (sync, FIFOs), the divider / square root, the ARM7's SPI (power manager, a made-up firmware — its user settings with their CRC —, the touch screen's TSC2046) and the RTC; `nds_cart.cpp` the card's commands after the boot (B7 reads with the 4 KB wrap, B8 the chip ID), the save chip on the AUXSPI — EEPROM 512 B / EEPROM-FRAM / Flash **found from the save file's size, else from the game's first write** (a whole page of one of them), in a buffer made once (8 MB at most) —, the secure area's KEY1 decryption; `nds_bios.cpp` the **direct boot** (the header's two programs where the BIOS puts them, the memory it leaves at 0x027FF800 / 0x027FFC00 / 0x027FFE00, the processors' registers) and the BIOS calls in C++ (IntrWait with the DTCM + 0x3FF8 flags, Div, Sqrt, CpuSet, CRC16, BitUnPack, the decompressions — those reading through the game's callbacks run them, `callGuest` —, the ARM7's sound tables `nds_tables.inc`); the interrupt entries are a few ARM words of ours at 0xFFFF0000 / 0 (as the real ones: they call DTCM + 0x3FFC / 0x0380FFFC). `nds_gpu2d.cpp` the two 2D engines a line at a time (text, affine, extended — 16-bit map, 256-colour and direct bitmaps —, large BG, the extended palettes, the sprites 1D / 2D / 256 colours / affine / bitmap, windows, blending with the 3D layer's and the bitmap sprites' own alpha, mosaic, master brightness), the display modes (VRAM, main memory) and the **display capture** (into an LCDC bank). `nds_gpu3d.cpp` the geometry: the

commands packed through GXFIFO or one per port, the four matrix stacks, lighting, the texture-coordinate transforms, the polygons (strips, culling, clipping to the view volume, the viewport) into the polygon RAM, the box / position / vector tests, the swap at the VBlank (the commands after a SWAP_BUFFERS wait for it); `nds_render3d.cpp` the rasterizer in software (the perspective, the seven texture formats, modulation / decal / toon / highlight, alpha test and blending, shadow polygons, edge marking, fog, the rear-plane bitmap), started at the VBlank — on **another app core** when the app gives `render3dKick` / `render3dWait` (ndsemu does), else inline. `nds_spu.cpp` the 16 channels (PCM8 / PCM16 / IMA-ADPCM with its loop state, the PSG squares, the noise), the mixer and the capture units, at 32728 Hz resampled to the host's rate. **The JIT** (`nds_jit.cpp`, AArch64 hosts; `Machine::jitEnable (codeAlloc)` with `kapi_code_alloc`'s memory, `useJit`): a block from an address to its branch out (48 instructions, within 1 KB); a **Thumb instruction is rewritten as the ARM one** that does the same (its PC read as Thumb's), so one translator serves both — BL pairs, the branches and the PC-relative loads apart. x19 = the processor (`Arm`), up to 9 guest registers the block uses most in w20–w28 (loaded at its start, stored at its exits); **the guest's NZCV is the host's** (the same meaning, carry included), a backward liveness pass skipping the flags nothing reads, the logical ops' C / V kept or made from the shifter (register shifts too: flag-free 64-bit shifts). Loads and stores read the page table inline; a missing page goes to the memory functions through a stub after the block that keeps the host flags; **the pages holding translated code have no write pointer**, so a write there takes the functions, which drop **exactly the blocks whose bytes were written** (a chunk list per KB) and stop the running block (`Arm::jitExit`, also set by an interrupt). The exits **chain** to the next block through a direct-mapped table (address | Thumb → code) read by the block itself while time is left and nothing asks to stop; else back to `jitRun` (C), which takes the interrupts and the halts. MSR / MRS, the coprocessor, SWI, the DSP multiplies, LDRD / STRD, LDM with ^ run in the interpreter (`execArm` / `execThumb`) and end the block. **Host tests** (`sh tools/tests/run_nds_test.sh`): `tools/tests/nds/ndstest.cpp <rom> <frames> [out.ppm]` runs a ROM headless (`NDS_KEYS`, `NDS_TOUCH`, `NDS_SAV`, `NDS_WAV`, `NDS_SHOTS`, `NDS_DUMP=<addr>,<words>`, `NDS_JIT=1` on an AArch64 host — or under `qemu-aarch64` with `aarch64-linux-gnu-g++ -static`; built with `-DNDS_DEBUG`, `NDS_WATCH9` / `NDS_WATCH7=<lo>,<hi>` stop when a processor leaves a range and print its last 4096 addresses). The test programs are ours (`tools/tests/nds/src`, built by `build.sh` with `arm-none-eabi-gcc`; `check.sh` runs them): **cputest** (C kernels — CRC, 64-bit arithmetic, division, shifts, sorting, struct copies, switch tables, fixed point — run on the ARM9 and the ARM7 in ARM and in Thumb must give the PC's own results; ARMv5 checks in assembly), **systest** (IRQs through the BIOS, IntrWait, timers, DMA, divider, IPC both ways, the card polled and by DMA, the chip ID, a Flash page saved and read back, the BIOS calls; the firmware, the touch screen, the clock and the keys read by the ARM7), **gfx2d** / **gfx3d** (pixels probed: bitmaps, sprites, a lit quad, the capture) and **sound** (440 Hz PSG on the left, a 1 kHz PCM loop on the right). **`jitfuzz`** (`tools/tests/nds/jitfuzz.cpp`, under qemu-aarch64) runs random ARM / Thumb sequences — every data-processing form and condition, the multiplies, loads / stores, block transfers — through the interpreter and the JIT from the same state and compares the registers, the flags and the memory; a mismatch is shrunk to its fewest instructions (`JITDUMP=<file>` writes its host code for `aarch64-linux-gnu-objdump -b binary -m aarch64`). **Doom** (`user/Ports/doom/`): doomgeneric (`third_party/doomgeneric`, GPL-2.0, only the portable sources; `ONYX.md` lists the three `#ifdef ONYX` changes) built against **newlib** like the `/bin` libc tools (`../libc/crt0libc.S` + `onyx_syscalls.c`, `main (void)` + `kapi_get_args`), by `user/Ports/doom/Makefile` (called from `user/Makefile`) into `user/doom.elf`. `doom_onyx.c`: the `DG_*` platform functions — the window canvas *is* `DG_ScreenBuffer` (640 × 400, no copy), full screen at 4:3, keys from `kapi_key_held` + modifiers + key events (Tab, F-keys…), `gamepad.h`, `rename`/`mkdir` on the kapi; `doom_uikit.cpp`:

window chrome + menu (uikit from C); **The engine runs on an app core** when one is free: `main` starts `doomgeneric_Tick` there after `doomgeneric_Create`; the main thread keeps the window, the input (a key queue), the pictures (a triple buffer: the engine swaps `DG_ScreenBuffer` between three slots), the sound output and the music. The engine's syscalls (malloc's `sbrk`, stdio, files, saves, `exit`) run on the main thread through the **libc RPC** (`libc/onyx_syscalls.c`: `onyx_rpc_enable`, `onyx_rpc3`, `onyx_rpc_serve` — any newlib program can use it for code on an app core; the main thread must then not use malloc meanwhile); `doom_sound.c`: `DG_sound_module` (16-channel mixer of the DS* lumps, run by the engine into a PCM ring that the main thread moves into `kapi_sound_write`) and `DG_music_module` (MUS and MIDI turned into one timed event list, played on the 16 kernel FM voices with the GENMIDI OPL patches converted to `kapi_fm_instrument`). **Host test**: `sh tools/tests/run_doom_test.sh` (headless on a virtual clock: frames, sound effects, FM notes). **Graphing calculator expressions** (`user/Apps/graphcalc/expr.h`): `gc::Parser::compile (src, program)` → an RPN `gc::Program` (`eval (x)`), `gc::fmt`; the maths of the BASIC core (`basic/basnum.h`). An app computing in `double` builds with FP: in `user/Makefile`, `graphcalc.elf: CXXFLAGS := $(CXXFLAGS_FP)`. Host test: `sh tools/tests/run_graphcalc_test.sh`. **HTTPS from a uikit app**: uikit apps are freestanding; do the TLS work in a newlib console tool and spawn it with pipes (`kapi_pipe`, `kapi_spawn`, write the request, `kapi_stream_eof`, poll `kapi_stream_read_nb` / `kapi_proc_done` from `Root::onTick`). Example: Lisa + `/bin/groq` (`user/Apps/lisa`, `user/BinUtils/groq.cpp`). **Wi-Fi scan (ABI v45)**: `kapi_wlan_scan (ap, max)` fills `struct kapi_wlan_ap` entries (ssid, bssid, security `WLAN_SEC_*`, channel, freq, level dBm, connected), strongest first; it blocks ~3 s. Examples: `user/BinUtils/wifiscan.c`, `wpaconf` (Scan button + Combobox). **WPF-style controls (P5)** — all in `uikit/uikit.h`, see `user/Apps/widgets` for each in use: `RadioButton (l, t, w, h, text, group, checked, cb)` — exclusive per `group` among its siblings (`uk_radio_checked (parent, group)`); `GroupBox (l, t, w, h, title)` — a titled frame, add controls as its children; `ToggleSwitch (…, text, on, cb)`; `NumericUpDown (…, min, max, value, step, cb)` — arrows, wheel, Up/Down, typed digits; `ListBox (…, onSelect, onActivate)` — `add`, `clear`, `item (i)`, `sel`, `setSel`; **`DataGrid (l, t, w, h)`** (`uikit/datagrid.h`) — a read-only table of rows and columns, virtual (a grid of any length costs what it shows): `setColumns (n)`, `setColumn (c, title, width, GRID_LEFT / GRID_RIGHT / GRID_CENTRE)`, `column (c)`, `autoSize (c, minW, maxW)` (the title's and the first rows' texts), `setRows (n)`, `sel`, `setSel (r)`, `ensureVisible`, `rowAt`; the app gives each cell's text (`cellText (grid, row, col, buf, cap)`) or draws it itself (`cellDraw (grid, cv, row, col, x, y, w, h, ink, selected)`: `cv` clipped to the cell; false → the text is drawn); a click on a title fires `onSort` (`clickedCol`: the app orders its rows and sets `sortCol` / `sortDesc`, the grid shows the arrow; `sortable = false`: plain titles), a title's right edge drags the column's width; `onSelect`, `onActivate` (a double click, Enter), `onContext` (a right click: `ctxRow` — -1 below the rows —, `ctxX`, `ctxY`); stripes, the row under the pointer tinted, both scroll bars (Shift + the wheel, Left / Right: sideways), `emptyText`, `user` (the app's). Used by Cardfile (its list, its design's fields); `TreeView (…, onSelect, onActivate)` — `add (parent, label)` → id, `expand`, `sel`, `label (id)`, `setUserData`; `Calendar (l, t, y, m, d, cb)` (size `CAL_W`×`CAL_H`) and `DatePicker (…, y, m, d, cb)` (`format (buf)` → `YYYY-MM-DD`); `ImageBox (…, IMG_FIT / IMG_FILL / IMG_NONE)` — `load (path)` (any `imgload` format) or `setPixels`; `uk_color_dialog (&color, title)` — RGB sliders + palette + preview, true = OK. `uk_file_open` / `uk_file_save` / **`uk_folder_open (out, cap, startDir)`** (a folder, no file name) — the file dialog (docs/04, *The file dialog*); its left column lists the **volumes** that are mounted (`SD:`, `SD1:` … `SD3:` — the SD card's partitions, `USB1:`, `USB1P2:`…). `startDir` may be a **file's** path (the program's last document): the dialog opens in its folder, and a save dialog proposes its name. Paths may start with any volume (`SD1:/roms/x.iso`); `kapi_fsize` is clamped to 4 GB − 1, **`kapi_fsize64`** (ABI v59) gives an exFAT file's real size; `kapi_rename` fails across volumes

(copy + remove instead). A `Textbox` holds **63 bytes** unless you raise its `maxLen` (up to `Textbox::TEXT_CAP - 1`, 511): IRC's chat line takes 400. Its `cb` fires on Enter (without one, Enter goes on to the dialog: its OK); `changed` after each edit (typed, pasted: a dialog's other field following it). **Tab** (`Widget::tabFocus`, done by `handleKey` when no widget takes the key): in a **dialog** (a `Modal`) Tab / Shift+Tab move the focus to the next / previous control (in the order they were added: `canFocus`, not hidden nor disabled); in a **window**, from a **text field** to the next text field (`isField ()`: `Textbox`, `Combobox`). `Modal::run` focuses the first text field when nothing has the focus. A focused `Button` takes Space and Enter; a `Checkbox` / `RadioButton` Space (Enter is the dialog's). A widget that wants Tab for itself (a text area, a terminal) takes it in its `onKey`. `Combobox (l, t, w, h, text, onEnter, onPick)` — an editable `Textbox` with a drop-down list of suggestions (`addOption`, `clearOptions`, `pick (i)`; arrow click or Down / Up; `onPick` fires with `picked` = the index). Used by the File Viewer's Connect dialog. `Dropdown (l, t, w, h, options, n, initial, cb)` — the non-editable sibling, same look (field + drop button): `sel`, `setOptions`; Up / Down change the selection, Enter / Space open the list, Esc closes it; `cb` fires when the selection changes. The option strings are not copied (they must outlive the widget). **Keys with modifiers**: Ctrl / Shift + arrows, Home, End, Page Up / Down arrive as the plain `KEY_*` code; test `kapi_get_modifiers () & MOD_CTRL` / `MOD_SHIFT` (e.g. the FM Tracker's Ctrl+Up / Down transpose, the text widgets' Shift selection). Inside a key handler, `kapi_get_modifiers` returns the modifiers held **when that key was typed** (the kernel stores them in the key event, taken from the xterm `ESC[1;<m>X` form, which `vncd` also sends), not the live state. F1–F12 arrive as `KEY_F1` .. `KEY_F12` (0x110..0x11B). Text widgets (`Textbox`, `Textarea`, `RichTextBox`) accept the printable Latin-1 range too (`é è à ç ù`… = 0xA0–0xFF, as the keymaps produce them). The euro sign (AltGr+E, …) arrives as **0x80**, Windows-1252's code for it (Latin-1 has none); uikit's font draws it there (`tools/fonts/gen_nssans.py`, `EXTRA`); the text widgets do not take it yet — the Spreadsheet does (`latin1_cp` → U+20AC). With a **text face** installed (below) `Textbox` and `Textarea` hold **UTF-8** instead: a typed Latin-1 key is stored as its UTF-8 (0x80 as U+20AC, the euro), the caret moves and deletes whole characters. **Text selection**: `Textarea` and `RichTextBox` select with Shift + navigation keys, a mouse drag, Shift+click and ^A; typing replaces the selection. `Textarea` has `hasSelection`, `selStart` / `selEnd`, `selectedText`, `deleteSelection`, `selectAll`, `copy` / `cut` / `paste` (system clipboard; ^C / ^X / ^V work by themselves when the app's menu does not take them). **The pointer's shape** (kapi v81): a widget calls `uk_cursor (KAPI_CURSOR_…)` from its `onMouse`, each time the pointer moves over it — `_TEXT` over text, `_HAND` over a link, `_SIZE_H` / `_SIZE_V` on an edge that drags, `_MOVE`, `_CELL`, `_CROSSHAIR`, `_WAIT`, `_NO` (`uikit/widget.h`). `Root` starts every pointer event with the arrow and tells the kernel when what was asked has changed: a widget that asks nothing shows the arrow, and nothing has to be put back. A widget that keeps the pointer during a drag (`catchOutside`) asks at each move of the drag. Already done by `Textbox`, `Textarea`, `RichTextBox` (the I bar), a `Splitter`'s grip and a `DataGrid`'s column edges (the two arrows). **Tooltips**: set `widget->tip = "text"`; the `Root` shows it after the pointer rests ~0.6 s. (No RTTI: `Widget::asRadio ()` identifies radio buttons.) `uikit::Root::onTick ()` (virtual) runs once per event-loop iteration — poll a mailbox, a spawned process or a timer there. (The `ask` window and `ask.h`, which only the Shelf used, were removed with it on 2026-10-05: a question is a uikit dialog.)

The output (kapi v84). An app never chooses a device: it plays into the one producer (44.1 kHz stereo), and the kernel's output — the jack, a USB audio device or HDMI — adapts (the rate, the sample format, the volume). `kapi_sound_output (-1)` says where the sound plays and which outputs are there (`KAPI_SND_OUT_NOW` / `_ASKED` / `_HAS`), `kapi_sound_output (KAPI_SND_OUT_USB)` chooses one; `volume_set_output` (`volume.h`) also keeps it in

`SD:/etc/sound.ini`. That is the Sound applet's and `/bin/volume`'s business: an app has no reason to call it. `sound_status`' rate stays 44100 whatever the output.

An app in another language (`uikit/lang.h`). The sources keep their English words, wrapped: **`TR ("Save")`** is the word in the system's language, else the English itself (a `const char *` valid for the app's life: it may be kept); **`TRC ("status", "Open")`** looks up `status|Open` first, for a word whose translation depends on where it stands. The catalogues are UTF-8 text, a line a word — `English<TAB>translation` (`\t \n \\` escaped, `#` a comment) —: **`SD:/res/lang/<code>.txt`** for uikit's own words (the dialogs' buttons, the file dialog, the months and days of `Calendar`, the window menu, Cardfile's editors), **`SD:/apps/<app>.app/lang/<code>.txt`** for the app's. The language is **the system's**: `SD:/etc/system.ini`'s `language=` (`en`, `fr`; none: English), chosen in the Control Panel's Language & Region applet (`langconf`) and in Setup, read through SystemKit (**`systemkit/locale.h`**: `locale_language ()`, `locale_set_language (code)`, the list `locale_language_count / _code / _name`; the time zones too, `locale_zone_*`). **`uk_lang_init ()`** first thing in `main` (after a text face is installed, if any) loads it; an app has **no language switch of its own** (but a port to another system: the Mac's Ledger keeps `SD:/apps/<app>.app/lang.txt` and its EN | FR, `#ifdef __APPLE__`). A program takes the language when it starts (the widgets are made with their words). An app drawing with the bitmap fonts gets the words converted to Latin-1 at load (as its text is drawn: one byte a glyph; the euro `0x80`), one with a face keeps UTF-8. A word missing from a catalogue stays English. A word kept in a table of English words and given to `TR (T[i])` where it is drawn is marked **`TRN ("word")`** in the table (the word itself), or listed in a comment `// TR: word` (a word that comes from a file or a kit). **`python tools/lang/check.py <app>`** (or `--all`) lists the words of the sources that a catalogue has not, and the catalogue's lines no source uses; `--keys <app>` prints an app's words to start a catalogue. **Every new app is translated** (English + French) **from its first version**; the older ones are translated as they are worked on -- done: Setup, the Control Panel and its applets, the Terminal, the menu bar (its own words; the categories of `app.txt` through `TR` where drawn, listed in `// TR:` comments), the Clock, 3DForge (its Windows version too), Ledger, Notes and Stickies (the dates' names in the catalogue, the model's English short dates translated where drawn: `notelist.h`'s `date_tr`), Turtle Quest (its own `L2 (en, fr)` pairs and its levels' `.fr` texts; `locale_language ()` at its start), Circuits (`TR` + `lang/fr.txt`, its packs' `.fr` keys), Pinball (`TR`, the reader's reasons as `// TR:` formats, its tables' `.fr` keys), Critters (the same; its levels' `.fr` keys, a no-break space before the French `: ? !`). `SHOTS_LANG=fr SHOTS_PNG=<folder> sh tools/tests/desktop_sim/shots.sh <app>` renders an app in French on the PC to check that the words fit. Wrap only what is shown — never a file's keys, paths, XML or a string the code compares.

Text faces — anti-aliased, proportional text in every widget (`uikit/text.h`). By default uikit draws with its bitmap fonts (8 × 16 cells); an app may install a **`TextFace`** instead and every text path of uikit goes through it: `Canvas::text`, `uk_text_l` / `uk_text_c` / `uk_text_w`, the line height widgets lay out with (**`uk_fh ()`** = the face's `height ()`; `uk_fw ()` = a digit's width), and the widths they compute — a `NumericUpDown`'s right alignment, tooltips, `Icon` labels, the `DataGrid`'s cells (cut with "..." by measure), `Calendar`, `PopupMenu`, the text boxes' carets. `Textbox` and `Textarea` place the caret, the clicks and the selection by the glyphs' real widths (a `Textarea` then scrolls sideways by pixels: `leftPx`). Without a face, nothing changes — byte for byte (the screenshots stay identical). Not through the face: the window's **frame** — its title has a face of its own, the same in every window, FreeType app or not: **`SD:/res/fonts/title.aaf`**, DejaVu Sans Bold at 13 px rendered ahead of time by the apps' own FreeType and gamma (`sh`

`tools/title_font/build.sh`: `gen_title_font.cpp` documents the format — Latin-1, anti-aliased bitmaps, kerning pairs), read once by `uikit/skin.cpp`'s `AafFace` (without the file: the bitmap font) —, `RichTextBox` (its own styled bitmap glyphs), and the explicit bitmap calls `Canvas::drawFont` / `uikit::draw_text`.

- The interface: `struct TextFace { virtual int height (); virtual int ascent (); virtual int width (const char *utf8, int style); virtual void draw (Canvas &cv, int x, int yTop, const char *utf8, unsigned color, int style); virtual int widthN (utf8, n, style); }` — styles 0 regular, 1 italic, 2 bold, 3 bold italic; text in UTF-8 (a stray byte is read as Latin-1); `draw` blends over the canvas, the line's top at `yTop`. **`uk_set_textface (f)`** installs it (0: back to the bitmap fonts), `uk_textface ()` returns it. Install it **before building the widgets** (some size themselves from `uk_fh ()` when made: a `DataGrid`'s rows, the dialogs).
- Measure and draw through it (they fall back to the bitmap fonts): `uk_tw (s, style)`, `uk_tw_n (s, n, style)` (a prefix: a caret's x), `uk_tpos (s, n, x, style)` (the character boundary nearest x: a click), `uk_text (cv, x, y, s, c, style)` (top-left), `uk_text_clip (…, cx, cy, cw, ch)` (clipped to a box), `uk_text_fit (s, w, out, cap)` (cut to w px with "..."), `uk_bfw` / `uk_bfh` (the bitmap cell whatever the face). UTF-8 helpers: `uk_u8_len / _get / _next / _prev / _put`, `uk_u8_key (k, out)` (a typed key's UTF-8). **`UkFaceScope sc (face);`** draws with another face until the end of the scope (a widget's captions, a display's large digits).
- **FreeType's face** (`user/Kits/fontkit/uikitface.h`, header-only, one translation unit; a **newlib** app linking `ft/libft.a`, as Letters — §5.1): **`ft_uikit_install ("DejaVu Sans", 13)`** at the start of `main` (before the `Root` and the widgets) makes an `FtTextFace` on `fontkit/fonts.h` (the card's TrueType families of `SD:/res/fonts` / `SD:/fonts`, anti-aliased, quarter-pixel positioned, kerned, a small width cache; bold / italic from the family's files or made) and installs it; false: no TrueType font (the bitmap fonts stay). DejaVu Sans at 13 px has a 16-px line, as the bitmap font's, so layouts keep their rows. More faces for large or small text: `FtTextFace *f = new FtTextFace; f->open ("DejaVu Sans", 24);` (e.g. an `LcdDisplay`'s `face`). `ft_uikit_face ()`: the installed one. The face holds no `fnt::Font *` between two calls (`fnt::trim` is safe).
- **Every new app uses the FreeType face** (unless told otherwise): add it to **`FT_APPS`** in `user/Makefile` — the newlib + `ft/libft.a` rule the Control Panel, its applets (Theme, Panel, Display, Sound, Keyboard & Mouse, Gamepad, Wi-Fi, App Settings), the Game Library, Setup, the menu bar and the File Viewer share (Paint, Letters, the Calendar... have rules of their own) (`FT_EXTRA_<app>`: libraries of its own) — and to the same list in `tools/tests/desktop_sim/shots.sh`'s `build`. Measure text in pixels (`uk_tw`, `uk_text_fit`), never in characters, and draw it through the face (`uk_text`, `canvas.text`), not `drawFont` (the bitmap fonts only).
- On the PC: **`sh tools/tests/desktop_sim/studio.sh [out]`** builds `gallery/studio.cpp` with the FreeType face and with the bitmap fonts, in the card's theme and in a dark palette, the Widget Showcase under the face (`gallery/ftwrap.cpp`), writes their pictures (default `/tmp/onyx_studio`), and runs `facetest.cpp` (the measures, the carets and clicks, UTF-8 editing).

Studio controls (for Koton's DAW, usable anywhere; drawn from the theme's colours — a light theme and a dark one alike —, anti-aliased with `uikit/vpaint.h`). All in `uikit/uikit.h` but the toolbar: **`#include "uikit/toolbar.h"`** yourself (Letters, the Spreadsheet and Cardfile have their own `ToolBar` / `ToolButton` next to `using namespace uikit`, so `uikit.h` leaves it out). See them in the Widget Showcase (`user/Apps/widgets`, its Studio group) and `gallery/studio.cpp`.

Widget	Make it	What it does
`Knob` (`uikit/knob.h`)	`Knob (l, t, w, h, min, max, value, onChange)`; `setLabel ("Gain")`,	A rotary control: a 270° track, the

Widget	Make it	What it does
	`showValue`, `format (v, out, cap)`, `setDefault (v)`, `bipolar`, `arcColor`, `step`, `face` (captions)	value's arc in the accent (from the start, or from 0 when `bipolar`: a pan), a cap with a pointer, the label and the value under it. Drag up / down (the range in 200 px; Shift: 1000 px), the wheel, a double click → the default; keys Up / Down / Left / Right, Page Up / Down, Home / End, Delete (the default). The dial is the width, less the captions' lines (about 24 … 64 px). `setValue (v, fire)`, `setRange`, `valueText`.
`VuMeter` (`uikit/vumeter.h`)	`VuMeter (l, t, w, h, vertical = true, stereo = true)`; `floorDb` / `topDb` (−48 / +6), `amberDb` / `redDb` (−12 / −3), `segPx` (3; 0 = continuous), `holdTicks`, `fallDb`, `peakFallDb`, `showPeak`, `showClip`	A level meter: segments on a dark well, green → amber → red along a dB scale, the peak held then falling, a clip light (a level over 0 dBFS; a click clears it). **`setQ16 (l, r)`** — linear, 65536 = 0 dBFS —, `setCdb (l, r)` (1/100 dB), `set (float l, float r)` in an FP-enabled unit only (uikit itself is integer-only). Call it at the UI's rate, silence included (the falls are timed by `kapi_get_ticks`); it repaints only when a lit segment or a peak moves. `uk_q16_to_cdb (v)`.
`SegmentedControl` (`uikit/segmented.h`)	`SegmentedControl (l, t, w, h, labels, n, selected, onChange)`; `equalWidths` (false: by the texts), `setLabels`, `setEnabled (i, on)`	Mutually exclusive segments drawn as one pill, the chosen one in the accent (bold). A click, the wheel, Left / Right. `selected`, `select (i, fire)`, `label (i)`, `count ()`. Labels copied (12 × 31 chars).
`TabStrip` (`uikit/tabstrip.h`)	`TabStrip (l, t, w, h, onChange)`; `add (title, data)` → its index, `remove (i)`, `setTitle`, `setMark (i, on)`, `data (i)` / `setData`, `closable`, `newButton`, `activeFace`, `onClose` (`closing` = the tab), `onNew`	A row of tabs over a view (the Terminal's shells): a title, a close cross, a dot (a mark), a "+" after them; the chosen one opens onto the content (`activeFace`). A click picks a tab, the wheel and `selectNext (±1)` step through them (round); the cross or a middle click only **asks** — the program decides and calls `remove`. `selected`, `select (i, fire)`, `count ()`, `tabAt (x)`. 32 tabs, titles copied (47 chars, cut with an ellipsis).
`ToolBar` + **`ToolButton`** (`uikit/toolbar.h`)	`ToolBar (l, t, w, h = 34)`: `add (w, gap)`, `addRight (w, gap)` (anchored right), `sep ()`, `space (px)`, `line`, `bg`;	A strip of small buttons: an icon, a label beside it or alone, a toggle (on: the accent's tint, or `filled` — a play

Widget	Make it	What it does
	`ToolButton (w, h, tip, onClick)` then `->setGlyph (WKT_PLAY)`, `->setIcon (fn, id)` (the app's drawer), `->setText ("Loop")`, `->setToggle (true, on)`, `->setSplit (arrowCb)`, `->fitWidth ()`; `filled`, `raised`, `onColor`, `iconColor`	button), a split arrow (a palette, a menu), a tooltip. Flat until pointed (`raised`: always a face). The icons: `WKT_NEW OPEN SAVE UNDO REDO CUT COPY PASTE PLAY PAUSE STOP RECORD TO_START TO_END REWIND FORWARD LOOP METRONOME PLUS MINUS SEARCH MIXER SPARK GEAR`, drawn at any size by `uk_tool_glyph (cv, kind, x, y, size, ink)`. Generalised from Letters' (which keeps its own).
`LcdDisplay` (`uikit/lcd.h`)	`LcdDisplay (l, t, w, h, text, caption)`; `setText`, `setCaption`, `setSub` (repainted only on a change), `face` / `smallFace`, `scale`, `ink`, `centred`	A time / position display: a sunken well (dark in a dark theme, the accent's pale tint in a light one), large digits in the accent — the `face` given (an `FtTextFace` at 24 px), else the bitmap font scaled as large as fits —, a small caption over a second line at its right ("BAR.BEAT.16" / "0:14.83").

The look: the modernised CDE (kapi v64). Every control, and every window's frame, is drawn by code from a few theme colours — no bitmap (docs/gui-redesign/README.md).

- **The palette** (`uikit/theme.h`): `C_BG` (an app's background: the face), `C_FACE`, `C_FACE_HI`, `C_FACE_DN`, `C_BORDER`, `C_TEXT` (on the face), `C_ACCENT` (focus, selection, checks), `C_DIS`, `C_FIELD` (a text field's, a list's background), `C_FIELD_TEXT`, `C_SEL_TEXT` (on the accent), `C_FRAME_ACTIVE` / `C_FRAME_INACTIVE` (the frames), `C_DOCK`, `C_BUTTON` / `C_BUTTON_TEXT` (a push button's, a drop-down's face: draw buttons with them), `C_MENUBAR`, `UK_OUTLINE`. They are **variables**, read once from `SD:/etc/theme.txt` by `uikit::init ()` (the `Root`'s constructor calls it): use them in drawing code, never copy them into a `static const` or a global initialised at start-up (that runs before the theme is read). The file: `theme` = Peach / Steel / Sage / Brick / Slate / Milk / Dark Coffee (the window in front's frame; `active` = any colour instead), `style` = cde / milk (**the style**, `UK_STYLE`: CDE's framed title buttons, or Milk's — Xfce's Milk theme, as OS X — the title buttons coloured beads, `uk_bead`, close red, minimise amber, maximise green, grey behind or for a button the window cannot use, and the frame melting into the window: the title's gradient from a light tone of the frame's colour down to the content's own, `C_BG`, the borders that colour, nothing between them — `uk_title_strip` likewise, down to `C_FACE`; a colour the file leaves out is the style's own, `uk_style_palette` — or the named theme's when it has colours of its own (`UkNamedTheme::pal`, `uk_theme_palette (i)`: **Dark Coffee**, Milk's dark sister — a dark frame's title goes from the frame's colour down to `C_BG`, its borders are black, a dark face's fields are darker than it and its grooves, `uk_etch_*`, darker too); a named theme chosen: `uk_theme_take (t, i)` (a style alone: `uk_theme_take_style (t, style)`)),
- **A list that draws its own scroll bar** (`uk_thumb`, `uk_draw_vscroll`: `uikit/widget.h`) must also give it the mouse: a `UkBarDrag` member, `if (bar.mouse (mx, my, bl, barX, barW, trackY, trackH, total, view, &pos)) { ...repaint; return true; }` at the top of its `onMouse` — the thumb drags, the groove turns a page, and the row under the bar never gets the click. `inactive`, `window` (the content: the face; `face` still read), `button`, `field`, `menubar` (these three follow the window's colour when absent), `accent`, `outline` = none / dark / black, `dock` — the Control Panel's Theme

applet writes it. As values: `UkTheme` (`UK_AUTO`: derived from the window's), `uk_theme_defaults`, `uk_theme_parse (text, t)`, `uk_theme_get (t)` (the palette in use), `uk_theme_set (t)` (the palette made from it: a preview), `uk_theme_write (t, out, cap)` (theme.txt's text), `uk_theme_reload ()` (a new theme applied: the dock, the menu bar). Text: `C_TEXT` on the face, `C_FIELD_TEXT` on a field, `C_SEL_TEXT` on the accent, `C_BUTTON_TEXT` on a button, `uk_ink_on (bg)` on any colour. Content keeps its own colours (images, games' boards, a terminal's screen); the chrome around it follows the theme.

- **The painter** (`uikit/paint.h`, integer only): `uk_tone (c, level)` (a shade: 128 = `c`, 255 = white, 0 = black), `uk_mix`, `uk_rbox` (a rounded box with a vertical gradient, its corners anti-aliased: per-radius tables), `uk_rline` (its outline), `uk_framed` (the push button: a raised face, the same as a drop-down's box — it had a frame and a well round it, hence its name), `uk_raised`, `uk_sunken` (a field), `uk_etch_h / _v / _box`, `uk_check_mark`, `uk_radio_mark`, `uk_switch_mark`, `uk_scroll_bar`, `uk_slider_mark`, `uk_progress_bar`, `uk_popup` (a floating panel, its corners keyed), `uk_hilite` / `uk_hilite_ink` (a selected row and its text), `uk_title_strip`, `uk_glyph` (`WKG_CHECK`, arrows, chevrons, close, minimise, maximise, restore, lock, gear, power, reload, home, history — a clock…), `uk_text_c` / `uk_text_l` (style 2 = bold). `tools/tests/desktop_sim/gallery/main.cpp` shows every control in every state; `gallery/studio.cpp` (`studio.sh`) the studio controls, with a FreeType face and without.
- **The frame**: `uk_decorate_window ()` (the `Root` calls it; an app drawing its own window calls it after `uk_win_resize`) draws the title bar, the borders, the rounded corners (their outside see-through in the chrome's top byte), the title buttons — the window menu, minimise, maximise, close, at the kernel's `KAPI_FRAME_*` places — and the title in bold, in the active and the inactive frames' colours. Close and minimise are the kernel's; the window menu and maximise come to the app as `GUI_EVENT_WINCTL`, handled by the `Root` (the frame's state: `uk_window_state (UK_WIN_MENU | UK_WIN_RESIZABLE | UK_WIN_MAXIMISED)`, kept by the `Root`; an app drawing its own window without a `Root` gets the window-menu button greyed). An app with its own pointer handler answers `GUI_EVENT_WINCTL` itself: `KAPI_FRAME_MENU` → `root.windowMenu ()`, `KAPI_FRAME_MAXIMISE` → `root.maximise (!root.maximised ())` (the terminal does).
- **Resizable windows**: an app whose layout follows its window's size (anchored widgets, layout containers, or its own `layout ()`) calls **`root.setResizable (true)`**: its maximise button (and a double click on the title) fills the work area — between the menu bar and the dock — and restores it (`Root::maximise`: `uk_win_resize2`, the canvas re-adopted, the frame redrawn), then **`virtual void onResized ()`**. Otherwise the button is greyed. The same call lets the user **drag the frame's edges and corners** (kapi v82: the kernel shows the outline, then `GUI_EVENT_WINRESIZE` → `Root::frameResize`: the canvas at the new size, the window moved, `layout ()`, `onResized ()`). The client area is never dragged smaller than **`root.setMinSize (w, h)`** — without it, half the size the window had when it became resizable (160 x 100 at least): give the size below which your layout breaks. The window menu (its button, top left): Restore / Maximise, Minimise, Move to *workspace* / On All Workspaces (the names: `SD:/etc/dock.ini`'s `desk =` lines), Close (`Root::windowMenu`). **`root.fitWorkArea ()`** (after the children are anchored): a window taller or wider than the work area is shrunk to it (a resizable one; else moved only) and moved into it — the Spreadsheet calls it at its start, so the dock does not cover its bottom.
- **The screen's size changing** (kernel v66, the Control Panel's Display applet): every window gets `GUI_EVENT_DISPLAY_RESIZE`; uikit calls **`virtual void onDisplayResize (int w, int h)`** at once (an app placed by the screen's size — a borderless one: the dock — places itself again there), then ~0.3 s later fits a framed window by itself: maximised, to the new work area again (its restore size kept inside it); else moved into the work area, shrunk if it is resizable and too big. A borderless window is left to `onDisplayResize`. Outside uikit: handle `GUI_EVENT_DISPLAY_RESIZE`

(`GUI_DISPLAY_W/H (value)`) in the pointer handler — the menu bar grows its canvas (`uk_win_resize2`), the notifications move to the new top right.

- **`PopupMenu (x, y)`** (`uikit/dialog.h`): a pop-up menu — `add (label, id, enabled, hint)`, `separator ()`, `run ()` → the id picked, −1 (a click elsewhere, Esc). A context menu.
- **See-through windows** (`WIN_FLAG_ALPHA`, borderless): the canvas's top byte is each pixel's transparency (0 opaque .. 255 see-through; a click on a wholly see-through pixel goes below). Clear to `0xFF000000`, then draw with **`uk_paint_alpha (true)`** so the anti-aliased edges over see-through pixels keep their colour (`uk_blend_px` for single pixels); `0xFE000000` (almost clear) still takes the clicks. Examples: `dock`, `menubar`, `agenda`. (Onyx Remote draws them over the other windows with the same transparency: rdpd sends them in 32 bits.)
- **A fixed window** (`WIN_FLAG_FIXED`, kapi v69): `Root (x, y, w, h, title, WIN_FLAG_FIXED)` — the user cannot move it, its frame has no buttons (uikit's `UK_WIN_FIXED`), and the kernel centres it again when the resolution changes (uikit does not fit it to the work area then). Setup's (`user/Apps/setup`: its pages in `main.cpp`, what it writes in `system.h`); with `kapi_screen_native` (the monitor's EDID size) and `kapi_set_timezone` (v69 too).
- **Present what you draw**: the compositor and the remote desktop (`rdpd`: a window is sent again when its counter changes) see a canvas change at `uk_win_present ()` — an app drawing in its own loop presents after drawing, and only when something changed (`eyes`: when a pupil moves).
- **Workspaces** (kapi v65): a window opens on the current desk; `uk_win_desk (set, count)` shows desk `set` and/or sets how many there are (−1 / 0 keep them) → `KAPI_DESK_CUR (r)`, `KAPI_DESK_COUNT (r)`, `KAPI_DESK_GEN (r)` (bumped at every change: poll it to redraw a pager); `uk_win_to_desk (id, n)` moves window `id` (0: yours) to desk `n` (−1: all; −2: ask). `uk_win_apps` and `uk_win_app_raise` see the current desk only; `uk_win_list` sees all, `KAPI_WIN_OFFDESK` and `KAPI_WIN_DESK (state)` in their state. The dock is the pager (`dockconf.h`: the desks' number and names).
- **Control Panel applets** (`user/Include/applet_proto.h`): any uikit app can be shown **inside** the Control Panel (`apps/control`) instead of in a window of its own. Started with `--applet <surface> <host pid>`, `uikit::Root`'s constructor sees it (**`uk_applet ()`**) and adopts the host's shared surface as its canvas (the pane's size, 700 × 470: lay out for it, or centre on `root.width`); the host copies it into its window at each present and sends the pointer and the keys over the mailboxes. `Root::run ()` does it all; an app with its own loop calls **`uk_pump ()`**, **`uk_present ()`** and **`uk_quit ()`** instead of `pump_events` / `uk_win_present` / `should_exit` (they fall back to those alone). `uk_applet_send (AP_THEME)`: the host restarts the applet (a new theme applied). An applet needs no menu (the host has one) and never calls `uk_win_create`. To list it, add a link file to `SD:/apps/control.app/applets/` (`name`, `icon`, `target`, `text`: the user guide §11) and give its `app.txt` `category = Settings` (the menu bar leaves those out). Examples: `theme`, `dockconf`, `soundconf`, `keyconf`, `config`, `padconf`, `wpaconf`. **Other hosts** (2026-10-03: Mail, showing Web as its HTML view): the host registers an IPC service of its own and adds its name — `--applet <surface> <host pid> <service>` — (the applet ends when that service is gone; without it: `"control"`, `AP_SERVICE`); its own message types (beyond `AP_*`, up to 512 bytes) reach the applet through **`uk_applet_on_message (fn)`** (called from `uk_pump`, the payload NUL-terminated). The surface may be bigger than the area the host shows: the applet shrinks its Root with `setBounds (w, h)` (the surface's stride kept).
- **Jet's web view** (`user/Apps/jet/webview.cpp`, `webview_proto.h`; docs/08 step 5): the browser's one program as an applet — `SD:/apps/jet.app/main --applet <surface> <host pid> <service>` —, the page alone (JavaScript off, a link clicked told to the host, not followed). The host makes the surface once, as big as the work area, and talks to it with: `WV_SIZE` (60, `int w, h`: the page's

size in the surface's top-left), `WV_HTML` (61, a path: an HTML file the host wrote — Mail: `RAM:/mailview-<pid>.html` —, read once), `WV_URL` (62, an address), `WV_COMMAND` (63, `"Copy"`, `"SelectAll"`), `WV_PING` (64: the host's send fails when the view is gone); the view answers `WV_LINK` (70, the URL clicked), `WV_STATUS` (71, the link under the pointer), `WV_LOADED` (72: the page is drawn), `WV_ENDED` (73: the engine did not start or its web process ended: show it your own way), plus the applet protocol's `AP_HELLO` / `AP_PRESENT` / `AP_EXIT`; the host sends `AP_PTR` / `AP_KEY` / `AP_CLOSE`. Mail's side: `user/Apps/mail/webview.h` (the HTML wrapped in a Content-Security-Policy, the `cid:` pictures as `data:` URLs; its own renderer when the program is absent, Mail cannot register its service — the desktop simulator —, or the view fails).

- **Shared settings headers**: `dockconf.h` (the dock's drawers, launchers and workspaces: `SD:/etc/dock.ini`, read / written by the dock and the Panel applet; `DOCK_MSG_RELOAD` to the IPC service `"dock"`), `wallpaper.h` (the wallpaper's modes and their painter: `SD:/etc/wallpaper.ini`, used by `voronoy` and the Theme applet's preview; a pattern: `wp_paint` paints its gradient, `wp_grey_cover` lays the grey picture over the area as "cover" does — box-averaged when it shrinks, bilinear when it grows — and `wp_multiply` multiplies the colours by it; the shipped patterns are made by `tools/gen_wallpapers.py`, `--preview` draws `screenshots/wallpapers.png`), `volume.h` (the master volume, `SD:/etc/sound.ini`).
- `Dropdown`'s list opens **above** its box when it does not fit below and fits better there — every parent clips its children, so a drop-down near a window's bottom (or in a group box) keeps its list visible. Put the controls whose lists must pass a group box's edge in the window (the Theme applet's Desktop box does: `desk_add`).
- **Text on any background**: `uk_ink_for (bg)` is `C_TEXT` on the window's face (and on what is as light, or as dark), else black or white — the ink `Label`s and `Checkbox`es need on a program's own colour (the BASIC runtime's controls; a `Checkbox` picks it by itself).
- **Dialogs**: `uk_file_open` / `uk_file_save` / `uk_folder_open` (a double-click on a file picks it and confirms), `uk_color_dialog` (R / G / B sliders, a palette), `uk_messagebox`.
- **The desktop simulator** (`sh tools/tests/desktop_sim/run.sh`): a uikit app built for the PC against a stand-in kernel (`fakekapi.cpp`: files read from `sdcard/` — what the app saves goes to `/tmp/onyx_sim_writes`, never to the card —, a script of pointer / key / menu events, `dump` writes its window — frame and client — with its transparency), laid over the wallpaper by `compose.py`; `wmtest.cpp` checks the kernel's window manager on the PC. `shots.sh` makes the documentation's screenshots with it (below).

Menus. Put commands in the **system menu bar**, not in button rows. After creating the `Root`, build a `uikit::Menu` once and publish it:

```
static Menu menu;
menu.menu ("File");
menu.item ("Open...", "^O", UK_CTRL ('O'), onOpen);   // label, shortcut text, key, void() ca
llback
menu.separator ();
menu.item ("Save",    "^S", UK_CTRL ('S'), onSave);
menu.publish ();                                       // uk_win_menu_set (ABI v39)
```

The `menubar` app shows the menus while your window is the active app and sends the chosen item back (`GUI_EVENT_MENU`); `uikit::Menu` runs the callback, then invalidates the root. Every key goes through `Menu::shortcut` first (so item shortcuts work anywhere in the app); **Ctrl-Q** quits. Commands and shortcuts are ignored while a modal dialog is open. `^I`, `^H`, `^M` share their key codes with Tab, Backspace and Enter: `Menu::shortcut` only takes them while **Ctrl** is actually held, so the plain key still reaches the focused text box (still, prefer other letters).

Minimal skeleton with the raw window API (a real app uses uikit, above: `uikit::Root`, its widgets and its `run ()` loop; the kernel draws no widget since v29). The window calls are UIKit's `uk_win_*` (`uikit/win.h`, C functions: §5.10.1); a C program links `lib/uikit.imp_c.a`, a C++ one `lib/uikit.imp.a`:

```
#include "appkit/appkit.h"
#include "uikit/win.h"

static void on_key (unsigned long sender, int ev, gui_value value)
{
    (void) sender; (void) ev;
    if ((int) value == 27) kapi_exit (0);              /* Esc */
}

int main (void)
{
    /* The canvas is mapped at 12 GB; fb[y*w + x] = 0x00RRGGBB. */
    unsigned *fb = uk_win_create (300, 200, "example");
    if (fb == 0) return 1;                             /* too big, or no memory */
    for (int i = 0; i < 300 * 200; i++) fb[i] = 0x00E0E0E0;
    uk_win_draw_text (10, 10, "Hello, Onyx", 0x00000000);
    uk_win_on_key (on_key);
    uk_win_present ();
    kapi_wait_for_exit ();   /* pumps the events until the window is closed */
    return 0;
}
```

Key points:

- `uk_win_create(w, h, title)` returns a pointer to the **canvas** (pixel buffer of `0x00RRGGBB`, width `w`). The app draws directly into it (no per-pixel call). The variant `uk_win_create_ex(x, y, w, h, title, flags)` is for explicit placement and `WIN_FLAG_BORDERLESS`. The client area is **at most the screen's size** (frame not counted; 1024 × 768 by default, `width=` / `height=` in `cmdline.txt`; before kernel v66, 1024 × 768 whatever the screen) — keep a window within 1000 × 700 or so (or size it from `kapi_screen_size`, as Paint), as Letters, so that it fits the default screen: over the limit (or out of memory) the call returns **0**. **Check it**: drawing into a null canvas faults and the app is killed (`el0: ... killed` in `kmsg`). (Before kapi v74, when apps ran at EL1, address 0 was the kernel's own memory: the Spreadsheet's first 1060-pixel window overwrote it and froze the whole Pi.) uikit's `Root` checks it: the app stops with `uikit: the window could not be made` in `kmsg`.
- **Widgets** are user-side (uikit); the kernel-drawn ones (`kapi_add_button`…) were removed by the v29 compat break.
- **Event loop**: either `kapi_wait_for_exit()` (blocking, simple), or your own loop `while (!kapi_should_exit()) { ...; kapi_pump_events(); uk_win_present(); kapi_msleep(16); }` when you animate the canvas yourself.
- **App-drawn UI**: to draw text over your canvas, use `uk_win_draw_text(x, y, s, color)` + `kapi_font_width/height()`. Capture the keyboard with `uk_win_on_key(fn)` (`GUI_EVENT_KEY` events, `KEY_*`/ASCII values) and the "outside-widget" clicks with `uk_win_on_click(fn)` (`GUI_EVENT_CANVAS_CLICK` / `..._MOTION`, coordinates encoded in `value`). Cursor position relative to the window: `uk_win_cursor_pos(&x, &y)`.
- **Yield anyway**: the scheduler preempts a busy app, so a loop that never yields no longer freezes the system, but it is treated as a CPU hog (short slices, served after the others) and its own window stops being redrawn and answering. Keep calling `present`/`msleep`/`pump_events`/`wait_for_exit` regularly.

See the demos `demoD.c` (widget gallery), `demoE.c` (textarea + scrollview), and the apps `tinypad.c`, `paint.c`, `mandelbrot.c` for complete examples.

The Archiver (`user/Apps/archiver`)

The archive manager (the plan and the user's decisions: `docs/archiver/README.md`; its use: docs/04 §9) is a **newlib** uikit app with FreeType text (`archiver.elf` in `user/Makefile`, with **zlib** built from `third_party/zlib-1.3.1` into `user/Libs/zlib/libz.a`), one translation unit:

File	What
`arc.h` (namespace `arc`)	The engine's base, no uikit: `Entry` (a path in UTF-8, sizes, CRC, DOS time, the format's fields), `Archive` (the interface a format implements: `open`, `extract (i, Sink &, Progress *)`, `rewrite (Plan, dest, Progress *)`, `writable`, `methodName`), `Plan` (a rewrite: the entries kept — renamed or not — and the new ones, a file of the disk or a folder), `Reader` (random access: `kapi_open` + `kapi_seek`, files of any size, not slurped like newlib's), `FileSink` (a `kapi_file_out` stream), CP437 → UTF-8, DOS dates, human sizes.
`zip.h`	ZIP: the central directory (zip64, Info-ZIP's Unicode path, a self-extractor's offset, a UTF-8 name without its flag), extract (Store, Deflate through zlib, ZipCrypto), rewrite (the kept entries copied packed under a new local header; the new ones stored, deflated in memory when small — stored if that is not smaller —, streamed with a data descriptor when big; zip64 records when needed).
`ops.h`	The operations whatever the format: `archive_open` (the format from the first bytes), `run_extract` (the layouts `LAY_FULL / LAY_FROM_CURRENT / LAY_FLAT`, the overwrite policy and its question, names made safe for FAT, no half file left), `apply_plan` (a new copy `*.part`, then swapped in), `op_add` / `op_delete` / `op_rename` / `op_new_folder`, `plan_add` (a folder walked, the names that exist replaced or kept).
`model.h` (namespace `ui`)	The archive as a tree for the view: a node per path part (implicit folders too), totals, children sorted (folders first, by the list's column), the search.
`icons.h`, `widgets.h`, `dialogs.h`	The vector icons; the widgets (`ToolStrip` of large buttons, `PathBar` with its `SearchBox`, `FolderTree`, `EntryList` — multiple selection, sort, drag out, the drop's banner —, `InfoCard`, `StatusBar`, `Welcome`); the dialogs (Extract, Add, progress, exists, text, properties).
`main.cpp`	The app: the jobs (a thread per job, `kapi_post` for the progress and the end, a question to the main thread answered through an event), drag & drop both ways, files opened into `RAM:` and watched to be put back, the menus.

`/bin/zip` and `/bin/unzip` (`user/BinUtils/zip.cpp`, `unzip.cpp`, `arccli.h`: the command line split, patterns) are C++ newlib tools on the same engine (`ops.h`: `plan_begin` / `plan_add_path` / `plan_finish` give each path its own folder in the archive, one rewrite for all), linked with the vendored `third_party/zlib-1.3.1/libz.a` (`ARC_PROGS` in `user/BinUtils/Makefile`).

A job works on **its own instance** of the archive (opened again): the view keeps reading the old one until the job ends and the archive is read again from the disk. **Host test**: `sh tools/tests/run_archiver_test.sh` — `tools/tests/archiver/arctool.cpp` (the engine on the simulator's kapi) driven by `test.py`: archives made by Python's `zipfile` and the `zip` tool (stored, deflated, a self-extractor, ZipCrypto, UTF-8 names), every result checked by `zipfile` and `unzip -t`; then `/bin/zip`

and `/bin/unzip` built for the PC, driven by `cli_test.py`. The screenshots: `sh tools/tests/desktop_sim/shots.sh archiver` (a sample archive made by `arc_sample.py`).

The packages: `pkg`, `user/Libs/pkg/pkglib.h`, `tools/pkg`

The design, the formats and the plan: `docs/pkg/README.md`. Done so far:

Part	What
`tools/pkg/packages.ini`	Which files of `sdcard/` make each package: `[onyx]` (the kernel, `bin/`, `etc/` as settings, the fonts, the Shell and Settings apps, the terminal, the File Viewer, the Task Manager, Tinypad, voronoy and imageview; `required`, `restart`), `[pi-firmware]`, `[demos]`, the samples each with its app (`[basic-samples]`, `[letters-samples]`... `needs` their app; their files are the user's: `config`), then `[*apps]`: every other app its own package (`[app.<name>]`: its files outside its bundle, its `needs` — the emulators `gamelib`, Letters / Sheet / Ledger `cardfile` —, its `config`).
`tools/pkg/versions.ini`	Each package's version, raised by `mkrepo.py --bump` (a package whose files changed at the same version is refused).
`tools/pkg/mkrepo.py`	`--out <onyx-packages checkout> --key <private key>`: the `.opk` of each package (a deterministic ZIP: the card's tree + `PKG/manifest.ini`), the icons, `index.txt` (version, size, SHA-256, needs — `kapi >= KAPI_ABI_VERSION` added —, the content's hash) signed into `index.sig` (ECDSA P-256 / SHA-256). `--db`: `sdcard/var/pkg/db/*.ini`, the card made "installed". `--lite sdcard_lite`: the card of the required packages only.
`tools/pkg/publish.sh`	The publishing, in one command (the skill `.claude/skills/onyx-packages`): the `onyx-packages` clone updated, `mkrepo.py --bump --db --lite`, the signature checked with `onyx.pub`, the host test, commit + push; the key from `ONYX_PKG_KEY` / `ONYX_PKG_KEY_FILE` / `~/.onyx/pkg-key.pem`.
`tools/pkg/keygen.py`	The key pair, once: the private key kept off the repositories, the public one `sdcard/etc/pkg/onyx.pub`.
`user/Libs/pkg/pkglib.h`	The library (`pkg`, later `pkgman` and `pkgd`): the ini text, the index fetched (`http.hpp` with TLS when `PKG_NET`, else a folder) and its signature checked (mbedTLS `pk_verify`), the database (`SD:/var/pkg/db`), `resolve` (the needs, the kernel's ABI), `install` (the download's size and SHA-256 those of the index, then the Archiver's ZIP engine: each file written beside and swapped in, a changed setting kept and the new one written as `.new`, the old version's other files removed — unless another installed package has them, `owned_elsewhere`, as when `bin/pkg` moved from `onyx` into `pkgman`), the staging of `restart` packages (`SD:/var/pkg/stage`) and `commit` (moved in, `kernel8-rpi4.img.old` kept), `remove` (the needs, an app running, the empty folders), `set_mode`, `refresh_desktop` (an app's files installed or removed: the dock killed and started again, called by `pkg`, `pkgman`, `pkgd` at the end of their job).
`user/Libs/pkg/pkgjob.h`	A job in a thread, the window alive (2026-10-09, out of `pkgman`): `job_start (J_CHECK / J_INSTALL / J_REMOVE / J_MODE, names, n, mode)` with `g_jobM` the `Manager`; what it says each frame in `g_job` (`cur`, `pct`, `msg`, `failed`, `staged`, `finished`, `done`, then `kapi_thread_join`). `pkgman` and the console's Packages page (`consolehome`'s `xset.h`) share it.

Part	What
`user/BinUtils/pkg.cpp`	The command (docs/04 §8 *Packages*); `PKG_PROGS` in `user/BinUtils/Makefile` (newlib + mbedTLS + zlib).
`user/Apps/pkgman`	The **Package Manager**, the Control Panel's applet (`70-pkgman.lnk`; FreeType): a snapshot of the index and the database (`Row`s, rebuilt after each job) drawn by its own widgets (`Tabs`, `PkgList`: the rows, the boxes, the mode pills, the buttons, the restart banner); a job (check, install, remove, mode) in a thread (`kapi_thread_create`), its progress read each frame (`onTick`: the package being done, its percentage — `http.hpp`'s new `progress ()` callback and the extraction —, the packages finished). Built by `pkgman.elf` in `user/Makefile` (newlib + FreeType + mbedTLS + zlib, `-DPKG_NET -DONYX_HTTP_TLS`).
`user/Apps/pkgd`	The **update daemon** (no window; `run pkgd` in `etc/autostart`): waits for the network and the time, then once a day (`SD:/var/pkg/lastcheck`) `refresh`, the "auto" packages installed (their new needs first; the system staged), `notify_action` for the others; `--once`: one round; `check = never` in `pkg.ini`: it ends.
`user/BinUtils/init.c`	The `wait <command>` builtin (`kapi_spawn` + `kapi_wait`): `wait pkg commit`, the autostart's first line.

Screenshots: `sh tools/tests/desktop_sim/shots.sh pkgman` — `tools/tests/desktop_sim/pkg_sample.py` makes a card of its own (the real one through symbolic links, its own `etc/pkg` and `var/pkg/db`: a few apps not installed) and a repository with four newer versions (a key made for it); the applet's four pictures (`pkgman`, `-installed`, `-available`, `-restart`: Install 4 Updates run to its end).

Host test: `sh tools/tests/run_pkg_test.sh` — mbedTLS and `pkg` built for the PC over the simulator's kapi (`SIM_SD` an empty card, the repository a folder of it), driven by `tools/tests/pkg/test.py`: a repository made by `mkrepo.py` with a key made for the test; install with the needs, already installed, update (a changed setting kept, `.new`, a dropped file removed), the modes and `upgrade`, removals refused (needed, running, the system), the system staged then committed (and the reboot), a changed index and a changed archive refused.

3DForge, the parametric CAD (`user/Apps/3dforge`)

The small CAD (the study, the mock-ups and the user's decisions: `docs/3dforge/README.md`; the user guide's *3DForge*). A newlib app as Paint, with libstdc++'s `operator new` (`ONYX_HOSTED_NEW`), linked with **Manifold** (`Libs/manifold/libmanifold.a`: `third_party/manifold-3.5.4` + Clipper2, `user/Makefile`'s `MF_*`).

File	What
`fdoc.h`	The document, portable: the **steps** (`Feature`: box, cylinder, sphere, torus, pyramid, prism, taper; sketch, extrude; fillet, chamfer; move — with its turns and scale, `moved ()` —, combine) and their **replay** (`Doc::rebuild`: each step's solid, then its operation on its body; a shape may be off its plane — `z` — and turned about its normal — `turn`); the **sketch** evaluated in order (`sketch_eval`: no solver; lines, arcs — `sweep` positive clockwise in memory, anticlockwise in the file, as it was first —, circles, rectangles from a corner or their centre, splines through their points (Catmull-Rom), points that are only marks); a body's **mesh as the tools want it** (`build_mesh`: the faces — flat patches, curved ones joined when their

File	What
	facets are strips of the same width —, the chains where two faces meet, each known as straight, a circle, an arc, or other; the triangles without area a boolean leaves are given to the face they lie along); the **corner solid** of a fillet or chamfer (`corner_solid`: the profile between the corner and the arc, extruded along a straight edge or revolved around a circle's or an arc's axis; added on an inner edge, cut on an outer one; an outer edge's cut goes on past its ends to the corner a neighbour was cut at — `corner_reach`, probing along the edge's line with `inside_mesh`: a mitre — and, where the edge ends on two faces already rounded, `corner_blend` adds the piece that turns those rounds around it: their profile revolved a quarter turn); **asymmetric** fillets and chamfers (`Feature::r2` > 0, `swap`; the `r2` line of the `.3df`): `corner_profile` takes a size per face — a chamfer's cut r on the first, r2 on the second; a fillet's setbacks r / tan (phi / 2) and r2 / tan (phi / 2), its section a rational quadratic arc from one tangent point to the other with the corner as control and the weight sin (phi / 2) (the circle itself when r == r2, a quarter ellipse at 90 degrees) —; the first face is `first_face`'s (the normal turned the most up, then +Y, then +X; a circle's flat face), `swap` turns them round; `corner_blend` only for equal sizes (an asymmetric corner is mitred); the **sketch**'s runs drawn apart joined where their ends meet, **construction** elements (`SkEl::cons`, the `cons 1` line) part of no outline; the **loft** (`F_LOFT`, its sketches in `sketch` and `tool`; `loft_solid`: each sketch's largest closed outline in space, turned the same way round the axis between their centres; corner to corner when they have as many corners (a turn of more than 15 degrees; the rotation that puts them the nearest), each stretch between two corners cut at the union of both's places along it, else along their whole lengths from the second's point nearest the first's start; joined by straight lines, the ends triangulated by Manifold's `Triangulate`, the mesh made a `Manifold`; its original id noted in `g_smoothOrig`, so `build_mesh` makes one curved face of its small facets though they fold up to 80 degrees where a straight side meets an arc); **any other edge** rounded or chamfered by `corner_sweep` (the corner's profile, of a fixed number of points, set across the edge at each of its vertices from the two faces' normals there, the profiles joined, the ends closed; one kind, outer or inner, along the whole edge; its facets one face through `g_smoothOrig`); `chain_mid` gives an open edge's point half way along it (an end is shared with the next edge); the `.3df` text; STL / OBJ. A face or an edge chosen is kept by its place (a plane; a point on the edge, found again by `chain_near`).
`frender.h`	The camera (orthographic), a frame as `kapi_gpu_vertex3` triangles in `kapi_gpu_batch` batches (the bodies with a matrix, lit per face; the edges and the grid as thin quads already in clip space, a depth bias toward the eye), drawn by **`kapi_gpu_render`** at twice the size then averaged down — or, without a GPU (and on the PC), by the z-buffer there over the same triangles; `pick` (a ray against the triangles).
(canvases)	`Canvas3` (`fdoc.h`: a picture's path, its plane and offset, its place, width, turn, opacity; the `canvas` / `canvasfile` lines of the `.3df`), drawn by `View::drawCanvases` over the frame — the view has no perspective, so a picture on a plane is an affine map: each pixel under it finds its point of the picture, blended at the canvas' opacity —, moved and sized by `View::canvasMouse`. The picture is read by UIKit's `ui::icon_load` (through ImageKit), reached as `picture_load` by its symbol: the header's namespace `ui` clashes with the app's `ui ()`.
`fview.h`	`App` (the tool and its step, the step being made and its preview, the sketch being drawn, the selection) and the `View` widget: turn / pan / zoom, the tools' clicks and moves, the overlays (values' tags, arrows, the sketch, the cube, the bodies' panel).
`fui.h`	The tools' bar and the shapes' fold-out, the timeline, the selection's panel (its fields bound to

File	What
	the step's values: rebuilt at the loop's next turn, never from a control's own callback), the status bar.
`fcam.h`	**Manufacture**, portable: `CamSetup` (the body, the stock, the origin among 27 points, X's direction, the tool, the machine, the operations `CamOp`), `cam_compute` → `CamPaths` (the moves of the tool's tip in the model: fast, cutting, plunging). A level's shape is the body's **shadow above it** (`cam_shadow`: `TrimByPlane` + `Project`), grown by the tool's radius (Clipper2's `Offset`); a **clearing** cuts rings from the outside in at each level (`cam_levels`: the step down, and the flat faces turned up), ramping into each ring, the stretches along the stock's edge left out (open passes that start in the air); a **contour** follows the outline pass by pass, its tabs a floor the pass may not go under; one face only: `cam_face_shape`. `cam_simulate` replays the moves over a height map (0.5 mm cells) and reports a fast move through matter, the body cut into, the lowest Z, the machine's travel, the tool's length. `cam_gcode` writes GRBL's text in the work coordinates (`cam_work`); `cam_save` / `cam_load`: the `cam…` lines at the end of the `.3df`; `cam_presets`: `cam.ini`.
`fprint.h`	**Manufacture for a resin printer**, portable: the file (`PmFile`, `pm_write` / `pm_read`: the "ANYCUBIC" 5.17 container — header, preview, layer table, the lift's stages, machine, software, model — and a layer's picture as runs, `PmRuns` / `pm_decode` / `pm_raster`); `PrintFormat` (a family's writer) and `print_machines` (the built-in printer, then `printers.ini`); `PrintSetup` (the body on the plate — `print_place` —, the resin, the supports' tips), `print_supports_auto` (a grid under what leans more than the angle, the low points, the low edges), `print_support_solid` (pillars, cones, a raft: one `BatchBoolean`), `PrintSlicer` (the layers a few at a time: `Slice`, the picture, the exposure, the time; a loop of a layer that touches nothing of the layer before is an island), `print_preview`, `print_save` / `print_load` (the `print…` lines of the `.3df`, and `print.ini`). The app: `App::gen` (0 milling, 1 resin) selects the generator's tools, pages and view.
`ffdm.h`	**A filament printer's path**, portable: `fdm_paths` (a layer's walls, solid fill, infill — lines, a grid, a honeycomb cut to the region — and skirt, from the body's sections; `fdm_shell`: the shell in millimetres), `fdm_gcode` (a plain Marlin G-code, **not offered by the app yet**: no printer's writer), `fdm_save` / `fdm_load`. Tested by `fdmtest.cpp`. In the app: `App::gen == 2`; it shares the plate, the placement and the layers' bar with the resin printer (`printer ()`), and plays a layer by drawing its paths up to `fdmDrawn` millimetres.
`fdraw.h`	What Export writes besides the meshes: `make_drawing` (the edges seen from a side, laid flat in millimetres; hidden parts found by walking each edge against the bodies' depth picture — the z-buffer of `frender.h` — and cut there), written as DXF (R12 lines, layers `VISIBLE` / `HIDDEN`), SVG, or PDF (`Libs/pdf/pdfwrite.h`'s `path`); `picture_png` (the shaded view, ImageKit's PNG writer).
`ficons.h`	The icons (`uikit/vpaint.h`).

Tests: `sh tools/tests/run_manifold_test.sh` (Manifold as linked for the Pi, under qemu-aarch64), `sh tools/tests/run_3dforge_test.sh` (the document on the PC: the sample bracket step by step, every volume, the other shapes, the file, the exports; then `camtest.cpp`: the bracket cleared and cut out, nothing hit, nothing cut into, the G-code's form and its extents, an operation on one face; then `printtest.cpp`: the bracket cut into layers (what is lit is its volume), the file written and read, supports that leave nothing in mid-air, an island found — `FORGE_PM_REF=<a file of the printer's slicer>` checks that it is written back the same to the byte; `FORGE_SAMPLE=sdcard/docs/3d/bracket.3df` writes the

sample). Screenshots: `sh tools/tests/desktop_sim/shots.sh 3dforge` (the scenes: `forge_scene.py` — its clicks are the tools' places in the bar: move a tool, move them). Icon: `tools/icons/3dforge_icon.py`.

Cosmo Zoom!, a 3D game on the GPU (`user/Apps/cosmozoom`)

The anti-gravity racer (docs/04 "Cosmo Zoom!"; its design: `docs/cosmozoom/DESIGN.md`, its models: `docs/cosmozoom/MODELS.md`) is the model of a **3D game on the V3D GPU** without shaders of its own: everything is coloured triangles (and a few textures) handed to **`kapi_gpu_render`** (kapi v53/v54: `struct kapi_gpu_vertex3`, `struct kapi_gpu_batch` — each batch its 4 × 4 matrix, texture, blending, depth test, culling). One translation unit (`main.cpp`) and headers, each a layer that the PC's tests use as they are:

Header	What it is
`math3d.h`	vectors, 4 × 4 matrices in the GPU's layout (row by row, clip = M · (x y z 1)), perspective, look-at; y up, a model faces +Z, its left is +X
`mesh.h`	the Wavefront `.obj` / `.mtl` reader (materials `paint1` / `paint2` recoloured, `*_glow` unlit and added, `glass` / `d < 1` see-through, `#@ name x y z` sockets, `o` groups), the soft light (smoothed normals, the creases kept), the black outline (the inverted hull)
`gfx.h`	a frame: vertices and batches in the kernel's layout; the software picture of the same frame (the BASIC's renderer, `user/Libs/basic/bas3d.h`)
`track.h`	a circuit: its `.trk` file, the Catmull-Rom centre line cut into 2 m segments with their frames (banked turns, loops spliced in, twists, tubes rolled down, forks and crossings: two branches with their own place across and height), the surface `surf (s, l, h)` everything stands on (`surfB` on a fork's branch)
`race.h`, `parts.h`	the race (physics in the track's coordinates — along, across, above —, the computer's pilots, the capsules and gadgets, the laps, the places; events for the sounds), the ships' parts and stats, the gadgets' table
`world.h`	the scenery made once per circuit, cut into pieces of 12 segments with a sphere each (only the pieces in front are sent): the road, the rails and force fields, the city, the ground, the sky
`scene.h`	one view's frame: the sky, the visible pieces, the ships (built from their parts, `buildShip`), shadows, trails, flames, sparkles, glow passes; the chase camera
`textures.h`, `audio.h`, `ui.h`, `draw2d.h`	the procedural textures, the music sequencer and the effects (AudioKit), the HUD and the menus' drawing

Each picture: per player, `Scene::view` fills one `Frame` (≈ 50–120 k vertices, 30–60 batches), then `render3d` (`main.cpp`) maps the game's texture numbers to the GPU's handles and calls `kapi_gpu_render` on that player's rectangle of the full-screen buffer (`kapi_gpu_frame.pixels` = its first pixel, `stride` the screen's); without a GPU (or `kapi_gpu_render` answering -1 / -3) `softRender` draws the same frame at half size. The HUD is drawn after, by the processor, on the same buffer (`ui.h`: UIKit's FreeType faces through a `Canvas` adopted on the buffer), then `kapi_present_fb`. Files are read through AppKit (`g_fileReader` = `kapi_open` / `kapi_read`); the race runs at a fixed 60 Hz step whatever the frame rate.

The tests on the PC, all on the same headers: `tools/tests/cosmozoom/racesim.cpp` (a whole race of the computer's pilots: results, events — `g++ -std=gnu++17 -O2 -Iuser -Iuser/Libs`), `racepic.cpp` (pictures of a race by the software renderer), `objview.cpp` (a model as the game draws it, four sides: the modeller's preview), `trackmap.cpp` (a circuit seen from above), and `tools/tests/cosmozoom/sim.sh OUT.png`

"SCRIPT" [ARGS] (the real app on the desktop simulator's stand-in kernel: menus, HUD, everything; SIM_SCREEN=1920x1080 for a 1280 × 720 window). The circuits are made by python3 tools/cosmozoom/trackgen.py [id...] (each circuit described as a drive -- straights, arcs, heights, features at marks -- written as sdcard/apps/cosmozoom.app/tracks/<id>.trk: change the generator, not the file; trackmap.cpp checks a layout, tracksheet.py makes docs/cosmozoom/tracks.png), the models by Python scripts (python3 tools/cosmozoom/models/build_all.py --sheets), the icon by tools/icons/cosmozoom_icon.sh.

Screenshot, the capture tool (user/Apps/screenshot)

The screen capture tool (the study, the mock-ups and the user's decisions: docs/screenshot/README.md; its use: docs/04 §12) is a **newlib** uikit app with FreeType text (screenshot.elf in user/Makefile), one file, main.cpp:

- **The capture.** New (or Print Screen) → after the delay (counted in the window's view), the window is minimised (uk_win_minimise (0)), ~0.35 s later the screen is grabbed (kapi_screen_grab, what the display shows); a rectangle or a window is chosen on the frozen screen shown **full screen** (uk_win_fullscreen_begin: every pointer and key event goes to the app, in screen coordinates; the app's own uk_win_on_pointer / uk_win_on_key, Root::attach () after uk_win_fullscreen_end gives the window its streams back; no cursor is drawn by the kernel in full screen: the app draws a crosshair, an arrow). The windows: uk_win_list (their frames: ow / oh, il / it), the backmost, system, minimised and other desks' left out. Then uk_win_app_raise ("screenshot").
- **The picture** (g_base), the **crop** (a rectangle of it), the **strokes** (in the base's 1/16 px: uikit/vpaint.h's units; a pen's opaque, a marker's at opacity 118 — the stroke filled once, so it does not darken where it crosses itself —; smoothed 1-2-1 twice when drawn), the **operations** (OP_ADD, OP_ERASE — a stroke taken away by the eraser —, OP_CROP with the rectangles before / after: undo / redo). doc_render makes g_doc (the crop with the strokes over it): what is copied and saved. The view shows it fitted, never enlarged (a box filter), the stroke being drawn over it.
- **Out**: clip_set_image (clipd's ring, an image item: clipboard.h), done at the tick after the window is drawn (clipd may have to start), and notify () (notifyd's bubble); **Save As** only (uk_file_save in SD:/Pictures/Screenshots, the last folder kept): img/pngsave.hpp's png_encode / jpeg_encode (quality 92) / bmp_encode by the name's extension (none: .png).
- **The service** screenshot (kapi_ipc_register): Print Screen, sent by Elegant, the graphics server (type 1, "now" / "window <id>"), read in onTick; started by Elegant: --now / --window <id>. The settings: SD:/etc/screenshot.ini (mode, delay, pen_colour, pen_size, marker_colour, marker_size, folder).

The icon: python3 tools/icons/screenshot_icon.py. The screenshots: sh tools/tests/desktop_sim/shots.sh screenshot — the simulator's SIM_GRAB=<file>.elsm is what kapi_screen_grab gives (screenshots/ desktop.png made an .elsm there), a full-screen app is dumped as its whole buffer, and SIM_WINS' windows may end with ,title.

Media Player, the music and video library (user/Apps/media)

The music and video library (the mock-ups and the user's decisions: docs/media/README.md; its use: docs/04 §12) is a **newlib** uikit app with FreeType text (media.elf in user/Makefile): main.cpp (the window) and headers. Its videos are played by **the media library** (user/Libs/av, below), compiled by the rule into Apps/media/obj/av/ with av/codecs.mk's AV_CODECS_CF **and -DAV_WITH_FFMPEG**, and linked with the codec libraries — libvpx.a, libdav1d.a, libopus.a (make -C user/Libs/av; committed in

`third_party/`) — and **FFmpeg 7.1.2**'s (`third_party/ffmpeg-7.1.2/onyx/aarch64/lib{avformat,avcodec,swscale,swresample,avutil}.a`, committed; made by `sh third_party/ffmpeg-7.1.2/onyx/build.sh pi`: every decoder and demuxer, `--enable-gpl`, no threads, no programs). **The Media Player is therefore GPL-2.0-or-later** (its files stay MIT; docs/LICENSING.md). The app is ~16 MB (FFmpeg: 10 MB of code, 20 MB of tables in its bss).

File	What
`codecs.h`, `codecs.c`, `vorbis.c`	The decoders from `third_party`: **minimp3** (`minimp3_ex.h`: an index for exact seeks), **dr_flac**, **dr_wav** (in `codecs.c`), **stb_vorbis** (alone in `vorbis.c`: its static names clash with minimp3's). C, built with the FPU into `Apps/media/obj/`; no stdio (`*_NO_STDIO`).
`decode.h`	`Src` (a file through the kapi: `kapi_open` / `kapi_read` / `kapi_seek` / `kapi_fsize64`, any size), `Decoder` (s16 stereo frames at the file's rate; mono doubled, more than two channels: the first two), one per format (an Ogg file read whole: stb_vorbis decodes from memory), `Stream` (a linear resampler to `SOUND_RATE`), `decoder_open` (by extension, then the first bytes).
`midi.h`	`MidiSong` (a Standard MIDI File, formats 0 / 1, RMID too: the tempo map, running status, the events in time order in frames, the notes for the view, the channels' General MIDI instruments), the SoundFont loaded once (`soundfont ()`), `MidiDecoder` (the events fired between `render`s of MeltySynth — Koton's `Apps/koton/synth`, linked from Koton's objects —; a seek replays the programs and controllers, no notes).
`player.h`	`Player`: a thread (`kapi_thread_create`) that opens the song asked for, decodes ahead and feeds `kapi_sound_write` (`kapi_sound_config (1024, 3)`: ~0.1 s queued, so a pause is heard at once), at the volume (a square law); the window asks (play, pause, resume, seek, stop) and reads `state`, `posMs` (what is heard: the frames sent less those queued), `lenMs`, `endedGen`, `errGen` under a `kapi_lock`. No output (the simulator): the song is decoded and timed silently.
`tags.h`	`read_tags`: ID3v2.2–2.4 (Latin-1, UTF-16, UTF-8; `APIC`: the cover's offset and length in the file) then ID3v1, the MP3's length (the Xing / Info / VBRI header, else the bit rate), FLAC's `STREAMINFO`, `VORBIS_COMMENT`, `PICTURE`, Ogg's comment header and the last page's granule, WAV's `fmt`, `data`, `LIST INFO`, MIDI's track name; the path fills the rest (`<artist>/<album>/<nn> - <title>`).
`lib.h`	`Library`: the songs (strings `strdup`'d), grouped by `build ()` into albums (album artist — else artist — and album), artists, genres, folders; `sort_idx` (a merge sort with a context: the scan's thread sorts too), `IntList`; `library.tsv` / `stats.tsv`; the scan (`scan_dir`, `scan_thread`): the folders walked (and `SD:/Videos`), a known file (same path, same size) copied, the others read; a folder's `cover.jpg` noted; the videos to `scan_video` (the scan reads a copy of the videos known: `VideoLib::clone`).
`covers.h`	`Covers`: a thread decodes the pictures (`img_load_mem` / `img_load`), cropped square at 320 px; each size drawn is made once (box filter, bilinear up) and cached (the oldest dropped); an album without a picture gets one drawn from its name; `round_corners` (polygons over the corners); `tone ()` (a cover's colour: the bands, now playing).
`videos.h`	`Video`, `VideoLib`: the videos (their facts, where each was left, watched, when) in `videos.tsv`; `probe_video` (the container's headers through `av_demux_*` — a few MB read at most, from where the demuxer wants: an MP4's `moov` at its end too —: length, size, codecs, `playable` = the build decodes them, `av_codec_list`); `video_names` (the title from the file's name, an episode's `S01E03` / `1x03`, the kind from the folders: *Films*, *Series*, *Clips*;

File	What
	else 40 min and more: a film); `scan_video` (the scan's: a known file — same path, same size — kept, else probed).
`thumbs.h`	`Thumbs`: a thread makes each video's frame — `video_frame_at` (`av_demux_seek` to a tenth of the video, at most a minute in, when the container has an index; else decoded on from the start, 4 s at most; `av_decoder_*`, `av_yuv_to_rgb`), cropped to 16:9 at 384 × 216, kept as `SD:/etc/media/thumbs/<key>.jpg` (`pngsave::jpeg_encode`; the key: the path and the size hashed) —; sizes cached as the covers'; it waits while a video plays (`busy`); a video not decoded here gets a frame drawn from its name.
`watch.h`	`VideoPlay`: a video playing — `av_player_new (AV_PIX_BGRA, 0)` (the kapi's sound) + `av_player_open_file` —, `poll ()` on the window's tick (the frame due copied: the library's is valid until the next poll; `statusGen` when what is shown changes), `draw` (fitted, black bars, a nearest-neighbour scaler: a frame each 33 ms on the Pi).
`ui.h`	The faces (DejaVu Sans 11 … 27), text cut to fit, the times, the icons (`uikit/vpaint.h`).
`main.cpp`	The settings, the playlists (`.m3u`), the queue (its order, shuffled or not; ids < 0: a file opened that is not in the library), the pages (`Page`, back / forward; `P_VIDEOS` a kind, `P_WATCH` a video: `page_changed` closes a video left — its position saved — and opens one come back to), the widgets — `Sidebar`, `TopBar` (`SearchBox`), `Content` (every page drawn in view coordinates, its hits listed for the clicks; the songs' table, its selection, its menu; the videos' tiles, the home's cards), `NowBar`, `MiniView`, `WatchView` (the video in the whole window: `draw_watch` — the frame, the controls while the pointer moves, loading, the end, an error — shared with **full screen**: `video_full_screen`, `uk_win_fullscreen_begin` + `kapi_present_fb`, its own event loop as the PDF Viewer's) —, the sound shared (`video_start`: the music's thread releases the output — `Player::release` — before the video's takes it; a song started closes the video), the positions (`video_save_pos`: every 15 s, at a pause, on leaving; near the end: watched, from the start next time), the mini player (`uk_win_resize2` + `uk_win_move` to the work area's bottom right), the scan's end (`scan_done`: the new library swapped in, the queue and the pages mapped by path / name; the videos' positions kept).

Tests on the PC: `sh tools/tests/run_media_test.sh` — sample files made by ffmpeg (MP3, Ogg, FLAC, WAV at 48 kHz, a 22 kHz mono WAV) and `tools/tests/media/make_midi.py`, decoded by `tools/tests/media/dectest.cpp` through the simulator's kapi: the length, the level, a seek, each. **The screenshots**: `sh tools/tests/desktop_sim/shots.sh media` over a sample library made by `tools/tests/media/make_library.py` (the mock-ups' albums and covers in every format, two MIDI albums, playlists, play counts; needs ffmpeg and `pip install mutagen`; and `Videos/`: VP9 + Opus WebM, AV1 MP4, an H.264 MP4, a film left half way — the app built with `user/Libs/av` and its codecs for the PC by `tools/tests/desktop_sim/av_host.mk`); the simulator got events (`event_create / set / reset / wait`, pthreads). The icon: `python3 tools/icons/media_icon.py`.

The media library (`user/Libs/av`): video and audio for Jet Browser and the Media Player

`user/Libs/av` plays media files and streams: containers, decoders, conversions, a store of coded frames (Media Source Extensions') and a player with its threads, sound and clock. Jet Browser's `<video>`, `<audio>` and MSE are built on it (WebKit's media engine for Onyx: docs/08 *Media*), and the Media Player's videos. Plain C (`av.h` is the reference), threads and locks through `av_os.h` (the kapi; pthreads with `-DAV_POSIX` for the PC tools). Compile every `user/Libs/av/*.c` with `-I user/Libs/av -I user -I user/Kits -I`

`user/Runtime -I user/Include -I user/Libs -I user/Emulators -I user/Ports -I kernel/include` (`-std=gnu11`); the large codecs' glue files compile to nothing unless their `AV_WITH_*` is given -- `user/Libs/av/codecs.mk` has their libraries' sources and flags and `AV_CODECS_CF` (the `-DAV_WITH_*` and include paths): include it with `TP` = `third_party` and `AV_ARCH` = `aarch64` (the Pi: NEON, dav1d's assembly; `make -C user/Libs/av` builds `libvpx.a`, `libdav1d.a`, `libopus.a`) or `generic` (C).

File	What
`av.h`	The API (below).
`av_demux.c`, `av_mkv.c`, `av_mp4.c`, `av_riff.c`, `av_flac.c`, `av_mp3.c`	The containers: WebM / Matroska, MP4 / MOV / fragmented MP4, WAV, FLAC, MPEG audio.
`av_codec.c`, `av_flac.c`, `av_mp3.c`	The codecs' table, PCM / A-law / mu-law / uncompressed I420, FLAC (built in), MP3 (minimp3); `av_type_supported` (canPlayType, isTypeSupported, MediaCapabilities).
`av_vpx.c`, `av_dav1d.c`, `av_opus.c`	VP8 / VP9, AV1, Opus on libvpx 1.15.2 / dav1d 1.5.1 / libopus 1.5.2 (`third_party/`, `user/Libs/av/codecs.mk`): with `-DAV_WITH_VPX` / `AV_WITH_DAV1D` / `AV_WITH_OPUS` and the library. One decoding thread each; 8-bit 4:2:0 video.
`av_ffmpeg.c`	With `-DAV_WITH_FFMPEG` (the Media Player; GPL-2.0+): **every codec libavcodec decodes** that Onyx's own do not (H.264, H.265, AAC, MPEG-4 Part 2, MPEG-2, WMV / VC-1, Theora, AC-3, DTS, Vorbis, WMA, ALAC...): a track's `ff_id` (libavformat's tracks, `AV_C_FFMPEG`) or its codec (H.264, H.265, AAC, Vorbis, MJPEG); the frames brought to 8-bit 4:2:0 (libswscale for 10-bit, 4:2:2, RGB...), the sound to interleaved float; `av__ff_identify` (the Matroska / MP4 codecs Onyx's parsers do not know: `A_AC3`, `V_MS/VFW/FOURCC`, `mp4v`, `ac-3`, an esds object type...). FFmpeg is built without threads: its first-time initialisations are not guarded, so the opens go one at a time under `av__ff_lock`. Our `av_packet_free` was renamed **`av_pkt_free`** (FFmpeg has the name).
`av_lavf.c`	With `-DAV_WITH_FFMPEG`: **`AV_FMT_LAVF`**, every other container through **libavformat** (AVI, MPEG-TS / PS, ASF / WMV, FLV, Ogg, RealMedia...: `av_probe` asks FFmpeg's probe after Onyx's, from 4 KB). libavformat pulls its bytes, the library pushes them: it runs on **a thread of its own**, its AVIOContext's read waiting for the bytes it needs (`av_demux_want` answers that offset; -1 when nothing is wanted now: its thread works on what it has), the parser's read copying the window into its cache and handing over the packets it queued; a seek is asked of the thread (`avformat_seek_file` on its index); the times start at 0 (MPEG-TS's start time taken off). The parser is **busy** (`av_fmt_ops.busy`) while packets may still come: the store takes them when asked where to feed (`av_store_want`), and is not "ended" before. `av_demux_set_size` / `av_store_set_size`: the file's size (its demuxers seek from the end).
`av_stub.c`	The tests' stand-ins (grey frames, silence) for the codecs not built in: `av_codec_enable_stubs ()`.
`av_yuv.c`, `av_resample.c`	YUV 4:2:0 to RGBA / BGRA (NEON on AArch64), the resampler (any rate / channels to s16 stereo).
`av_store.c`	The coded frames by source and track (MSE's semantics), buffered ranges, removal, quota.

File	What
`av_player.c`	The player: audio and video threads, the kapi's sound, the clock, the frame due now; the file mode's reader thread (the file read in 64 KB pieces where the demuxer wants them, ~30 s ahead: `av_os.h`'s `av_file_*` — the kapi's `kapi_open` / `kapi_seek` / `kapi_read` on Onyx, any volume and any size; newlib's `fopen` would load the whole file —, stdio with `AV_POSIX`).

Playing a file (the Media Player's case):

```
#include "av.h"
struct av_player *p = av_player_new (AV_PIX_BGRA, NULL);   /* NULL: the kapi's sound */
if (av_player_open_file (p, "SD:/Videos/film.webm") != AV_OK) { /* not a format we read */ }
av_player_play (p);
for (;;) {                                          /* the window's loop, every ~10-20 ms */
    struct av_player_status st; struct av_video_out v;
    if (av_player_poll (p, &st, &v))                /* a new frame is due: v.pixels (v.width x
 v.height,
       blit (v.pixels, v.width, v.height, v.stride);   v.stride bytes a row) until the next
poll */
    /* st.time, st.duration, st.paused, st.ended, st.waiting, st.ready (AV_HAVE_*), st.width
/ height
     * (the display size), st.error (AV_EUNSUP: a codec not built in), st.decoded / st.droppe
d */
}
av_player_seek (p, 90 * AV_US);  av_player_pause (p);  av_player_set_volume (p, 0.8f, 0);
av_player_set_rate (p, 1.5);  av_player_free (p);
```

Feeding it yourself (a network stream, Jet's `<video src>`): `av_store_add_source (av_player_store (p), "video/webm", 0)` then `av_store_feed (store, src, offset, bytes, n)` from where `av_store_want (store, src)` says, `av_store_feed_end` at the end. **MSE**: `av_store_append` per SourceBuffer (`av_store_remove`, `av_store_set_offset`, `av_store_buffered`, `av_store_reset_parser`, `av_store_change_type`, `av_store_set_eos`). **A queue** (Web, the WebKit browser: WebKit does MSE's bookkeeping itself — its SourceBuffer parses with `av_demux_append` / `av_demux_read` / `av_demux_reset`, keeps the coded frames, and hands over those to decode, in decode order, a few seconds ahead): `av_store_add_queue (store)` is a source without a parser; `av_store_queue_track (store, q, track)` defines a track (again at each new initialization segment), `av_store_queue_put (store, q, t, packet)` adds a frame, `av_store_queue_level` says how far the frames reach after the playback position (the host stops putting past ~3 s), `av_store_queue_flush` empties a track (a seek: the host puts again from a random access point, then `av_player_seek`), `av_store_queue_end` tells the track's last frame is there; the player drops the frames it has played. **Lower layers alone**: `av_demux_new` / `av_demux_feed` / `av_demux_read` (packets with their track, pts / dts in microseconds, key flag), `av_decoder_new (track)` / `av_decoder_send` / `av_decoder_receive` (frames: I420 planes or float PCM), `av_yuv_to_rgb`, `av_resampler_new` / `av_resample`.

What decodes: VP9, VP8, AV1, Opus (with `codecs.mk`'s libraries), FLAC, MP3, PCM (WAV), the tests' I420, and with FFmpeg (the Media Player) nearly everything else -- `av_codec_list ()` says, `av_decoder_supported_track (track)` for a track. A file whose codec is not there plays nothing: `st.error == AV_EUNSUP` (in a page: MediaError 4). **Decode times**: Matroska keeps none (its blocks are in decode order, their times are the pictures'): its video packets come with `dts = AV_NOTIME` and the store makes them (after the last one: H.264 / H.265 with B-frames). In the file mode a jump forward in the times is the stream's own (a variable frame rate, Theora's repeated frames left out), neither a discontinuity nor a hole to wait for. **Tests**: `sh tools/tests/av/run.sh` (the library alone, PC and AArch64 under qemu; then, with

the `ffmpeg` command, `tools/tests/av/fftest.c`: FFmpeg for the PC -- `third_party/ffmpeg-7.1.2/onyx/build.sh host` -- and clips made in H.264 / AAC (MP4, TS), H.265, Xvid AVI, MPEG-2, WMV, FLV, Theora OGV, AC-3, 10-bit 4:2:2: each read and decoded whole, played in the file mode, a seek), `tools/webkit/tests/video-file.html` and `video-mse.html` (in Jet Browser, on the Pi; the VP9 / AV1 / Opus clips of `tools/tests/av/clips`, made by `mkcodec.py` with PyAV), `sh tools/tests/av/bench.sh <clips>` (the decoders' speed, PC and AArch64 under qemu, the frames' checksum the same on both).

PDF Viewer, and the PDF export (`user/Apps/pdf`, `user/Libs/pdf/pdfwrite.h`)

The reader of PDF documents (the mock-ups and the user's decisions: `docs/pdf/README.md`; its use: docs/04 §12) is a **newlib** uikit app on **MuPDF 1.28.5** (`third_party/mupdf-1.28.5`: only its `fitz` and `pdf` parts, the 14 standard fonts, jbig2dec and openjpeg — the tarball's other formats, JavaScript, HarfBuzz, lcms2, the Noto / CJK fonts left out). **The app is AGPL-3.0**, MuPDF's licence (docs/LICENSING.md).

File	What
`mupdf.mk`	MuPDF as one static library, `libmupdf.a`, for the Pi (`user/Makefile`: `MU_ONYX=1`, newlib's gaps in `mu/onyx_mucompat.{h,c}` — `quad`, `timegm`, `stat`, `ftruncate`, `getentropy`, no folder "archives") or the PC (`make -f user/Apps/pdf/mupdf.mk MU_ROOT=. MU_CC=gcc MU_OUT=...`). The `FZ_ENABLE_*` switches turn the other formats off; `TOFU` drops the Noto fonts. It holds **its own FreeType** (Onyx's 2.14.3 with the Type 1 / CFF / CID drivers PDF needs, `mu/onyx_muftmodule.h`, plus the apps' TrueType + autofit): the app links it instead of `ft/libft.a`. Onyx's zlib and libjpeg (jpeg-9f, its names hidden: `FZ_HIDE_INTERNAL_JPEG`).
`engine.h`	MuPDF's side: the contexts (one a thread, `fz_clone_context`; MuPDF's locks on `kapi_lock`), the files through the kapi (`KStream`: an `fz_stream` on `kapi_open` / `kapi_read` / `kapi_seek` — any volume, and the simulator), `Doc` (a document, its lock: a page becomes a **display list** under it, the slow drawing happens outside), the pages' sizes, the outline flattened, the links (read with the lists), `render_page` (a part of a page at a scale, turned, into 0x00RRGGBB), `page_text` (the structured text: the selection, the search), the facts and the fonts (Properties), and the **`Worker`**: a thread that draws what the window wants now (`set_wants`: the view's pages first, then the neighbours, the thumbnails, the recent documents' first pages) and searches page after page (`fz_match_stext_page`, a regular expression for *Whole words*), each result handed over with `kapi_post`.
`ui.h`	The faces, text cut to fit, the hit lists, the icons (`uikit/vpaint.h`).
`main.cpp`	The tabs (`Tab`: a document and how it is shown — layout, zoom, rotation, scroll, the selection, the search's hits), the bitmaps kept (`Bmp`: a page or, past 2600 × 2600 px, the part seen on a 256-px grid; another scale's shown, scaled, until the right one comes; 18 M pixels at most, the least used dropped), the layout (`lay_out`: continuous, one page, two pages with the first alone), the widgets — `TabBar`, `ToolBar` (`SearchBox`, the page's field), `SidePanel` (Pages, Contents, Find), `View` (the pages, the hits and the selection over them, the links, the scroll bars), `Home` (the recent documents, the folders) —, the dialogs (the password, `PropsBox`), full screen (`uk_win_fullscreen_begin`: the next page drawn ahead), the settings and `recent.tsv`.

The PDF writer — `user/Libs/pdf/pdfwrite.h`, header-only, **MIT** (Onyx's own code: docs/LICENSING.md), used by Letters' and the Spreadsheet's *File ▸ Export as PDF*: `pdfw::Writer` writes PDF 1.7 — pages of rectangles, text as **glyphs** of TrueType fonts (Type 0 / CIDFontType2, `Identity-H`: the codes are the glyphs' numbers; a **subset** of each font embedded — the glyphs used, and those their composites take, keep their numbers, the others are left empty, `loca` rewritten long —; a **ToUnicode** map so the text can be selected and found; a made bold as fill + stroke, a made italic slanted), images

(Flate with a soft mask for their transparency, or JPEG through `img/pngsave.hpp`), links (the web, a page), bookmarks (by level), the document's facts. Its streams are deflated with `pngsave::deflate`. The apps **draw their pages again into it**: Letters' `PageView::paintPage` (the screen's drawing, its three primitives — `fillClip`, `glyph`, `drawImage` — sent to `g_pdf` when it is set; 100 %: a px is 0.75 point), the Spreadsheet's `paint_pane` (`out_rect`, `out_glyph`, the lines' runs; the used cells cut into page bands, the charts drawn twice as large into images).

Tests on the PC: `sh tools/tests/run_pdf_test.sh` — MuPDF built with gcc and checked on the manuals (`tools/tests/pdf/mutest.c`: the pages, a page drawn, its text, a word found, the outline, the links), then a PDF written by `pdfwrite.h` (`tools/tests/pdf/writetest.cpp`: two fonts, an image with transparency, links, bookmarks, a landscape page) and read back. **The screenshots**: `sh tools/tests/desktop_sim/shots.sh pdf` (the manuals of `sdcard/manuals`; `letters` and `sheet` take their export's dialog). The icon: `python3 tools/icons/pdf_icon.py`.

Mail, the mail client (`user/Apps/mail`, `user/Libs/mail`)

The mail client (the mock-ups, the plan and the user's decisions: `docs/mail/README.md`; its use: docs/04 §12) is a **newlib** uikit app (`mail.elf`: FreeType's text, mbedTLS as Courier, uikit's image codecs). **MIT**, all of it. Two layers: `user/Libs/mail/` (header-only, the protocols and the HTML renderer, reusable by any app) and `user/Apps/mail/` (the app).

File	What
`mail/util.h`	`Buf`; base64, quoted-printable; the charsets met in mail (UTF-8, Latin-1, Windows-1252, ISO-8859-15) to UTF-8; RFC 2047 words both ways; RFC 5322 dates; address lists (`parse_addrs`: split first, then each name decoded).
`mail/conn.h`	`Conn`: a server's connection over the kapi sockets — TLS at once or after STARTTLS (`tls/onyx_tls.hpp`, the certificate checked against `SD:/res/ca-bundle` unless the account says not to), lines and literals, a time-out (`kapi_clock_us`; `softTimeout` for IDLE), a cancel flag.
`mail/imap.h`	IMAP4rev1: LOGIN / AUTHENTICATE PLAIN / **XOAUTH2** (SASL-IR), CAPABILITY, LIST with SPECIAL-USE (and the usual names), modified UTF-7, SELECT, UID FETCH (ENVELOPE, BODYSTRUCTURE — the part to preview, the attachments —, `X-GM-THRID`, the References), FETCH by number (a folder's newest), sections and partial bodies, UID STORE, UID MOVE (else COPY + \Deleted + EXPUNGE), APPEND (LITERAL+), IDLE. A response is read whole and parsed into a small tree (`Resp`, `Val`).
`mail/pop3.h`, `mail/smtp.h`	POP3 (CAPA, STLS, USER / PASS, AUTH PLAIN / XOAUTH2, STAT, LIST + UIDL, TOP, RETR — the dot-stuffing undone —, DELE, QUIT); SMTP submission (EHLO, STARTTLS, AUTH PLAIN / LOGIN / XOAUTH2, SIZE, MAIL / RCPT / DATA — the dots doubled —, the recipients refused reported).
`mail/mime.h`	`Mime`: a message's parts' tree (multipart, `message/rfc822`, RFC 2231 names), a part decoded and its text in UTF-8, the part to show (HTML or text), the attachments, `cid:`; `build`: a message made (text + HTML as `multipart/alternative`, inline pictures as `related`, attachments base64, UTF-8 headers, a Message-ID).

File	What
`mail/oauth.h`	Microsoft's **device code** flow (`start`, `poll`, `refresh`; the "Onyx Mail" client id built in, `SD:/etc/mail/oauth.ini` overriding it), HTTPS POST / GET over `conn.h`, a JSON field reader.
`mail/html_dom.h`, `html_css.h`, `html_layout.h`, `html.h`, `html_ft.h`	**Mail's own HTML renderer** (the user: "simple, not Jet, HTML 4 at least with CSS 2"): a forgiving parser (HTML 4's implied ends, character references), CSS 2 (a UA sheet with HTML's quirks for tables, the presentational attributes, `<style>` with `@media` by width, selectors, specificity, `!important`, `style=`), a layout to a display list (blocks and margins, inline lines, inline boxes and inline-blocks — the mail "buttons" —, floats, lists, tables with the automatic widths, colspan / rowspan, `align=center`), painted by `FtHost` with the card's fonts; the links hit-tested; remote pictures only when the host gives them.
`Apps/mail/accounts.h`	The accounts (`SD:/etc/mail/accounts.ini`), the providers known by their domain, the secrets encrypted (AES-256-GCM, `SD:/etc/mail/key`).
`Apps/mail/store.h`	The cache on the card (`SD:/mail/<id>/`: `folders.tsv`, each folder's `index.tsv`, the `.eml` fetched), the previews.
`Apps/mail/sync.h`	The **worker thread**: jobs (sync, a body, flags, move, delete, send, a draft, a check, OAuth, a picture) one after another, the IMAP sessions kept open, the tokens renewed, each result posted (`kapi_post`).
`Apps/mail/contacts.h`	Contacts: Cardfile's model (`Apps/cardfile/model.h`) on `SD:/Documents/Contacts.card` (Latin-1 there), the addresses written to, the completion.
`Apps/mail/model.h`	The state apart from the window: the jobs from what the user does, the results applied, the lists — a folder, all the inboxes, starred, a search — grouped in conversations (Gmail's thread id, else the References), the replies from Sent joined.
`Apps/mail/ui.h`, `read.h`, `compose.h`, `wizard.h`, `contactsui.h`, `main.cpp`	The window: the toolbar, the folders, the list, the reading pane (an HTML too wide drawn once and averaged down), writing (completion), the wizard and the settings, the contacts.

Tests on the PC: `sh tools/tests/run_mail_test.sh` — `tools/tests/mail/fakemail.py` (a small IMAP + POP3 + SMTP + Microsoft-endpoints server; `--demo personal|work`: the screenshots' made-up mailboxes, `demo_mail.py`) and three programs on the stand-in kernel (`SIM_REALNET`, `SIM_REALCLOCK`: real sockets and clock): `mailtest.cpp` (the protocols, MIME, OAuth, TLS refused / accepted: 98 checks), `htmltest.cpp` (the renderer: 22 checks, PNGs to look at), `modeltest.cpp` (accounts, secrets, the worker, conversations, the cache read back: 42 checks). **The screenshots**: `sh tools/tests/desktop_sim/shots.sh mail` (two demo servers, `tools/tests/mail/mkaccounts.cpp` writing the accounts and the contacts with Mail's own code). The icon: `python3 tools/icons/mail_icon.py`.

Telegram, the instant messenger (`user/Apps/telegram`)

The Telegram client (its use: docs/04 §12 *Telegram*) is a **newlib** uikit app (`telegram.elf`: FreeType's text, mbedTLS's cryptography, FileKit's inflate, uikit's image codecs). **MIT**, all of it. It speaks **MTProto 2.0** itself — no TDLib (threads, SQLite, OpenSSL: too much for Onyx) — and draws a window in the way of

Windows Live Messenger. The core (the first four headers) is portable C++ with no uikit: the PC's tests build it with `tools/tests/telegram/tgplat_host.h` in place of `tgplat.h`.

File	What
`schema.inc`	The API's **TL schema** (layer 229: 2520 constructors and functions), generated by `python3 tools/telegram/mkschema.py` from `tools/telegram/scheme/{api,mtproto}.tl` (Telegram Desktop's copies of the schema Telegram publishes). A newer layer: replace the two files, run the script.
`tl.h`	The **TL codec**, driven by the schema read at start: any object decoded into a tree of values (an `Arena` each), fields found by name (`v["message"].str ()`), absent fields as empty; any request made by name (`tl::make (a, "messages.sendMessage")`, `set ("peer", ...)`) and encoded, the `flags` fields made from the fields present. Vectors (boxed, bare), bare types, `gzip_packed`, a function's result read by its type.
`tgcrypto.h`	On mbedTLS: SHA-1 / SHA-256, **AES-256-IGE**, **RSA_PAD** (the servers' keys and fingerprints), the factoring of `pq` (Pollard-Brent), the Diffie-Hellman checks (the prime by value, else tested), **SRP** (the cloud password: PBKDF2-HMAC-SHA512, 100 000 rounds), the random generator: a CTR_DRBG seeded from an SHA-256 pool (kapi_random, the counter's jitter around memory work, the keys typed, `seed.bin` renewed at each start) — `kapi_random` alone is a timer-seeded PRNG (kernel/sys/kapi.cpp).
`mtproto.h`	`mt::Session`: one data centre — the **authorization key** made (`req_pq_multi`, `req_DH_params`, `set_client_DH_params`, non-blocking), the encrypted messages (msg_key, the KDF, msg_id, seq_no, the salt), `invokeWithLayer (initConnection (...))` on a connection's first request, the containers, `rpc_result` / `rpc_error`, `bad_server_salt`, `bad_msg_notification` (the clock corrected), `new_session_created`, acks, pings (`ping_delay_disconnect`), the pushed updates; `mt::Transport` (the platform's).
`client.h`	`tg::Client`: the sign-in (`auth.sendCode`, `signIn`, `signUp`, `account.getPassword` + `auth.checkPassword`), `*_MIGRATE_X` (the request sent again on the data centre named), the sessions by data centre (the photos' ones: `auth.exportAuthorization` / `importAuthorization`), the model (users, chats, conversations and their messages, statuses, typing, read marks) kept by the updates (`updateShort*`, `updates`, `getDifference` after a cut), sending (`random_id`, `updateMessageID`), **photos sent** (`sendPhoto`: the JPEG in parts of 128 KB, `upload.saveFilePart`, then `messages.sendMedia` with `inputMediaUploadedPhoto`, its progress) and **shown** (`msgPhoto`: the size up to 800 px, `inputPhotoFileLocation` from its data centre, `cache/m<id><type>.jpg`), reading, typing, the status, a contact added by phone number (`addContact`: `contacts.importContacts`, the answer in `added`), the bar of a person not in the contacts (`showBar`: `users.getFullUser`'s `blocked` and `settings`, `updatePeerBlocked` / `updatePeerSettings`; `addToContacts`: `contacts.addContact`; `block`: `contacts.block` / `unblock`; `hideBar`: `messages.hidePeerSettingsBar`), the profile photos (`upload.getFile`, `cache/<id>.jpg`), `session.dat`. The window watches `rev`.
`tgplat.h`	Onyx's side: the UTC clock (`kapi_clock_info`), `log.txt` (a ring written every 5 s), the entropy, `seed.bin`, FileKit's `fk_inflate` (gzip), **TCP's "intermediate" transport**

File	What
	(`0xeeeeeeee`, then each packet's length) on `kapi_tcp_*` to port 443.
`look.h`, `emoticons.h`, `rich.h`	The look (the sky, the glass frames by status, the buddies, the avatars: photos cut square and kept, else initials on Telegram's seven colours); the **emoticons drawn by vectors** (`uikit/vpaint.h`: 32 pictures, the emoji's code points and MSN's typed forms; any other emoji a badge; joiners and skin tones skipped); a message laid out (words wrapped, emoticons inline, links).
`buddylist.h`, `chatview.h`, `chatinput.h`, `signin.h`, `demo.h`, `main.cpp`	The window (`main.cpp`: the app's own api_id / api_hash, `TG_API_ID` / `TG_API_HASH` -- the project's, public by the user's choice; config.ini may give another): the contact list; the header, the messages, the display pictures; the editor (wrapped, emoticons as pictures while typed, Enter / Shift+Enter, Ctrl+V) and the picker; the picture to send (`main.cpp`: a file, the clipboard's picture or file, a drop; brought to 1280 px, JPEG by ImageKit through `pngsave.hpp`); the sign-in's steps; `--demo`'s made-up model; the layout, the menus, the notifications (`notify_action`). **Several windows** (kapi v94): a conversation's pane (`TgPane`, `buddylist.h`: its conversation, its attached picture, its header, bar, messages, line to write in, pictures' column, picker) is the main window's right side or a `ConvWindow` (`main.cpp`: a `Root (NewWindow)`, one a conversation, up to 16); the pane being served is `g_pane` — `g_open`, `g_chat`, `g_input`, `g_att`... are its fields (macros) — and every widget of a pane names its own (`m_pane`) and makes it the current one before it draws, ticks or answers an event (`tg_use`). `config.ini` `windows=0`: the conversation beside the list.

Tests on the PC: `sh tools/tests/telegram/run_tgclient_test.sh` — `tgunit_test.cpp` (the schema, TL both ways with flags / vectors / bare types / gzip, AES-IGE's known vector, `pq`, the keys' fingerprints: 21 checks), `tgmodel_test.cpp` (the client's model fed the server's objects as they come over the wire — dialogs, a history, `updateShortMessage`, a chat message, an edit, a deletion, typing, statuses, read receipts, getDifference, a message sent and confirmed both ways, a photo sent in parts then confirmed, a photo received asked for, a contact added (and a number without an account), a stranger's message and its bar (added, blocked, unblocked), the session file — and **SRP checked against the server's side**: 56 checks), then, with `TG_API_ID` / `TG_API_HASH` set (my.telegram.org's), `tgclient_test.cpp` **against Telegram's servers** over MTProto's HTTP transport through the environment's proxy: the authorization keys with the test and the production servers, the code sent, a wrong code refused, `PHONE_MIGRATE_1` followed, a production data centre refusing an impossible number. The test servers refuse their own sign-in codes since late 2024 (`PHONE_CODE_INVALID` for every client, tdlib/td#3083), so the sign-in and what follows it are tried only with `TG_TEST_LOGIN=1`; the model test covers them offline. **Not tried here**: the TCP transport (the PC's sandbox allows HTTP only) and a real account — on the Pi. **The screenshots**: `sh tools/tests/desktop_sim/shots.sh telegram` (`--demo`). The icon: `python3 tools/icons/telegram_icon.py`.

PocketQI, Pocket the desk assistant (`user/Apps/pocketqi`)

PocketQI (its use: docs/04 §12 *PocketQI*) is the Onyx port of the user's project **PocketQI** (github.com/stephaneweg/PocketQI): Pocket, an assistant whose mind is a large language model, whose hands are small apps in Onyx BASIC and whose face is two animated eyes. PocketQI runs on a robot (a bare-metal Circle kernel), an ESP32 handheld and Windows; on Onyx it is a **newlib** uikit app (`pocketqi.elf`: FreeType's text, mbedTLS for https:// as Courier, Onyx BASIC's compiler and VM built in). **MIT**, all of it (PocketQI's code is the author's own, placed under MIT here; mbedTLS is taken under Apache-2.0).

File	What
`core/groq_proto.c`, `.h`	**PocketQI's LLM protocol, unchanged** (C): the system prompt (in French: the model answers in the person's language), the history, the masked memory (`groq_mask` / `groq_unmask`: a value the person said becomes `{nom}` before it leaves, and back), the request's JSON (`chat/completions`, `response_format: json_object`), the answer's `{"langue": ..., "outcomes": [{"app": ..., "args": [...]}]}` parsed and checked (the moods, the gestures, the variables to remember), the providers (Groq, Mistral). Its texts can be replaced without rebuilding: `SD:/apps/pocketqi.app/prompt/<name>.txt` (`groq_set_lire_texte`).
`core/pocket_humeur.c`, `.h`	**PocketQI's moods, unchanged**: 19 named moods x 3 intensities, and the two axes (valence x arousal) the skills' `SetMood` uses; the way back to neutral after 12 s (`pocket_humeur_tick`).
`core/pocket_visage.c`, `.h`	**PocketQI's face, unchanged**: the eyes as signed distances, every parameter on a spring (the transitions), blinks, glances, the little signs (tears, a vein, "z z z", a heart), the states (`VISAGE_VEILLE` idle, `_ECOUTE` waiting, `_REFLECHIT` the hopping dots, `_PARLE` bobbing); drawn line by line in ARGB at any size (`pocket_visage_image_argb`).
`text.h`	UTF-8 <-> Latin-1 <-> code page 437 (Onyx BASIC's strings: `basic/basfont.h`'s tables), the files (AppKit's `kapi_open` / `kapi_save_file`: the same calls in the desktop simulator), the dates (days since 1970 from `kapi_get_datetime`).
`net.h`	One request: NetKit's `HttpClient` (`netkit/http.hpp`) with `ONYX_HTTP_TLS` (`tls/onyx_tls.hpp` over mbedTLS); the certificate is not verified (as `/bin/llm`). `POCKETQI_NO_TLS`: http:// only (the desktop simulator).
`skills.h`	**The skills** (PocketQI's `pocket_apps.cpp`, ported): the manifests (`apps/<id>/app.txt`: `llm.<action> = <types> : <what>`, `cron =`), the outcome checked against them (an action known, the arguments' count and types -- `date` YYYY-MM-DD, also a spoken date, `heure` HH:MM, `nombre`, `texte`), the program compiled with `lib/galo.bi` after it (`WITH HEADERS ... END WITH` made `WebHeader` calls first), kept compiled (`saveBax` / `loadBax`, six at most), run by a `bas::Host` whose devices are PocketQI's: `SAY:` (each line shown, then a pause to read it, as the voice would take), `MOOD:`, `LLM:` (SendLLM), `ASKINFO:`, `CRON:` / `CRON:STOP`, `[METHOD ]http(s)://...` with `HEADERS:` / `STATUS:` / `RESPONSE:`, `PRINTER:` (`PRINTER:LIST`; a text file printed through `natives.h`'s `print_lines`: 202 sent, 404, 415 a PDF, 503 no printer), and Onyx's `LANG:` (the system's language); `MAIL:` answers 501, `SCREEN:` "0 0". The skills' `SD:` is the app's folder: their paths (`SD:/apps/agenda/agenda.txt`, `SD:/etc/memoire.txt`) are PocketQI's, unchanged.
`natives.h`	**The native skills: Pocket in Onyx's own apps**, told to the LLM by manifests as the BASIC ones (the same checks, a `long` type for a note's or a mail's text) but run in C++ on the apps' data through their own models: `agenda` and `todo` -- the Calendar's events and tasks (`calendar/model.h`, included in `namespace calm`; then `calendar/agendafiles.h`'s `agenda.txt` and `reminders.txt`, shared with the Calendar, which reads `calendar.ics` again when it changed) --, `notes` (`notes/notesmodel.cpp`; Stickies told

File	What
	STK_MSG_RELOAD), mail (a draft handed to mail --compose: the person sends it), contacts (Cardfile's cf model over SD:/Documents/Contacts.card: find, add, edit, delete; the fields given as champ: valeur in French or English -- contact_field, a value alone guessed: @ an e-mail, digits a phone --; the names matched with the accents folded, several as likely: asked; no file: Mail's form, the same as mail/contacts.h's), reveil (clock/alarms.cpp; clockd told CLOCKD_MSG_RELOAD), imprimer (**PrinterKit** without a dialog: print_setup_default, the printer named found in print_printers, print_lines lays a title, the day and the lines -- wrapped with print_text_width, a new page when full -- and print_end queues the job to printd, which speaks IPP to the printer or writes the PDF; PocketQI's own IPP client is not taken: the kit does it). Mail, the Clock and Cardfile keep their files while they run: app_running (kapi_list_tasks) and Pocket asks to close them. A native skill takes the place of a BASIC one of its name (PocketQI's agenda and todo are not on the card). Their sentences are TR words.
brain.h	**Pocket's mind** (PocketQI's pocketqi_win.cpp: chat (), run_outcomes (), the cron, the stories): one turn -- the LLM asked (the server of config.ini url, else the provider chosen, then the other one), the outcomes run in order (say, humeur, geste, retenir, wiki / askinfo / ask with their second turn, next / reprendre, maketext, cron, a skill), the history kept in markers, forgotten after 10 minutes. It talks to the window through BrainUi and its fixed words are English in the source, marked TRN and translated by the window.
main.cpp	The window: FaceView (the face at 25 images a second), ChatView (the bubbles, wrapped by uk_text_wrap), the field (Entry), three questions to try, the menus, the settings (FormDialog), the memory. **The brain runs on a thread** (kapi_thread_create, 1 MB of stack): a sentence handed over with an event (g_jobEv), the person's answer to a question with another (g_ansEv); everything it shows comes back through kapi_post (on_msg: a line, a mood, a state, the end of the turn), so the face keeps moving while the network or a skill works. ONYX_HOSTED_NEW: libstdc++'s operator new (the brain keeps its text in std::string).

A skill of your own: a folder SD:/apps/pocketqi.app/apps/<id>/ with main.bas and app.txt (name, description, one llm.<action> = <types> : <what it does, for the model> line per action; cron = <actions> for those the cron may run again). The arguments come in COMMAND$, separated by tabs (Arg$ (1) is the action); Say "..." shows a sentence; Ask$ ("...") asks the person and returns the answer ("" when none); ForLLM is 1 when the skill runs for the model (askinfo: what it says goes back to the model as data); Web$, WebSend$, WebHeader, WebStatus, JsonText$ reach a web API; SetMood 60, 40 or SetMoodName "happy", 2 set the face; LANG: read gives en or fr. PocketQI Studio (PocketQI's Windows IDE) writes such skills too. The skills shipped are PocketQI's own (courses, meteo, humeur) and speak French; their genpdf / genmail actions are left out of the manifests. **A native skill of your own** (an Onyx app's data): its manifest in NATIVE_MANIFESTS, its do_<id> in natives.h, the call in native_run; its words in TR.

Following PocketQI: the three core/ files are PocketQI's commit 060b797 as they are, but for their two-line MIT header -- copy a newer one over them (keep the header). skills.h and brain.h follow

PocketQI's `pocket_apps.cpp` and `pocketqi_win.cpp`; `sdcard/apps/pocketqi.app/{apps,lib}` its `system/sd/{apps,lib}` (LF line ends here).

Tests on the PC: `sh tools/tests/desktop_sim/shots.sh pocketqi` (and `SHOTS_LANG=fr`): the window with a turn whose LLM answer is canned (`SIM_NET`: a chat completion; `config.ini url =` points the request at it) -- an appointment written in the sample Calendar and a line added to a note (the native skills; their files in the writes' folder); `sh tools/tests/server_sim/pocket640.sh <out> pocketqi`: the pocket mode at 640 x 480 by the keys alone (a question to try chosen with the D-pad, asked with A). The icon: `python3 tools/icons/pocketqi_icon.py`. **Not tried here**: Mistral and Groq themselves, over TLS, on the Pi.

Photos, the photo library (`user/Apps/photos`)

The photo library (the mock-ups, the plan and the user's decisions: `docs/photos/README.md`; its use: docs/04 *Photos*) is a **newlib** uikit app (`photos.elf`: FreeType's text, uikit's image codecs, `img/pngsave.hpp` to write JPEG and PNG, `pdf/pdfwrite.h` for the contact sheets). **MIT**. Headers included by `main.cpp`:

File	What
`exif.h`	A picture's facts without decoding it: the size (JPEG's SOF, PNG, GIF, BMP, WebP, PCX headers), the EXIF (TIFF) fields — DateTimeOriginal (local time), Make / Model, f-number, exposure, ISO, focal length, **orientation**, the camera's preview —, a date in the file's name; `jpeg_with_exif` puts the original's EXIF into a new JPEG (orientation 1, no preview).
`lib.h`	`Library`: the photos (`Photo`, a path hash index), `SD:/etc/photos/library.db` (tab-separated lines), the folders watched (`SD:/Pictures`, its `Camera`, each volume's `DCIM`, `folders.txt`), the albums (`albums/<name>.txt`, paths), the **scan thread** (walks the folders; what is known by path + size is not read again; batches posted with `kapi_post`; the gone dropped, an absent volume's kept aside).
`imgops.h`	`Pix`; orientation, quarter turns, straighten (bilinear, enlarged), crop, scaling (area average down, bilinear up), the adjustments as one tone curve (`ToneMap`: exposure, contrast, highlights, shadows) + saturation, warmth, an unsharp mask, the filters, `auto_enhance` (from the histogram).
`thumbs.h`	`Thumbs`: a thread making the thumbnails (320 px, turned the right way, kept as `thumbs/<key>.jpg`; the key = path, size, orientation) and decoding the photo shown big first (`want_full` → `onFull`); when idle, the **backlog** (`set_backlog`, at start and after each scan): every thumbnail missing on the card made, `blDone` / `blTotal` drawn as the status line's bar; a cache of the sizes drawn.
`ui.h`, `app.h`	Faces, icons (vpaint), hit lists, the colours (the library follows the theme; viewer and editor dark); what is shown (`g_src`, the search), the list by day, the selection.
`grid.h`, `viewer.h`, `editor.h`, `share.h`, `main.cpp`	The toolbar, the left column, the days' grid (only what shows is drawn; the years' strip), the albums' page; the viewer; the editor (works on a 1600 px copy, the whole photo rendered when saved); favourites, albums, trash, the lossless rotation (the EXIF orientation rewritten), Send by Mail (`mail --attach <list>`), the wallpaper (`wallpaper.h`; a photo not on `SD:` or stored turned copied upright to `SD:/res/wallpaper.<ext>`), the Clipboard, the PDF, the slideshow (`uk_win_fullscreen_begin`, a cross-fade).

On the PC: `python3 tools/tests/photos/make_samples.py <dir>` makes a library (drawn photos as JPEGs with their EXIF — some standing, orientation 6 —, a screenshot dated by its name, a `library.db` with favourites, albums); `sh tools/tests/desktop_sim/shots.sh photos` the screenshots. The mock-ups: `python3 tools/screenshot/mockup_photos.py`; the icon: `python3 tools/icons/photos_icon.py`.

A large app: Koton, the studio (`user/Apps/koton`)

Koton (the DAW: `docs/daw/README.md` has its plan and the user's decisions) is a **newlib** uikit app (`koton.elf` in `user/Makefile`: the engine, MeltySynth and the plugin host are separate objects in `Apps/koton/obj/`, the UI is headers included by `main.cpp`), with FreeType text (`fontkit/uikitface.h`: `ft_uikit_install ("DejaVu Sans", 13)` before the Root). Its parts:

Folder	What
`engine/` (namespace `kt`)	`kbase.h` (Vec, Str, .NET's seeded `Random` reproduced bit for bit, banker's rounding); `model.h/.cpp` (the song: tracks of items — a module after a silence, positions relative —, the chord track pinned last; `.sq` / `.kson` read and written with `json.hpp`); `theory` (the modes, 35 qualities, degrees and colours, 30 cadences, voice leading, the next-chord suggestions, key changes); `gen_*.cpp` (every module rendered to notes: the chord styles and grids, the articulation, the drums, the euclidean rings, the melodic line engine); `compile` (the song flattened to events per track, the tempo map); `engine` (the audio engine: one MeltySynth per track, a lock-free command ring, the mix, the preview voice, the metronome — no allocation, no kapi call in `render`); `ai*` (Koton's AI: prompts, replies placed).
`synth/`	MeltySynth (C#) ported to C++: the SoundFont reader and the synthesizer, reverb and chorus.
`plug/`	The plugin host: plugins as processes (`kplug_proto.h`, `kplug.h`), rendered ahead through shared rings, effects with their latency compensated, generators, editors shown as applets.
`ui/` (namespace `kui`)	`palette.h` (the studio's colours as a uikit theme, the drawing helpers); `doc.h` (the song open, undo / redo as JSON snapshots, the timeline's edits); `audio.h` (the SoundFont found and loaded, the engine started on an **app core**, else a real-time thread, else the UI's tick; the song compiled when the document's revision changes); `arrange.h` (the arrangement: ONE widget the size of the view, drawing only what shows); `grid.h` (NoteGrid: every editor's grid — voice rows, drum lanes, the piano roll); `editor.h` + `ed_chord.h`, `ed_rhythm.h`, `ed_riff.h` (the block editors: a form of uikit controls in columns, a grid or a wheel on the right); `ops.h` (the editors' operations: the drum catalog, euclidean rhythms, the degree vocabulary, cadences, the next chord); `dialogs.h`, `ai_dialog.h`, `chain.h` (a track's sound chain, a plugin's editor floating), `chrome.h` (the transport bar, the browser, the editor's host, the status bar).

Conventions worth keeping:

- **The document's revision**: every edit is `g_doc.checkpoint ()` (the undo step), the change, then `g_doc.changed ()`; the views and the audio host follow `g_doc.revision` (the song is compiled again once the edits pause — a drag recompiles a few times a second).
- **An editor never deletes itself**: a change that needs other controls sets `g_rebuildEditor`; the Root's tick makes the editor again from the model (the same after an undo: the pointers into the song are stale then).

- **The audio thread / core** makes no kapi call and allocates nothing; the UI posts commands (`Engine::post`) and frees the songs the engine hands back (`retired ()`).
- **Memory**: JSON documents are arenas (`json.hpp`), freed as a whole; the song and its modules are owned values (an `Item` owns its module, copies clone it). On the PC the app runs in the desktop simulator (`sh tools/tests/desktop_sim/shots.sh koton`) and under valgrind with the host allocator (`-DONYX_CPP_HPP -include new`: `onyxpp.hpp`'s allocator left out) — the engine's and the AI's own tests run under ASan / UBSan / LSan (`sh tools/tests/koton/engine_run.sh`, `ai_run.sh`, `synth_run.sh`). (ASan cannot run the simulator itself: its shadow memory takes the kapi table's fixed address.)
- **Static objects** that allocate must not be made before `main` (the kapi table is not there yet on Onyx): Koton makes its document in `main` (`g_doc` is a reference to storage constructed there).

Koton's plugins: processes over IPC (`kplug.h`, `kplug_proto.h`, `Apps/koton/plug`)

Koton's instruments, effects and generators beyond the SoundFont are **plugins**, each a process of its own — the way the Control Panel hosts its applets: a crash in one never takes Koton down.

File	What
`user/Apps/koton/plug/kplug_proto.h`	The protocol (plain C, **append-only** like the kapi ABI: a message number or a field never moves; `KP_PROTO_VERSION` grows): the shared region (`KpShm`), the rings, the mailbox messages.
`user/Apps/koton/plug/kplug.h`	The plugin's side: a plugin is its parameters, a few callbacks and `KPLUG_MAIN (desc)`; the runtime does the rest (the region, the handshake, the render thread, the parameters, the state, a generator's requests, the editor). Small DSP helpers (`KpAdsr`, `KpSvf`, `KpBiquad`, `KpNoise`, `kp_sin`, `kp_mtof`, `kp_db`).
`user/Apps/kp_<name>/`	A plugin: `main.cpp` + `plugin.json` (fm2, subsynth, pluck; delay, reverb, chorus, eq3, drive; arp, euclid, automaton — the catalogue: docs/04, *Koton's plugins*).
`Apps/koton/plug/plughost.h`	The host: the catalogue, the processes, the engine's side of each, parameters, states, generators, editors, crashes.
`Apps/koton/plug/plugshm.h`	The engine's side of a plugin — `ShmSource` (a `kt::ExternalSource`), `ShmEffect` (a `kt::Effect`) — pure memory (they run on the app core).
`Apps/koton/plug/plugctx.h`	A generator's request (the context JSON) and reply (notes → a riff); a module's state (base64 of the plugin's JSON, as Koton's files).

Packaging. `SD:/koton/plugins/<name>/main` (the program, a newlib app with the FPU) and `plugin.json`: `{ "name", "kind": "instrument" | "effect" | "generator", "text", "vendor", "aliases": [other ids: Koton's "koton.arpeggiator"...], "editor": { "w", "h" } (optional), "params": [{ "id", "name", "min", "max", "default", "unit", "step" (1: whole numbers), "choices": [names] }] }` — the host lists the plugins from the manifests without starting them; a project stores a plugin by its folder's name (`PluginSlot::id`) or an alias. `user/Makefile` builds each `Apps/kp_<name>/main.cpp` into `Apps/kp_<name>/kp_<name>.elf` (`make plugins`; not a `user/*.elf`, which `make stage` would make an app of) and `make stage` copies it with its manifest to `sdcard/koton/plugins/<name>/`. The manifest must say what the program says: `sh tools/tests/koton/plug_run.sh --manifest <name>` prints the program's own list (the tests compare them).

Starting. The host makes the plugin's **shared region** — a surface (`kapi_surface_create`) of 512 × 224 "pixels" used as 448 KB of bytes, `KpShm` — fills its identity (kind, sample rate, lead, latency, the initial state as JSON), and runs `kapi_exec_as ("…/main", "--kplug <surface> <host pid>", "kp.<name>.<n>")`. The plugin maps the region, loads the state, calls `prepare`, writes its parameter list into it, sets `ready` and sends `KP_HELLO`. The host is the IPC service `kotonplug` (one Koton hosts plugins at a time): a plugin whose host is gone ends by itself.

Real time without a round trip per block. The engine runs on an app core: no kernel call — it cannot wake anybody. But it is also the sequencer and knows the notes in advance, so it **renders ahead**:

- It counts every frame it outputs on a **stream clock** (never reset: a seek or a loop does not move it). For a track played by a plugin, a second cursor dispatches the song's notes `lead` **frames ahead** of the frame being played (4096 by default: 92.9 ms at 44.1 kHz), stamped on that clock, into the region's **event ring** (1024 events), and moves `want` (every event before it is in the ring) and `kick`.
- The plugin's **render thread** (`kapi_thread_priority`: real time) sleeps on `kick` (`kapi_wait_word`: the kernel re-reads sleeping words at each 10 ms tick, so the app core's write wakes it within 10 ms) and renders the frames `[done, want)` into the **output ring** (16384 frames), cutting its blocks at the events' frames (sample-accurate). The engine reads frame `P` of the ring when it plays `P`. The lead covers the worst wake (a tick) and the rendering many times.
- A **play** or a **seek** starts a new pre-roll: the plugins' voices are reset at the new ahead point, what they had rendered before is muted (a 3 ms fade), the played cursor waits `lead` frames, and both start together. A **loop** needs none (the ahead cursor wraps as the played one will). A **live note** (`CMD_EXT_NOTE_ON`, a MIDI keyboard on a plugin track) is stamped at the ahead point: heard `lead` late.
- An **effect**: the engine writes the track's dry block of frames `[t, t + n)` into the region's **input ring** (`want = t + n`), the plugin processes it into the output ring, and the engine takes back the frames `[t - latency, ...)` (4096 by default). While an IPC effect is connected, the engine **delays every other track by the same latency** (plugin delay compensation: a delay ring per latency, up to 4; two IPC effects in series on a track: twice as much) — the song stays aligned and is heard that much later; the playhead (`Engine::position`) is the song's frame heard now, the metronome clicks with it.
- A frame not rendered in time is **silence and a count** (`KpShm::underruns`, `Engine::pluginUnderruns`): the engine never waits. A plugin late behind the engine skips to where the engine reads (`lateFrames`).

The engine's side (`engine.h`): `ExternalSource` — `lead ()`, `noteOn / noteOff / allOff / reset (at, ...)` stamped on the stream clock, `flushTo (upTo)`, `render (l, r, n, at)` → false when not all was rendered in time; `Effect` — `latency ()` and `processAt (l, r, n, at)` besides `process` (a built-in effect keeps no latency); `CMD_EXT_NOTE_ON / OFF` (a live note on a plugin track); read by the UI: `renders` (the blocks rendered: a source or an effect taken out is free once it moved by 2), `streamClock`, `pluginUnderruns`, `extLead`, `pdcFrames`. With no plugin connected the engine renders exactly as before (the same samples: `tools/tests/koton/engine_run.sh`).

The messages (mailboxes; the requests' payloads and replies in the region's `data[]`, 128 KB of JSON, a request posted with `kp_req_post` and answered when `repSeq` = its number — the mailbox message is only a doorbell):

Message	Direction	What

Message	Direction	What
`KP_HELLO` (100)	plugin → host	`KpHello {version, kind, nparams, shm}`: it is up.
`KP_SET_PARAM` (101)	host → plugin	`KpParamMsg {index, value}`.
`KP_PARAM_CHANGED` (102)	plugin → host	Its editor moved a parameter.
`KP_STATE_GET` / `KP_STATE_SET` (103 / 104)	host → plugin	Its state, JSON: `{"v":1, "params": {id: value...}, ...its own keys}` (a flat `{id: value}`, Koton's, is read too).
`KP_GENERATE` (105)	host → plugin	A generator: the context → `{"notes": [[start, len, MIDI note, velocity]...]}`.
`KP_PARAMS` (106)	host → plugin	Its parameter list (the `plugin.json` shape).
`KP_EDITOR` (107)	host → plugin	`KpEditor {surface, w, h, themed, window, button, field, accent}`: draw your editor into that surface (the host's colours), as an applet — then the **applet protocol unchanged** (`applet_proto.h`: `AP_HELLO`, `AP_PRESENT`, `AP_EXIT` back; `AP_PTR`, `AP_KEY`, `AP_CLOSE` in).
`KP_DIRTY` (108)	plugin → host	Its state changed otherwise than by a parameter.
`KP_BYE` (109)	both	Please end / it is ending.

(The plugin's editor is drawn by the plugin's runtime itself into the surface — uikit widgets under a panel that adopts the surface — rather than by uikit's applet mode, whose `Root` checks the Control Panel's service.)

Writing a plugin (`user/Apps/kp_<name>/main.cpp`):

```
#include "../koton/plug/kplug.h"
enum { P_CUT, P_RES, NP };
static const KpParamDef P[NP] = {
    { "cutoff", "Cutoff", 20, 18000, 1200, "Hz", 0, 0 },     // id, name, min, max, default,
unit, step, choices
    { "res", "Resonance", 0.5f, 12, 1, "", 0, 0 },
};
static KpSvf s_f[2]; static int s_rate;
static void prepare (int rate) { s_rate = rate; }
static void setParam (int, float) { for (int c = 0; c < 2; c++) s_f[c].set (kp_param (P_CUT),
 kp_param (P_RES), s_rate); }
static void process (float *l, float *r, int n)                  // an effect: in place
{ for (int k = 0; k < n; k++) { l[k] = s_f[0].tick (l[k], 0); r[k] = s_f[1].tick (r[k], 0); }
 }
static const KpDesc desc = { .name = "Filter", .kind = KP_EFFECT, .params = P, .nparams = NP,
                             .prepare = prepare, .process = process, .setParam = setParam };
KPLUG_MAIN (desc)
```

- An **instrument** gives `noteOn (note, velocity)`, `noteOff`, `allOff (hard)` (hard: a seek, cut every voice) and `render (l, r, n)` (it writes `n` frames); an **effect** `process (l, r, n)`; a **generator** `generate (const KpContext &, const float *params, KpNotes &out)` — the context: the

block's length and absolute position, the key (tonic, mode, its scale), the meter (`ternary`), the tempo, the chords under the block (each its root, Koton's quality, its intervals `iv`, Koton's plugin contract `basic` / `biv`, its bass: `chordAt (beat)`), a seed and the module's whole state; the parameters of *that module* are in `params` (the process serves every block using the plugin); `out.add (start, length, MIDI note, velocity)` in beats from the block's start.

- **Threads**: `prepare`, `setParam`, the notes and `render` / `process` run on the **render thread** — no allocation, no lock, no wait, no kernel call (a NaN or a blow-up is zeroed before the mix anyway); `kp_param (i)` reads a parameter there (changes arrive between blocks, `setParam` tells). `generate`, `saveState` / `loadState` (its own keys beyond `params`) and the editor run on the main thread.
- **The editor**: made from the parameters when the plugin gives none — a knob per parameter, a drop-down for `choices`, a check box for `{"Off", "On"}` —, laid out in the size the host asks (`PlugHost::editorSize` guesses it; `editorW / editorH` or the manifest's `editor` say better). An own editor (`desc.editor (root, w, h)`) builds uikit widgets under `root` and binds parameters with `kp_knob / kp_choice / kp_toggle` (they follow a change from the host); `kp_set_param (i, v)` moves one (the host is told), `kp_dirty ()` says the state changed otherwise.
- **Tests on the PC**: define `KPLUG_DSP_ONLY` and `KPLUG_TEST_SYM=<name>`: no kernel, no uikit; `KPLUG_MAIN` exports a `KpTestApi` (attach to a region, one render pass, a request, a parameter) — see `tools/tests/koton/plug_test.cpp`.

The host, for an app (the UI thread — the one that posts the engine's commands; the generator hook may run on another):

```
PlugHost host;
bool PlugHost::init (int sampleRate);                // false: no plugins here (the service t
aken, an old kernel, the simulator)
int  PlugHost::scan (const char *dir = KP_DIR);      // the catalogue: count (), info (i), fi
nd (id or alias), countKind (kind)
void PlugHost::attach (Engine *e);                   // (0: the engine was deleted)
void PlugHost::installGeneratorHook ();              // kt::g_generatorHook -> generate (): o
ne process per generator plugin,
                                                     //   its replies cached by request; ende
d when unused for a minute
bool PlugHost::syncTrack (int track, const Track &t);   // its instrumentPlugin and inserts
(slots 0..3) made, kept, replaced,
                                                        //   connected (a disabled one: kept,
 disconnected); a running one keeps
                                                        //   its live state (after an undo: s
etState)
void PlugHost::releaseTracks (int from = 0);         // the tracks' instances from `from` on,
 ended
void PlugHost::poll ();                              // every UI tick: the mailbox (handleMes
sage (from, type, data, len) for an
                                                     //   app that reads it itself -- the res
t to `foreign`) and tick ()
PlugEditorView *PlugHost::openEditor (PlugInstance *p, uikit::Widget &parent, int x, int y, i
nt w, int h);
PlugEditorView *PlugHost::openGeneratorEditor (const GeneratorModule &m, uikit::Widget &paren
t, int x, int y, int w, int h);
bool PlugHost::pullGeneratorState (PlugInstance *p, Project &song);   // at its onParam / onD
irty: the module's state from it
void PlugHost::closeEditor (PlugEditorView *v);      // (deleting its parent does too)
void PlugHost::saveTrack (int track, Track &t);      // before saving the song: the live stat
es into its PluginSlots
void PlugHost::shutdown ();                          // the engine stopped (or deleted: attac
h (0) first)
```

and, lower down: `create (id, stateJson, wait)`, `destroy`, `restart` (a crashed one, its last state, reconnected), `connectInstrument (p, track)`, `connectEffect (p, track, slot)`, `disconnect`, `setParam`, `getState`, `setState`, `generator (id)`, `setLead` / `setLatency` (1024 .. 8192 frames, for the instances made after); a `PlugInstance` says its `info ()` (the manifest), `state ()` (`PLUG_STARTING / READY / FAILED / CRASHED / ENDING`), `error ()`, `param (i)` (its value now), `underruns ()`, `dspUs ()`, `track ()` / `slot ()`. The callbacks `onCrash` (a plugin died or hung 3 s: an effect is taken out of its track, which goes on dry; an instrument's track is silent), `onParam` (its editor moved a parameter), `onDirty`. A plugin process is ended (`KP_BYE`, killed a second later) when its slot is emptied, when a generator is unused for a minute, and at shutdown.

Limits: the kernel's 64 surfaces for the whole system (a plugin takes one, two with its editor open); 128 instances a host; 64 parameters a plugin; 1024 events in flight; 128 KB of JSON a request; the delay compensation up to 16128 frames (366 ms: three IPC effects in series on a track at the default latency) and four different latencies at once; a plugin's fault halts the Pi like any app's (§12). The editors' *Listen* (the engine's preview voice) plays a plugin track's block with the SoundFont.

Tests: `sh tools/tests/koton/plug_run.sh` — under ASan / UBSan / LSan: the rings with a producer and a consumer thread; the engine rendering ahead with fake sources (the pre-roll, a note heard exactly when played, a loop, a seek while playing, a live note, an underrun, the delay compensation and the position heard); every plugin's DSP to `/tmp/koton_plug_<name>.wav` (no NaN, no silence, the parameters' extremes); the generators from a project's context (the arpeggiator's notes are the chords' tones; Koton's states; twice the same); the states' round trips and the manifests; a real-time run (the engine paced like the audio pump, kp_fm2 and kp_delay on threads that sleep between 10 ms ticks: no underrun). Then `plug_host_run.sh`: the host itself and the plugins' kernel side (`kplug_main`, the render thread, the editor) over the desktop simulator's stand-in kernel with a small multi-process layer (processes are threads): syncTrack, a song played through them, a parameter, the states, the editor drawn and a knob dragged, a generator block and its cache, a plugin killed and started again, the shutdown (`KPLUG_SHOT=prefix` writes the editors' pictures).

7. Writing a `/bin` tool

A `/bin` tool follows the **same app model** (an ELF at EL0, §5) but reads `stdin`, writes `stdout`, and exits (no window). It is composable via the terminal's pipes.

```c
#include "appkit/appkit.h"     /* the system's calls; ax_puts, ax_putln, ax_strlen, ax_itoa,
 ... */

int main (void)
{
    char args[128];
    kapi_get_args (args, sizeof args);   /* la ligne d'arguments (chaîne) */

    /* Lire stdin et le réécrire en majuscules, par exemple : */
    char buf[256];
    int n;
    while ((n = kapi_stdin_read (buf, sizeof buf)) > 0)
    {
        for (int i = 0; i < n; i++)
            if (buf[i] >= 'a' && buf[i] <= 'z') buf[i] -= 32;
        kapi_stdout_write (buf, (unsigned) n);
    }
    return 0;   /* code de sortie récupérable via kapi_wait dans l'appelant */
}
```

- **`kapi_get_args(buf, size)`**: the entire argument line as **a single string** separated by spaces (parse the words yourself). The shell (`/bin/cmd`) applies the quotes and escapes itself (`"…"`, `'…'`, `\`: docs/04 §7) and spawns with `spawn_ex` and the exact argv; this string is rebuilt from it by the kernel — the words joined by blanks, a word with a blank in double quotes (`grep "two words" f` → `"two words" f`), at most 1023 characters. A program that needs the words exactly (one holding a `"`, a long line) reads **`kapi_get_argv`** (v75: the block `"path\0arg1\0…\0\0"`), as the POSIX programs' `argv` does. The parser is `user/BinUtils/cmdparse.h` (host test: `sh tools/tests/run_cmd_test.sh`).
- **`kapi_stdin_read(buf, n)`**: reads the task's stdin (`0` = EOF). **`kapi_stdout_write`**: writes stdout.
- To read a file passed as an argument: `kapi_open/read/close`. To list a directory: `kapi_opendir/readdir/closedir`.

The existing tools to study: `ls`, `cat`, `grep`, `wc`, `echo`, `page`, `rm`, `mkdir`, `touch`, `cp`, `mv`, `ps`, `kill`, `run`, `keyb` (in `user/BinUtils/`).

A tool written as **portable POSIX C** (`main (argc, argv)`, `open`/`read`, `pthread_create`, sockets, `poll`) is built on libonyxposix instead (§5.4): add it to `POSIX_PROGS` in `user/BinUtils/Makefile` (`posixtest` is the example). The ports' tools (`sqlite3`, `xmllint`, `curl`) come from `make -C user/BinUtils ports` (§5.5).

`memset` / `memcpy` in freestanding apps

Apps built with `-ffreestanding -nostdlib` (every uikit app and the plain `/bin` tools) have no libc, yet GCC may still emit calls to `memset`/`memcpy`/`memmove` on its own (e.g. `char buf[64] = "";`, struct copies). Since ABI v36 the kapi table has them — since v74 as **user-side** routines of the EL0 code page (`kernel/arch/aarch64/el0blob.S`: no system call), before as Circle's kernel implementations; `user/Kits/appkit/appkit.h` defines them as weak `kapi_memset`/`kapi_memcpy`/`kapi_memmove`, and `user/Makefile` / `user/BinUtils/Makefile` alias the C names onto them at link time (`KAPI_ALIASES`: `-Wl,--defsym,memset=kapi_memset …`). A new freestanding link rule must add `$(KAPI_ALIASES)`; newlib programs must not (they keep newlib's versions).

The AI helper `/bin/llm` and Koton's AI composition (`engine/ai.h`)

Koton's "compose with AI" is split like Lisa + `/bin/groq`: the app builds the prompts and places the reply; a newlib + mbedTLS helper, **`/bin/llm`** (`user/BinUtils/llm.cpp`, in `TLS_PROGS`), does the HTTPS.

- **`user/Apps/koton/engine/ai.h`** (namespace `kt`; `ai.cpp` = the prompts, `ai_place.cpp` = the replies) is a port of Koton Studio's `Engine/AI` (`AiArrangement*`, `AiArrangementPlacer`, `AiPolyArrangement`, `AiPolyrhythm`, the clients) and the `ChordModelOps` / `TimelineHelper` parts they call (`AddAiChord`, `ApplyAiDrum`, `ApplyAiRiff`, `ChordsUnder`, `AddSectionMarkers`). The **French prompts are Koton's, byte for byte** (they are tuned; `tools/tests/koton/ai_run.sh` looks every generated line up in Koton's C# sources when they are on the machine).
  - `AiRequest` — `kind` (`AI_COMPOSE` a new piece, `AI_DEVELOP` develop a theme riff after the end, `AI_ADD_TRACK` one voice over the whole piece, `AI_ADD_DRUMS`, `AI_DRUM_GROOVE` one drum module, `AI_RIFF` one riff (+ a progression when no chord is under it), `AI_POLYRHYTHM` a polyrhythmic piece), `style`, `intention`, `measures`, `fullMelody` (riffs vs rhythm-only melodic lines), `drums`, `chordsVoice`, `polyChords`, `polyDrums`, `english` (the labels the model writes), `track` / `item` (the theme, the drum module, the riff; -1 = a new one / the last riff).
  - `aiBuildPrompt (project, req, system, user)` — including the whole piece as text for "add a track" (every track's notes in beats, the chords as `[bar,degree,quality]`) and the

theme for "develop"; `aiFullPrompt` = "Copy the prompt" (works with no key: paste the answer back).
    - `aiCheckReply (req, text, summary, cap)` — parse only, a one-line summary for the dialog.
    - `aiApplyReply (project, req, text, err, cap)` — tolerant parsing (a ``` fence, text around the object, trailing commas, notes as `[a,b,c]` or objects, numbers as strings, one object for a list); placement as Koton's placer: the chord track pinned last with one degree-locked chord a bar (colour from `colourForQuality`; a chord that is not its degree's own — V in minor, V/V — stays fixed), one chord articulation a section on an "Accompaniment" track (the AI's one-bar motif, saved as a user chord style, and its melodic cell), melodic lines or riffs (`PlayRiff` + `Riff`, the out-of-harmony notes snapped) grouped by role in 4-bar blocks, drum modules from the motif (`CompressPeriodic`), polychords / polydrums in 1–4-bar modules, the sections as markers, the key / meter / tempo. `AI_COMPOSE` / `AI_POLYRHYTHM` replace the project; on an error the project is unchanged. Deliberate differences with Koton are marked `Onyx:` in the sources (the default track names follow `english`; "Do" / "Ré" and `7#9` are read right; a develop's accompaniment lies under the development).
    - `aiBuildRequestJson (provider, model, key, system, user, temperature, thinking, writer[, url])` — the request for `/bin/llm`; `aiParseLlmOutput (out, len, text, error, &lastProgress)` — its answer; `aiBuildFetchJson (url, outPath, writer)` / `aiParseFetchOutput (out, len, &bytes, error, &progress)` — the same for its download mode.
    - `g_aiProviders`, `aiProviderLabel`, `aiProviderDefaultModel` — Koton's providers.
- **`/bin/llm [request.json] [-o result.json]`** reads ONE JSON document (stdin, or the file): `{ "provider": "gemini"|"groq"|"mistral"|"claude"|"openai-compatible" (+ "deepseek", "grok", "openai"), "model", "key", "system", "user", "json": true, "temperature": 0.7, "thinking": -1, "url"?, "maxTokens"?, "timeout"? }` and writes ONE JSON line: `{"ok":true,"text":"..."}` or `{"ok":false,"error":"..."}` (exit code 0 / 1). Gemini: `POST …/v1beta/models/{model}:generateContent`, `x-goog-api-key`, `responseMimeType: application/json`, `thinkingConfig.thinkingBudget` when `thinking` ≥ 0; the text = `candidates[0].content.parts[*].text`, `MAX_TOKENS` = an error. Groq, Mistral, DeepSeek, Grok, OpenAI and any OpenAI-compatible `url`: chat/completions with `response_format: json_object`. Claude: the messages API (`max_tokens` 32000, no temperature). A **download** mode: `{ "fetch": "https://…", "out": "SD:/…" }` → the body written to `out` (redirects followed, folders made) and `{"ok":true,"bytes":N}` — Koton fetches its SoundFont with it. Progress lines go to stderr — on Onyx the same stream as stdout — as `llm: connecting to …`, `llm: sending N bytes`, `llm: waiting for the answer (N s)`, `llm: receiving N bytes [of M]`; the result is the last line starting with `{`. The response buffer grows (16 MB for an answer, 512 MB for a download; sized from `Content-Length` at once). The TLS certificate is **not verified** (no CA bundle on the card). After the handshake the TLS receive is made non-blocking (onyx_tls's own gives up after 20 s of silence; a thinking model may be silent for a minute). A busy model's answer (429, 500, 502, 503, 504 -- Gemini's free models say 503 "high demand" often, even for a short prompt) is asked again after 5, 10, 20 and 40 s (`llm: Gemini answered 503 (busy): asking again in 5 s (1/4)`) before its error is given.
- **From the app** (uikit, freestanding): build the prompt and the request, then

```
json::Writer rq (false);
kt::aiBuildRequestJson ("gemini", model, key, sys, usr, 0.7, -1, rq);
void *in = kapi_pipe (), *out = kapi_pipe ();
```

```
void *proc = kapi_spawn ("SD:/bin/llm", "", in, out);
// write rq.data () to `in` in one go (a pipe holds 8 KB, the write yields until llm h
as read it:
// llm reads all of its stdin first, as groq does for Lisa), then kapi_stream_eof (in)
// each tick: kapi_stream_read_nb (out, ...) into a growing buffer (the answer can be
100+ KB);
//   kt::aiParseLlmOutput (buf, n, text, error, &progress) -> the status bar shows `pr
ogress`
// when kapi_proc_done (proc): kapi_wait, close the pipes, then
//   if (kt::aiParseLlmOutput (buf, n, text, error)) kt::aiApplyReply (project, req, t
ext, err, sizeof err);
```

In Koton (`ui/ai_dialog.h`) this runs inside the open dialog (`runLlmIn`: a nested pump, the dialog redrawn, Cancel sets the flag that kills `llm`); `aiCheckReply` sums the reply up there, and *Apply as a new song* places it on a fresh `Project` (COMPOSE / POLYRHYTHM) or a copy of the song (the additions), taken by `Doc::adopt` -- untitled, dirty, no undo history -- after `askSave`. The app keeps the provider, the model, the API key and the dialog's last request (`aiLast`: kind, style, intention, bars, the options) in `SD:/koton/settings.json` (plain text on the card — the key field's tooltip says so); its dialog is `ui/ai_dialog.h`.

- **Tests on the PC**: `sh tools/tests/koton/ai_run.sh` — every kind of prompt, canned replies shaped like Gemini's (placed, then checked: tracks, chords through `chordAt` and a key change, every module rendered, `compileSong`, a `.kson` round trip), malformed replies, the `/bin/llm` protocol per provider (`#define LLM_PROTO_ONLY` + include `llm.cpp`) and a 60+ KB reply end to end, under ASan / UBSan / LSan; then the AArch64 compile and `make -C user/BinUtils llm.elf`.

8. AppKit's small services (formerly `applib.h`)

Declared in `appkit/appkit.h`, with the calls to the system; their bodies are AppKit's (`user/Kits/appkit/appkit_lib.inc`, in `SD:/lib/appkit.so`). Any program has them, with or without a C library, and nothing more to include. *(Until 2026-10-05 they were `user/applib.h`, a header of inline functions copied into every program; that header is gone — kapi v87.)*

- **Strings**: `ax_strlen`, `ax_streq`, `ax_strcat(dst, cap, &pos, src)` (concat without overflow), `ax_app_path(dst, cap, name, suffix)` (builds `SD:apps/<name><suffix>`), `ax_itoa(v, buf)`, `ax_fmt2(d, v)` (2-digit decimal).
- **Console**: `ax_puts(s)`, `ax_putln(s)` (to stdout).
- A minimal **`.ini` reader**: `app_ini_load("config.ini")` (from the app's folder via `kapi_app_dir`) or `app_ini_load_path("SD:/etc/theme.txt")`; then `app_ini_get(section, key, default)` and `app_ini_get_int(...)`. Sections `[xxx]`, lines `key=value`, comments `;`/`#`. The entries in the file's order: `app_ini_count()`, `app_ini_section(i)`, `app_ini_key(i)`, `app_ini_value(i)`. One file at a time a program (the store is the program's own, in AppKit's private data): 64 entries, 63 characters a name or a value, 2 KB.
- **Keyboard layout**: `ax_load_keymap("FR")` (`SD:/etc/keymaps/<name>.kmap`, else a map the kernel has).

The app-drawn widgets `applib.h` had (`ax_dropdown`, `ax_colorpick`, `ax_fill`, `ax_frame`) are gone: UIKit has the widgets (`Dropdown`, `ColorPicker`, `uk_color_dialog`). Mandelbrot, whose picture is its own canvas, keeps the few lines of its drop-down in its own source.

There is also `user/BinUtils/httpc.h` — a header-only **HTTP/1.0 client** over the v21 TCP socket calls. It does **no allocation** (the caller passes the response buffer, so all memory stays in the app's address space): `http_get(url, buf, cap, &resp)`, `http_post(...)`, or the general `http_request(method, url,`

`headers, body, len, buf, cap, &resp)`. Plain HTTP only (no TLS). Used by `/bin/wget`. This is the recommended pattern for application-level protocols: build them in a user library on top of the kernel's transport kapis, rather than adding them to the ABI.

For **REST / web-API** clients there is a reusable C++ class, `user/Include/http.hpp` (`HttpClient`/`HttpResponse`) — a header-only, freestanding **HTTP/1.1** client that works in any app (integer-only or newlib). It adds, over `httpc.h`: custom default headers (chainable `.bearer(token)`, `.accept(type)`, `.header(name,value)`), JSON helpers (`post_json`/`put_json`), response-header lookup (`r.header("Content-Type", out, cap)`), and **chunked** transfer decoding. Same no-allocation model (the caller passes `char buf[N]`; the body points into it, NUL-terminated). Errors are a negative `r.status` (`HttpError`). Example: `HttpClient api; api.bearer(tok).accept("application/json"); auto r = api.get(url, buf, sizeof buf); if (r.ok()) …`. See `/bin/httpget` for a working demo. **HTTPS:** the class has a transport seam (`http://` = plain kapi TCP; `https://` = TLS). TLS is provided by `user/Libs/tls/onyx_tls.hpp` — **mbedTLS ≥3.6.3** over the kapi sockets, with a **buffered** BIO (Circle's `CSocket::Receive` discards the remainder of a TCP segment on a short read, so we read whole segments) and **software-only crypto** (the Pi 4's Cortex-A72 has no ARMv8 crypto extensions). **Verified end-to-end on real hardware** — `httpsget` downloads real pages. To enable it in a **newlib** app: `#define ONYX_HTTP_TLS` before `#include "http.hpp"` and link the cross-built mbedTLS libs — `make -C user/Libs/tls` then `make -C user/BinUtils MBEDTLS_DIR=../tls/mbedtls` (see `user/Libs/tls/README.md`). The freestanding default (no `ONYX_HTTP_TLS`) keeps `http://` only and returns `HTTP_ERR_HTTPS` for `https://`. Demo: `/bin/httpsget`. **Not yet secure:** the RNG is a software PRNG (not the HW RNG, which stalls on the Pi 4) and certificate verification is OFF — see the TLS README.

A full HTTP client app: Courier (`user/Apps/courier`, the user guide's *Courier, the HTTP client*) — Postman for Onyx. It keeps `http.hpp`'s `Transport` (plain / TLS) but has its own engine (`net.h`): the request prepared from the model (`{{variables}}` resolved, auth, raw / urlencoded / multipart / binary bodies, the cookie jar), sent on a **thread** (`kapi_thread_create`; the result handed to the UI with `kapi_post`; `Job::cancel` polled by the read loop), the response read into a growing buffer (to 16 MB; the end found by Content-Length or the last chunk, not only by the close), gzip / deflate inflated with `img_inflate`, redirects followed with their cookies, timed with `kapi_clock_us`. Its data are Postman's own formats (`model.h`: collection v2.1, environment), read and written with `json.hpp`. The pieces: `util.h` (Str, Vec), `model.h`, `vars.h`, `net.h`, `tools.h` (tests, snippets, cURL, JSON / XML formatting), `widgets.h` (LineEdit with `{{variable}}` pills, CodeEdit — a code editor with colours, undo, selection —, KVTable with Bulk Edit, TabBar, DocTabs, Btn, Choice), `views.h`, `rail.h` (the sidebar), `dialogs.h`, `main.cpp`. Built by `user/Makefile`'s `courier.elf` rule: a newlib app as Letters (FreeType's text through uikit) linking mbedTLS (`COURIER_MBEDTLS`, default `third_party/mbedtls-3.6.3`). On the PC it builds with `-DCOURIER_NO_TLS` (`shots.sh courier`).

For **images** there is a reusable decoder, `user/Kits/imagekit/img/image.hpp` (`onyximg::decode(data, len, &w, &h)`) — built on the cross-compiled **zlib + libpng + libjpeg**. It sniffs the format (PNG signature / JPEG SOI) and decodes a byte buffer into a freshly `malloc`'d array of `0xAARRGGBB` pixels (8-bit alpha in the top byte, which the canvas ignores when blitting). Same split as TLS: the libraries are cross-built once (`make -C user/Kits/imagekit/img`, sources pinned in `user/Kits/imagekit/img/README.md` — zlib 1.3.1, libpng 1.6.44, libjpeg IJG v9f), the Onyx glue is header-only. It is a **newlib** component (uses `malloc` + the libs), and no program of the card uses it any more (its demo `/bin/imgtest` was removed on 2026-10-05): pictures are ImageKit's (§5.9), tested by `/bin/iktest`. Note: `image.hpp` is for full-colour web images; keep `user/Kits/uikit/bmp.h` for the magenta-keyed `0x00RRGGBB` icons loaded from SD.

The browser is Jet, on WebKit: the port's own document is `08-WEBKIT-PORT.md` (the POSIX layer it stands on, the patch series, the builds, its media engine and compositor). Until 2026-10-04 Jet was a port of NetSurf (`user/netsurf`, `third_party/netsurf` and its libraries — libcss, libdom, libhubbub, QuickJS, wasm3 —, `user/nsfb`, the PC bench `tools/tests/netsurf`, a Windows build): all of it left the tree when the WebKit browser took its name; the history has it (`git log -- user/netsurf`, the commit before the removal: `git show 3e2eb270:docs/06-JET-BROWSER.md`).

Two things that work came with the browsers' benches and stay in the desktop simulator: its stand-in kernel implements `kapi_file_out` (a file written in pieces, in `SIM_WRITES`) and lets `kapi_remove` delete what an app wrote there (never the card) — uikit's `FileDialog` takes Enter (Open / Save) and Esc (Cancel), its name box focused in save mode; `fileName ()` reads the box — and its clipboard is a real one: `SIM_CLIP` sets it at the start (text, or `files:PATH` for `CLIP_FILES`), `SIM_CLIPFILE` receives each copy's bytes, and every set logs `SIM-CLIPBOARD type=T len=N`. An app that pastes a copied picture reads it as Paint does (`clip_get_file` + `img_load`).

The widgets are user-side: **uikit** (`user/Kits/uikit/`, §6) is drawn entirely in the app's canvas, driven by the kernel's **pointer stream** (ABI v22: `set_pointer_handler` → `GUI_EVENT_PTR_MOVE/DOWN/UP/ENTER/LEAVE/WHEEL` with client coords; `GUI_EVENT_PTR_WHEEL` carries a signed notch delta in the `lValue` wheel field, decoded with `GUI_PTR_WHEEL`). (The first C toolkit, `uikit.h`, and the kernel-drawn widgets — `add_button`…, removed by the v29 compat break — are gone: every graphical app is on uikit.) New widgets are added there, in userland, with no kernel/ABI change.

Dynamic memory + C++ apps

Apps have **no `malloc` by default** — they use static buffers + the stack (both in the app's address space). For dynamic memory, include `user/Runtime/umm.h`: a small user allocator (size-class free lists + a `kapi_sbrk` arena). `umm_malloc` / `umm_free` / `umm_calloc` / `umm_realloc`. The heap lives at `USER_HEAP_BASE` (10 GB); its pages are owned by the address space, so they are **freed automatically when the app exits** and show up in the app's page count (`ps`, the Task Manager). `kapi_sbrk` is the underlying primitive (rarely called directly). `/bin/heaptest` exercises it.

Memory is filled on first touch (kernel v75). The heap, the stacks (the main one: 8 MB by default, `app.txt` `stack = 16M` for more; a thread's) and `kapi_vm_map`'s regions cost RAM only for the 64 KB pages actually touched; a large `sbrk` or `malloc` is cheap until written. What changes for an app: a growth larger than the free memory fails at once (`sbrk` → −1, `malloc` → 0); a page the system cannot give when touched **kills the app** ("out of memory" in `kmsg`) rather than the system; a stack overflow is a clean kill ("stack overflow"). Code that runs on an **app core**: the heap is filled as it grows once a core is acquired, and the job's stack top at `core_run`; other untouched memory a job writes is filled through core 0 (slow: up to 10 ms a request) — `kapi_vm_advise (addr, len, KAPI_MADV_WILLNEED)` it first. `kapi_vm_map` / `vm_protect` / `vm_advise` (`mmap` / `mprotect` / `madvise`) and `kapi_vm_stats` are in docs/02 §8 "v75: memory"; `/bin/memtest` exercises them.

C++ apps are supported (freestanding subset — no exceptions, no RTTI, no STL):

- Name the source `*.cpp`; the user `Makefile` builds it with `g++` (`-fno-exceptions -fno-rtti -fno-threadsafe-statics -fno-use-cxa-atexit`).
- Include `user/Runtime/onyxpp.hpp`: it defines `operator new`/`delete` (on `umm`) and the runtime stubs (`__cxa_pure_virtual`, `__dso_handle`, `__cxa_atexit`/`atexit` no-ops). Global constructors run via `crt0.S` (the `.init_array` walk); static destructors are **not** run (the app exits and its address space is reclaimed).

- Classes, inheritance and virtual methods (vtables) work; `new`/`delete` go through the user heap. No `std::string`/`std::vector` — write small containers on `umm` as needed. See `user/Apps/cppdemo/main.cpp` for a working example.

9. Packaging an app: `.app`, icons, `config.ini`

Layout of an application on the card (produced by `make stage`):

```
SD:apps/<nom>.app/
  main           l'ELF de l'app, sans extension (obligatoire)
  icon.bmp       icône 40×40, BMP 24 bpp ; le magenta 0xFF00FF est transparent (optionnel)
  app.txt        métadonnées (display name + catégorie) lues par le launcher (optionnel)
  config.ini     configuration de l'app, lue via app_ini_load() (optionnel)
```

An app may also be written in **BASIC**: `main.bas` (or a compiled `main.bax`) instead of `main`. The kernel only loads ELFs: **AppKit's `lx_*` functions** (`appkit/appkit.h`; they were `user/launch.h`) resolve the rest from **`SD:/etc/runners.ini`** ("extension = program", e.g. `bax = SD:/bin/basic`), then from the **apps' own `app.txt`** (`lx_app_for`: an emulator's `games = Game Boy Color: gbc; Game Boy: gb` — the extensions after each system's name —, any app's `opens = wad`; the app's `main` must be there), then a built-in list (those that exist): `lx_launch (name, args)` starts an app (its `main`, else the first `main.<ext>` with a runner), `lx_open (path, args)` a program file (an ELF, or by its runner), both through `kapi_exec_as` so the process is named after the app. The launchers use it: the menu bar, `run`, `fileassoc.h` (File Viewer, the dock), the dock, the Game Library. A new format = a `games =` / `opens =` line in the app's `app.txt` (installed with its package: nothing to change elsewhere), or one line in `runners.ini`. **The Game Library's systems** come from the same keys (`load_systems`: every app with `games =`, its sections sorted by `order =`); a system whose emulator is not one of the cores it carries (GB, GBA, NES, SNES; the N64's label, the GameCube's banner) shows its emulator's icon on the cards. An app written in BASIC and shipped compiled is listed in `BASIC_APPS` of `kernel/Makefile`: `make stage` compiles it with `tools/basc` (the host build of the same compiler) to `apps/<name>.app/main.bax` (Arkanoid); one in `BASIC_STANDALONE` becomes a native program, `apps/<name>.app/main` = `basc --standalone ../user/BinUtils/basic.elf prog.bas main` (the runtime of this build with the program attached, `bas::attachBax`: what the editor's *Make App* > *Standalone* makes on the Pi) -- Antigrav. See *Onyx BASIC* below.

The **app name** is the base name of the `.app` folder (without the suffix). That is what you put in `/etc/autostart` and what `kapi_list_apps` returns.

`app.txt` — friendly metadata for launchers (`key = value`, no section, read with `app_ini_load_path` / `app_ini_get(0, …)`). Every app under `SD:apps/` ships one:

```
name     = Text Editor          ; display name shown under the icon
category = Productivity          ; Productivity, Internet, Graphics, Programming, Games, Demo
s, System, Settings, Emulators, Shell
stack    = 8M                    ; optional: the app's stack (bytes, K or M), read by the KER
NEL
```

Every app runs at **EL0** (docs/02 §6, *Protected mode*): `stack` sizes its **user** stack (mapped in its own address space; 1 MB at least, 64 MB at most); its kernel stack is always 256 KB. (A `mode =` line, from kapi v73, is ignored.)

`stack` is the one key the kernel reads (`AppStackSize`, `kernel/kernel.cpp`, when it creates the app's task): an app runs on its kernel task's stack, **256 KB** by default; a bigger one is asked for here — rounded up to 64 KB, at most 64 MB (smaller: ignored). The stack is kernel memory (it must stay mapped when the

address space switches), taken from the kernel heap and reused after the app ends. Jet Browser asks for 8 MB (its JavaScript engine may use 4).

The menu bar's **Onyx** menu and the dock's drawers group the apps by `category` and show their `name`. Three categories are **not listed** there: `Shell` (the desktop's own parts: `menubar`, `dock`, `notifyd`, `agenda`, `stickies`, `lock`, the services without a window -- `clipd`, `clockd`...), `Settings` (the Control Panel's applets: reached through it) and `Emulators` (reached through the Game Library, which starts the right one for a game). A shell component also creates its window with **`WIN_FLAG_SYSTEM`** (`uk_win_create_ex` / the positioned `uikit::Root` constructor), so it is left out of `uk_win_apps` (the menu bar's Open Windows, the dock's running dots) and shown on every workspace.

Icons — `tools/gen_assets.py` procedurally generates the 40×40 BMPs (BGR bottom-up, 4-byte padding) for all the apps (a document for `tinypad`, a calculator for `tinycalc`, a glider for `life`, etc.) and the "9 squares" glyph of the *apps* button in the panel. `tools/preview_icons.py` produces a PNG preview montage. Workflow: run `gen_assets.py` (writes the `icon.bmp` files into `sdcard/apps/<name>.app/`), then `preview_icons.py` to check visually.

`config.ini` — example read by `inidemo` / `voronoy`:

```
; commentaire
greeting = bonjour
[app]
name = Mon App
version = 1
[display]
barwidth = 40
```

Screenshots and documentation

- **Screenshots (the real apps)**: `tools/tests/desktop_sim/shots.sh` builds each documented app **for the PC** against the desktop simulator's stand-in kernel (`fakekapi.cpp`, uikit with its real image codecs and fonts), plays a short script of events (clicks, keys, menu commands; a canned shell session for the terminal `SIM_PIPE`, an IRC server `SIM_NET`, the mailbox messages a process would send `SIM_MBOX` — IRC's conversation window —, sample files from `tools/tests/desktop_sim/sd/` `SIM_OVERLAY`), dumps its window — the frame uikit drew and the client area — and writes `screenshots/<name>.png` (`shot.py`: the window alone, its rounded corners transparent) or lays several over the Voronoi wallpaper (`compose.py`: `desktop.png`, `menubar.png`, `volume.png`, `clock.png`, `wifimenu.png`, `dock.png`, `agenda.png`). Run `sh tools/tests/desktop_sim/shots.sh` (all) or `sh tools/tests/desktop_sim/shots.sh paint dock` (some); ~15 s, needs g++ and Pillow + numpy. **To add an app**: add it to `APPS` and a line `sim <app> <name> "<script>" …; png <name>`. The script's steps: `wait`, `down / up / move / wheel X Y`, `rdown / rup`, `key CODE`, `mods N` (the modifiers held from then on: 1 Ctrl, 2 Shift, 4 Alt — Shift+arrows select...), `menu N` (the app's menu item N: its items counted from 0 in the order the app adds them), `winctl N`, `dump FILE`, `quit` (the window closed: the app's loop ends and what it does before leaving `main` runs — `exit` stops the process on the spot), and drag & drop from another app: `dragover X Y [FLAGS]` (`GUI_EVENT_DRAG_OVER`; FLAGS 1 Ctrl, 4 the drag left) and `drop X Y PATH|PATH... [FLAGS]` (`GUI_EVENT_DROP`, the paths a `DND_FILES` payload that `uk_win_drag_data` returns), and `waitlog N TEXT` (the script stays on this step, one main-loop turn at a time, until the file `SIM_LOG` -- where the app's output is redirected -- holds TEXT, or N turns: a test waits for what it expects, `console: done` in mediatest.sh, rather than a fixed count of `wait`s a loaded machine may not be enough for). Files: what an app writes goes to `SIM_WRITES` (never the card) — `kapi_save_file`, `kapi_mkdir`, the streams `kapi_file_out` / `kapi_file_in` (a `FILE *` behind the handle) —, and `kapi_remove` /

`kapi_rename` work on `RAM:` and on those written files. Like the kernel, the simulator makes no window over 1024 × 768 (`uk_win_create` returns 0). **Threads and the network**: the simulator runs an app's threads (kapi v67) as pthreads — `kapi_post`'s calls run at the main thread's next `pump_events`, a thread's `msleep` only sleeps (the script is the main thread's) — and with **`SIM_REALNET=1`** its TCP sockets are the PC's: an HTTP client against a local server (`python3 -m http.server`...), with `SIM_SLEEP=1` for the answers to come in real time. Without it, `SIM_NET` is what any connection receives (Courier's screenshots: a canned HTTP response). **Services, the card, a drag** (AutoDev round 1, for Notes and Stickies; the full list in `fakekapi.cpp`'s header): `SIM_SERVICES="notify,stickies"` (or `-`) — `kapi_ipc_lookup` finds only those services, any `kapi_ipc_register` succeeds, and each `kapi_mailbox_send` to one is logged `sim: send <name> type <t> "<payload>"` (and `notify ()` does not wait for a notifyd); a `SIM_MBOX` line `@<ticks>:type:pid:payload` comes only that many ticks after the start (2 a script step); `SIM_ROFS="SD:/Notes"` makes a folder read-only (a full card: root ignores `chmod`); `SIM_CURSOR=follow` gives `uk_win_cursor_pos` in screen coordinates, so a widget that drags itself moves; the step `copy SRC DST` puts a host file on the card mid-run (a file changed by another program); `SIM_STAT=1` answers `kapi_path_stat` / `kapi_clock_info`; **`SIM_CLOCK=YYYYMMDDHHMMSS`** (AutoDev round 6, the Clock) makes the date a real one that **advances with the ticks** (unset: frozen at Monday 2026-09-28 12:34:00), `kapi_clock_info` then answering with UTC = that wall time − `SIM_TZ` minutes (default 120), and `kapi_set_timezone` changing the offset for the run (always logged `sim: set_timezone <m>`); each window is logged with its flags (`sim: window <title> flags 0x33`). Tests built on them, asserting on `SIM_WRITES` and the log: `tools/tests/run_notes_sim_test.sh`, `run_stickies_sim_test.sh` (its cards read back from the dump's pixels by `tools/tests/notes/cards.py`). **The Pi's own binary on the PC**: `sh tools/tests/desktop_sim/elfrun.sh <app> [stack bytes]` (the same `SIM` script) runs `user/<app>.elf` — newlib and the code the Pi's compiler made — under `qemu-aarch64` with the simulator's kapi (`elfrun.cpp`: the ELF's segments at their addresses, the stack with a guard page); needs `g++-aarch64-linux-gnu` and `qemu-user`. Letters' and the Spreadsheet's are built with the apps' FreeType (the same sources, for the PC). (`nintendoemu.png` and `arkanoid.png` — an emulator, a BASIC program — still come from the older, simulated renderer `tools/screenshot/render.py`.)

- **Manuals** (a big app's user manual, for its users): `sdcard/manuals/<app>/<Name>.md`, its pictures in `images/` and its PDF `<Name>.pdf` next to it (the folder is `manuals`, not `Docs`: it would be the same folder as `docs/` on Windows' and FAT's case-insensitive disks). Written in English, in a small Markdown subset (headings, paragraphs, bold / italic / code, links, lists, quotes, tables, figures — a picture alone in its paragraph, an italic line after it its caption), the one `tools/manuals/build_manuals.py` documents at its top. Its pictures: a script per manual, `tools/manuals/ledger_shots.sh` (as `shots.sh`, the demo's data, the stand-in kernel's fixed day: the same pictures every time; Letters run on what Ledger printed), `annotate.py` adding numbered callouts in margins. The PDF: `python tools/manuals/build_manuals.py [its .md]` — Python only; a headless Chrome, Chromium or Edge prints it (`$CHROME` to choose one), A4, the docs' colours, Selawik (`sdcard/res/fonts`), bookmarks from the headings. Translations: `<Name>.<lang>.md` next to it (`Ledger.fr.md`, `Ledger.nl.md` → `Ledger.fr.pdf`, `Ledger.nl.pdf`), the same pictures (the apps' screens are in English: the manual names their buttons as on the screen, then translates them). Ledger's: `sdcard/manuals/ledger/Ledger.md`, `.fr.md`, `.nl.md`. Koton's: `sdcard/manuals/koton/Koton.md`, `.fr.md`, its pictures by `tools/manuals/koton_shots.sh` (the app in the simulator on its demo song, 1920x1080, clicks at fixed places; the plugin editors are the pictures `tools/tests/koton/plug_host_run.sh` made, the simulator running no plugin).

- **Word/PDF exports**: `docs/build_docs.py` converts each `.md` in `docs/` into `.docx` (via `pandoc`) then into `.pdf` (via Word/`docx2pdf`), in `docs/exports/`. Run `python docs/build_docs.py` after any modification to the `.md` files. Prerequisites (once): `pip install python-docx docx2pdf pypandoc_binary`.
- **Reminder of the project rule** (see `CLAUDE.md`): any modification of a `kapi` function or of an app **updates the docs in the same go**, regenerates the affected screenshot if the visual changes, then regenerates the exports.

Onyx BASIC (the interpreter)

- **Core** (`user/Libs/basic/`): `bas.h` (API: `bas::compile`, `bas::run`, the `bas::Host` interface), `bascomp.cpp` (lexer + one-pass compiler to bytecode, with a pre-scan of the SUB / FUNCTION / DEF FN headers and of the TYPEs), `basvm.cpp` (the stack VM: tagged values, ref-counted strings, arrays and records -- a record shared at a store is copied first, so TYPEs have value semantics; a CLASS's objects are shared, see *Classes* below -- references (by-ref arguments, the address of an element or a field: `OP_REFG` / `OP_AADDRG` / `OP_FADDR`, stored through with `OP_STREF`), a frame per call), `basnum.cpp` (number formatting / parsing and the math functions, no libc). Numeric sub-types are a compile-time matter: values are doubles; a store into an INTEGER / LONG gets an `OP_CONV` (round half even, Overflow), a fixed string an `OP_FIXSTR`; a DOUBLE is only a printing precision (the compiler tracks `dblSeen` per expression). **Errors**: `VM::fail` fails the op; before the next one `trap ()` looks for the resume point (setjmp-like): with `ON ERROR GOTO` it unwinds the frames and the value stack and jumps to the handler; `RESUME` uses the statement table (`Program::stmts`) to restart or skip the statement that failed. **Events** (`ON TIMER` / `ON KEY`) are checked every 32 ops and after each blocking statement, and fire as a GOSUB (the interrupted expression stays on the stack). **CHAIN**: `bas::run` loops, compiling the next program and moving the COMMON values and the open files into the new VM. Portable C++ (only `new` / `delete`): built into `basic/libbasic.a` with FP/SIMD (`-fno-math-errno`), and on a PC for the tests.
- **Runtime** `/bin/basic` (`basic/runtime.cpp`): the Onyx `bas::Host` — console (stdio) or a `uikit::Root` window (a text/graphics framebuffer + uikit controls, pumped by the VM through `Host::poll`, `Root::attach ()`). A windowed app (`WINDOW`) asks the platform its colours (`ScreenHost::windowColours`: Onyx = the theme's `C_BG` / `C_TEXT`, the PC none) and takes them as its background, text and drawing colours unless the program set `COLOR` first. Options `-d <dir>`, `-i` (report errors to the editor: mailbox to the IPC service `qbasic`, payload `line\0message\0`), `-s <service>` (the same to another service: QBStudio's `qbstudio`). A resizable window (`WINDOW t$, w, h, 1`: `Host::windowFlags`) reallocates its pages on a resize (`ScreenHost::userResized`) and queues the event **-2**; `MOVECONTROL` / `SHOWCONTROL` / `ENABLECONTROL` / `FOCUSCONTROL` and `MENUITEM` (a uikit `Menu` rebuilt, its items' callbacks template thunks) are `Host` virtuals.
- **Editor** `apps/qbasic` links `libbasic.a` for the syntax check.
- **QBStudio** `apps/qbstudio` (`user/Apps/qbstudio/`, a newlib app: the `qbstudio.elf` rule compiles the core's sources into it): `form.h` (the `.form` text read / written, the layout engine: `form_layout` places every element for a size), `gen.h` (the controls' library -- `#import UIKit`, `CLASS Control` with PROPERTYs over UIKit's flat functions, each control's object made by `DIM SHARED name AS Control ()` --, `generate ()`: the window's code, each place an affine function of the window's size, found by laying the form out at two sizes; the program's parts, `part_of ()` turns a line of the whole program back into its file's), `codeedit.h` (UIKit's `CodeEdit` set up for QBStudio: its faces, BASIC's words, the `.form` flags, the completion -- the editor itself moved into UIKit on 2026-10-06, `uikit/codeedit.h`), `designer.h` (the window drawn with real uikit widgets under a transparent `Overlay` that takes the mouse; the toolbox), `props.h` (the properties). Host test: `sh`

`tools/tests/run_qbstudio_test.sh` (the example project read, written back, laid out, generated, compiled and run on a scripted host).

- **Properties** (`PROPERTY T.Name AS type ... END PROPERTY` the getter, `PROPERTY T.Name (v AS type)` the setter): `rewriteProperties ()` (bascomp.cpp) turns them, before the pre-scan, into a `FUNCTION T.Name` and a `SUB T.SETPROP_Name`; `methodStatement` sends `x.Name = v` to the setter, and a read of `x.Name` finds the getter as a method.
- **A host's own words: dialects** (`bas::setDialect`, 2026-10-06; made for Turtle Quest): a program that hosts BASIC adds statements and functions of its own without touching the language -- a table of `bas::ExtWord` (`name`, `id`, `kind` -- `'s'` a statement, `'n'` / `'$'` a function giving a number / a string -- and `args`, the built-ins' spec), word **aliases** (pairs `"AVANCER", "FORWARD"`, `"SI", "IF"`: the lexer replaces the word -- another language's keywords; the word before a dot too: `CECI.n` is `THIS.n`), **`Dialect::twoWords`** (such a language's ELSEIF and WEND written in two words: at a statement's start `ELSE IF` is `ELSEIF` and `END WHILE` is `WEND` -- French's `SINON SI`, `FIN TANTQUE`; inside a one-line IF, `ELSE IF` stays a nested IF). Turtle Quest sets one dialect whatever the language (a program compiles in French as in English).
- **BASIC in French** (the library's, 2026-10-06; for the hosts that want it -- **Turtle Quest and GPIO Lab**, not QBasic, QBStudio nor `/bin/basic` for now: the user's choice): `bas::FRENCH[]` (`bascomp.cpp`) holds the pairs `"SI", "IF"` ... of the language's words, its types and the 40-pin header's (`BROCHE`, `MODEBROCHE`...; a word's first French name is the one shown), `bas::frenchDialect ()` a ready `Dialect` for a host with no words of its own (GPIO Lab), `Dialect::aliases2` a second table for a host's own words beside it (Turtle Quest's `AVANCER`...), and `bas::frenchMessage (msg, out, cap, more)` says a compiler's or a run's message with BASIC's words in French ("FOR without NEXT" -> "POUR sans SUITE"). `PINMODE`'s modes are also "SORTIE", "ENTREE", "RAPPELHAUT", "RAPPELBAS", "LIBRE". **`python tools/lang/bas_fr.py`** makes the French GPIO examples (`sdcard/basic/examples/fr/gpio_*.bas`) from the English ones: the keywords translated, the comments, strings and names from `tools/lang/gpio_fr/gpio.fr.txt` -- run it again when an example changes; `run_basic_test.sh` compiles them.
- **Two kinds of numbers** (the user, 2026-10-06; `basint.h`'s `NT_*`): `INTEGER` (= `INTEGER64`, `%`, `DEFINT`: `NT_I64`, a whole number rounded at a store, kept as a double -- exact up to 2^53) and `REAL` (= `REAL64`, a name without a type: `NT_SNG`, a double shown with 7 digits); `BYTE` (`NT_BYTE`, 0..255); the sizes, for what is written byte for byte (a TYPE's field, GET / PUT: `ntKind ()` -> `LK_*`, `ntSize ()`): `INTEGER16` (`NT_INT`), `INTEGER32` and QBasic's `LONG`, `&` (`NT_LNG`), `REAL32` and `SINGLE`, `!` (`NT_R32`), `DOUBLE`, `#` (`NT_DBL`: a REAL shown with 15 digits). `LK_I64` is a whole number of 8 bytes, `LK_BYTE` one byte. The JIT rounds an INTEGER inline (`roundWhole`, no range to check); a store into a BYTE leaves the function to the VMare rounded at a store and which keep every digit; the JIT leaves a function that converts to them to the VM and the block **`REPEAT n ... END REPEAT`** (`Dialect::repeat`; the count is evaluated once, into a hidden variable). The words are reserved while the dialect is set and take precedence over a built-in of the same name (a turtle's `COLOR`). They compile to `OP_ST S_EXT` / `OP_BI B_EXT` with the word's id as the first argument; the VM calls **`Host::ext (id, args, argc, result, why, cap)`** -- false is a run-time error at the word's line, `why` its message. No new opcode: the `.bax` format and the machine code are unchanged (the translator leaves `B_EXT` / `S_EXT` to the VM). `bas::wordList` lists the dialect's words too (an editor's colours). `setDialect (0)`: plain BASIC.
- **The statement hook** (2026-10-06): `Host::lineHook = true` before `bas::run` and **`Host::onStatement (line)`** is called before each statement the program starts (the starts of `Program::stmts`, marked once at the run's start) -- false stops the program there (`run` returns 0). Such a run is the VM's (no machine code), like TRON's. A loop with no statement inside (`DO: LOOP`)

only calls `Host::poll` (every 4096 instructions): a host that limits a run counts there too. Made for Turtle Quest's playback; the base of a debugger for QBasic / QBStudio (breakpoints: stop at a line; stepping: stop at the next statement).

- **Adding a function**: an entry in `BFNS[]` (bascomp.cpp: name, id, result type, argument spec `N`/`S`/`?`, `[` = optional from here), a `B_*` id (basint.h), its case in `VM::builtin` (basvm.cpp); something the VM cannot do itself goes through a new `bas::Host` virtual (default no-op) implemented in runtime.cpp. Statements: `simpleStatement` (fixed arguments) or a dedicated `st*` parser, `S_*` id, `VM::statement`.
- **BASIC and the kits** (`#import <kit>`, 2026-10-05): a BASIC program calls the functions of any kit by their name, and **nothing in BASIC names a kit** — a kit made tomorrow is imported like the others.
    - *What a kit says of itself*: a shared library only has places in a table (docs/03 *Shared libraries*: no name is looked up when a program runs). So each kit has a **description**, `SD:/lib/<kit>.bi`, one line per function BASIC can call — `<name> <place> <result> <arguments or -> <C name>` — made by **`tools/kitbi/kitbi.py`** from the kit's `.abi` (the places) and its headers (the C prototypes: the types). `user/Makefile` runs it for every kit of `user/Kits` (`lib/kits.bi.stamp`, part of `libs`), `make stage` copies `lib/*.bi` beside the `.so`, and each kit's package carries its `.bi`. `kitbi.py -v` says what it leaves out and why: C++ classes and overloads, a struct by value, a reference, variable arguments, more than 8 whole-number or 8 floating-point arguments, a function with no prototype in the kit's headers (the codecs AudioKit carries; FreeType in FontKit). `<name>` is the C name less the kit's prefix when at least 80 % of its functions share one (`fk_copy` → `copy`); BASIC takes both. A sixth word, optional, gives the arguments' names (`button 690 l piiiisc uk_button window,x,y,w,h,text,on_click`): BASIC does not read it, QBStudio's completion does (`main.cpp`: `imported_kits`, `kit_members`).
    - *The types*, one letter each (`basint.h`, above `struct Program`): a result `v` none, `i` / `u` 32 bits, `l` 64 bits or a pointer, `b` / `c` a byte, `h` / `w` 16 bits, `f`, `d`, `s` a `const char *` (copied into a BASIC string); an argument `i` a whole number, `p` a pointer, `c` a function, `s` a C string, `f`, `d`, `I` / `L` / `F` / `D` a pointer to numbers (`int *`, a 64-bit number's or a pointer's address, `float *`, `double *`) — a number (the address) or `BYREF variable`.
    - *Structures*: the `.bi` also lists the structures the functions take or return by pointer — `struct <name> <size> <C name>`, then `field <name> <offset> <kind> [<length> | <structure>]` (kinds `b c h w i u l f d`, `a` a text in a `char` array, `t` another structure). `kitbi.py` works the layout out itself (AArch64: natural alignment), from the kit's headers and what they include directly (`kern/kapi_abi.h` for AppKit), and **the compiler checks it**: `--check` writes `lib/<kit>.bi.check.cpp`, a file of `static_assert`s on every size and offset, which the same Makefile rule compiles (`-fsyntax-only`) — a wrong layout stops the build. Left out: a structure with a union, a bit field, C++ members or an array whose size is an expression; an array that is not a text has no field (its bytes are kept, zero from BASIC). `#import` makes each one a **TYPE of the program** (`KIT.NAME`, and under its C name: `Compiler::kalias`), its fields numbers, strings (`a`) and nested TYPEs; `Program::kstructs` / `kflds` keep the C places (`.bax` format 5). Where a function takes a pointer (`p`), a variable of such a TYPE — or an element, or a **whole array** (`name ()`: the C array) — is packed into a C structure before the call and unpacked after it (`VM::kitPack` / `kitUnpack`); `PEEKT` / `POKET address, variable` do the same at an address (a structure the kit keeps, or one a callback receives). `LEN (variable)` is the C structure's size.

- *The compiler* (`bascomp.cpp`, *kits*): `prescanImports` reads the descriptions through `bas::setKitSource` (Onyx: `bas::onyxKitSource`, `baskits.cpp`; the PC tests: a kit of their own in `host_main.cpp`), `findKitFn` finds `KIT.NAME`, `kitCall` compiles the arguments by the function's letters and emits `OP_KCALL function argc`. Only what the program calls goes into the compiled program (`Program::kits`, `Program::kfns`; `.bax` format 4), with the smallest table that has them (`KitRef::minVer`) — so a `.bax` or a standalone app runs without the `.bi`. The words `ALLOC`, `DEALLOC`, `CSTR$`, `PEEKx`, `POKEx`, `PEEKT`, `POKET`, `ADDRESSOF`, `BYREF` exist only in a program that has an `#import` (`kitsOn`): an older program that uses them as names still compiles.
- *The call* (`VM::kitCall`, `basvm.cpp`): the kit opened at its first call (`Host::kitOpen`; Onyx: `bas::onyxKitOpen` = `kapi_lib_open` + the library's `init`, AppKit at `APPKIT_TABLE_VA`), then **one C call through one function type** — eight whole numbers then eight doubles: on AArch64 each kind has its own registers (x0–x7, d0–d7), so the same call serves every signature; a `float` goes in the low half of its register. No assembly, and the same code runs the tests on x86-64 (six whole-number registers there). Strings go as Latin-1 copies that live as long as the call; a result `s` comes back as CP437. The machine code (`basjit.h`) leaves `OP_KCALL` to the VM like any instruction it does not translate.
- *Callbacks* (`ADDRESSOF (Name)`): what the kit gets is one of `CB_MAX` (48) **relays** — C functions made by a template, each bound to a SUB / FUNCTION the first time its address is taken. A relay hands its arguments (up to 8 whole numbers or pointers; a `$` parameter takes a C string) to `VM::callback`, which runs the SUB **on the VM** until it returns (`stopNf`: the loop ends when the frames are back) and gives a FUNCTION's number back; where the program was (`pc`, `opPc`, the loop's budget) is kept, so a kit may call back from inside a call or from the window's pump. An error in the SUB ends it and fails the kit call that led to it (`ON ERROR` sees it there). A kit must call back **on the program's own thread**.
- *UIKit for BASIC* (`user/Kits/uikit/flat.h`, `flat.cpp`): UIKit's table is C++ (classes to derive), so it has a **flat layer** — C functions on handles, `uk_window`, `uk_button`, `uk_set_text` … (42 entries of `uikit.bi`; table version 722). A handle is the `Widget *` itself, its `tag` saying what it is (the window: a `Root` subclass, `FlatWin`, with its menus and its resize callback). A widget's `Action` (`void (*) (Widget &)`) is given the BASIC relay as it is — a reference is the widget's address, the relay's first argument. The loop is the program's: `uk_window_wait` = one round of `Root::step ()` (new: what `Root::run ()` repeats) and 16 ms — **in BASIC the loop is BASIC's** (`DO WHILE UIKit.window_wait (win) : LOOP`), not `uk_window_run`: an error or an `END` in a SUB the window called must come back to the VM, which a loop inside the kit would never let it do. One window a program: `uk_window` gives 0 if the runtime's own screen is open, and `runtime.cpp` no longer opens its screen once the program has a UIKit window (`Root::current ()`: `openWindow` refuses; `MSGBOX`, the file dialogs and the error's box go over that window). **QBStudio's generated code is on it** (`gen.h`): the controls' library is `#import UIKit` + `CLASS Control` (a `handle`, PROPERTYs over `UIKit.get_text` …), `<Form>_Create` makes the window and each widget with `ADDRESSOF (<name>_<Event>)` when the project has that SUB, `<Form>_Sized` is the window's resize callback, `<Form>_Run` the loop. The test (`tools/tests/qbstudio/qbstudio_test.cpp`) runs the sample project against a UIKit of its own placed by the **real** `uikit.bi` (made by `kitbi.py` in the test's script): a function renamed or re-typed in `flat.h` fails it. `/bin/basic`, `qbasic` and `qbstudio` are linked against table 722: their packages need `uikit >= 1.722`.
- *User controls and Hosts in QBStudio* (2026-10-05): a form whose root is `UserControl` (`form.h`: the root El is a `K_WINDOW` with `uc` set — read, laid out and drawn as a window is) is

a **panel of controls**; a `Host` (`K_HOST`: an area of the layout, `is_area`) shows one. UIKit's flat layer has the panel (`uk_panel`: every maker's `window` is the window **or a panel**; `uk_set_parent`, `uk_width`, `uk_height`; table 726). The library: `CLASS Panel` (handle, id, the Host that shows it) and on a Host's `Control` `Load` / `Unload` / `Content` / `Resized` — the Host keeps the number and the panel of what it shows, the user control its Host, so that no two objects hold each other (reference counts: a cycle would never be freed). What is specific to each user control is reached by its number through `QBS_Build` / `QBS_Layout` / `QBS_Shown` / `QBS_Left` (`generate_panels`: one part of the program, **before the forms' code** — it has the user controls' `DIM SHARED`, which a window's `_Create` may name); `generate_start` numbers them and runs the main window (`project.ini`: `main =`). A user control is made at its first `Load` (`<Name>_Create` in the panel the Host made for it), hidden — not freed — when another takes its place, re-parented when another Host shows it. **The resize chain is the generated code's**: a window's `_Layout` moves a Host and calls `host.Resized w, h`, which calls the shown user control's `_Layout` (and so on for its own Hosts) — UIKit has no layout to run. The layout engine (`form.h`) gained `halign` / `valign` (`aligned ()`) and a Grid's `widths=` / `heights=` (`tracks_of`: pixels, `*` shares, auto); all of it stays affine in the window's size, which is what `generate ()` needs (the places are found by laying the form out at two sizes). The designer (`designer.h`) keeps its widgets from one change to the next while the elements and what they show are the same (`sync ()`: places and sizes only; `sig_of ()`: an element's look without its layout properties) and makes them again when the tree or a look changes; it scrolls (`origin ()`, `scrolled ()`: two `Scrollbar`s) without making anything. `qbstudio -bench [project]` measures a step of a drag (made again / kept; drawing; showing). In the app (`main.cpp`): `main_form ()` is now the form **shown** (the designer, the events' lists, the completion follow it), `start_form ()` the program's window; `cmd_add_usercontrol`. Test: `qbstudio_test.cpp`, `pages ()` — the project `sdcard/projects/pages` laid out, generated, compiled and run on the test's UIKit (a click replaces a page, a resize, a page shown again is not made twice).
  - *Not checked*: a wrong pointer ends the program (not the system) — as in any compiled BASIC. Memory a kit returns is freed by the program with the kit's own function (`FileKit.free`).
  - Tests: `tools/tests/basic/progs/t24_kits.bas` (PC, `.bax`, AArch64 in machine code and on the VM); on the Pi: `SD:/basic/examples/kits.bas`.
- **Keys**: the kernel delivers F1-F12 as `KEY_F1` .. `KEY_F12` (0x110..0x11B); the runtime turns them into QBasic's `INKEY$` codes (`CHR$(0) + CHR$(59..68)`, 133, 134).
- **Runtime graphics** (`runtime.cpp`): a mode table (size, text cell height, pages, colours, display scale), up to 8 page buffers (draw `apage`, show `vpage`), a 256-entry palette (the VGA default; `PALETTE` recolours the pixels already drawn), a text-cell buffer (`SCREEN ()`), `VIEW PRINT` rows, a clip rectangle (`VIEW`), scanline `PAINT`, `readRect` / `writeRect` for `GET` / `PUT`; `FULLSCREEN` uses `uk_win_fullscreen_begin` and scales the visible page (aspect kept; whole-number zoom when it covers >= 85 %). `PLAY "MB"` notes go to a 32-note queue that `bgTick ()` plays from `pump ()`; `KEYDOWN` maps its key to a `KEY_*` code for `kapi_key_held`. `PAD` / `STICK` / `STRIG` call `Host::padButtons` / `padAxis` (Onyx: `user/Include/gamepad.h`; the PC: winmm `joyGetPosEx`).
- **3D** (`basic/bas3d.h`, `basic/bas3dscene.h`): the VM's `Scene3D` turns the 3D statements into vertices + batches in the layout of kapi v53 (`G3Vertex` = `kapi_gpu_vertex3`, `G3Batch` = `kapi_gpu_batch`, same flag bits): the vertices stay in model space, each batch carries projection × view × model (a new batch when the matrix, the texture or the state changes); the shapes are lit on the CPU (the normal turned by the model matrix). `RENDER3D` calls `Host::render3d`; `ScreenHost` keeps a copy of

each texture (`GRAB3D` → `Host::texture3d`) and asks the platform (`gpuTexture` / `gpuRender`: Onyx's runtime = `kapi_gpu_texture` / `kapi_gpu_render` into the active page), else draws with `swRender` (the same semantics in software: near-plane clipping, perspective-correct, depth, culling, blending). Host test: `sh tools/tests/run_basic3d_test.sh` (a window-less `ScreenHost` saves each `RENDER3D` as a PPM and checks some pixels).

- **Machine code** (`user/Libs/basic/basjit.h`, AArch64 only, included by `basvm.cpp`; the user's decisions: on by default, the VM for what is not translated, `-m` / "Managed" for the VM). At the start of `VM::run ()` `nativeTranslate ()` turns the whole bytecode into AArch64 in memory from `Host::codeAlloc ()` (`kapi_code_alloc`, v58; no memory, another CPU, `Host::managed` or `Program::managed` -- `OPTION MANAGED`, the compile dialogs, `bas::setManaged ()` -- = the VM's loop as before). **The machine code works on the VM's own state** (value stack, frames, globals), so it need not know every instruction: one it does not translate is a call to `VM::nativeStep (pc)` = `loop (1)` (that instruction, by the same `switch`) + what the loop does between two instructions (`trap ()`, an event raised by a statement, a destructor), which returns the pc to go on at; the machine code compares it with the next instruction and otherwise goes through `dispatch` (`table[pc]`, the code of each *entry point*; none: back to the VM, which steps until the next entry). So PRINT, strings, files, graphics, controls, SUB calls, GOSUB, errors and events **are** the VM's. **Translated**: numeric constants and variables (globals, locals, by-reference parameters: a `VR` tag check), `+ - * /`, `\`, `MOD`, the comparisons (fused with the `JZ` / `JNZ` that follows), `AND OR XOR EQV IMP NOT`, `OP_CONV` (INTEGER / LONG stores), the elements of numeric arrays of 1 or 2 dimensions, the jumps; the numeric functions (`builtin ()`: ABS SGN INT FIX SQR CINT CLNG CDBL MIN MAX in line, SIN COS TAN ATN EXP LOG by a direct call to `basnum`'s functions); a reference pushed for a by-reference argument (`OP_REFG` / `OP_REFL`); **SUB / FUNCTION calls** (`call ()`): the frame is made in line exactly as `VM::enter ()` makes it -- its locals from the VM's block of locals (`VM::arena`: the frames' locals are no longer one `new []` per call, on the VM either), each with the tag it starts with, the arguments moved from the value stack -- when the SUB's locals need nothing but a tag (`plainFrame ()`: no fixed string, no TYPE record), then a direct branch to the SUB's code; and **returns** (`ret ()`) in line when the SUB's locals are all numbers (`plainReturn ()`: nothing to free), the result pushed and the caller found through `dispatch`. Otherwise, and whenever a check fails at run time (too deep, the block full, a GOSUB pending in the SUB), the stubs call `VM::nativeCall ()` / `nativeRet ()` -- `enter ()` / `popFrame ()` themselves, without the loop. A call counts down to a tick like a jump back (recursion must stay interruptible). The numbers on top of the stack live in `d8`..`d15` (`regs`, callee-saved: a call to the VM keeps them) and are written back (`flush`) before a jump, an entry point or the VM; an instruction that cannot go on (division by zero, index out of range, overflow, array not DIMmed, the stack full) **bails out**: its operands go back on the stack and the VM does it -- the error, its line and `RESUME` are the VM's. Entry points (`scan ()`): jump targets, SUB entries, after every instruction left to the VM, every statement's start and end. A jump back counts down (`w25`); at zero `VM::nativeTick ()` pumps the window and the events (at most every millisecond on the Pi). `TRON` leaves the machine code for good. `opLen ()` (`basint.h`) is the operand count of every opcode -- **keep it right when adding one** (`-DBAS_CHECK_OPLEN` makes the VM check it; an opcode the translator does not know is simply left to the VM). **Tests**: `sh tools/tests/run_basic_native_test.sh` builds the host test for AArch64 with the *bare-metal* toolchain (newlib, its system calls done by Linux ones: `tools/tests/basic/a64/linux_shim.c`) and runs it under `qemu-aarch64`: every `progs/*.bas` in machine code, then with `MANAGED=1`, then the fuzzer `a64/fuzz.py` (random programs -- every numeric type, arrays with bad indices, loops, SELECT, GOSUB, SUBs with by-reference arguments, errors under `RESUME NEXT` -- machine code against VM: 1 500 programs identical on 2026-10-05).

Measures: `basic -p` (`bas::Profile`), `tools/tests/basic/bench/calc.bas`, `tools/tests/basic/pi_prof.py`.

- **Standalone apps** (the user's form (c); no linker on the machine): `bas::attachBax ()` makes *the runtime's executable + a `.bax` + a 16-byte trailer* (`"OBAXAPP1"`, the program's offset and length); the kernel's ELF loader ignores what follows the segments, and `/bin/basic` (`attached_program ()` in `runtime.cpp`) starts by reading its own file's tail (`kapi_get_argv`: argv[0] is its path): a trailer = that program is run, every argument is the program's, the title is the app folder's. "Make App" with **Standalone** (`qbasic`, `qbstudio`, `pc/OnyxBasic`) copies `SD:/bin/basic` so; the tests' `.bax` replay goes through `attachBax` / `attachedBax`.
- **Compiled programs** (`basbax.cpp`): `saveBax ()` writes a `Program` table by table (little-endian; header "OBAX", the format and the VM's opcode / builtin / statement counts, so a `.bax` from another VM is refused), `loadBax ()` reads it back, `load ()` takes a file's bytes (a `.bax`, else source to compile) -- used by `/bin/basic` (`-c` compiles), `CHAIN`, the PC `obcore.dll` (`ob_compile`, `ob_run` with a length). `runners.ini` sends `.bax` and `.bas` to `/bin/basic` (an app's `main.bax` before its `main.bas`). The tests run every program a second time through a `.bax` (`BAX=1`). **Add opcodes / builtins / statements at the end** of their enums: the counts in the header change, old `.bax` files are then refused cleanly.
- **Methods**: `SUB Class.Name` (the part before the last dot is a CLASS) is a procedure `CLASS.NAME` whose first parameter is `THIS`, the object; `fieldPath ()` turns an unknown last part of a path into `Ref.method`, and the call pushes the object (`emitThis`) then the arguments (`callMethod`). `SUB Class.new` is the constructor: `DIM v AS Class (args)` and `NEW Class (args)` (`OP_NEWREC`: a fresh object) call it. A TYPE only holds data: a method on a TYPE is refused (the user's decision, 2026-10-05; before, a TYPE had methods with `THIS` by reference). Errors: `trap ()` keeps the SUB frames (each remembers its value-stack depth, `sp0`), so `RESUME [NEXT]` continues inside the procedure, as in QBasic.
- **Classes** (`CLASS` / `INTERFACE`; the language: docs/04 §13): a class and an interface are `TypeInfo`s like a TYPE (`kind` `TK_CLASS` / `TK_IFACE`, `parent`), so `TY_REC + index` types a variable of any of the three. **The compiler**: `prescanTypes ()` first registers every class / interface name (a field may name a class defined further down), then lays the fields out — a child's start with **a copy of its parent's**, so a field keeps its index down the hierarchy; `prescan ()` reads the modifiers (`procModifier`: `VIRTUAL` / `OVERRIDE` / `ABSTRACT`, kept in `PDecl.mod`) and the methods declared in an `INTERFACE` block (procedures `IFACE.NAME` without a body); `buildClasses ()` then fills, per class, `Program::vtab` — the parent's table copied, the overrides put in place, the new virtual methods after (`PDecl.vslot`) — and `Program::itab` — per interface implemented (the parents' included) the pair *interface, where its methods' procedures start in `vtab`* — and checks the rules (OVERRIDE needed / without a target, the same parameters, every interface method present, abstract classes). `findMethod ()` walks up the parents; `callMethod ()` emits `OP_CALL` (a plain method, `BASE.Name`, a constructor), `OP_VCALL slot argc` or `OP_ICALL interface slot argc`. `THIS` of a class's method is the **object by value** (`emitThis`), and an object argument is never passed by reference. `conv ()` is the assignment rule (up: nothing; down or through an interface: `OP_CAST type`). The class words are not in `KEYWORDS`: they are known by their place (`classHeader`, `procModifier`, `isBaseCall`, `NOTHING` when no variable has the name). **The VM**: an object is a `Rec` like a record but tagged `VO` instead of `VT` — `own ()` (the copy before a store) only copies `VT`, which is the whole difference between a value and a reference; `p = 0` is `NOTHING`; slots, array elements and fields whose type is a class start as `VO` / 0 (`initSlot`, `newArr`, `newRec` look at `types[].kind`). `OP_VCALL` / `OP_ICALL` find the object under the arguments and take the procedure from its class's tables (`Rec::type`); `OP_ISTYPE`, `OP_SAMEOBJ`, `OP_CAST` use `Program::isA ()`; `OP_NIL` pushes `NOTHING`. **Destructors**: `rrel ()` does not free an object whose class (or a parent) has a `SUB Class.delete`: it keeps it by one

reference in `dtorQ`, and the loop, before the next instruction, calls the destructor like a SUB (`enter ()`, the code shared with `OP_CALL`); when that frame ends the object comes back to `rrel ()`, which queues the next parent's destructor or frees it (`Rec::fl` = how far it got). Nothing runs after the program's end. **`.bax`**: format 2 adds the classes' fields of `TypeInfo`, `vtab` and `itab` after the tables of format 1, format 3 the program's flags (managed); older files still load. Tests: `t20_classes`, `t21_cls_*` (the compile errors), `t22_classwords` (the words stay free).

- **Character set**: BASIC text is code page 437. `user/Libs/basic/basfont.h` (generated by `tools/gen_basfont.py` from Debian console-setup's VGA fonts) holds the 8 × 8 / 14 / 16 glyphs that `glyph ()` plots, and the Latin-1 <-> CP437 tables: the lexer translates string literals and `DATA`, the host translates keys, and the VM's `cstr ()` / `pushL1 ()` translate what goes to or comes from the system (controls, paths, clipboard, dialogs); `craw ()` keeps screen text (`DRAWTEXT`) and `ENVIRON` as they are. A PC render of it: `run_games_test.sh BASICRT` (programs in `tools/tests/basic/rt/`).
- **The PC tools** (`pc/`, built on Linux by `sh pc/build.sh` into `pc/dist/`, which is committed): `obcore.dll` (`pc/obcore/obcore.cpp`, mingw-w64) is the core + `basscreen.h` with a Windows `ScreenHost` (SD:/ mapped to a folder, waveOut sound, callbacks for the window, the picture, the controls and the dialogs) and a C API (`ob_check`, `ob_words`, `ob_run`, `ob_key` / `ob_keyheld` / `ob_mouse` / `ob_event` / `ob_stop`); `OnyxBasic.exe` (`pc/OnyxBasic`, .NET Framework 4.8 WinForms, `EnableWindowsTargeting`) is the editor (`EditorForm.cs`: modules split / composed like `qbasic`) and the runtime (`Runner.cs`: the VM on a worker thread, the page drawn scaled into a bitmap, WinForms controls). Wine + wine-mono can run both on Linux for a check.
- **Koton for Windows** (`pc/Koton`, built on Linux by `sh pc/Koton/build.sh` into `pc/dist/Koton/`, committed; `pc/build.sh` runs it too). Not a port: **the Onyx sources, unchanged** -- `user/Apps/koton`, `user/Kits/uikit`, FreeType, MeltySynth, the plugins `user/Apps/kp_*`, `user/BinUtils/llm.cpp` + mbedTLS -- built with MinGW-w64 over `pc/Koton/winkapi.cpp`, the kernel's ABI table on Win32: put at `KAPI_TABLE_VA` (`VirtualAlloc`) by a constructor that runs before all the others (`init_priority`), so `KT->...` works as on the Pi. What it gives: a Windows window whose client area is the canvas (`get_chrome` answers "no frame": uikit draws none), the app's menu bar as a Windows menu (`set_menu`), its size as the work area (`win_geometry`: a maximised uikit `Root` follows the window -- `GUI_EVENT_DISPLAY_RESIZE` on `WM_SIZE`), the events queued by the window procedure and handed out by `pump_events` (the app's thread, as on Onyx); files with `SD:/` = the exe's folder (`ONYX_SD`, inherited by the programs it starts) and `C:/...` a Windows path, `.../main` and `SD:/bin/llm` being `main.exe`, `llm.exe`; threads, `wait_word` (`WaitOnAddress` re-checked every millisecond, as the Onyx kernel re-reads a sleeping word: a word another process changed wakes its sleeper); the sound ring played by WASAPI (shared mode, event driven; winmm, then a silent real-time drain, when not); winmm MIDI inputs as the USB MIDI events; processes in a job (they end with Koton), pipes for `spawn`; named services, mailboxes (a ring per process in a named mapping) and surfaces (named mappings) for the plugins; Winsock for `tcp_*` (the AI helper's HTTPS). `pc/Koton/onyxwin.h` (forced first: `-include`) stands for `onyxpp.hpp` (the C runtime's `new` / `delete`), `-DIMG_HOST_TEST` for the image codecs' libc stubs. Three things in the shared sources exist for it, none changing Onyx: `gui_value` (`kapi_abi.h`: an event's 64-bit value -- `long` on Onyx, `long long` where `long` has 32 bits) with the `GUI_PTR_*` macros in 64 bits, uikit's handlers declared with it; `kapi_memset/memcpy/memmove`'s weak definitions left out on `_WIN32` (PE has no weak symbols worth the name); uikit's file dialog lists `SD:` and the drives `C:`... on `_WIN32`. Checked under Wine (`Xvfb` + `wine explorer /desktop=onyx,1920x1080 ...\Koton.exe`, driven by `xdotool`): the demo song, playback, the editors, the plugins (a generator, an effect, an instrument: processes, editors, their sound), Compose with AI (`llm.exe` over HTTPS), the file dialog, closing. `pc/Koton/README.txt` is the user's page (copied into the folder).

- **Ledger for macOS** (`pc/macOS`, built **on a Mac** by `sh pc/macOS/build.sh` into `pc/dist/macOS/Ledger.app` + `Ledger-macOS-arm64.zip` -- the Xcode command-line tools only; not built here, so not committed). As Koton for Windows, **the Onyx sources unchanged** -- `user/Apps/ledger`, `user/Apps/letters` (it prints Ledger's documents from their templates), `user/Kits/uikit`, FreeType -- over the kernel's ABI table on macOS, in two halves: `pc/macOS/hostkapi.cpp` (POSIX: the table at `KAPI_TABLE_VA` by `mach_vm_allocate (VM_FLAGS_FIXED)` -- taken only if free --, files, time, threads, `wait_word`, processes, arguments) and `pc/macOS/cocoa.mm` (an `NSWindow` whose view draws the canvas with CoreGraphics, 1 point a pixel; the events queued and handed out by `pump_events` -- the app's thread pumps Cocoa's events itself, no `[NSApp run]`; the app's menu bar as the Mac's; `NSPasteboard`; files dropped or opened from the Finder as `GUI_EVENT_DROP`). Mach-O has no `init_priority`: `hostkapi.o` is linked first (Mach-O and ELF run the files' initialisers in the link's order) and the screen's half keeps no C++ object at file scope (they would be made after `gui_setup`). **The card** is two folders over the bundle's read-only one (`Contents/Resources/sd`, filled by `pc/macOS/card.sh`): `SD:/docs` is `~/Documents/Onyx Ledger` (`ONYX_DOCS`), the rest of `SD:/` `~/Library/Application Support/Onyx Ledger` (`ONYX_SD`; the app's `apps/<app>.app` -- Ledger's templates -- copied there at the first start); read from the user's folder else the bundle's, written in the user's (the demo company, saved, lands in Documents); `HOME:/` is `~`, `MAC:/` the Mac's `/` (a file the Finder gives: `MAC:/Users/...`). **Programs**: `SD:/apps/letters.app/main` is `Ledger.app/Contents/Helpers/Letters.app` (`ONYX_HELPERS`), the arguments passed whole in `ONYX_ARGS`; one the Mac lacks (the Spreadsheet, the File Viewer) has its file or folder shown by `open` (Numbers / Excel, the Finder). **Keys**: Cmd+letter is the Onyx Ctrl+letter (the real Ctrl too), Cmd+Left / Right Home / End, Cmd+Up / Down Ctrl+Home / End (each event carries its modifiers: `get_modifiers` answers the event's while it is handled), Option is Alt, the text through `NSTextInputClient` (dead keys). One thing in the shared sources exists for it: uikit's file dialog lists `SD:`, `HOME:`, `MAC:` on `__APPLE__`. Checked on Linux by **`sh pc/macOS/check.sh`**: the same `hostkapi.cpp` under Ledger and Letters with a screen-less half (`pc/macOS/headless.cpp`, a script of events, the window written as a picture) -- the demo company opened from the bundle's card, a quote printed (Letters started from `Helpers/`, its `.rtf` in the user's `SD:/docs/Quotes`), a document saved (the books written in the user's folder, the bundle's untouched), the templates copied, a host path opened as `MAC:/...`; `cocoa.mm` syntax-checked against GNUstep's headers (the Mac-only calls aside). `pc/macOS/README.txt` is the user's page.
- **Volume and Wi-Fi from the menu bar** (ABI v60): `user/Include/volume.h` (`volume_save` / `volume_restore`: `SD:/etc/sound.ini`) for the menu bar's volume box and `/bin/volume` (`kapi_sound_volume (vol, mute)`, −1 keeps). The Wi-Fi menu is its own app, `Apps/wifimenu` (a borderless window: it takes the keyboard for the password, unlike the TOPMOST menu bar): `kapi_wlan_scan`, the known networks parsed from / written back to `SD:/etc/wpa_supplicant.conf` (several `network={}` blocks, `priority`), then `kapi_wlan_reconnect`; it closes when another window has the keys (`uk_win_list`, `KAPI_WIN_KEYS`).
- **NintendoEMU** (`pc/NintendoEMU`): `nemucore.dll` (`core/nemucore.cpp`, mingw-w64) builds the emulator cores **unchanged** (`user/Emulators/gb`, `gba`, `nes`, `snes`, `n64`, `gc`) behind a C API — `ne_open` (the system from the extension; a GameCube disc read on demand through `discRead`), `ne_set_keys` (the Windows key states) + XInput pad 0, mapped per system exactly as the Onyx apps do, `ne_run_frame (h, draw)`, `ne_audio`, `ne_video` (the last picture, double-buffered under a lock: the game runs on its own thread), `ne_thumb` (the library's picture), `ne_save` (`<rom>.sav`, as on Onyx; a GameCube's is its **memory card** in slot A, a 2 MB raw image — Dolphin's `.raw` sizes accepted — inserted before the boot and written when `card[0].dirty`), waveOut (`ne_audio_*`; the GameCube's sound through `Machine::audioRead`). The N64 `GFrame`s go to **OpenGL**

(`ne_gl_attach (h, hwnd)` from the game's thread: a WGL context on the picture control, GLSL 1.20 — colour = texel × colour + colour2, alpha test, the batch's matrix, blending / depth / cull / wrap from the flags, as `kapi_gpu_render`; the machine's dirty textures uploaded; a CPU-drawn framebuffer as a textured quad), else `bas::swTriangles` + `g3raster` in bands of 16 rows on every core (`user/Libs/basic/bas3d.h`) at `ne_set_scale` × their size. A GameCube game with OpenGL 3.3 gets **`core/gxgl.cpp`** as its `Machine::gpu` (the GX's real pipeline on the GPU): the EFB is a framebuffer (RGBA8 + depth 24, `ne_set_scale` × 640 × 528, its rows the EFB's); a draw's vertices, the XF memory and the `GxState` go to three ring buffers (persistently mapped, their quarters fenced; else unsynchronized maps) and two uniform blocks; the vertex shader transforms, lights (the two channels, 8 lights: spot / specular attenuation, diffuse functions) and makes the texture coordinates (texgens, dual texture); the fragment shader is the TEV in integers as the hardware (the lerp with c + c >> 7, the scale inside it, compare modes, swap tables, konst colours, the last stage into PREV, & 255), indirect texturing (matrices, wraps, bump alpha), the alpha test, the z texture, the fog (with its range adjustment), RGBA6 / RGB565 EFB formats, the destination alpha (dual-source blending); GX blending / logic ops / depth / culling / scissor are GL state. Each TEV configuration gets a **specialized program** (the ubershader's code with the stages unrolled and the state's choices as constants — ~1.6× faster on an integrated GPU), compiled by the driver's threads when it has `KHR_parallel_shader_compile` (the ubershader meanwhile), else at once (the driver caches them on disk: a pause only the first time a configuration is seen). Consecutive draws of one state are one draw call; textures are uploaded with their mipmaps, sampled through cached sampler objects. An EFB copy to a texture renders the rectangle into a texture of ours converted as the texture decoder would read it (intensity Y = 0.257 R + 0.504 G + 0.098 B + 16, RGB565, RGB5A3, the depth's bytes for Z formats), box-filtered when halved; a copy to the XFB is the picture, presented into the window (`gxgl_present`). `NEMU_GX_UBER=1` forces the ubershader, `NEMU_GX_SYNC=1` compiles at once, `NEMU_GX_DUMP=<prefix>` writes the GPU's picture every 250 fields (+ what GL is). Pads: XInput, else the first Windows joystick (winmm `joyGetPosEx`) through `ne_pad_map` (PAD_* bit → button / axis end / hat direction; `PadDialog.cs`). `NintendoEMU.exe` (.NET 4.8 WinForms): `MainForm` (the library, `library.txt`, the pictures made by a worker thread), `GameForm` (the game thread: paced by the sound queue — ~70 ms — when the game makes sound, else by the clock, a picture skipped to catch up; the window draws with `StretchDIBits`). View ▸ 3D Renderer ▸ *Use the High-Performance Graphics Card* (`GpuPreference` in `GameForm.cs`) writes Windows' own per-application choice (`HKCU\Software\Microsoft\DirectX\UserGpuPreferences`, what Settings ▸ Display ▸ Graphics writes) for a laptop with two graphics cards: a .NET program cannot export `NvOptimusEnablement`. Test: `sh tools/tests/run_nemu_test.sh` (the C API on Linux: each core a few frames, a picture, the library pictures; `NEMU_ROMS=<folder>` of ROMs).

- **3DForge for Windows** (`pc/3DForge`, 2026-10-18): Onyx's 3DForge as a native Windows program, a WPF window (.NET Framework 4.8) over `forgecore.dll` (MinGW-w64, `sh pc/3DForge/build.sh` → `pc/dist/3DForge/`). The DLL is `core/forgecore.cpp` + `user/Apps/3dforge`'s headers **unchanged** (fdoc, fcam, fprint, ffdm, frender, ficons, **fview** — the tools, their gestures, the view's drawing — and fdraw) + Manifold / Clipper2, compiled over `core/shim/` (`appkit/appkit.h`, `uikit/*.h` lead to `forgeshim.h`: UIKit's `vpaint.cpp` itself, the rest — `Canvas`, `uk_rbox`, `uk_tone`, the text by GDI+ in Segoe UI, `ui::icon_load` by GDI+, `kapi_get_ticks` — the shim's). What UIKit drew on Onyx is told to the host: the view draws into the WriteableBitmap's pixels (`fc_view_draw`, `fc_view_mouse`); its frames are drawn by the **graphics card**: frender.h's `scene_show` hands the batches built for the Pi's GPU to `FORGE_HOST_SHOW` (`core/glview.cpp`: OpenGL on a hidden window's context, a multisampled framebuffer the view's size — 8 samples, else 4, the smoothness the CPU gets from drawing twice as large —, resolved, read back into the view's

pixels, y turned over by the projection); without OpenGL 3 framebuffers (an old driver, a remote desktop, Windows' "GDI Generic") or with `FORGE_SOFT=1`, `kapi_gpu_render`: frender.h's z-buffer cut in bands of rows, one thread a band (a row's span from its edges: a grid line's quad is not tested over its box). frender.h's limits for its GPU path (2048 px, the kernel's vertex count) are `FORGE_GPU_MAX_*`, raised by the shim — past them it used to fall back on one thread: 1.2 s a frame for a maximised window. A view of 1600 × 900: ~15 ms a frame (the CPU's threads: ~30 ms); the shim's `uk_rbox` / `uk_rline` fill a large box's straight parts directly (the painter works over a shape's whole box: the view's frame cost 10 ms); fui.h's right panel is a list of items (`fc_form`: head, labels, fields bound to the values, check boxes, segmented choices, drop-downs, buttons, the operation, the colours, the origin's 27 points, summaries; `forgecore.cpp` lists them) that `FormPanel.cs` makes once for a layout and fills again in place (a field being typed keeps its text while its number is the same); the timeline (`fc_tl_*`, `Timeline.cs`), the bar (`fc_state`), main.cpp's commands (`fc_cmd`) and dialogs (Export, G-code, the print file: computed by the core, `Dialogs.cs`). The icons are ficons.h's, drawn twice by the core (over black, over white) for their alpha. `3DForge.exe --shot out.png [--size WxH] [--do "tool:1;click:x,y;..."]` draws the window **off the screen**, plays the steps (`Shot.cs`), saves its picture and ends — `sh pc/3DForge/tools/shots.sh` makes `screenshots/3dforge-win*.png` so (never a capture of the screen). A change to `user/Apps/3dforge` comes to Windows by rebuilding; a new panel control kind needs its item in `forgecore.cpp` and its class in `FormPanel.cs`. **MCP** (2026-10-18): the window listens on the named pipe `\\.\pipe\3DForge` (`Control.cs`: a JSON request a line, `{"tool", "args"}`, done on the window's thread through the same `fc_*` calls as its controls — the panel's fields found by their names in `fc_form`, never a dialog —, the answer the state after it, a PNG drawn off the screen for `screenshot`); `3DForge.exe --mcp` (`Mcp.cs`, JSON-RPC over stdio, `Json.cs` a small codec) lists the tools (`get_state`, `get_panel`, `screenshot`, `command`, `set_tool`, `set_field`, `set_option`, `press`, `pick`, `view_mouse`, `look`, `key`, `timeline`, `make_shape`, `new_part`, `open_file`, `save_file`, `add_canvas`, `export`, `write_gcode`, `write_print_file`, `close_window`) and relays each call, starting the window (its own handles, `UseShellExecute`) when the pipe is not there. A size field typed with a `-` turns its way (a cylinder down: Depth). Test: `python pc/3DForge/tools/mcp_test.py` (`FORGE_OFFSCREEN=1`: the window it starts is drawn off the screen; it refuses to run while a 3DForge window is open — it would drive it). Its language: `Lang.cs` loads, when Windows speaks French, Onyx's own catalogue (`sdcard/apps/3dforge.app/lang/fr.txt`, shipped as `lang\fr.txt`) and the window's (`pc/3DForge/lang/fr.txt`, shipped as `lang\fr-win.txt`), and gives both to the core (`fc_lang`, before `fc_init`): the shim's `TR` / `uk_tr` read them (`forgeshim.h`), so 3DForge's headers translate as on Onyx; the C# says `Tr ("...")` (`{0}` formats). `fdoc.h` defines `TR` / `TRN` as nothing when no UIKit is there (the tests on the PC): its words stay English; the tables built before `main` (KIND_NAME, OP_NAME, the bar's tools, the cube's faces) are `TRN` and translated where they are drawn.

- **Tests**: `sh tools/tests/run_basic_test.sh` builds the core with a console host (`tools/tests/basic/host_main.cpp`, graphics / controls logged as text) under ASan + UBSan and compares `tools/tests/basic/progs/*.bas` with their `.out` (`--update` rewrites them); then `tools/tests/basic/dialect_test.cpp` (a dialect's statements, functions, aliases, `REPEAT`, errors; the statement hook's lines and its stop).
- **Turtle Quest** (`user/Apps/turtle/`, a newlib app as QBStudio: the `turtle.elf` rule compiles the core's sources into it; docs/04 §13). `world.h` is the game without its window: the **turtle's dialect** (`WORDS`, the French aliases `ALIASES_FR` -- the turtle's words and BASIC's keywords), the lessons (`CONCEPTS`), the **packs** (`parse_pack` / `write_pack`: `.turtle` text files, their format in the header), the **world** (the map, the turtle, the pen's lines, the painted tiles, the keys, coins and gems, the portals' pads) and **`run_program`**: the program compiled with the dialect and run **at once** by a `Recorder` --

a `bas::Host` that is the turtle (`ext ()`: a move is checked square by square against the walls; a wall, a locked door, nothing to pick is a run-time error with a sentence for the player) with the statement hook on, so that every statement started is an event (`EV_LINE`) beside the moves, turns, picks, doors, prints and portal jumps (`EV_TELEPORT`, from the pad to its twin, emitted in `Recorder::move` right after the step's `EV_MOVE` that ended on a pad: the steps left go on from the twin, the heading kept, no `Seg` across -- the window animates it); a run is stopped past 40 000 events or 3 000 polls (about 12 M instructions: "the turtle is tired"). The window (`main.cpp`) **plays the record back**: `playback_tick ()` (Root::onTick) applies the events to its own world at the speed chosen, lights each `EV_LINE`'s line in the editor (UIKit's `CodeEdit`, `hiLine`), and in step mode pauses at the next one -- so the program never runs while the window waits, and a step is exact. A drawing level's figure is its solution's lines (`target_of`); a program's lines must cover them and stay on them (`same_drawing`: points every 0.1 square, 0.15 apart at most); a `draw = color` level (`Level::draw` 2; 1 is the shape only) checks the shape first, then the colours (`same_colours`: each point near a line of the same pen), so the player is told which is wrong. The **level checks** are pure functions there too, shared by `parse_pack` and the editor so they never disagree: `check_level` (gems 1 … N without a gap or a repeat, each portal letter twice or not at all; the fault's kind and its cell), worded by `level_fault_reason` (the pack's refusal, English) and `level_fault_text` (the editor's red message, EN/FR); the editor's Gem and Portal tools are `edit_gem` / `edit_pad` (host-tested). The VM's raw "Out of stack space" becomes the friendly recursion sentence (`friendly`). Stars: `count_instructions` (the statements, not the blocks' ends nor the comments) against the level's `par`. The progress is a small `(section, key) = value` store in `SD:/apps/turtle.app/progress.ini` (a section a player: `<id>` the stars, `<id>.best` the fewest instructions of a won run -- `new_best` --, `<id>.code` the last program, `seen.<concept>`; not yet on FileKit's `fk_kv`, see docs/HANDOFF.md). Host test: `sh tools/tests/run_turtle_test.sh` -- every level of the card's packs solved by its solution with three stars, written back and read again the same, and the player's errors (a wall at its line, a locked door, an endless loop, a syntax error, French words, a figure too small; gems out of order, portal jumps, colour drawings, a recursion without a stop, the editor's tools), a lint of every level (ids unique across the packs, both languages' texts present and not cut, a colour level's pens) and exactly 48 levels. Screenshots: `shots.sh turtle` (the players' progress from `tools/tests/desktop_sim/turtle/*.ini`: 8 shots, `turtle-portals`, `turtle-fractal`, `turtle-rainbow`, `turtle-editor-gems`, `turtle-fr-gems` among them).

- **Circuits** (`user/Apps/circuits/`, a FreeType app: `FT_APPS`, `FT_EXTRA_circuits = Apps/circuits/circuit.cpp`; docs/04 §12, AutoDev round 2). The **engine** `circuit.h` / `circuit.cpp` (`namespace circuits`) has no UI and no file I/O: the parts and wires on a 40 × 30 grid (`Circuit`: `add`, `move`, `connect` -- the one place a wire is made, so the loop test is there --, `route`, `pinAt`, `wireAt`), the evaluation (`evaluate` live, `evaluate_to` depth by depth for step mode), `check` (every row of the truth table, the stars by gate count), the circuit text (`write_text` / `read_text`, canonical), the undo `History` (64 steps), the packs (`parse_pack_kv` over a FileKit `fk_kv` document read with `FK_KV_PIPES`: every error with its line), the progress (`Progress` over an `fk_kv` with `FK_KV_ESCAPES`, written back with `fk_kv_save`) and the unlocking. `lessons.h` holds the 18 lesson cards in English and French. The window is `main.cpp` + `gates.h` (the colours, the gates' outlines as integer curves into `VPath`), `board.h` (the board and its drag state machine: no capture in UIKit, `catchOutside` while a button is held) and `views.h` (the list, the card, the truth table, the cards). Calls only the inline FileKit parts on the PC (`fk_kv_*`, `fs_*`), so the generic FT build of the simulator links it. Tests: `sh tools/tests/run_circuits_test.sh` (the engine and the card's packs: every level solved by its solution with three stars and the Min gate count; ASan + UBSan), `sh tools/tests/run_circuits_sim_test.sh` (the window driven in the simulator: 61 checks on

`progress.ini`, the clipboard and the dump's pixels), `python3 tools/lang/check.py circuits`; screenshots: `shots.sh circuits` (fixtures `tools/tests/desktop_sim/circuits/*.ini`).

- **Pinball** (`user/Apps/pinball/`, a FreeType app: `FT_APPS`, `FT_EXTRA_pinball` = its core + `lib/audiokit.imp.a`, built with `-ffp-contract=off`; docs/04 §12, AutoDev round 4). The **core** has no UI and no I/O (`namespace pinball`, testable on the PC): `table.h` / `table.cpp` (the `.table` reader over FileKit's `fk_kv`: every block and key, the cross-checks, each error with its line; `LoadError::fmt` + `arg` so the window translates the reason, the formats listed in `// TR:` comments), `physics.h` / `physics.cpp` (`World`: a fixed 1/60 s frame cut into sub-steps of at most half a ball radius, segments / arcs / posts with the side kept from the sub-step's start -- no tunnelling --, flippers found in their rotating frame and striking with ω × r, the plunger, the toys, ball to ball, a seeded generator; of libm only `sqrt` (exact): `pb_sin` / `pb_cos` are its own, no fused multiply-add, so the PC and the Pi give the same bits), `rules.h` / `rules.cpp` (`Game`: score, bonus, multiplier, balls, ball save, extra balls, multiball, tilt, the `[rule]` counters; `GEvent`s for the window, cleared by the next `frame`) and `scores.h` / `scores.cpp` (the top 5 per table over `fk_kv`). The window is `main.cpp` (a `GameView` for the playfield only, stepped by a microsecond accumulator, at most 3 frames a tick; the flippers and the plunger polled with `kapi_key_held`, a key event without a hold = a tap), `draw.h` (the static layer drawn once per table and size with `VPath`, the lamps, toys, flippers and balls each frame), `panel.h` / `picker.h` (drawn widgets beside UIKit's `LcdDisplay`, `ToolButton`, `Button`, `Textbox`). `--seed N` and `--start multiball` are for the tests. Tests: `sh tools/tests/run_pinball_test.sh` (AC 1-27: the reader and its errors, containment and no tunnelling over thousands of launches, every toy, the rules, the scores; ASan + UBSan, then -O2 with the same determinism fingerprint), `sh tools/tests/run_pinball_sim_test.sh` (the window in the simulator: a refused table, a table opened, the tap and the hold, the pause, a whole game to the name entry and `scores.ini`, in French too), `python3 tools/lang/check.py pinball`; screenshots: `shots.sh pinball` (the simulator's `hold` / `release` steps hold a key: `kapi_key_held` answers them; fixtures `tools/tests/desktop_sim/pinball/scores.ini`, `sd/docs/pinball/*.table`). The tables are generated by `python3 tools/pinball/mktables.py` (edit them there).
- **Critters** (`user/Apps/critters/`, a FreeType app: `FT_APPS`, `FT_EXTRA_critters` = its core + `lib/audiokit.imp.a`; docs/04 §12, AutoDev round 5). The **core** has no UI, no I/O and **no floating point** (`namespace critters`, testable on the PC; the test script greps it for `float`, a random number, a clock, `kapi_`, UIKit): `terrain.h` (a material byte and a colour a pixel, the map's edges, dig / burst / brick, the changed rectangle, an incremental hash), `level.h` (the `.level` reader over FileKit's `fk_kv`, each error with its line, `LoadError::fmt` + `arg` for the window's `TR`, the formats in `// TR:` comments; `build_terrain`: shapes in order, polygons by scanlines at pixel centres), `world.h` (`World::tick` = one step of 1/20 s in a fixed order, the six roles, `assign` / `can_take` / `pick`, the nuke, the end, a per-step `checksum`; `Clock::due` tells the window how many steps to run -- pause and fast forward change when steps run, never what a step does), `solution.h` (the `.sol` reader, `Replay`, `run_solution`) and `progress.h` (`progress.ini` over `fk_kv`: no Critters type in its API -- it can move to FileKit). The window is `main.cpp` (a `GameView` for the 800 x 320 play area: the terrain blitted x2, the world stepped by `Clock`; `--replay <sol> --until <step|end>` runs the world headless to a step, the sounds dropped -- the simulator's scripts and the screenshots reach a mid-level state in one step), `draw.h` (the creatures, hatch, exit and roles' pictures in `VPath`, the creatures' frames pre-rendered once -- drawn on black and on white, the opacity is the difference -- and blended), `bar.h` (the skill slots, the status line, the minimap), `picker.h` (the level list, the preview, the facts). Tests: `sh tools/tests/run_critters_test.sh` (AC 1-24, 29-31: the reader and its errors, every rule pixel by pixel, the 12 levels each won by its recorded solution in `tools/tests/critters/solutions/` and lost when nothing is done; ASan + UBSan, then -O2 with

the same determinism fingerprint; `tools/critters/crsim` is built too: the headless runner to make a solution -- `--trace`, `--where STEP`, `--search`, `--ppm`), `sh tools/tests/run_critters_sim_test.sh` (the window in the simulator: a refused level, a level opened, a replay to its end and `progress.ini`, a role given by the mouse and by Tab / Enter, the cards, a restart, a locked level), `python3 tools/lang/check.py critters`; screenshots: `shots.sh critters` (fixtures `tools/tests/desktop_sim/critters/`, `sd/docs/critters/`). The levels are generated by `python3 tools/critters/mklevels.py` (edit them there; a geometry change re-checks the solutions: `crsim --search`).

- **Clock** and **clockd** (`user/Apps/clock/`, `user/Apps/clockd/`; docs/04 §12, AutoDev round 6) -- **an app and its service without a window**, the pattern for any program that must act while its app is closed. **One UI-free core beside the app** that both programs (and the menu bar's bell) link: `alarms.h` / `alarms.cpp` (the model of `SD:/apps/clock.app/alarms.txt` over FileKit's `fk_kv` -- `fk_kv_block_new / _get / _set` write the many `[alarm]` blocks, the unknown keys kept --, `alarm_next`, `alarms_next`, snooze / stop / missed, the `[timer]` hand-over and its states, `alarms_missed_since`, and the **Ringer**: each alarm minute rung once, within 2 minutes, never the past at its start, a jump back of the clock rung nothing twice) and `clocktime.h` / `clocktime.cpp` (days and wall minutes, the world rows, the timer and the stopwatch from the ticks, the texts); no `kapi_`, no UIKit in it (`run_clock_test.sh` greps). `clock_proto.h`: the services `clock` / `clockd` and their messages. **clockd** (`clockd.elf`: newlib as `clipd`, `category = Shell`, `run clockd` in `etc/autostart`) registers its service (a second one quits), reads the file at its start, on `CLOCKD_MSG_RELOAD` and when its signature changed (every 30 s), steps the Ringer every 0.5 s (`trusted` = a real date and 90 s since the boot, `--grace N`), and **has no words**: a ring is handed to the Clock (`CLOCK_MSG_OPEN "--ring <id>"` to the running one, else `lx_launch ("clock", "--ring <id>")`), which notifies and shows it in the system's language; only when the Clock can be neither told nor started does clockd send a word-free bubble. The once alarms missed while the Pi was off go the same way at its start (`--missed <id> <minute> ...`: all in one, a Clock started for it notifies, writes `missed =` and ends without a window). It also calls SystemKit's `locale_zone_sync ()` at its start and once a minute (the summer time applied to the system's clock). **The rule for a close (03 R-1)**: a window's close cannot be refused (the close box and the menu bar's Quit, `MENU_QUIT`, end the loop), so **no exit may lose anything** -- the Clock asks nothing and hands over at every exit: a running timer to clockd (`[timer]` with its end in ticks and in UTC, RELOAD; taken back by a Clock started before its end), the stopwatch's start and laps in `config.ini`. The window is `main.cpp` (the tabs, the menus, the keys -- Ctrl+1...4 in `onKey`, not as menu keys: `UK_CTRL ('1')` is ^Q --, one Clock at a time, the arguments) and its views (`world.h`, `alarmsview.h`, `timerview.h`, `swview.h`, `ring.h`, `sounds.h` -- the alarms' sounds on AudioKit's FM voices, `ak_out_open (0, 0)` probing the output first --, `ui.h`: the `Veil` cards over the window, modal, the loop and `onTick` going on under them; `Spin`, a `NumericUpDown` that passes Ctrl, Enter and Esc on). Tests: `sh tools/tests/run_clock_test.sh` (the simulator's clock, SystemKit's zones at an instant and their sync, the core: alarm and clocktime cases with UBSan, -O1 and -O2), `sh tools/tests/run_clock_sim_test.sh` (clockd and the Clock built for the PC with `-Wall -Wextra`, no warning allowed, and driven through `SIM_CLOCK`, `SIM_SERVICES`, `SIM_MBOX`: rings, reloads, the summer-time sync, the timer handed over, the missed alarms, every tab, the keys, the hand-over at a close, the menu bar's bell), `python3 tools/lang/check.py clock` (and `menubar`); screenshots: `shots.sh clock` (fixtures `tools/tests/desktop_sim/clock/`: `clock-<tab>.png` and each `-fr`; `clock.png` is the menu bar's calendar).

10. Extending the `kapi` ABI

The ABI is **append-only**. To add a kernel function callable by apps, there are **5 points** to touch (all in the same direction, at the end):

1. **`kernel/include/kern/kapi_abi.h`** — add the function pointer **at the end** of `struct TKApiTable` (never in the middle, never reorder), with a version comment, and **increment `KAPI_ABI_VERSION`**.

   ```
   // --- v22 additions ---   (next free version; v21 added the TCP sockets)
   int (*ma_fonction) (int arg);
   ```

2. **`kernel/sys/kapi.cpp`** — implement `extern "C" int kapi_ma_fonction(int arg)`. It runs in the app's context (use `CurrentAS()` for the current address space if needed).

3. **`kernel/sys/kapitable.cpp`** — declare the `extern "C"` prototype and **assign the pointer** in `KApiTableInit()`: `t->ma_fonction = kapi_ma_fonction;`.

4. **AppKit** — declare it in `user/Kits/appkit/appkit.h` and give its body in `user/Kits/appkit/appkit_calls.inc` (§5.10); commit `user/Kits/appkit/appkit.abi`, which the build completes:

   ```
   KAPI_FN int kapi_ma_fonction (int arg);                                    // appkit.h
   KAPI_CALL (int, kapi_ma_fonction, (int arg), { return KT->ma_fonction (arg); })    // appkit_calls.inc
   ```

5. Rebuild (`make`). The apps that want the new function use it; the old ones keep working (they ignore the new field).

Apps at EL0 (v73/v74) — what a new kapi must also do:

- **Check every pointer an app hands it, at the `kapi_*` entry point** (not in the helpers below it, which the kernel calls with its own memory): `kern/uaccess.h` — `CUserStr` for a string, `UserCopyIn` / `UserCopyOut` / `UserGet` / `UserPut` for a structure or an out parameter, `UserReadable` / `UserWritable` for a buffer the kernel works in directly (a file read, a socket) — and fail with the function's usual error value. These accept the user VA range only (anything from a kernel caller). Never dereference an app pointer with plain C before that check.
- **App memory may not be there yet (v75)**: the heap, the stacks and `vm_map`'s regions are filled on first touch (docs/02 §4 *Demand paging*). `UserReadable` / `UserWritable` fill the range and **pin** it until the system call returns (another thread's `vm_unmap` cannot take it away meanwhile), and the copies fill and retry — so never touch an app buffer in place without a probe first, and never allocate in an exception handler (fill in C, then retry).
- **Return no kernel pointer as a handle**: put the object in the process's handle table (`kern/handle.h`: `HandlesCurrent()->Add (pObj, type, kind)`, `Get`, pinned with `CHandleUse` across a call that may yield) so it is checked and closed when the process ends.
- **At most 8 integer or pointer arguments**, no structure returned by value, nothing in floating-point registers: a protected app reaches the kapi by `svc` with x0–x7 only.
- **No call back into the app from the kernel** (a handler, a callback): app code cannot run at EL1 (its pages are PXN). Queue it and let the app run it from its event pump, as `pop_event` / `pop_post` do.

- A new pure-computation entry (like `memcpy`) that must not cost a system call needs a user-side implementation in `kernel/arch/aarch64/el0blob.S` and its slot added to the refused list in `sys/el0.cpp`.
- Regenerate `user/BinUtils/kapi_names.h` (`python3 tools/gen_kapi_names.py`: the slot names `sysstat` shows), and after a build run `tools/el0scan.sh` on the card: an app must contain no instruction EL0 cannot run.

From v75 (the POSIX calls, `docs/POSIX-PLAN.md`): a new entry returns ≥ 0 on success and **`−KAPI_Exxx`** on failure (`kapi_abi.h`: newlib's errno values, so a libc does `errno = -r`); 64-bit values are `long long` / `unsigned long long`, never `long` (32 bits in the Windows build); a structure's size and its slot are `static_assert`ed in `kapi_abi.h` (`KAPI_CHECK_SIZE`, `KAPI_CHECK_SLOT`); and AppKit's body (`appkit_calls.inc`) tests the version **and** the slot — `return KT->version >= 75 && KT->x ? KT->x (…) : -KAPI_ENOSYS;` — since the host tables (the PC simulator, `pc/`) leave the v75 slots 0. The files and processes block (`file_*`, `path_*`, `dir_read`, `stream_write_nb`, `spawn_ex`, `proc_wait`, `get_argv`, `get_env`, `getpid`, `clock_info`, `sleep_us`) is implemented: its semantics are in docs/02 §8 *v75: files and processes*; `/bin/filetest` and `/bin/proctest` exercise it on the Pi, `tools/tests/run_ofile_test.sh` on the PC. An app can use these directly (64-bit offsets, pread / pwrite, a file unlinked while open, an argv / environment for a child) — but should not mix them with the old `open` / `save_file` on the same file at the same time.

BSD sockets and `poll` (v75, docs/02 §8 "v75: sockets and poll"): `kapi_sock_*` are the kernel half of `socket`/`connect`/`send`/`recv`… (IPv4 TCP and UDP; a port in `struct kapi_sockaddr` is in host order). A socket number is not a handle: poll it as `KAPI_PK_SOCKET`, a pipe or stream handle as `KAPI_PK_STREAM`. The `tcp_*` handles are numbers of the same table (they can be polled). With `netcore=0` nothing announces a socket's change, so a wait looks again every 10 ms; with `netcore=1` a change wakes within a tick. `shutdown (SHUT_WR)` sends no FIN (the connection ends at `close`). Test on the Pi with `/bin/nettest` (+ `tools/tests/nettest_peer.py` on the PC).

IPC between processes (v76, docs/02 §8 "v76: IPC", `docs/POSIX-PLAN.md` §14): `kapi_sock_pair` makes two connected **local sockets** (STREAM / SEQPACKET / DGRAM) whose numbers are handles of the caller (≥ `KAPI_SOCK_LOCAL_BASE`) and work with every `kapi_sock_*` call and `poll`; `kapi_sock_sendmsg` / `kapi_sock_recvmsg` carry up to 256 handles (`struct kapi_handle_xfer {h, kind, tag, fd, flags}`: an open file, a stream — `KAPI_HXF_WRITER` for a pipe's write end —, a local or IP socket, a shm object) into the receiver's table; `kapi_shm_create` / `shm_open` / `shm_ctl` / `shm_map` give **shared memory** (the same frames in every space mapping it; `kapi_vm_unmap` and the other `vm_*` work on it); `kapi_spawn_ex2` gives a child handles at chosen descriptors, the child reads them with `kapi_get_handles`; `kapi_handle_close` closes a shm (or any passable) handle. A new kapi that takes a handle should accept one received this way (it is an ordinary entry of the caller's table). Tests: `/bin/ipctest` on the Pi, `sh tools/tests/run_ipc_test.sh` (the kernel code on the PC) and the posixsim bench.

Program images (v77, docs/02 §7 *Program images* and §8 "v77: program images"): a program is read from its file **once** and the pages of its read-only segments are shared by all its processes; a second process of a running program, or any process of a preloaded one, starts without reading the card. For a program's author this means:

- **its code and constants are really read-only and shared** — a program must not write there (it never could at EL0; a kapi asked to write there fails as before). Keep the linker's layout (`-z max-page-size=0x10000`, `user.ld` / `onyx-posix.ld`): two segments in one 64 KB page are refused at load (`two segments share a page` in the kernel log);
- **its writable data is private** as always (each process gets its own copy of `.data`, a zero `.bss`);

- **the image's key is the program's path** (lower-cased, the volume first). Replace a program the way `pkg` does — write the new file beside it, remove the old, rename — or simply overwrite it: the file calls drop the old image (the processes running the old version keep it and go on; the next start reads the new file). While you iterate on a program over FTP nothing more is needed;
- `kapi_image_preload (path)` / `kapi_image_unload (path)` / `kapi_image_list (path, out, cap)` (`struct kapi_image_info`) are what `/bin/preload`, `/bin/unload` and `pkg` use;
- the kernel log has one line per start (`kmsg`): `image <path>: loaded in N ms, mapped in N ms (… KB shared, … KB private)` — `loaded` = read from the card, `shared` = already in memory. Tests: `sh tools/tests/run_image_test.sh` (the kernel code on the PC); on the Pi, `preload` and the log lines.

> **Golden rule:** never change the signature or the order of an existing field. If some semantics must change, add a **new** entry. An app can query `((const struct TKApiTable *)KAPI_TABLE_VA)->version` to find out what is available.

If you add a new **GUI event** or a **window flag**, keep the values synchronized between `user/Servers/common/wm/kern/gui/window.h` (the graphics servers' window manager) and the `#define`s in `user/Kits/appkit/appkit.h` (commented "must match").

11. Coding conventions

- **Kernel (C++)**: Circle style. `CXxx` classes, `m_Xxx` members, CamelCase methods, `boolean`/`TRUE`/`FALSE` and Circle's `u8/u16/u32/u64` types. No exceptions or RTTI. `new`/`delete` go through Circle's heap.
- **Userland**: the apps are C++ on uikit (freestanding subset, §8 *Dynamic memory* + *C++ apps*; newlib for the big ones), the `/bin` tools freestanding C. `ax_` prefix for the AppKit's `ax_*` helpers. Globals `g_xxx`. New code carries the MIT notice (docs/LICENSING.md).
- **kapi**: `extern "C"` functions named `kapi_xxx` on the kernel side; inline wrappers `kapi_xxx` on the app side.
- Respect the **comment density** and the **idiom** of the file you are modifying.
- **Git**: commit into the **Onyx repo** explicitly — the current working directory (cwd) drifts; a bare `git` may land in the wrong repo. (`circle/` is a submodule: commit a Circle change in the fork first, then the new pointer here — §1.)

12. Debugging on hardware

Bring-up is done **directly on the Pi 4** (no QEMU raspi4b). Tools:

- **On-screen exception dump**: a **kernel** fault (EL1) paints a panic + register dump on the HDMI framebuffer (`PanicToScreen` + Circle's handler) and is kept in `SD:/etc/lastcrash.txt` at the next boot (docs/02 §13). Note the `ELR` (faulting PC). An **app** fault (EL0) only kills the app: a line `el0: <name> (pid N) killed: …` in `kmsg` (with the PC and the fault address: `aarch64-none-elf-addr2line -e user/<name>.elf <pc>`), then a line `el0: <name> backtrace: …` with the callers' return addresses (give them to `addr2line -f -C` with the unstripped program), and a notice on the desktop. `/bin/faulttest` and `/bin/el0test` exercise both paths.
- **`addr2line`**: `aarch64-none-elf-addr2line -e kernel8-rpi4.elf <ELR>` to locate the faulting line (keep the unstripped `.elf` next to the `.img`).
- **Remote shell**: `/bin/telnetd` (autostarted, TCP port 23) serves the `cmd` shell over the network; from the dev machine, `python tools/onyx-telnet.py <pi-ip>` (or any telnet client) — handy to run `kmsg`, `ps`, `kill`, tests, without the Pi's keyboard. Its server side uses the ABI v37

`kapi_tcp_listen`/`kapi_tcp_accept`. A session whose client goes without `exit` is ended with what it runs (`shellend.h`: Ctrl-C, the end of the shell's input, its output still read; `kill_pid` after 3 s — the kernel ends a dead process's children): a script that must leave a daemon on the Pi starts it with `run SD:/bin/<tool> …`, not in the foreground of its session. A pipe has no "reader gone": a program writing to one nobody reads waits for ever (in `ps`: state R, no system call), which is what dropped sessions used to leave. A connection from which not one byte came in 5 minutes is ended too (the kernel's "deaf" connections, `docs/HANDOFF.md`, *Testing on the Pi*).

- **Remote desktop**: `/bin/vncd` (autostarted, VNC port 5900) — watch and drive the GUI from any VNC viewer, no monitor needed. It grabs the screen with `kapi_screen_grab` and injects input with `kapi_inject_pointer`/`kapi_inject_key` (ABI v38); it is a newlib program linked with the vendored zlib (`ZLIB_PROGS` in `user/BinUtils/Makefile`).
- **Window-level remote desktop**: `/bin/rdpd` (autostarted, port 3390) + the Windows client `pc/OnyxRemote` (.NET Framework 4.8, built by `sh pc/build.sh` into `pc/dist/OnyxRemote.exe`): a connection dialog first (`ConnectDialog.cs`: the address, the options, Full screen, Terminal, Connect -- the connection made on a thread, `MainForm.Begin`; `Settings`, shared with the session's window), then one MDI window (+ `TelnetForm`: a telnet console on telnetd, its own window -- IAC dropped, `ESC [2J` clears, the line sent whole + CR LF, telnetd echoes it) -- the Onyx menu bar across its top (stretched: `BarView`, its drop-down menus in a colour-keyed layer over the children), each Onyx window a child window (native frame or the Onyx one), the desktop + bubbles as the MDI area's background -- composited by the PC: nothing is composited for it on the Pi. `rdpd` lists the windows (ABI v56 `uk_win_list`), reads only those whose `gen` changed (`uk_win_read`), compares their 64 × 64 tiles with what the client has and sends the changed runs, LZ4-compressed (its own compressor, the standard block format), 32 or 16 bits a pixel; a frame when `chromeGen` changes; the desktop (`KAPI_WIN_DESKTOP`, the wallpaper + the backmost windows) only when the client asks for it (message 7). The hello carries the kernel's kapi version (the client warns below 56). The client sends the pointer in window coordinates (rdpd adds the window's place, raises it when clicked), keysyms (`user/BinUtils/remotekeys.h`, shared with vncd: specials, modifiers, letters / digits as held keys only) and the characters typed (the PC's layout). The protocol is described at the top of `user/BinUtils/rdpd.c`. **Loss tolerance** (Wi-Fi): a round is sent only when something changed, and only while the server has **credit** (each READY gives one); a client setting hello option bit 2 gets `CAPS` (9: protocol 2 since kapi v96 -- the client may send `MOVE` (client 9: u32 id, s16 x y), a window dragged on the PC put at the same place on the Pi, `uk_win_place`) and 3 rounds in flight (an older client: lock-step, its READYs alone); a round unanswered for 250 ms makes rdpd send small `PING`s (10) -- the PC's dup ACKs make Circle resend its lost tail segment at once instead of after its 1 s minimum RTO, and the client's `PONG`s (client 8) do the same for a READY the PC lost; a pipelined client also gets a PING every 2 s when idle and is dropped after 12 s of silence. **COPY** (server 12: u32 id, u16 x y w h, s16 dx dy -- move a rectangle of the client's copy of a window's content, the two places may overlap): only to a client that sets hello option bit 3 (an older rdpd ignores the bit). `send_content` maps the changed tiles first; with 8 or more, `copy_find` takes 6 anchors (16 pixels in the middle of changed tiles, along a row and down a column), looks for each on the same row / column of the client's copy (`prev`), grows the rectangle where the current pixels are the old ones shifted (checked pixel for pixel: a wrong guess costs only the search), sends the largest (at least 2 tiles) and does it in `prev` -- up to 3 a window a round; the tile diff then sends what is new. A window where nothing is found is searched again after 2, 4 .. 16 rounds (a game, a video). Per window, so the desktop's windowed mode and the console's full screen alike. Onyx Remote (`Connection.cs`) sends from its own thread, coalesces the pointer's moves (one per 16 ms, the latest; buttons, wheel and keys at once, in order), applies a round to its model only at its END, validates every message (a damaged one is

skipped, a damaged stream reconnects) and reconnects by itself (0.5 .. 8 s back-off, the windows kept). Host tests: `sh tools/tests/run_rdpd_test.sh` (a mock kapi, a Python client; `rdpd_copy_test.py` in it: a band slid, a window scrolled, the client's copy right after every round with and without bit 3), `sh tools/tests/run_rdpd_pipeline_test.sh` (rdpd against AppKit with fake windows -- `tools/tests/rdpd/rdpdhost.c`, `RDPD_ANIM=2` a window scrolling --, the current and an older rdpd when it still builds, a Python client both ways, and Onyx Remote's own `Connection.cs` when the .NET SDK is there: COPY applied right after every round), and `unshare -rn python3 tools/tests/rdpd/loss_bench.py` (rounds a second over a loopback that drops 20 % of the packets).

- **PocketTerm Simulator** (`pc/PocketTermSim`, `pc/dist/PocketTermSim.exe`; docs/04 *The PocketTerm35 on a PC*): the Waveshare PocketTerm35's face on a PC, a second client of `rdpd`. It **links** Onyx Remote's `Connection.cs`, `KeyMap.cs` and `Lz4.cs` (no copy), composites the windows into one screen itself (`Compositor.cs`, as `Screenshot.cs`) and sends every control as rdpd's `KEY` (X11 keysyms: rdpd has no gamepad message; the chords in `PocketTermSim.buttons.ini`). Build: `sh pc/PocketTermSim/build.sh` (the .NET SDK; it touches nothing else in `pc/dist`). Test, with no Pi: `python pc/PocketTermSim/test/run_test.py` -- a fake rdpd (`test/fake_rdpd.py`: the hello, one picture, a log of the input received) and the program's `--selftest`, which presses every control (the maker's 55 keys, the Fn layer, the six combinations the device keeps): 190 steps checked against the default mapping. `python pc/PocketTermSim/test/fake_rdpd.py` alone, then `PocketTermSim.exe 127.0.0.1`, shows the simulator over a mock-up's picture.
- **Serial console**: `config.txt` must have `enable_uart=1` (PL011 clock). The boot log goes **also** to the HDMI screen (`CScreenDevice`) so it is readable without a serial cable.
- **Post-mortem console**: on a kernel panic, the compositor stops and the logger is shown on the framebuffer (see Kernel internals §13).

12.1. The Pi's binaries on a PC: the app runner `onyxrun` (`tools/onyxrun`)

`onyxrun` runs Onyx's programs **as they are on the card** -- their AArch64 ELF, AppKit and the kits (`SD:/lib/*.so`), Elegant -- on a PC, Windows (`onyxrun.exe`, one static program) or Linux. Their code runs on Unicorn Engine (the AArch64 target of QEMU's translator); their system calls are answered by the runner, which stands for the kernel: the same kapi page as the kernel's (`KAPI_TABLE_VA`, the stubs `mov x8, #slot; svc #0`, the kernel's own EL0 blob from `el0blob.S`, AppKit's table at `APPKIT_TABLE_VA`), the same address map (the program at 8 GB, the libraries at 16 GB+, the windows' buffers at their slots), SD: on a folder of the PC (the repository's `sdcard/` by default), RAM: on another. The plan and the choices are `docs/APP-RUNNER-STUDY.md`.

Built, ready to use on Windows: `tools/onyxrun/dist/onyxrun.exe` (one static .exe, nothing to install; rebuilt and committed when the runner changes: `make -C tools/onyxrun win`, then `x86_64-w64-mingw32-strip -o tools/onyxrun/dist/onyxrun.exe tools/onyxrun/build/onyxrun.exe`). Run from the repository it finds `sdcard/` by itself (beside the .exe or above the current folder); elsewhere give it `--root`: `tools\onyxrun\dist\onyxrun.exe clock`. It carries Unicorn (GPL-2.0): the .exe is distributed under GPL-2.0, its sources stay MIT (docs/LICENSING.md).

```
sh tools/onyxrun/get_unicorn.sh      # once: Unicorn built for Linux and (with mingw-w64) Win
dows
make -C tools/onyxrun                # build/onyxrun          (Linux)
make -C tools/onyxrun win            # build/onyxrun.exe      (Windows, cross-built; one .exe)

onyxrun echo hello                   # a /bin tool: its output on the PC's console
onyxrun cmd -c "ls SD:/etc | sort"   # the shell, its pipes, its children
```

```
onyxrun clock                          # an app: Elegant started first, the screen in a window
of the PC
onyxrun --headless --shot out.bmp --shot-after 8000 clock      # no window: the screen writte
n after 8 s
onyxrun --headless --input "wait 5000; click 640 300; key hello; enter; shot a.bmp" notes
onyxrun --desktop --root copy-of-sdcard   # the whole desktop: Elegant, the session, the serv
ices without network
```

The programs write into the folder SD: stands for (Setup at the first start, the settings, the apps' files): give `--root` a copy of `sdcard/` when the repository's must stay as it is.

Options: `--root DIR` (the folder for SD:), `--ram DIR`, `--cwd PATH`, `--screen WxH` (1280x800), `--console` / `--gui`, `--trace` (every system call on stderr; also `ONYXRUN_TRACE=1`), `ONYXRUN_PROFILE=ms` (where the threads run, every ms). The PC's keys reach Elegant as a Pi's USB keyboard gives them (UTF-8, Enter `\n`, Backspace 0x7F, the VT100 escapes); `--input` plays a script (`wait MS`, `move` / `click` / `rclick` / `down` / `up X Y`, `wheel X Y N`, `key TEXT`, `enter`, `esc`, `tab`, `bs`, `mods N`, `shot FILE.bmp`, `waitfile FILE` + `include FILE`: a test looks at a shot, then says where to click).

How it is made (the sources' comments say the rest): every Onyx process lives in the one host process, its user range one host reservation (a pointer of a system call is translated); one Unicorn engine and one host thread a guest thread, one thread of a process running at a time (its lock), the processes in parallel; `k_*.cpp` are the kapi's slots (`KAPI (slot, function)`), `k_ws.cpp` the graphics server's side (`kern/wsrv.h`), `display.cpp` the PC's window (Win32) or the headless screen.

What it does not show: the kernel (not run), the weak memory ordering (an x86 PC orders more than the A72: a missing barrier passes), the caches (a missing cache clean passes), the timing. The GPU's stock calls (`gpu_draw`, `gpu_texture`, `gpu_render`: gpucomp, the 3D apps) are drawn by the PC in native code (`k_gpu.cpp`); the network is the PC's sockets (`k_net.cpp`). Not there yet (their calls answer as a Pi without them would): the apps' own QPU programs (`gpu_program`: gcemu, n64emu take their CPU path), the sound, the gamepads, USB, GPIO. The Pi stays the reference.

13. Known pitfalls

- **`uk_win_resize` keeps the buffer's size and row pitch.** The window buffer is made once, at the size given to `uk_win_create`; resizing changes the size shown (clamped to that buffer), not the buffer, and the compositor reads its rows with the **creation width** as pitch. Draw with that pitch: `canvas.adopt (uk_win_resize (w, h), w, h, creationWidth)`. To offer several sizes, create the window at the largest one (the emulators: their 4x zoom). Adopting with pitch = the new width gave a doubled, interlaced picture in `gbemu` at zoom 2x. The window's **frame follows the new size**: call `uikit::uk_decorate_window ()` after the resize so the title bar and borders are redrawn at it (else the frame keeps the old drawing).

- **`kapi_mkdir` / `kapi_remove` / `kapi_rename` return 0 on success** (-1 on failure), like POSIX — not a boolean. Test `== 0` for success (a `!kapi_rename (…)` "failure" check silently treated every successful move as failed; fixed in `trash.h`, `fsutil.h`, the File Viewer and `ftpd`).

- **`RAM:` is lost at a restart** (docs/02 §16, §5.3): keep only what can be rebuilt there, and expect a file there to be missing. `kapi_vol_info` returns −1 when there is no `RAM:` (an older kernel, `ramfs=0`): fall back to the card.

- **`kapi_save_file` returns the bytes written** (≥ 0; 0 for an empty file) or -1 — test `< 0` for a failure, or `== n` (a full card writes less). A `!= 0` test reported every save failed on the Pi (Koton's

`Doc::save`), and `fsutil.h`'s copy treated an empty file as a failure; the desktop simulator's fake answered 0 and hid it (it answers the length now, as the kernel).

- **`kapi_tcp_send` returns the bytes queued.** A short count means a 5 s timeout (the network's queue full) after those bytes: they will be sent, so send the rest again (or give up). Before 2026-10-01 the count was wrong after a timeout part-way (Circle's `CSocket::Send` answered the error, its earlier chunks already queued: `docs/05` §22), and resending "the rest" put bytes twice in the stream.

- **Host tests** (`tools/tests/`): `run_ramfs_test.sh` (the kernel's RAM file system, `kernel/sys/ramfs.cpp`, built for the PC under AddressSanitizer: random operations against a model, a full volume, files removed while open, the memory given back — run it after touching `sys/ramfs.cpp`), `run_circlenet_test.sh` (the Circle fork's TCP: duplicate ACKs, the RTO, `CSocket::Send`'s count -- run it after touching `circle/lib/net`), `run_fs_test.sh` (the Circle fork's FatFs + `diskio.cpp` sector cache on a RAM disk: 4000 random file operations checked against a model, cache on and off, same disk image — run it after touching `circle/addon/fatfs`), `run_trash_test.sh` (trash.h + fsutil.h against a mock kapi with the kernel's return conventions) and `run_ftpd_test.sh` (ftpd over real sockets, driven by Python's `ftplib`: login, LIST/NLST, RETR/STOR round trip, MKD/RMD, RNFR/RNTO, DELE, root jail, PORT, two concurrent sessions) and `run_ftpfs_test.sh` (the ftpfs FTP client against pyftpdlib — `pip install pyftpdlib` — and against ftpd: LIST/MLSD, RETR, a 300 KB STOR round trip, MKD, RNFR/RNTO, DELE/RMD, a missing file, reconnecting after a dropped control link). The socket mock reproduces Circle's "one segment per receive, the rest is dropped". Run them after touching those files.

- **Hardware float is opt-in.** Apps are integer-only by default (`-mgeneral-regs-only`); the kernel now saves the full FP/SIMD state on every trap, so an app may opt into `float`/`double` by building without `-mgeneral-regs-only` and with `-mcpu=cortex-a72` (see §5). Preemption saves the full FP state too (`El0IrqEntry` builds the whole frame, q0–q31 included, before yielding); keep it that way if the preemption path changes — the cooperative `TaskSwitch` only keeps `d8–d15`.

- **L3 tables shared with the kernel.** On the kernel side, never free an L3 table from the user area without checking that it is not shared with the kernel's L2 (cf. Kernel internals §4). Otherwise: global corruption.

- **Do not free the EL0 table and code pages** (the kapi table at 14 GB, the stubs + blob at 14 GB + 64 KB). They are global to the kernel, shared by every process; the destruction of an address space already skips them.

- **`DEPTH=32` for Circle.** `GImage` renders 32-bit; forgetting `-d DEPTH=32` (or changing `DEPTH` without `make clean` in `circle/lib`) gives wrong colors/breakage.

- **The kernel is not preemptive.** Apps are preempted, but kernel code (a `kapi_*` call, a kernel task) is not: a long kernel loop that never yields still stops every other task. Put `if (IsReschedPending()) Yield();` in any such loop (the stall watchdog's `stall:` log lines point at them).

- **Threads and the kernel** (v67): a kapi that waits (`Yield`, an event) may now find another task of the **same** process inside the kernel when it resumes — per-process state touched across a wait (the window's event queue, a dialog, a file handle) is no longer the caller's alone. And `CScheduler::EnumerateTasks`'s callback must not yield (the task list is a linked list the reaper frees

nodes from). An app's tasks are one group (`TerminateTask` on one ends them all; `TerminateGroup`); list the process once (`pAS->GetMainTask ()`).

- **Circle LF renormalization.** On Windows, Circle is checked out in CRLF; renormalize once (cf. §2) otherwise the build breaks.
- **The right Circle.** Patch `Zircon/circle`, not another clone.